Developing a Centralized Role Based Event Coordination Platform for Multi Departments University Workflow Management.

By

Osamah Ahmed Mohammed Al-Naggar

TP078781

APD3F2601CS

A report submitted in partial fulfilment of the requirements for the degree of

B.Sc. (Hons) Computer Science

at Asia Pacific University of Technology and Innovation.

Supervised by Mr. MUSTAFA OTHMAN

2nd Marker: Ms. AIDA RAIHANAH

DECLARATION OF THESIS CONFIDENTIALITY

Author's full name: Osamah Ahmed Mohammed Al-Naggar

IC No./Passport No.: [IC No. / Passport No.]

Thesis/Project title: Developing a Centralized Role Based Event Coordination Platform for Multi Departments University Workflow Management

---

I declare that this thesis is classified as:

- ☐ CONFIDENTIAL
- ☐ RESTRICTED
- ☑ OPEN ACCESS

I acknowledged that Asia Pacific University of Technology & Innovation (APU) reserves the right as follows:

1. The thesis is the property of Asia Pacific University of Technology & Innovation (APU).
2. The Library of Asia Pacific University of Technology & Innovation (APU) has the right to make copies for the purpose of research only.
3. The Library has the right to make copies of the thesis for academic exchange.

Author's Signature: Osamah Ahmed Mohammed Al-Naggar

Date: 1 September 2026

Supervisor's Name: MR. MUSTAFA OTHMAN

Date: 1 September 2026

Signature: [Signature]

LIBRARY CATALOGUING DETAILS

First Name: Osamah
Middle Name (only if applicable): Ahmed Mohammed
Last Name: Al-Naggar
Title of the Final Year Project / Dissertation / Thesis : Developing a Centralized Role Based Event Coordination Platform for Multi Departments University Workflow Management
Abstract : This project presents the design and development of a centralized, role-based event coordination platform aimed at addressing the inefficiencies inherent in manual and fragmented event management processes within academic institutions. The research problem that motivates this project is the absence of a structured digital infrastructure capable of managing multi-departmental coordination, resulting in communication breakdowns, task duplication, and lack of process transparency during event organization. To address this problem, a comprehensive investigation was conducted through a review of existing event management systems, analysis of similar works, and a study of relevant coordination and workflow management theories and technologies. The preliminary findings indicate that current systems fail to support independent departmental workflows within a unified platform, creating significant operational gaps. The proposed system resolves these gaps by providing a structured environment where a single event request is distributed across multiple departments simultaneously, each operating through its own independent approval and execution workflow. The outcome of the development stage is a fully functional platform, built with Angular, Flask, and a PostgreSQL database, that enhances institutional coordination efficiency, reduces manual overhead, and improves task accountability across all departments involved. The significance of this project lies in its direct contribution to SDG Goal 9: Industry, Innovation and Infrastructure, by establishing a resilient digital coordination infrastructure within the university while introducing process-level innovation through structured, role-based workflow management.
A few keywords associated with the work : Event Management, Role-Based System, Workflow Coordination, Departmental Collaboration, Digital Infrastructure, SDG 9
General Subject: Computer Science
Date of Submission : 1st September 2026

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to all those who contributed to the completion of this report.

First and foremost, I would like to thank my supervisor, **MUSTAFA OTHMAN**, for his guidance, continuous support, and for giving me the opportunity to undertake this project. His direction throughout the semester has been invaluable in shaping the direction and quality of this work.

I would also like to extend my appreciation to **AIDA RAIHANAH**, second marker, for the time and effort that will be dedicated to reviewing and evaluating this report.

I am deeply grateful to **my parents** for their unwavering support, encouragement, and patience throughout this journey. Their belief in me has been a constant source of motivation.

Finally, I would like to thank my friends, **Adel**, **Sohaib and Ibrahim**, for their companionship, encouragement, and for helping me carry the heavy weight of this semester throughout the academic year.

ABSTRACT

This project presents the design and development of a centralized, role-based event coordination platform aimed at addressing the inefficiencies inherent in manual and fragmented event management processes within academic institutions. The research problem that motivates this project is the absence of a structured digital infrastructure capable of managing multi-departmental coordination, resulting in communication breakdowns, task duplication, and lack of process transparency during event organization. To address this problem, a comprehensive investigation was conducted through a review of existing event management systems, analysis of similar works, and a study of relevant coordination and workflow management theories and technologies. The preliminary findings indicate that current systems fail to support independent departmental workflows within a unified platform, creating significant operational gaps. The proposed system resolves these gaps by providing a structured environment where a single event request is distributed across multiple departments simultaneously, each operating through its own independent approval and execution workflow. The outcome of the development stage is a fully functional platform, built with Angular, Flask, and a PostgreSQL database, that enhances institutional coordination efficiency, reduces manual overhead, and improves task accountability across all departments involved. The significance of this project lies in its direct contribution to SDG Goal 9: Industry, Innovation and Infrastructure, by establishing a resilient digital coordination infrastructure within the university while introducing process-level innovation through structured, role-based workflow management.

Keywords: Event Management, Role-Based System, Workflow Coordination, Departmental Collaboration, Digital Infrastructure, SDG 9

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

CHAPTER 1: INTRODUCTION

1.1 Introduction

In contemporary academic institutions, the organization of events plays a vital role in fostering community engagement, academic collaboration, and institutional development. Managing such events, however, requires the coordinated effort of multiple departments, each responsible for a distinct set of services ranging from catering and logistics to audio-visual support and transportation. As universities grow in scale and the frequency of events increases, the demand for a structured, efficient, and transparent coordination mechanism becomes increasingly critical.

Currently, event coordination in many academic institutions relies on informal communication channels, manual request processes, and disconnected tools that are not

purpose-built for multi-departmental collaboration. This results in recurring challenges such as miscommunication between departments, duplicated efforts, lack of real-time progress visibility, and an overall absence of accountability within the event management process.

This project proposes the development of a centralized, role-based event coordination platform designed for university-wide event management at Asia Pacific University of Technology and Innovation (APU). The platform enables an applicant to create a single structured event request that is automatically distributed to all relevant departments within the institution. These departments include Logistics and Facilities, Transport Services, Photography Services, A/V Services, Student Services, and Food and Beverage services. Each department operates through its own independent workflow, allowing it to review, approve, and execute assigned responsibilities without interference from other departments.

The platform is developed using Angular for the frontend, Flask with Python for the backend, and a PostgreSQL database hosted on Supabase. Beyond the internal coordination workflow, the platform also publishes approved events to a public-facing portal where members of the university and invited external guests can discover and register for events, and it includes an AI assistant that answers questions about events and requests within the boundaries of each user's own access rights. This project directly contributes to **SDG Goal 9: Industry, Innovation and Infrastructure** by establishing a resilient digital coordination infrastructure within the university.

1.2 Problem Background

1.2.1 The Core Problem:

Event coordination within academic institutions requires the continuous exchange of information, task follow-up, and collaboration among multiple departments. However, when coordination relies on informal communication channels, manual request processes, and direct interactions, the workflow becomes fragmented and difficult to manage. Such fragmentation disrupts communication flow, increases task-switching, and reduces overall efficiency, as

activities are handled across multiple unstructured channels instead of within a unified system (Moy et al., 2023).

This situation is further intensified by the absence of a centralized system that integrates communication, tracking, and data management. Without centralization, event-related actions, approvals, and updates are not consistently recorded, resulting in limited visibility and weak transparency. Consequently, event coordinator faces difficulties in monitoring progress, ensuring accountability, and analyzing performance. Prior studies highlight that the lack of integrated data systems restricts the ability to generate meaningful insights and support informed decision-making, ultimately weakening operational effectiveness (Matheus et al., 2020; Khan et al., 2022).

1.2.2 Problem Statements

The following problem statements highlight critical limitations in current event coordination practices. These issues contribute to inefficiencies and operational challenges in event management.

1.2.2.1 Fragmentation of Communication and Coordination Processes

The use of multiple disconnected communication channels, including messaging applications, phone calls, and verbal instructions, leads to fragmented coordination processes. Information becomes scattered across different platforms rather than being maintained within a centralized system, increasing the risk of miscommunication, missed requests, and inconsistencies in execution. This aligns with findings that workflow fragmentation significantly disrupts communication and coordination efficiency (Moy et al., 2023).

1.2.2.2 Inefficiency and Delays in Event Preparation Workflows

Manual and sequential handling of event requests limits the ability to process tasks concurrently, resulting in delays in event preparation. relevant parties involved in event coordination often depend on prior confirmations before proceeding, which slows down workflow execution. Research indicates that fragmented and non-integrated workflows contribute to inefficiencies and delays due to increased task dependency and interruptions (Moy et al., 2023).

1.2.2.3 Lack of Role Clarity, Structured Processes, and Guided Workflow Execution

The absence of structured digital workflows and clearly defined roles creates ambiguity in task responsibilities. Without a guided process, relevant parties involved in event coordination may duplicate tasks or overlook essential responsibilities, leading to coordination errors. Studies on fragmented workflows emphasize that lack of structured processes increases operational complexity and reduces workflow consistency (Moy et al., 2023).

1.2.2.4 Weak Accountability and Lack of Transparency in Task Ownership and Decision-Making

In manual coordination environments, approvals, actions, and task ownership are often not systematically recorded. This lack of traceability reduces transparency and weakens accountability, making it difficult to identify responsibility for decisions or delays. Research shows that systems lacking traceability and structured data records negatively impact transparency and accountability in organizational processes (Khan et al., 2022; Matheus et al., 2020).

1.2.2.5 Limited Data Availability, Insights, and Analytical Capabilities for Decision-Making

The absence of centralized data collection and analytical tools limits the ability to monitor event progress and evaluate performance. Without access to real-time data and historical insights, stakeholders cannot make informed decisions or identify areas for improvement. Prior studies highlight that data-driven systems and dashboards are essential for generating insights and supporting effective decision-making (Matheus et al., 2020).

1.3 Project Aim

To develop a centralized, role-based event coordination platform using Angular, Flask, and PostgreSQL to be employed in university-wide multi-departmental event management at Asia Pacific University of Technology and Innovation (APU).

1.4 Objectives

1. To reduce the complexity and difficulty of managing university events by replacing manual, disconnected communication channels including WhatsApp, phone, and verbal instructions. and implement structured digital coordination system.
2. To improve efficiency and reduce delays in event preparation by enabling simultaneous request distribution and independent processing by different departments.
3. To minimize task duplication and miscommunication by clearly defining responsibilities and ensuring that each department only handles tasks relevant to its role.
4. To strengthen accountability across departments by maintaining a clear record of approvals, actions taken, and task ownership throughout the event lifecycle.
5. To support data-driven decision-making in event coordination through the implementation of a dashboard that visualizes key analytics related to event activities and user engagement.

1.5 Scope

This section defines the boundaries of the proposed system by clearly outlining its functionalities and limitations. It helps to clarify what the system is intended to achieve within the project and ensures that the development remains focused on the defined objectives without extending beyond its intended context.

What will be included:

The proposed system focuses on the development of a centralized, role-based event coordination platform specifically designed for Asia Pacific University (APU). The system is tailored to align with the institution's internal operational structure, ensuring that all event coordination activities are managed within a unified and controlled digital environment.

1- **End-to-End Event Coordination Lifecycle**: The system supports the complete lifecycle of event coordination, starting from event creation and submission by the applicant, who may be a student, lecturer, or staff member. Once an event is created, it is automatically distributed to the relevant departments for processing. Each department operates through an independent approval workflow, ensuring that decisions are made according to their internal procedures. The system also supports revision and resubmission handling, allowing requested changes to be tracked and managed within the same workflow. Additionally, tasks can be assigned and executed by department staff, while real-time progress tracking ensures visibility across all ongoing activities. All completed events are stored in a structured event history log for future reference and analysis. Approved events may also be published to the university's event portal, so that the lifecycle continues beyond internal task completion into event announcement, attendance registration, and post-event records.

2- **Support for Multiple University Departments**: The system is designed to accommodate multiple operational departments within APU, each department is responsible for handling its own domain-specific tasks, ensuring clear separation of responsibilities and reducing operational overlap. This separation ensures that each department operates within clearly defined boundaries, which minimizes confusion in task ownership and prevents duplication of work. By structuring departmental

involvement in a centralized platform, coordination between departments becomes more systematic and traceable, improving overall operational alignment during event execution. The routed service departments are Logistics and Facilities, Transport Services, Photography Services, A/V Services, and Student Services. Food and Beverage requirements are routed separately according to role, while funding and purchase requirements are captured on the request for financial review rather than routed to a service department.

3- **Role-Based Access Control**: The system implements twelve distinct user roles: Student, Lecturer, Staff, Head of School, Head of Department, CFO, Cafeteria Admin, Cafeteria Manager, Cafeteria Staff, Club Admin, System Admin, and External User. Roles are not held globally but are assigned per organisational unit, so the same person may carry different responsibilities in different departments. The pages each role can reach are controlled through configurable access grants rather than being fixed in the interface, allowing an administrator to adjust visibility without changing the system itself. This role-based structure is essential for maintaining system security, preventing unauthorized actions, and ensuring that each user interacts only with functions relevant to their responsibilities. It also strengthens workflow governance by clearly defining decision-making authority at each level of the system.

4- **Flexible Task Handling Mechanism**: Once a department approves its part of a request, the Head of Department assigns the resulting tasks to individual staff members. Assignment is made for each individual requirement rather than for the request as a whole, so several staff members may work on the same request at once, while requirements that can only be handled by one person are limited to a single assignee. This flexibility allows departments to adapt task execution based on workload and operational needs. It also improves resource utilisation by allowing work to be distributed according to urgency, complexity, and staff availability, ensuring more efficient task completion and workload balancing.

5- **Responsive Web Accessibility:** The platform is designed as a web-based system that is fully responsive across desktop and mobile devices. This ensures accessibility

without requiring installation of native applications. By supporting multiple device types, the system enhances usability and ensures that users can access the platform anytime and anywhere. This is particularly important in even coordination scenarios where real-time updates and rapid response are required, enabling continuous interaction with the system regardless of device constraints.

6- **Public Event Portal and Registration**: Approved events may be published to a portal where members of the university, and external guests holding a self-registered account, can browse and search events, view an event calendar, save events of interest, and register to attend. Registrations are approved by the organiser, and attendees may choose which reminders they wish to receive.

7- **Club Management**: The system maintains a directory of university clubs organised by category, together with club membership, join requests with approval or rejection, requests to change a club president, and the activation or deactivation of a club. Club records are managed by a dedicated Club Admin role.

8- **Cafeteria and Food & Beverage Operations**: Cafeteria outlets are managed within the system, including the assignment of managers and staff, menu and pricing management, orders generated from approved food and beverage selections on event requests, and the tracking of those orders through preparation to fulfilment.

9- **Venue and Catalogue Management**: Venues and the selectable options used across request forms are maintained as managed catalogues rather than free text, so that requests are built from controlled and consistent data and reporting remains reliable.

10- **Automated Email Notification**: The system sends email notifications at each decision point in the workflow, together with scheduled reminders for upcoming events and escalation notices when an approval is running late relative to the event date.

11- **Role-Based Analytics Dashboards**: Dashboards are provided for departments, cafeterias, the Food and Beverage unit, and the CFO, each presenting the measures relevant to that role across areas such as demand, capacity, workflow progress, timeliness, and cost. Panels can be filtered and exported for reporting purposes.

12- **AI Assistant**: An assistant built into the platform answers questions about the system and about the user's own records in ordinary language. Every answer is restricted to information the user is already permitted to see, and all interactions are recorded so that administrators can review how the assistant is used.

13- **System Administration and Configuration**: Administrators manage users, organisational units, and role assignments, control which pages each role can reach, maintain the option catalogues used on request forms, and configure institutional policies. Deleted records are retained for a recovery period during which they may be restored before permanent removal.

14- **Security Controls**: Access to the system is protected by token-based authentication with rotating sessions, hashed passwords, controlled registration, and request rate limiting. Every permission decision is made by the server rather than the browser, so the interface cannot be used to obtain access that the user has not been granted.

What Will Not Be Included:

1- **Financial and Budget Management**: The system does not include payment processing, invoicing, or the reconciliation of expenditure against an institutional budget, as the focus is on coordination workflows. Cost information for requested items is captured on the request, and requests above a configurable attendance threshold are routed to the CFO for financial approval, but the movement and reconciliation of money remain the responsibility of separate institutional financial systems. By narrowing the scope in this way, the system avoids unnecessary complexity and maintains a clear focus on event coordination processes.

2- **Multi-Institution Deployment:** The platform is designed exclusively for APU and will not be extended to other universities or external institutions. This decision ensures that the system is highly customized to APU's internal workflows, policies, and operational structure. By limiting deployment to a single institution, the system can be optimized for specific requirements rather than generalized for multiple environments.

3- **External System Integration:** Integration with third-party platforms or existing university management systems is not included in the current implementation. However, it may be considered in future development phases depending on institutional requirements. This limitation ensures that the current system remains focused on core functionality while maintaining flexibility for potential scalability and future enhancements.

4- **Native Mobile Applications**: The platform is delivered as a responsive web application accessed through a browser. No separate Android or iOS application is developed, as browser access already covers the required devices without the additional overhead of maintaining native clients.

1.6 Potential Benefits

The implementation of a centralized event coordination platform is expected to deliver both operational and organizational improvements by replacing fragmented manual processes with a structured digital workflow system. This section explains the advantages of the proposed system by presenting the expected improvements in event coordination practices, which are categorized into tangible and intangible benefits to distinguish measurable outcomes from qualitative organizational impacts.

Tangible Benefits:

1- **Reduced Coordination Time**: The system reduces coordination time by replacing fragmented communication channels such as email, phone calls, and verbal instructions with a centralized workflow system. This eliminates repeated follow-ups and ensures that event requests are processed efficiently through a structured digital pipeline. As a result, coordination becomes faster and more streamlined, reducing delays caused by manual communication dependencies and improving overall event preparation efficiency.

2- **Structured Approval Tracking**: Every event request follows a clearly defined approval workflow where all decisions, status changes, and actions are recorded. This creates an auditable trail that improves transparency and ensures that approval processes are consistently followed across departments. The presence of structured tracking also reduces uncertainty in decision-making and allows stakeholders to clearly understand the progress and status of each request.

Reduced Task Duplication: Role-based access control ensures that users only interact with tasks relevant to their assigned responsibilities. This prevents overlapping work and reduces redundancy in task execution, improving overall operational efficiency. By clearly defining task ownership within the system, duplication of effort is minimized and workflow execution becomes more organized and controlled.

3- **Centralized Event History**: All completed events and their associated workflows are stored in a centralized system database. This allows the institution to maintain

structured historical records, which can be used for future planning and reference. The availability of historical data also supports institutional learning by enabling analysis of past events to improve future coordination strategies.

4- **Faster Issue Resolution**: The system includes a revision and feedback loop mechanism that allows departments to flag issues and resubmit requests with clear explanations. This structured approach reduces delays in resolving operational problems by ensuring that feedback is handled within the same system workflow rather than through external communication channels. As a result, issue resolution becomes more efficient and traceable.

5- **Automated Notification and Reminders**: Decisions, task assignments, and upcoming deadlines are communicated automatically by email, removing the need for manual follow-up messages and ensuring that the person responsible for the next step is informed as soon as it becomes theirs.

6- **Reporting and Analytical Output**: Role-based dashboards convert the records held in the system into measures that each unit can act on, and any panel can be exported for use in institutional reporting, so that coordination data becomes a planning resource rather than an administrative by-product.

7- **Faster Access to Information**: The AI assistant allows users to obtain answers about their own events, requests, and tasks by asking in ordinary language, reducing the time spent locating information across different screens while keeping every answer within the user's existing access rights.

8- **Wider Event Reach and Attendance Visibility**: Publishing approved events to a shared portal gives the university a single place where events are announced and attendance is registered, improving visibility of what is taking place on campus and giving organisers an accurate picture of expected attendance.

Intangible Benefits:

1- **Improved Departmental Accountability**: By recording all actions, approvals, and task assignments within the system, each department develops a stronger sense of responsibility. This traceability ensures that accountability is clearly defined and enforceable, reducing ambiguity in task ownership and improving discipline in task execution across departments.

2- **Enhanced Inter-Departmental Trust:** Transparent workflows allow departments to view the progress of interdependent tasks, reducing miscommunication and operational uncertainty. This visibility fosters trust and improves collaboration between departments as all stakeholders operate within a shared and observable system environment, reducing reliance on informal communication.

3- **Better Staff Experience**: The system provides a simplified and structured interface that displays only relevant tasks to each user. This reduces confusion and improves efficiency by allowing staff to focus solely on their assigned responsibilities. As a result, cognitive load is reduced and users can perform their tasks more effectively without being overwhelmed by unnecessary information.

4- **Institutional Professionalism**: The adoption of a structured digital coordination system enhances the institution's operational maturity and reflects a more professional and organized approach to event management. This modernization of workflow practices demonstrates improved governance and aligns with contemporary digital transformation standards in higher education institutions.

5- **Alignment with SDG Goal 9:** The system contributes to Sustainable Development Goal 9 by promoting innovation and improving institutional infrastructure through digital transformation in event coordination processes. This alignment demonstrates the project's contribution to building resilient and efficient institutional systems that support sustainable technological development

1.7 Overview of the Report

This report is structured into six chapters, each designed to systematically present the research, analysis, design, implementation and evaluation of the centralized event coordination platform developed for APU.

Chapter 1 — Introduction

Chapter 1 introduces the overall context of the project, including the domain of event coordination within academic institutions. It presents the problem background supported by academic literature and defines the project aim, objectives, scope, and potential benefits. This chapter also provides an overview of the report structure and outlines the overall project direction.

Chapter 2 — Literature Review

Chapter 2 focuses on domain research across multiple relevant areas related to event management systems. It includes a comparative analysis of existing systems to identify limitations and gaps in current solutions. Additionally, it provides technical justification for the selected technologies and frameworks used in system development.

Chapter 3 — Methodology

Chapter 3 outlines the selected Waterfall-Agile Hybrid with Evolutionary Prototyping development methodology and provides justification for its suitability. It also details the survey-based data collection approach and presents the quantitative analysis of responses from 30 APU students and staff members.

Chapter 4 — Conclusion

Chapter 4 documents the design and implementation of the delivered system. It presents the system architecture, the security and identity model, the use case, activity and sequence models, the database design and data dictionary, the interface design, screenshots of every implemented screen with discussion, and annotated extracts of the source code.

Chapter 5 — Results and Discussions

Chapter 5 presents the testing carried out on the delivered system. It sets out the test plan, the unit test cases designed against the individual programs of the system and the user acceptance testing conducted with representative users, and it discusses the results obtained, the defects identified and the corrections applied.

Chapter 6 — Conclusion

Chapter 6 concludes the project. It critically evaluates what was achieved against the objectives stated in Chapter 1, states the contribution of the project and its strengths, sets out the limitations of the delivered system honestly, and recommends the directions in which the work should be taken forward.

The four chapters above describe the structure of this Investigation Report. The complete Final Year Project documentation continues from this foundation into system design and implementation, testing and evaluation of the delivered platform, and a final conclusion reflecting on the outcome against the objectives stated in this report.

1.8 Project Plan

Table 1: *PROJECT PLAN*

Task	Duration	Start Date	End Date	Status
1. Project Proposal Form	2 Days	09 Feb 2026	10 Feb 2026	Completed
1.1 Introduction	1 Day	09 Feb 2026	09 Feb 2026	Completed
1.2 Problem Background	1 Day	10 Feb 2026	10 Feb 2026	Completed
1.2.1 The Core Problem	1 Day	10 Feb 2026	10 Feb 2026	Completed
1.2.2 Problem Statements	1 Day	10 Feb 2026	10 Feb 2026	Completed
1.3 Project Aim	1 Day	11 Feb 2026	11 Feb 2026	Completed
1.4 Objectives	1 Day	11 Feb 2026	11 Feb 2026	Completed
1.5 Scope	1 Day	12 Feb 2026	12 Feb 2026	Completed
1.6 Potential Benefits	1 Day	13 Feb 2026	13 Feb 2026	Completed
1.7 Overview of IR	2 Days	16 Feb 2026	17 Feb 2026	Completed
1.8 Project Plan	1 Day	17 Feb 2026	17 Feb 2026	Completed
2. Chapter 2	11 Days	18 Feb 2026	28 Feb 2026	Completed
2.1 Introduction	1 Day	18 Feb 2026	18 Feb 2026	Completed
2.2 Domain Research	4 Days	19 Feb 2026	22 Feb 2026	Completed
2.3 Similar Systems	3 Days	23 Feb 2026	25 Feb 2026	Completed

2.4 Technical Research	2 Days	26 Feb 2026	27 Feb 2026	Completed
2.5 Summary	1 Day	28 Feb 2026	28 Feb 2026	Completed
3. Chapter 3	15 Days	01 Mar 2026	15 Mar 2026	Completed
3.1 Introduction	1 Day	01 Mar 2026	01 Mar 2026	Completed
3.2 Methodology	3 Days	02 Mar 2026	04 Mar 2026	Completed
3.4 Methodology Discussion	2 Days	05 Mar 2026	06 Mar 2026	Completed
3.5 Justification	2 Days	07 Mar 2026	08 Mar 2026	Completed
3.6 Activities & Processes	3 Days	09 Mar 2026	11 Mar 2026	Completed
3.7 Data Gathering Design	2 Days	12 Mar 2026	13 Mar 2026	Completed
3.8 Analysis	2 Days	14 Mar 2026	15 Mar 2026	Completed
3.9 Summary	1 Day	15 Mar 2026	15 Mar 2026	Completed
4. Chapter 4	5 Days	16 Mar 2026	20 Mar 2026	Completed
4.1 Achievements	1 Day	16 Mar 2026	16 Mar 2026	Completed
4.2 SDG Alignment	1 Day	17 Mar 2026	17 Mar 2026	Completed
4.3 Gaps & Solutions	1 Day	18 Mar 2026	18 Mar 2026	Completed
References	2 Days	19 Mar 2026	20 Mar 2026	Completed

Appendices	4 Days	21 Mar 2026	24 Mar 2026	Completed
Review & Finalization	5 Days	25 Mar 2026	29 Mar 2026	Completed
Submission	1 Day	22 Apr 2026	22 Apr 2026	Completed
5. FYP Part 2 — System Development	55 Days	27 Apr 2026	10 Jul 2026	Completed
5.1 System and Database Design	10 Days	27 Apr 2026	08 May 2026	Completed
5.2 Backend Development	20 Days	11 May 2026	05 Jun 2026	Completed
5.3 Frontend Development	20 Days	08 Jun 2026	03 Jul 2026	Completed
5.4 AI Assistant and Dashboards	10 Days	06 Jul 2026	17 Jul 2026	Completed
6. Testing and Evaluation	15 Days	20 Jul 2026	07 Aug 2026	Completed
7. Final Report Documentation	20 Days	10 Aug 2026	04 Sep 2026	In Progress
8. Final Submission	1 Day	11 Sep 2026	11 Sep 2026	Pending

Note. This table presents the detailed project plan and timeline for the development of the system, including all major chapters, tasks, durations, start and end dates, and their completion status throughout the project lifecycle.

CHAPTER 2: LITERATURE REVIEW

2.1 Introduction

This chapter presents the findings of a comprehensive literature review conducted to establish the academic and technical foundation for the proposed centralized, role-based event coordination platform at APU. The review is structured to progress from broad domain knowledge to specific technical considerations, ensuring that all relevant aspects of the project are examined and substantiated through credible academic sources.

The chapter begins with a domain research section covering event management in academic institutions, multi-departmental coordination and workflow management, role-based access control, centralized digital platforms for institutional operations, and SDG Goal 9. This is followed by a review of four similar existing systems, and a technical research section justifying all development components. The chapter concludes with a summary identifying four key research gaps and the proposed system's solutions.

2.2 Domain Research

2.2.1 Event Management in Academic Institutions

As universities expand in scale and operational complexity, the need for structured digital systems to support these activities has become increasingly evident in academic literature.

Hadiwiyanti et al. (2022) report that universities can organize tens to hundreds of activities within a single academic year and note that a web-based Campus Event Management Information System was developed and implemented to provide better managerial services for managing campus activities for both internal and external parties. Their study further indicates that the evaluated system had not been fully accepted by users, identifying specific difficulties in graphical user interface navigation as a contributing factor (Hadiwiyanti et al., 2022).

Jeny et al. (2022) identify that Event Management Systems are used to manage all facets of an event and present a web-based platform that supports event scheduling across multiple parameters including date, time, location, and equipment, while providing real-time alerts to organizers when stakeholder interactions occur. The authors observe that the motivation for

such systems stems from the inadequacy of existing approaches in managing multi-faceted coordination requirements through a unified interface (Jeny et al., 2022).

Arriesgado et al. (2023) present TICaP Hub, an event management system for an academic institution published through Springer, which integrates cloud-based infrastructure and automated task scheduling. The authors identify that academic event management systems benefit from centralized digital platforms that consolidate task tracking and departmental responsibilities into a single accessible environment (Arriesgado et al., 2023). This finding supports the argument that centralization must accompany role-based workflow structure for the system to be operationally effective.

2.2.2 Multi-departments Coordination and Workflow Management

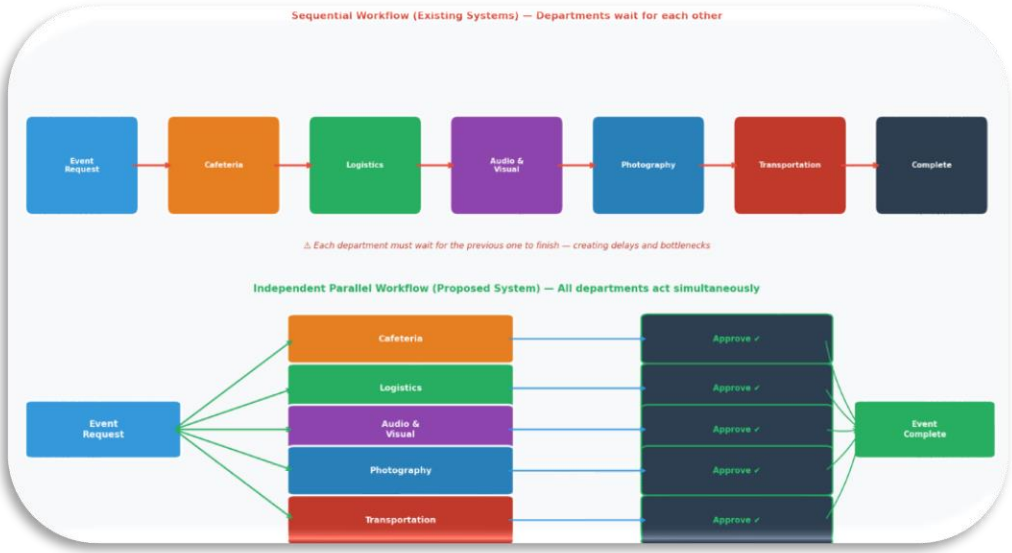


Figure 1: *Sequential vs Independent Departmental Workflo.*

Workflow management in higher education relies on structured process design that can support the movement of tasks through an organization in a controlled and consistent manner. In the university context, the absence of sufficient automation and an unclear organizational structure can slow down administrative processes and reduce operational efficiency. Research on e-university delivery models shows that process automation can significantly shorten procedures and improve efficiency, particularly where universities move toward digitally managed workflows rather than manual handling (Ismail et al., 2019).

Business process management literature also shows that digital innovations are increasingly reshaping how processes are designed and executed. Instead of treating workflow

as a set of isolated tasks, BPM emphasizes the integration of process logic, digital support, and structured control so that organizational activities can be managed more effectively. In this sense, workflow management is not only about speed, but also about ensuring that work is routed through a coherent process that supports consistency and coordination across different units (Ahmad & Van Looy, 2020).

In higher education institutions, digital transformation has become a key enabler for improving administrative effectiveness and supporting more coordinated institutional operations. A systematic review on digital transformation in higher education institutions highlights that HEIs are increasingly adopting digital approaches to improve internal processes and strengthen institutional performance (Benavides et al., 2020). By inference, this supports the use of a centralized event coordination platform, because event requests can be routed, reviewed, and tracked through one managed workflow rather than handled through disconnected communication channels.

Overall, the literature suggests that effective coordination across multiple organizational units depends on automation, structured workflow logic, and digital process management. For the proposed system, this means that event coordination should be designed as a centralized workflow where task routing, approval handling, and progress visibility are controlled within one platform. This approach is consistent with the direction of BPM and higher education digital transformation research, which emphasizes efficiency, process clarity, and improved institutional coordination.

2.2.3 Role-Based Access Control (RBAC) in Information Systems

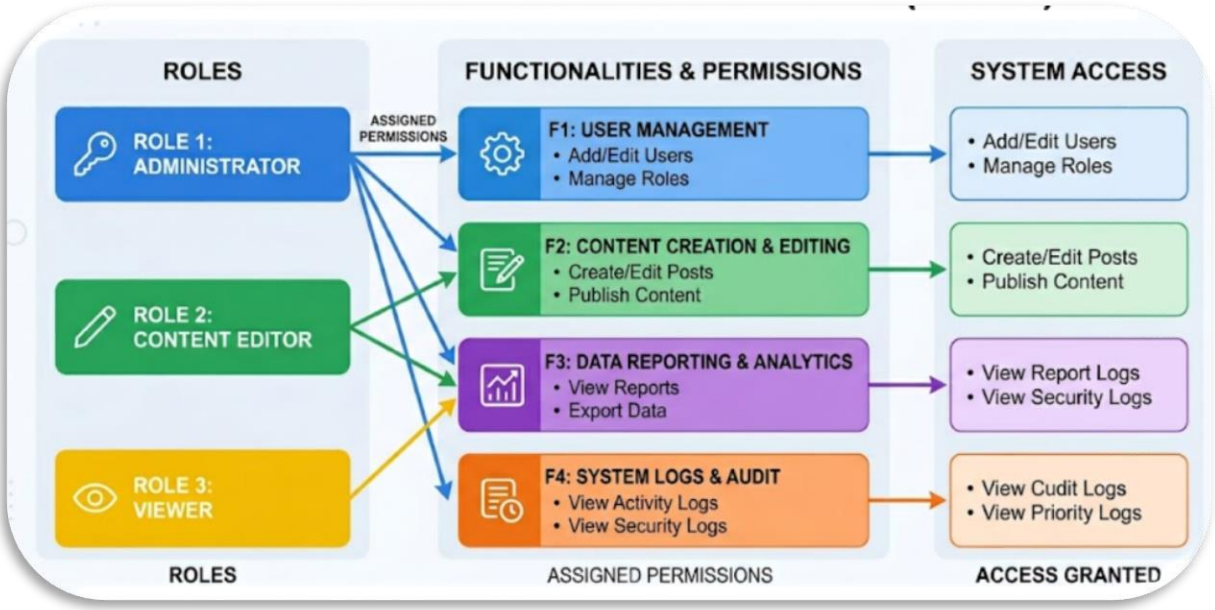


***Figure 2**: RBAC Hierarchy Model — Users, Roles, Permissions.*

Role-Based Access Control (RBAC) is an access control model in which permissions are assigned to roles rather than directly to individual users. Recent literature describes RBAC as a structure that simplifies access management by aligning permissions with organizational units such as job titles, security levels, or departments, which makes it suitable for systems with clearly separated responsibilities (Anderer et al., 2023). RBAC is widely considered more scalable than traditional discretionary access control because it reduces administrative overhead when managing large numbers of users by grouping permissions under defined roles instead of handling them individually.

In multi-domain and multi-user environments, RBAC supports structured role separation by distinguishing between role categories and enforcing controlled access across different operational boundaries. Recent research on cross-domain RBAC models shows that role design can support platform-domain isolation, inter-domain management, and fine-grained sharing, which is especially relevant when a system must coordinate several groups without allowing unrestricted access. Research also shows that RBAC must remain adaptable in real business environments because role assignments can change as organizational responsibilities evolve (Jiao et al., 2022). This adaptability is particularly important in dynamic institutional environments such as universities, where users may shift roles depending on operational needs, semesters, or administrative restructuring.

For the proposed event coordination platform, RBAC is therefore appropriate because it allows each user type to be limited to the functions relevant to their responsibilities while keeping workflows organized across departments. This means that the system can support separate access boundaries for roles such as Event Manager, Department Manager, Department Staff, Admin, and Super Admin without exposing unrelated functions to all users. In this way, RBAC helps maintain controlled workflow execution, clearer responsibility allocation, and more consistent system governance (Zubair et al., 2024). Additionally, RBAC contributes to operational security by ensuring that sensitive actions such as approvals, task assignments, and system configuration changes are only accessible to authorized roles, reducing the risk of unauthorized modifications.

Beyond access restriction, RBAC also plays an indirect role in supporting workflow clarity in multi-user systems. By defining what each role can see and execute, it reduces ambiguity in task execution and ensures that users interact only with relevant system components. This improves system usability and reduces cognitive load on users, as they are not exposed to unnecessary functions outside their operational scope. As a result, RBAC contributes not only to security enforcement but also to improving the efficiency and structure of system interaction in complex organizational workflows.

2.2.4 Centralized Digital Platforms for Institutional Operations

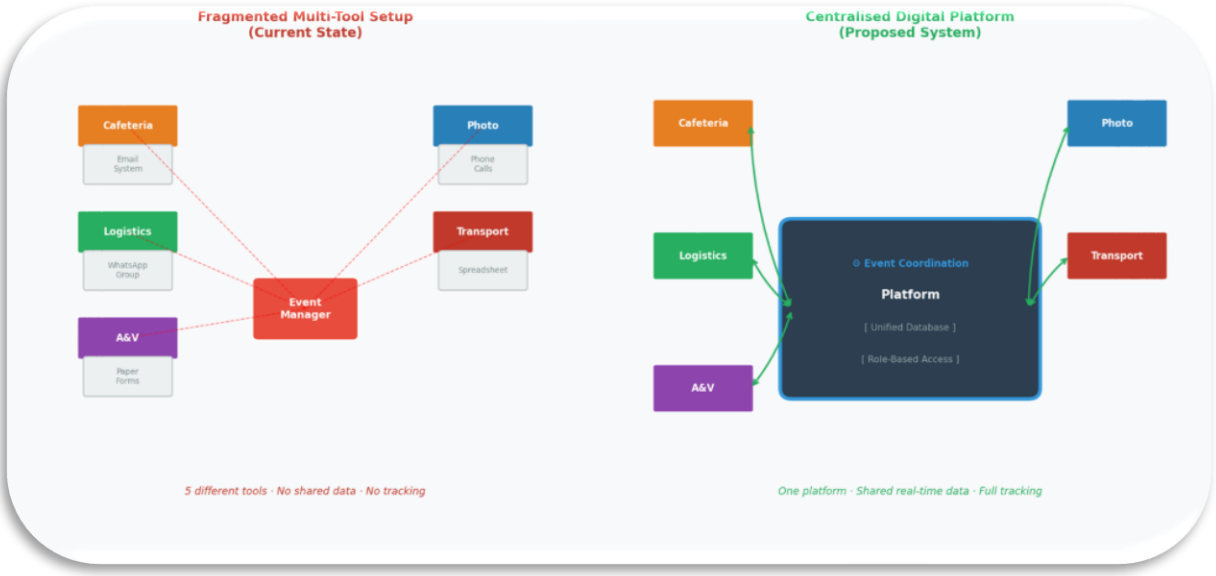


Figure 3: *Centralized vs Fragmented Platform Architecture.*

Abdul Rahman et al. (2024) emphasize that centralizing institutional event management into a unified digital platform enhances administrative visibility by consolidating all events within a single system. They further note that the absence of a centralized platform can result in unreported events and reduced oversight, particularly in large academic institutions where coordination is distributed across multiple departments. This demonstrates that fragmented systems can lead to inconsistencies in event tracking and weakened institutional control.

Similarly, Habib et al. (2021) highlight that decentralized university administrative systems often lead to duplicated operational activities and inefficiencies due to the lack of a unified coordination structure. Their findings suggest that when institutional processes are distributed across multiple disconnected tools, additional effort is required to reconcile inconsistencies, increasing administrative overhead and reducing operational efficiency.

From an infrastructure perspective, Fernández-Portillo et al. (2023) explain that universities are increasingly adopting centralized cloud-based platforms to integrate institutional resources and support inter-departmental collaboration. Cloud technologies enable scalable access and centralized resource management, reducing system fragmentation. However, they also highlight that effective centralization requires appropriate system governance to ensure that integration does not introduce organizational complexity or reduce clarity in user interactions.

Hadiwiyanti et al. (2022) evaluate a centralized Campus Event Management Information System using the System Usability Scale (SUS) and report a score of 70.80, indicating an "OK/Fair" usability level. The study further identifies usability limitations, particularly difficulties in navigating the graphical user interface, suggesting that the system has not yet achieved optimal user acceptance. Although the system provides a centralized environment for managing event-related activities, the findings indicate that centralization alone does not necessarily guarantee clarity or ease of use in multi-user systems. In complex institutional environments, where multiple user groups interact with the same platform, exposing all functions uniformly to all users may increase interface complexity and reduce overall usability. This highlights the importance of incorporating structured role-based access control and permission mechanisms, where system functionalities are selectively exposed based on user roles, reducing cognitive load and improving interaction efficiency within centralized systems.

In summary, the literature consistently supports the adoption of centralized digital platforms in institutional environments due to their ability to improve coordination, reduce duplication, and enhance visibility of operations. However, it also indicates that centralization must be complemented with structured access control and role-based system design to ensure usability, clarity, and effective user interaction in multi-user environments such as universities.

2.2.5 SDG Goal 9 and Digital Infrastructure in Higher Education

***Figure 4**: SDG Goal 9 and vision ISGlobal (n.d.).*

SDG Goal 9 — Industry, Innovation and Infrastructure — calls for the development of resilient infrastructure, the promotion of inclusive and sustainable industrialization, and the fostering of innovation. Mualla and Mualla (2024), in a Springer publication on digital transformation in higher education, note that institutions in developing regions lacking structured digital administrative systems face significant obstacles in meeting SDG 9 targets, and emphasize that digital transformation in university administrative operations represents a direct contribution to the infrastructure development dimension of SDG 9 (Mualla & Mualla, 2024).

Kaputa et al. (2022), in a Springer publication on social innovation in higher education, reports that digital transformation in higher education institutions serves as a driver of social-oriented innovations and contributes to broader societal goals, including those outlined under SDG 9 (Kaputa et al., 2022). The authors argue that digital transformation in this context represents more than operational improvement — it constitutes a structural contribution to the sustainable development of the institution and the communities it serves.

The proposed centralized, role-based event coordination platform for APU directly contributes to SDG Goal 9 by establishing a purpose-built digital coordination infrastructure that supports SDG 9's targets for resilient and sustainable institutional infrastructure.

2.3 Similar Systems / Works

A review of existing academic and commercial systems reveals that various platforms have been developed to support event management, workflow coordination, and institutional administrative processes. These systems typically focus on specific aspects such as event scheduling, task management, approval processing, or user collaboration. However, they are generally designed with a limited operational scope and do not fully address the complexity of multi-departmental coordination required in academic institutions.

While these systems provide useful foundations in areas such as centralized event listing, workflow automation, and role-based access, they often lack integration between independent departmental workflows, structured approval hierarchies, and event-specific task execution mechanisms. As a result, there remains a clear gap in existing solutions when it comes to managing end-to-end university event coordination within a unified and role-driven system architecture. This section critically evaluates selected systems to identify these limitations and establish the justification for the proposed centralized event coordination platform.

Table 2: *SIMILAR SYSTEM*

Research Name & Author (Year)	Research Description	Methods / Technologies	Gaps and Issues	Proposed Improvements
Alesandro et al. (2025). Development of a Campus Project Management System to Enhance Collaboration Using the Agile Method. JAIEA. DOI: https://doi.org/10.59934/jaiea.v4i3.1158	A web-based campus project management system that helps lecturers, students, and administrative staff manage their projects from one place. The system lets users create projects, add tasks, set deadlines, assign work to team members, and track progress step by step. It also provides **simple** communication tools (comments, updates, and notifications) so everyone can share information, discuss tasks, and see what has been done and what is still pending.	Agile/Scrum methodology; web-based development	The system does not have a clear step-by-step approval flow between different units (for example: program, faculty, finance). There is no way for one unit to officially approve or reject a request in the system before work starts. The system cannot handle several separate approval processes at the same time for different departments on the same request	The proposed system sends each event request to all related departments at the same time. Each department can **separately** review the request, approve it, or send it back, following its own approval steps in the system.
User-Centered UiTM e-Event using Predictive Software Development Life Cycle (SDLC) – Noorihan Abdul Rahman et al. (2024)	UiTM e-Event web system, an online platform that gathers all student event information for UiTM Machang in one place. The system allows students to easily browse and search for events, while organizers can create, manage, and promote their events through	Uses a predictive SDLC (planning, analysis, design, implementation, testing, maintenance) and user-centered design to build and evaluate a web-based event management system	The paper mostly talks about putting all event information in one place and making it easier to communicate and use the system for UiTM Machang events.	Extend the system into a platform where many departments can work together, each with its own steps and roles. Add features like edit/resubmit requests, live progress tracking, a full event history log, and build it using a modern technology stack (Angular,

DOI: https://doi.org/10.24191/jcrinn.v9i2.468	a **user-friendly** and user-centered interface..	for event creation, management, and viewing.	It does not really talk about harder things like many departments working together, different user roles with different steps, or advanced approval and editing processes. It only shows a simple admin approve/delete feature. It also does not clearly link the system to SDG 9 or explain technical choices like using the cloud or how the system is built in detail.	Flask, PostgreSQL on Supabase) with an Agile/Scrum development approach.
Cvent (2025). Cvent — All-in-one event management and marketing platform. Retrieved from Link :https://www.cvent.com	Cvent is a website that helps people plan and run events. You can use it to find places, register participants, make event pages, manage guests, run online events, and see how well your event is doing. It works on the cloud and can connect to other systems like Salesforce.	Cloud-based SaaS; web platform; mobile applications; integrations with Salesforce and marketing tools	Cvent mostly helps with planning and promoting events for people outside the organization, like guests, customers, or partners. It does not really help with the inside work of an institution, like many approval steps, different roles deciding, and	The proposed system solves this gap by focusing on use inside the institution. It adds separate approval steps for each department, lets users revise and resubmit requests, and supports task assignment and progress tracking after approval. This helps departments work together smoothly before and after an event is approved, which is not what Cvent mainly focuses on.

			different departments working together on the same request.	
Essel, D. D., Techie-Menson, H., Oppong, S. O., & Alakuu, A. (2023). Digitalising the Clearance Processes of Higher Education Institutions through an Online Clearance System. IJSRM. DOI: https://doi.org/10.18535/IJSRM/V11I07.AS01	An online clearance system that uses a web platform to replace manual, paper-based graduation clearance. Clearing officers in different offices log in to the system and, based on their role, can review student records and either approve or reject the student's clearance.	ASP.NET MVC 5; MSSQL 2014; JavaScript; HTML; CSS; Agile methodology	The system is mainly about clearing final-year students before graduation, not about wider event processes like planning and running graduation ceremonies or other institutional events. Each office only does a simple one-step approval or rejection, and the system does not support more complex approval steps inside a department.	The proposed system builds on role-based access and multi-office approvals and extends them to event-specific workflows for each department. In this system, every department can approve requests, assign its own service tasks, and carry them out, with each task tracked by clear status values until the event is completed. .

Note. This table compares similar existing systems in terms of their purpose, technologies, limitations, and identified gaps, and highlights how the proposed system addresses these shortcomings through improved multi-department workflows and coordination features.

Conclusion — Summary of Similar Systems:

The analysis of existing systems demonstrates that although significant progress has been made in event management systems and workflow-based platforms, current solutions remain fragmented in their functional coverage. Systems such as those proposed by Alesandro et al. (2025) and Essel et al. (2023) primarily focus on structured task execution and approval processes; however, these implementations are limited to single-domain workflows and do not support complex inter-departmental coordination or parallel approval structures required in university-wide event management.

Commercial platforms such as Cvent (2025) provide scalable and feature-rich event management capabilities, particularly for external and public-facing events, but they are not designed to support internal institutional workflows where multiple departments independently evaluate, approve, and execute tasks within a single event lifecycle. Similarly, academic systems such as Abdul Rahman et al. (2024) focus on centralized event information management and usability, but they do not incorporate layered approval mechanisms, revision workflows, or structured task execution across departments.

Overall, the key limitation identified across all reviewed systems is the absence of a fully integrated framework that combines multi-departmental independent workflows, role-based access control, event-specific task distribution, and revision-based approval cycles within a single unified platform. Furthermore, none of the systems explicitly align their operational design with institutional digital infrastructure development goals such as SDG Goal 9. The proposed system addresses these gaps by integrating centralized coordination with structured role-based workflows, enabling efficient, transparent, and scalable event management tailored specifically for academic institutions.

2.4 Technical Research

This section identifies and justifies all technical components selected for the development of the proposed centralized, role-based event coordination platform for APU. Each component is presented in the same form. The alternatives that were realistically available for that decision are named, the reason each was set aside is given, and the selected option is then defended against them rather than in isolation. This form is used deliberately, because a list of a tool's advantages is not a justification: the rejected alternatives carry advantages of their own, and in several cases the rejected option is the more common industry choice. What settles each case is the fit between what a tool does well and what this particular system demands, which is a multi-departmental approval workflow whose access rules are unusual, whose data is strongly relational, and which is built and maintained by one developer within a single academic year.

2.4.1 Hardware and Software Requirements

Hardware:

The proposed system is designed as a web-based platform, meaning that **no specialized hardware is required** on the user side. Users can access the system through any standard device capable of running a modern web browser, including desktops, laptops, and mobile devices. This ensures broad accessibility and ease of use without imposing additional hardware requirements on end users.

The system is developed and executed on a standard Windows workstation, which runs both the Angular client and the Flask application server during development. No dedicated institutional server hardware is required at this stage of the project. The only externally hosted component is the database, which is provided as a managed PostgreSQL service by Supabase. Using a managed database service removes the need to install, secure, and back up a database server on campus, while keeping the data in a single authoritative location that both the development and demonstration environments share.

Software Components:

1- **Frontend: Angular framework**

Angular is used to build the user interface of the system. It follows a component-based structure, allowing the interface to be organized into reusable parts. It also supports

dynamic data binding, enabling real-time updates such as task status and progress tracking without reloading the page.

2- **Backend:** Flask with Python

Flask is used to handle the server-side logic of the system. It processes user requests, manages workflows such as approvals and task assignments, and connects the frontend with the database. Its lightweight and flexible design makes it suitable for implementing customized system logic.

3- **Database:** PostgreSQL (hosted on Supabase)

PostgreSQL is used to store and manage system data in a structured format. It handles information such as users, events, tasks, and approval records, ensuring data consistency and enabling efficient retrieval for tracking and reporting purposes. The database instance itself, together with the connection pooling used by the application, is provided by Supabase.

2.4.2 IDE — Visual Studio Code

Figure 5: Visual Studio Code logo WebCatalog (n.d.).

Selected IDE: Visual Studio Code (VS Code) by Microsoft

Key Characteristics:

4- Lightweight, open-source, and cross-platform (Windows, macOS, Linux)

5- Supports TypeScript, Python, HTML, and SCSS — all languages used in this project

6- Extensive extension ecosystem for linting, debugging, and version control integration

Justification: Tan et al. (2023) report that Visual Studio Code ranked as the most popular developer environment tool in the Stack Overflow 2021 Developer Survey, with 71.06% of over 80,000 respondents indicating its use. The authors attribute this widespread adoption to its cross-platform compatibility and extensive extension ecosystem, which make it particularly well-suited for multi-language development projects. This is directly relevant to the proposed system, as development must be conducted simultaneously across Angular (TypeScript/SCSS) and Flask (Python) within a single unified environment (Tan et al., 2023).

Three alternatives were considered against it. The JetBrains family offers the deepest language-specific tooling of the group, but its coverage is split across two separate products, PyCharm for the Flask tier and WebStorm for the Angular tier, so adopting it would have meant two editors, two sets of configuration and two paid licences for a single codebase. Visual Studio provides comparable depth on Windows but is weighted towards the .NET ecosystem, which this project does not use. Lightweight editors such as Sublime Text and Vim start faster and cost nothing, but language support, debugging and version control must be assembled from separate plugins by the developer rather than being present by default, which shifts effort from writing the system to configuring the tool used to write it.

The deciding factor is the shape of the work rather than the strength of any one editor. This project is written by a single developer who moves between an Angular component and the Flask route that serves it many times in the same session, so the cost that matters is the cost of crossing between TypeScript, SCSS, Python and SQL, not the depth of the tooling within any one of them. Visual Studio Code is the only one of the four candidates that covers all four languages in one free installation with an integrated debugger and version control, which is why it was preferred over tools that are individually stronger across a narrower range.

2.4.3 Libraries and Tools

A — Angular (Frontend Framework)

Figure 6: Angular logo Angular Press Kit (n.d.).

Selected Frontend framework: Angular

Key Characteristics:

- **TypeScript-Based Ecosystem:** An open-source framework developed by Google that utilizes TypeScript to build robust Single-Page Applications (SPAs).
- **Structured Modularity:** Designed with a modular environment that allows for the organization of complex application logic into manageable units.
- **Inherent Scalability:** Provides a comprehensive set of tools and architectural patterns specifically engineered to handle high-complexity systems (Vashisht, 2021).

Justification:

As stated by Vashisht (2021), Angular is distinguished by its ability to manage highly complex applications through a strictly structured and modular development platform. This characteristic is particularly important for the proposed system, where twelve user roles must be supported within a single application. The framework's modularity enables role-specific scalability, ensuring that the views used by applicants, Heads of School and Heads of Department, the CFO, cafeteria users, club administrators, and system administrators can remain independent while still operating within the same codebase. The client is built on the principle that it displays what the server permits and decides no access rule of its own, so that the interface remains a presentation layer rather than a security boundary. This supports the development of a scalable and well-organized system architecture aligned with the project's multi-role requirements (Vashisht, 2021).

Four alternatives were weighed against Angular. React is the most widely used of them, but it is a library rather than a framework: routing, form handling, HTTP access and state

management are supplied by separate packages chosen by the developer. For a team with established conventions that freedom is an advantage; for a single student building a twelve-role application to a fixed deadline it becomes a series of architectural decisions that must each be researched and defended before any feature is written. Vue offers a gentler learning curve and a lower initial cost, but it is less opinionated about structure and treats TypeScript as an option rather than as the default. Svelte compiles to smaller and faster output, and Vashisht (2021) reports exactly that advantage in a direct comparison with Angular, while identifying Angular's own advantage as the completeness of its tooling for large and complex applications. Building the interface in HTML and JavaScript with no framework at all was also considered, and rejected because routing, validation and session handling would then all have been written by hand.

Angular was selected because the properties this system needs are the ones it supplies as first-party parts of the framework rather than as choices to be made. The application has twelve roles whose menus differ, a six-step proposal form with staged validation, and an HTTP layer in which every outbound request must carry the caller's token and every authentication failure must be handled identically. Angular provides routing with guards, reactive forms, dependency injection and HTTP interceptors as one documented mechanism each. The route guard and the token interceptor described in Sections 4.6.3 and 4.6.4 are configurations of framework features; in React or in plain JavaScript the same behaviour would have been the developer's own code, and the access model would then have depended on that code being written correctly in every place it was needed. Svelte's advantage in output size was judged the less valuable of the two, because the platform is used by staff on campus networks rather than by anonymous visitors on constrained connections, so the characteristic it improves is not the one under pressure here.

B — Flask (Backend Framework).

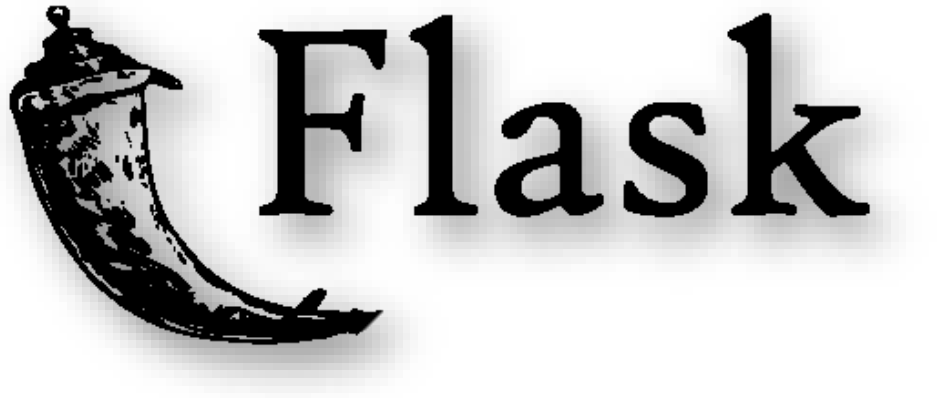


Figure 7: *Flask logo Wikimedia Commons (n.d.).*

Selected backend: Flask

Key Characteristics:

- Lightweight Python microframework for building REST APIs
- Ranked among the most popular Python web frameworks (Albesher & Alfayez, 2024)
- Commonly used for database integration, API development, authentication, and deployment (Albesher & Alfayez, 2024)
- Modular design supports clear routing and business logic separation

Justification:

Flask is suitable for building RESTful APIs for the proposed system, as it provides a lightweight and flexible framework for developing backend services that can efficiently handle client–server communication (Albesher & Alfayez, 2024). Its design supports the creation of modular and scalable API endpoints, which is essential for managing interactions between the frontend and backend components of the system. In addition, Flask supports core system requirements such as user authentication, database integration, and API request handling, making it well-suited for implementing the functional architecture of the platform. These capabilities allow the system to manage user roles, process event-related requests, and maintain structured data flow across different components, ensuring reliable and efficient backend operations (Albesher & Alfayez, 2024).

In the developed system, the backend is organised into resource-based modules served under a single versioned API path, with the rules of the approval workflow held in a separate service layer rather than inside the request handlers. Database access is written directly in SQL through

a managed connection pool instead of an object-relational mapper, which keeps the multi-stage approval queries explicit and easier to reason about. Every change to a request, its departmental tasks, and its history record is applied within a single transaction, so a workflow decision is either recorded completely or not recorded at all.

Four backend alternatives were considered, and the strongest of them was rejected for a reason specific to this system. Django supplies an object-relational mapper, an administrative interface and a complete authentication and permission system out of the box, and on that basis it is the more productive starting point for most applications. Its permission model, however, is defined per model and held globally per user, whereas authority in this system is held by a user in a role within an organisational unit, as described in Section 4.2.3. Adopting Django would have meant overriding its permission system rather than using it, which removes most of the advantage that made it a candidate. FastAPI is newer, generates its own API documentation, and handles requests asynchronously, which is a real advantage for workloads dominated by concurrent input and output; this workload is a departmental approval queue with a bounded user population and database-bound requests, where that advantage does not apply, and its body of published guidance is smaller than Flask's (Albesher & Alfayez, 2024). Node.js with Express would have allowed one language across both tiers, but it would have placed the assistant, the query guard and the scheduled jobs in JavaScript and away from the Python libraries they depend on, including the Gemini software development kit and the password hashing and migration tooling used here. Spring Boot and ASP.NET Core are both proven for institutional systems of this kind, but each carries a configuration and build burden disproportionate to a project written by one person.

Flask was selected precisely because it imposes little. The three-layer separation described in Section 4.2.2, in which a route may not issue SQL and a change to a request must pass through the workflow service that writes its audit record, is an arrangement designed around this system's workflow rather than inherited from a framework's conventions. A more complete framework would have supplied its own answers to authorisation, data access and request routing, and in each of those three places this system needs a different answer: authority scoped to a unit, explicit SQL for the multi-stage approval queries, and an audit write bound to the same transaction as the state change it records. The absence of those defaults is the reason Flask was chosen over Django, and it is equally the cost of the choice, since everything a fuller framework would have provided had to be built instead and is accounted for in the implementation described in Chapter 4.

C — Supporting Libraries and Tools

Beyond the two principal frameworks, the system depends on a set of supporting libraries and tools that provide database access, authentication, validation, documentation, and testing. These are summarised below. Among these, authentication is handled through JSON Web Tokens together with one-way password hashing, an approach widely applied to secure REST APIs and manage sessions between a separate client and server (Dalimunthe et al., 2023).

Library / Tool	Version	Purpose in the system
psycopg2	2.9	PostgreSQL database driver and connection pooling
PyJWT	2.10	Issuing and validating authentication tokens
bcrypt	4.2	One-way hashing of user passwords
Flask-Cors	5.0	Controlling which origins may call the API
Flask-Limiter	3.10	Rate limiting of incoming requests
Marshmallow	3.26	Validation of incoming request data
apispec	6.8	Generation of the API documentation
Gunicorn	23.0	Application server used to run the Flask API
pytest	8.3	Automated testing of the backend
google-genai	1.x	Access to the Google Gemini model used by the AI assistant
RxJS	7.8	Handling of asynchronous data streams in the Angular client
Vitest	3.2	Automated unit testing of the Angular client
Prettier	3.6	Consistent code formatting across the project

Table 3: SUPPORTING LIBRARIES AND TOOLS

Two of the decisions represented in the table above are worth setting against the alternatives they were chosen over. The first is data access. The conventional pairing with Flask is SQLAlchemy, an object-relational mapper that maps tables onto Python classes and removes

most hand-written SQL from the application. It was rejected here because the queries that matter most in this system are the multi-stage approval queries and the scope-filtered list queries, and in both of them the filtering condition is the security boundary rather than a convenience. An object-relational mapper would either generate those conditions out of sight or require raw SQL to be embedded within it anyway, while adding a mapping layer that must itself be maintained. Writing parameterised SQL directly keeps the condition that enforces access visible in the query text, at the acknowledged cost of portability between database engines, which this project does not need because the engine is fixed.

The second is session handling. Server-side sessions carried in cookies are the older and simpler approach, and they hold one clear advantage over tokens: a session can be destroyed on the server at the moment access is withdrawn, whereas a signed token remains valid until it expires. They were nevertheless rejected, because the Angular client and the Flask API are served as separate origins, and cookie-based sessions across origins require cross-site cookie configuration together with separate protection against cross-site request forgery. JSON Web Tokens avoid both, and are an established approach for exactly this separation of client and server (Dalimunthe et al., 2023). The revocation weakness that argues for sessions was answered in the design rather than ignored: as described in Section 4.2.3, the caller's roles are read from the database on every request instead of being trusted from the token's claims, so withdrawing a role takes effect on the next request rather than when the token happens to expire.

D — Google Gemini (AI Assistant)

Selected model: Google Gemini, accessed through the google-genai software development kit

Key Characteristics:

- Answers questions written in ordinary language, without the user needing to know how the data is stored
- Two answer paths: a curated knowledge base for questions about how the system works, and a generated database query for questions about individual records
- Every generated query is checked and restricted to the data the asking user is already permitted to see

- All interactions, including refused requests, are recorded so that administrators can review how the assistant is used

Justification:

Large language models are increasingly used as a natural language interface to relational databases, translating a question written in ordinary language into a database query and returning the result to the user. Shi et al. (2025) review this text-to-SQL approach and note that it lowers the barrier for users who need information held in a structured database but are not able to write queries themselves. This is directly relevant to the proposed system, where staff across several departments need answers about events, requests, and tasks without navigating reporting screens belonging to roles other than their own. Google Gemini was selected because it is reached through a documented software development kit, requires no local model hosting, and can be operated within the resource constraints of an individual final year project. Because a generated query would otherwise be able to reach any record in the database, every query produced by the model is validated and constrained to the asking user's own access rights before it is executed, so the assistant cannot return information that the user could not already open in the interface (Shi et al., 2025).

The first alternative to the assistant is not another model but no model at all. The same questions could be answered by adding further reporting screens, and that option was considered first because it introduces no external dependency and no new failure mode. It was set aside because the number of screens grows with the number of roles and each screen answers only the questions it was designed for, whereas the questions staff actually ask cut across roles and were not known in advance. A self-hosted open-weight model such as Llama or Mistral was the next candidate, and it is the stronger option on confidentiality, since no institutional data would leave the host, and on cost, since there is no per-request charge. It was rejected on the resources available: it requires a continuously running machine with a suitable graphics processor, which this project does not have, and accuracy on the text-to-SQL task for models of the size such a machine could host remains materially below that of the current hosted models (Shi et al., 2025).

Among the hosted models, Gemini, the OpenAI models and Anthropic's Claude are close enough in capability on this task that the choice between them is not a capability judgement. Gemini was taken because it is reachable through a documented software development kit at a usage tier the project can afford, and because two credentials can be configured with automatic

failover between them, which matters for a component that has to work during a live demonstration. The more important point is that the design does not depend on which of the three was chosen. The model is only permitted to propose a query; the guard described in Section 4.6.11 then validates that query and constrains it to the asking user's own access rights before it is executed. Because the safety of the assistant rests on the guard rather than on the model behaving well, the provider can be replaced without weakening it, and the selection was made on that understanding.

2.4.4 Database Management System — PostgreSQL (Supabase)

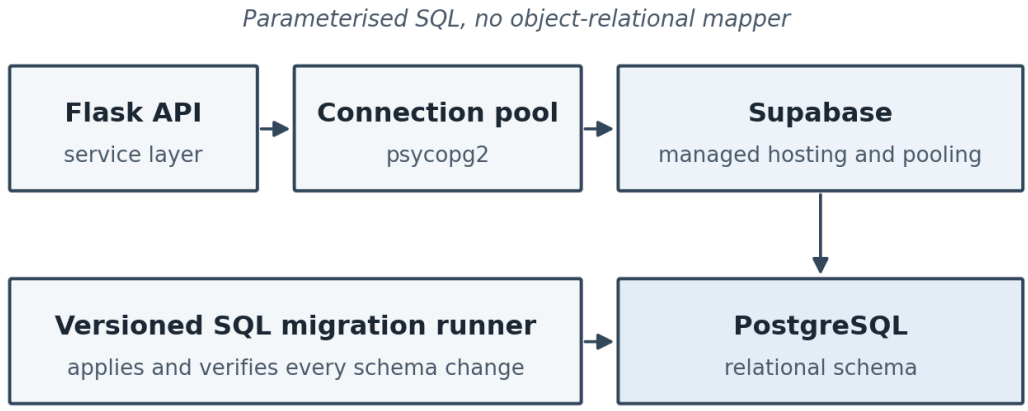


Figure 8: PostgreSQL data access through Supabase

Selected DBMS: PostgreSQL (open-source relational DBMS), hosted on Supabase

Key Characteristics:

- Structured relational database for events, departments, users, tasks, and approval history
- Supports foreign keys, constraints, and transactions for data integrity
- Accessed through parameterised SQL rather than an object-relational mapper, keeping the multi-stage approval queries explicit
- Schema changes applied through a versioned migration runner that records and verifies every file it applies

Justification:

PostgreSQL is well-suited for managing structured relationships within multi-departmental workflows, as it provides a robust relational model that represents entities such as events, departments, users, and tasks. Its support for primary and foreign keys and for table-level constraints ensures data integrity by keeping the relationships between connected records valid, and its transactional behaviour preserves consistency when several users act on the same request at the same time. These capabilities are supported by comparative studies reporting strong query performance for PostgreSQL relative to other widely used database systems (Salunke & Ouda, 2024; Urnikienė et al., 2026). Three further decisions shaped the way the database is used in this project. First, the application accesses the database through parameterised SQL rather than an object-relational mapper, which keeps the multi-stage approval queries visible and predictable. Second, every schema change is applied through a versioned migration runner that records and verifies each file it applies, so the database can be rebuilt from scratch in a known state. Third, the database is hosted on Supabase as a managed service, which supplies the instance, its backups, and the connection pooling used by the application, removing the need to administer a database server as part of the project.

This choice was made in two parts, and each part had realistic alternatives. On the question of data model, MongoDB was the principal non-relational candidate. A document store is the better fit where records are self-contained and their shape varies between them; here a single event request refers to a user, an organisational unit, a role assignment, a set of departmental tasks, a venue and, where catering is ordered, a cafeteria menu. Keeping those references valid is the property the system depends on, and in a document store it would have to be enforced by the application rather than by the database. Urnikienė et al. (2026) also report PostgreSQL's advantage over MongoDB on precisely the filtering and analytical query patterns that the dashboards in this system perform. MySQL was the closest relational alternative and would have served, but Salunke and Ouda (2024) report stronger performance for PostgreSQL in direct comparison, and PostgreSQL's stricter treatment of constraints was preferred for a system whose correctness claims rest on them. SQLite was used during early development and rejected for delivery, because it is a single-writer file database and this system has concurrent approvers acting on the same request while a scheduled escalation job writes to it in the background.

Firebase, with its Firestore database, was considered separately, because it would have supplied hosting, authentication and file storage as one service and removed several other decisions at once. It was rejected on the same ground that ruled out Django's permission system. Firestore

expresses access control as per-document security rules, whereas the model here is a role held within a unit, applied as a filtering condition on list queries so that records outside a caller's scope are never retrieved at all. That condition is a single clause in SQL and an awkward reconstruction in a document rule language. Firestore also offers no equivalent of the multi-table transaction on which the audit guarantee depends, since a state change and the history record describing it must either both be written or neither be written.

The second part of the choice was where the database runs, and it was decided separately from which database it is. Running PostgreSQL on the development machine keeps everything local and free, but it ties the institution's data to one student's computer and provides no backup. Running it on university infrastructure would be the correct answer in production and was not available, because it requires a provisioning decision outside a student project's control. Supabase was taken because it supplies the instance, its backups and the connection pooling used by the application, which are the parts of database administration that a single developer is least able to guarantee, while leaving the schema itself under the project's own versioned migration runner rather than under the vendor's control.

2.4.5 Operating System

The system is web-based and is not bound to any specific client operating system. Users may access the platform from Windows, macOS, Linux, Android, and iOS. On the development side, the application is written and executed on Windows, and the Flask application server runs unchanged on Linux, which is the environment assumed for the scheduled background jobs described in the following section.

Development could equally have been carried out on Linux or macOS, and containerising the application with Docker was considered as a way of making the development environment identical to the deployment environment. Containers were set aside as disproportionate for a single-developer project with one deployment target, since the isolation they provide addresses a problem that arises when several services and several developers must agree on an environment, and neither condition holds here. Windows was used because it is the machine available, and the risk that ordinarily argues for containers is contained instead by two facts about this system: the Flask application depends on no platform-specific behaviour, and the database is the same managed instance from every environment, so there is no local copy that can drift out of step with it.

2.4.6 Deployment and Hosting Environment

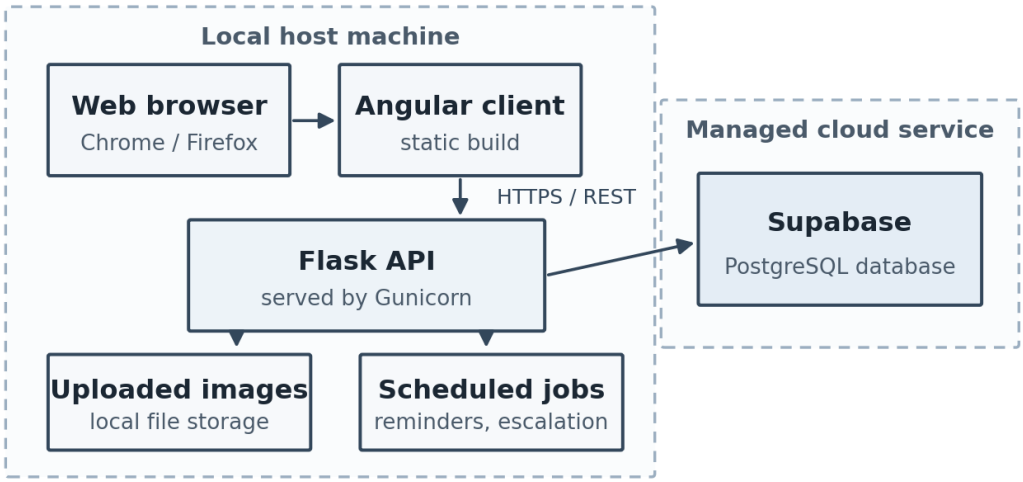


Figure 9: Deployment and hosting environment of the platform

Selected deployment approach: locally hosted application with a managed database service

Key Characteristics:

- The Angular client is served by the framework's own development server and is compiled to static files for release
- The Flask API is run through Gunicorn as its application server
- PostgreSQL is provided as a managed service by Supabase, the only externally hosted component of the system
- Uploaded images are stored on the file system of the machine running the API, and scheduled background jobs handle event reminders and approval escalation

Justification:

The platform is deployed as a locally hosted application supported by a managed database service. The Angular client and the Flask API run on the same host machine, with the client compiled to static files and the API served through Gunicorn, while the PostgreSQL database is hosted by Supabase. This arrangement was chosen deliberately for the scale of the project. The coordination workload is institutional rather than public, so the demand placed on the application tier is predictable and does not require elastic scaling, whereas the database is the component that benefits most from managed hosting because it carries the data that must

survive, be backed up, and be reachable from every environment used during development and demonstration. Separating the two in this way keeps the hosting cost and administrative effort proportionate to the project while ensuring that no institutional data depends on a single developer machine. Two limitations of the current arrangement are stated deliberately. Uploaded images are written to the file system of the host machine rather than to an object storage service, so they are not shared between hosts; and the scheduled jobs that send event reminders and process approval escalation depend on an operating system scheduler on a continuously running host, which is why the system also allows a System Admin to trigger the same sweep manually. Both are addressed by moving the application tier to a hosted environment, which is identified as future work rather than claimed as delivered.

Four hosting arrangements were compared before this one was adopted. A fully managed application platform such as Render or Railway would have hosted the API alongside the database and removed server administration entirely; it was rejected because the free tiers that fit this project's budget suspend an idle instance, and a suspended instance cannot run the scheduled sweep that sends event reminders and escalates overdue approvals, which is a behaviour the system is assessed on. A virtual machine on a cloud provider was the next candidate, and it is the route taken by Arriesgado et al. (2023) for a comparable campus event management system; it gives full control and a permanently running host, at the price of transferring operating system patching, certificate renewal and backup onto one student for the duration of the project, for a system that carries no external users during assessment. A containerised deployment under an orchestrator was rejected as disproportionate for the same reason containers were rejected for development, since orchestration exists to scale and coordinate many service instances and this system runs one of each. Hosting the whole platform on university infrastructure would be the correct production answer and remains the recommendation in Section 6.3, but it depends on an institutional provisioning decision that a final year project cannot make.

The arrangement adopted splits the decision rather than answering it once, and it splits it along the line that separates what can be rebuilt from what cannot. The application tier can be reconstructed from source on any machine in minutes, so it was left local, where it costs nothing and is always available for development and demonstration. The database holds the only state that cannot be reconstructed, so it is the component placed under managed hosting, where it is backed up and reachable from every environment. This is why the split was preferred over the two consistent alternatives: hosting everything locally would have made the data dependent on

a single machine, and hosting everything remotely would have added recurring cost and administration for a tier that needs neither.

2.4.7 Web Browser — Chrome and Firefox

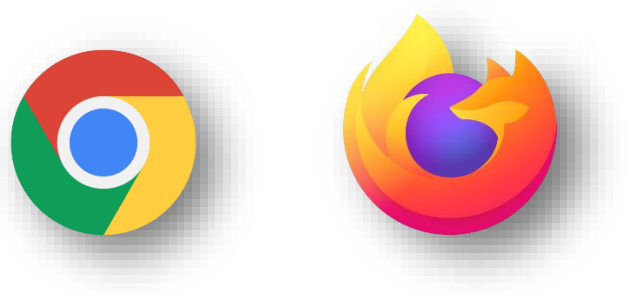


Figure 10: *Googe Chrome, Firefox Mozilla Firefox (n.d.); Be Connected (n.d.).*

Selected Browsers: Google Chrome and Mozilla Firefox

Key Characteristics:

- Both browsers fully support modern web standards: HTML5, CSS3, and JavaScript ES6+, required for the Angular-based frontend
- No installation is required on the user's device — access is entirely browser-based
- Both browsers are available on desktop and mobile, ensuring accessibility for all APU internal staff regardless of device

Justification:

Cross-browser inconsistencies in how web applications are interpreted and rendered are a well-documented issue in web development, as differences in browser implementations can lead to variations in application behavior across different platforms (Choudhary, Prasad, & Orso, 2012). These inconsistencies can negatively affect the user experience, particularly in complex web applications that rely on modern frameworks and dynamic user interfaces. Based on this, the use of modern, standards-compliant browsers such as Google Chrome and Mozilla Firefox is recommended to reduce compatibility issues and ensure more consistent execution of web applications. This is especially important for applications built using frameworks such as Angular, where consistent rendering and JavaScript execution are essential for maintaining interface stability and functionality across different devices and operating systems.

The alternatives here concern how wide the supported set should be rather than which browsers exist. Targeting a single browser is the cheapest option and would have halved the verification work; it was rejected on the evidence Choudhary et al. (2012) present, since a system verified on one engine carries no evidence about its behaviour on another. Declaring support for every current browser, including Safari and the older engines still present on some managed machines, was rejected for the opposite reason: each additional engine is a further full pass of the test plan, and Safari in particular cannot be exercised from the Windows development machine used for this project, so support for it could have been claimed but not demonstrated. Building a native desktop or mobile application instead of a web client was also considered, and rejected because it would require a separate build for each platform and an installation step on university machines that staff are not permitted to perform themselves, while the responsive web client already reaches mobile devices without one.

Chrome and Firefox were chosen as the pair because they are built on independent rendering engines, Blink and Gecko. That is the property that makes the pair meaningful, rather than the popularity of either browser: verifying on Chrome and Microsoft Edge would have covered a large share of users while testing the same engine twice and demonstrating nothing about cross-engine behaviour. Two independent engines is therefore the smallest set that provides real evidence of the portability the system claims, and the largest set this project can re-test on every release without the verification effort displacing the development it exists to support.

2.5 Summary

This chapter presented a literature review supporting the proposed centralized, role-based event coordination platform for APU. It covered three main areas: domain research, similar systems, and technical justification. Domain research examined event management in academic institutions, multi-departmental coordination, RBAC systems, centralized platforms, and SDG Goal 9 in higher education. Similar systems included studies by Alesandro et al. (2025), Abdul Rahman et al. (2024), and Essel et al. (2023), as well as a review of Cvent (2025), focusing on campus event management and clearance management systems. Technical research justified the use of Visual Studio Code, Angular, Flask (Python), PostgreSQL hosted on Supabase, the supporting libraries used across both tiers, the Google Gemini model behind the platform's AI assistant, and the deployment and hosting arrangement adopted for the system. Each of those decisions was argued against the alternatives available for it rather than

on its own merits alone, and where the rejected option is the more common industry choice, the reason it was set aside is stated in terms of what this system requires rather than in terms of what the tool lacks.

Several key gaps were identified. Existing systems lack independent multi-department workflows, role-specific interfaces, structured revision loops between coordinators and departments, and post-approval task assignment and execution tracking. To address these, the proposed system introduces independent departmental workflows within a unified platform, role-based interfaces for different users, a selective revision loop for rejected departments, and extended functionality for task tracking beyond approval until completion.

CHAPTER 3: METHODOLOGY

3.1 Introduction

The selection of an appropriate research and development methodology is a fundamental prerequisite for the successful delivery of a complex software system. This chapter establishes the structural framework used to guide the project from the initial investigation through to the final deployment of the centralized event coordination platform. It provides a comprehensive analysis of various Software Development Life Cycle (SDLC) models, evaluating their theoretical foundations and their practical utility in the context of institutional workflow management. By examining traditional, iterative, and blended approaches, this chapter outlines the roadmap for managing departmental requirements, designing multi-layered approval logic, and ensuring the platform remains robust through its development lifecycle. Finally, a specific methodology is justified and selected as the optimal path for this individual-led project.

3.2 Evaluation of Candidate Methodologies

3.2.1 The Waterfall Model (Sequential Development)

The Waterfall model is the most established paradigm in software engineering, predicated on a linear and sequential flow. In this model, development is divided into distinct phases: Requirement Analysis, System Design, Implementation, Integration and Testing, Deployment, and Maintenance. Progress flows strictly in one direction, requiring the complete sign-off of one phase before the next can commence.

Theoretical Foundation and Scholarly Perspective: Academic literature often classifies Waterfall as a "plan-driven" approach. According to Srivastava et al. (2019), the model's primary strength lies in its discipline and the high level of documentation it produces. This makes it an ideal choice for projects where requirements are static and the technical environment is highly stable. The structured nature of Waterfall provides a clear project timeline and a measurable set of milestones, which is traditionally favored in large-scale administrative or government-level software procurement where accountability and fixed-scope delivery are paramount.

Project-Specific Applicability: For a centralized event coordination platform, the Waterfall model offers a strong framework for the initial design of core database structures and role-based permissions, which are unlikely to change significantly. However, its rigidity presents a significant risk for this project. Since the specific nuances of departmental workflows—such as the exact fields required for the "Cafeteria" or "Transportation" requests—may only become clear during the actual build, a pure Waterfall approach would make it difficult and costly to integrate late-stage refinements or feedback from university stakeholders.

3.2.2 Agile Methodology (The Scrum Framework)

Agile is an iterative and incremental development philosophy that prioritizes flexibility and frequent stakeholder feedback over rigid initial planning. Scrum is the most prominent framework within the Agile umbrella, utilizing time-boxed development cycles known as Sprints (typically 1–4 weeks) to deliver functional increments of software.

From a theoretical perspective, Alami and Krancher (2022) observe that Scrum has become the dominant industry standard because it directly addresses the volatility of software requirements. By focusing on "User Stories" and a "Product Backlog," Scrum allows developers to pivot based on user testing and evolving priorities. Academic studies emphasize that Scrum significantly improves final software quality by ensuring that testing is an ongoing activity rather than a terminal phase. This iterative "inspect and adapt" cycle is recognized as a critical success factor in modern web-based application development.

Building on this iterative nature, the Scrum approach is highly beneficial for implementing the complex coordination logic of the event platform. It allows for the incremental building of departmental modules, ensuring that the distribution logic is tested and verified before moving to the next feature. However, because this is an individual Final Year Project (FYP), the standard Scrum ceremonies (Daily Stand-ups, Sprint Retrospectives, and specific roles like Scrum Master) are designed for teams. For a solo developer, these elements can introduce unnecessary administrative overhead, necessitating a modification of the framework to fit an individual workflow.

3.2.3 Rapid Application Development (RAD)

Rapid Application Development (RAD) is a methodology that emphasizes speed and user involvement through the frequent creation of prototypes. It de-emphasizes the traditional

planning phase in favor of "getting a version of the software in front of the user" as quickly as possible.

According to Thesing et al. (2021), RAD operates through a “development–feedback–refinement” loop that enables stakeholders to interact with a working version of the system at an early stage, rather than waiting for a fully completed product. In academic contexts, this approach is widely recognized for its ability to bridge the “requirement gap” between what is initially designed by the developer and what is needed by the end users, by using prototypes as a form of visual agreement between stakeholders.

Building on this theoretical foundation, RAD is highly relevant to this project due to the integration of high-fidelity prototyping tools such as Figma. By developing detailed visual representations of the system interface and departmental dashboards before implementing backend functionality, it becomes possible to validate the system flow with both the supervisor and potential users at an early stage. This is particularly important in the context of a centralized event coordination platform, where multiple user roles such as Logistics, AV, and Photography must interact with the system in a clear and intuitive manner. Through this approach, the core distribution logic and interface structure can be validated visually before the implementation of complex backend components such as Flask and PostgreSQL, ensuring better alignment between system design and user expectations.

3.2.4 Hybrid Methodologies (Structured-Iterative Blends)

Hybrid methodologies seek to combine the planning and documentation strengths of Waterfall with the flexibility and speed of Agile or RAD. This is often referred to as "Water-Agile-Fall" or "Agile-Waterfall Hybrid."

Hron and Obwegeser (2018) explain that hybrid development models are becoming increasingly common in both academic and complex institutional systems. These approaches combine the strengths of different methodologies by typically applying a Waterfall structure for high-level architectural planning and requirement definition at the “macro” level, while adopting Agile-based Sprints for detailed implementation and feature refinement at the “micro” level. In this way, hybrid models provide a stable structural foundation while still recognising that software development is inherently iterative and may require continuous adjustments throughout the development process.

Building on this theoretical perspective, a hybrid approach is particularly well suited to a university project environment. The structured Waterfall component ensures that the system meets academic expectations in terms of documentation, planning, and clearly defined architecture at the early stages of development. At the same time, the Agile component supports incremental progress, allowing each functionality to be developed and validated individually until it operates effectively before moving on to the next. This balanced approach helps manage the complexity of a multi-departmental system by breaking the development process into smaller, manageable iterations, reducing cognitive overload and enabling steady, controlled progress throughout the project lifecycle.

3.2.5 Comparative Analysis of Methodologies

The candidate methodologies are evaluated based on their alignment with the technical requirements and administrative characteristics of the proposed centralized event coordination platform. This comparison considers key factors such as the ability to handle evolving requirements, suitability for individual development, support for complex workflow logic, academic relevance, and effectiveness in maintaining stakeholder alignment.

Table 4: Comparative Analysis of Software Development Methodologies.

Factor	Waterfall	Agile (Scrum)	RAD	Hybrid (Proposed)
Handling Uncertainty	Poor; requires fixed specs.	Excellent; thrives on change.	High; uses prototyping.	Balanced; handles evolution.
Solo Developer Efficiency	High; clear linear path.	Low; team-heavy ceremonies.	High; focus on UI/UX.	High; tailored for one person.
Workflow Logic Depth	High; well-documented.	Variable; depends on Sprint.	Focuses on appearance.	High; planned yet refined.
Academic Suitability	High; satisfies IR/Design.	Moderate; requires team simulation.	Moderate; UI-centric.	Very High; covers all phases.
Stakeholder Alignment	Minimal until end.	Frequent but time-consuming.	Very high via prototypes.	Strategic; at key milestones.

Note: This table compares four development methodologies and shows that the Hybrid model provides the best balance of structure and flexibility for this project.

3.4 Methodology Discussion (Pure vs Hybrid Approaches)

In a pure Waterfall approach, development follows a linear sequence of phases with limited flexibility for changes once requirements are defined. In contrast, a pure Agile approach focuses heavily on iteration and flexibility but may lack strong upfront architectural control in complex system environments.

However, recent developments in software engineering indicate that purely applying a single methodology is often insufficient for systems with mixed or evolving requirements. Alami and Krancher (2022) highlight that real-world projects benefit more from tailored or hybrid approaches rather than rigid adherence to one methodology, particularly when dealing with constraints that require both structure and adaptability.

In institutional systems such as university-based platforms, requirements are typically distributed across multiple stakeholders and are subject to refinement over time. Kadenic et al. (2023) further emphasize that stakeholder acceptance is strongly influenced by early validation and visual confirmation of system behavior. This is where hybrid approaches become more effective, as they combine the structured planning of traditional models with the iterative feedback cycles of modern development practices.

Therefore, compared to pure methodologies, hybrid approaches provide a more balanced solution by addressing both the need for architectural stability and the requirement for continuous refinement based on stakeholder input.

The case for a hybrid should not be made without stating its risk, since a process that borrows from two models can as easily inherit the weaknesses of both, and become a licence to plan less than Waterfall requires while iterating less than Agile expects. Hron and Obwegeser (2018) describe adaptation of this kind as ordinary practice rather than as a failure of discipline, but adaptation is only defensible when what is taken from each model, and what is left behind, are stated in advance. That is the condition applied in the following section. The Waterfall element is confined to the phases whose output is a fixed artefact, the Agile element is confined to implementation, and the prototyping element is confined to the interface, so that each part of the process is governed by one model rather than by a negotiation between two.

3.5 Selected methodology and Justification

The **Waterfall-Agile Hybrid with Evolutionary Prototyping** is selected as the optimal methodology for this project. This choice is justified by the following evidence-based factors:

1. **Requirement Refinement via Prototyping:** Since the project requirements are currently in an early conceptual stage, the use of evolutionary prototyping through tools such as Figma plays a critical role in clarifying functional expectations. This approach enables early visualization of system components and user interactions before implementation begins. As a result, requirement gaps, inconsistencies, and overlooked functionalities can be identified at an early stage. This reduces the likelihood of costly revisions during later development phases and supports a more accurate translation of stakeholder needs into system features.

2. **Structural Integrity vs. Functional Flexibility:** The Waterfall component of the methodology ensures that essential foundational activities—such as initial investigation, system analysis, and high-level architectural design—are completed in a structured and sequential manner. This is particularly important for ensuring that the centralized, role-based architecture of the system is properly defined prior to implementation. At the same time, the integration of Agile principles introduces flexibility at the functional level, allowing iterative refinement of modules without compromising the overall system structure. This balance ensures both design stability and adaptability.

3. **Optimization for Individual Execution:** The adaptation of Agile principles into short, focused development cycles supports continuous progress evaluation and quality assurance. Each functional component is developed, tested, and validated before proceeding to the next stage. This iterative workflow enhances system reliability and ensures that each feature meets its intended functional requirements. It also reduces dependency bottlenecks typically associated with large-scale team coordination, allowing development to progress in a controlled and self-managed manner while maintaining consistent quality standards.

4. **Institutional Workflow Alignment:** The hybrid methodology is well aligned with the multi-departmental coordination structure required by the system. The Waterfall component supports the documentation of stable, organization-wide rules governing role permissions and request distribution logic. In parallel, the Agile and prototyping

components allow iterative refinement of user interfaces and departmental interaction flows based on continuous supervisor feedback. This ensures that while the core system logic remains consistent and well-defined, the user-facing components evolve in response to practical usability considerations and institutional expectations.

The three pure alternatives were rejected on specific grounds rather than on general preference, and each was rejected only in part. Waterfall was set aside because its central assumption does not hold for this project. It is at its strongest where requirements are settled before design begins, and Srivastava (2019) identifies those conditions as the ones that favour it. Here the departmental requirements were not settled: the fields a cafeteria request needs, and the point at which a rejected proposal returns to its applicant, were established during implementation, in discussion with the supervisor and by observing where the interface failed. Under a pure Waterfall process those discoveries would have arrived after the design phase had been signed off, and each would have been a change against a frozen specification rather than an ordinary revision. What Waterfall does supply, a documented architecture fixed before implementation begins, was retained in the hybrid rather than discarded along with it.

A pure Scrum implementation was rejected for a reason of scale rather than of principle. Scrum's mechanisms are designed to coordinate a team: the daily stand-up synchronises people who cannot otherwise see each other's work, the retrospective surfaces problems in how a group is working together, and the separation of Product Owner from Scrum Master divides the decision about what to build from the decision about how to build it. Kadenic et al. (2023) tie the benefit Scrum delivers to team maturity and to the presence of those components. With one developer none of the coordination problems they solve exist, and performing the ceremonies would produce a record of a team's dynamics where there is no team. The parts of Scrum whose value does not depend on team size, short time-boxed iterations and testing carried out continuously rather than at the end, were kept.

Rapid Application Development came closest to being adopted outright, since prototyping is central to this project and the requirement gap it addresses was real. It was rejected as a complete methodology because it de-emphasises the planning phase, and the elements of this system that must not be revised late are precisely the ones it would defer: the identity model that pairs a user, a role and an organisational unit, and the transactional guarantee that binds an audit record to the state change it describes. Both are foundations that everything else is built against, and reworking either after implementation had begun would

have propagated through the whole system. Thesing et al. (2021) treat the choice between plan-driven and iterative approaches as a decision to be made for each project against its own characteristics rather than adopted wholesale, which is the reasoning applied here: prototyping was taken from RAD and applied to the interface, where late change is cheap, while the architecture was settled first under the plan-driven part of the process, where late change is not.

Overall, the selected methodology provides a comprehensive framework that supports both rigorous system design and adaptive development. It ensures that the system evolves in alignment with stakeholder requirements while maintaining structural consistency and implementation efficiency throughout the project lifecycle.

3.6 Activities and Processes Within Each Phase

The selected **Waterfall–Agile Hybrid with Evolutionary Prototyping** structures the development process into a sequence of high-level phases, while incorporating iterative refinement within each stage. This ensures that the system maintains a stable architectural foundation while allowing continuous improvement based on feedback and evolving requirements.

The initial phase, **requirement analysis and investigation**, focuses on identifying the core functionalities of the centralized event coordination platform. This includes defining the roles of different departments (e.g., Logistics, AV, Transportation), understanding request workflows, and establishing the overall system scope. At this stage, information gathered from the Initial Report (IR) and stakeholder discussions is translated into structured requirements.

Following this, the **system design phase** establishes the architectural foundation of the platform. This includes designing the database schema, defining role-based access control, and outlining the request distribution logic that ensures each department receives relevant tasks. High-level system diagrams and data flow structures are developed to guide implementation.

To address the uncertainty in user requirements, **prototyping activities** are integrated early in the process. Using tools such as Figma, high-fidelity interface designs and dashboard layouts are created to simulate user interaction. These prototypes are reviewed and refined

through feedback, ensuring that the system interface aligns with user expectations before backend development begins.

The **implementation phase** is conducted iteratively, following Agile-inspired cycles. Functional modules—such as event request submission, departmental dashboards, and approval workflows—are developed incrementally. Each module is tested individually before integration, ensuring correctness and reducing system-level errors.

During the **integration and testing phase**, individual components are combined into a unified system. Testing is performed continuously rather than as a final step, including validation of request routing logic, user role permissions, and system usability. Feedback from supervisors is incorporated to refine both functionality and interface design.

Finally, the **deployment and evaluation phase** involves preparing the system for practical use, ensuring stability, and documenting the final implementation. Any remaining adjustments are made based on final validation, ensuring that the system meets both technical requirements and institutional workflow expectations.

Overall, this structured yet flexible process enables the system to be developed in a controlled manner while still accommodating iterative improvements, ensuring alignment with both the technical objectives and the user requirements identified in the project.

3.7 Data Gathering Design

3.7.1 Data Collection Method

The data gathering process for this project was conducted using a **survey-based approach**, selected due to its effectiveness in collecting structured and quantifiable feedback from a relatively large number of participants within a limited timeframe. This method enables the identification of common trends, patterns, and user expectations, which are essential for requirement analysis. In the context of a university-based system, surveys are particularly suitable as they allow input to be gathered from a diverse group of users with varying levels of interaction with event management processes.

Three alternatives were considered before the survey was chosen. Semi-structured interviews would have produced a richer account of how coordination currently fails and would have allowed each answer to be followed up; they were set aside because the question at this stage was whether the difficulties are widespread rather than how one person experiences them, and the number of participants who can be interviewed within the project timetable is too small to settle that. Focus groups were rejected on two grounds: assembling students and staff from several departments at one time is difficult within a semester, and a group setting removes the anonymity that matters when participants are asked to describe shortcomings in their own institution's processes, with the further risk that a senior participant's view shapes the responses of the others. Direct observation of the existing process would have given the most faithful picture of the three, but a single event cycle runs over several weeks and passes through departmental steps that a student has no access to observe.

Analysis of existing documentation was considered as a fourth option and could not be carried out, for a reason that is itself a finding: there is no single set of forms or records to analyse, because the process is spread across email, spreadsheets and messaging applications, which is the fragmentation stated as a problem in Section 1.2.2.1. The survey was therefore selected as the only method that could reach a group large enough for its answers to be read as a pattern, within the time available and without asking participants to identify themselves. Its known weakness is that a fixed set of questions cannot capture what the person writing them did not think to ask, which is why the open-ended question described in Section 3.7.4 was included, and why the survey results were treated as the starting point for the requirements rather than as the whole of them, with the remainder refined through prototyping and supervisor feedback during development.

3.7.2 Target Participants

The target participants consisted of **students and staff members of Asia Pacific University (APU)**. These groups were selected because they represent the primary users and stakeholders of the proposed event coordination platform. Students often participate in or organize events, while staff members are typically involved in approval processes, logistics, or administrative coordination. Their combined perspectives provide a comprehensive understanding of both operational challenges and user expectations within the current system.

3.7.3 Sample Size and Distribution

A total of **30 participants** were involved in the survey, fulfilling the minimum requirement for survey-based data collection. The survey was developed using **Google Forms**, which allowed for efficient distribution, response collection, and data organization. To reach the intended audience effectively, the survey link was shared through commonly used communication channels such as **email, WhatsApp, and Excel-based contact lists**. This multi-channel distribution approach increased accessibility and encouraged participation from individuals with different communication preferences.

3.7.4 Survey Structure and Design

The survey was carefully structured into four main sections to ensure comprehensive and organized data collection, with each section targeting a specific aspect of the research:

- **Section 1: Participant Information**
  This section collected demographic data such as age, gender, role (student or staff), and year of study. These details are important for categorizing responses and understanding how different user groups perceive event coordination processes. As required, this demographic information is included in the appendix for reference.
- **Section 2: Current Event Coordination Issues**
  This section focused on identifying existing challenges in event planning and management within the university. Participants were asked about their frequency of participation in events, the tools currently used for coordination (e.g., email, paper forms, WhatsApp, Excel), and the types of issues they encounter. Likert-scale questions were used to measure factors such as clarity of the process, satisfaction levels, and frequency of difficulties, allowing for both qualitative and quantitative analysis.
- **Section 3: Proposed Solution – Usefulness and Acceptance**
  In this section, participants were introduced to the concept of a centralized, role-based event coordination platform. They were asked to evaluate its potential usefulness, their likelihood of using such a system, and its expected impact on improving efficiency and reducing coordination issues. This helps assess user acceptance and the feasibility of implementing the proposed solution.

- **Section 4: Additional Features**
  This section gathered user preferences regarding potential system enhancements. Participants were asked to rate the importance of features such as mobile accessibility, real-time notifications, and fully digital request management. An open-ended question was also included to allow participants to suggest additional features, ensuring that user input is not limited to predefined options.

3.7.5 Considerations

Ethical guidelines were strictly followed throughout the data collection process. Participants were required to provide informed consent before proceeding with the survey, confirming that participation was voluntary and that they were 18 years of age or older. All responses were collected anonymously to protect participant identity, and the data was used solely for academic purposes. Additionally, all collected data was stored securely and handled with confidentiality, with approval obtained from the project supervisor in accordance with institutional ethical requirements.

3.7.6 Purpose of Data Collection

The primary purpose of the data collection was to identify current limitations in event coordination practices, evaluate user acceptance of the proposed system, and gather insights into desired features and improvements. The collected data provides a foundation for defining system requirements and ensuring that the developed platform aligns with the needs of its intended users. Furthermore, it supports evidence-based decision-making throughout the design and development phases of the project.

3.8 Analysis

3.8.1 Quantitative Data Presentation

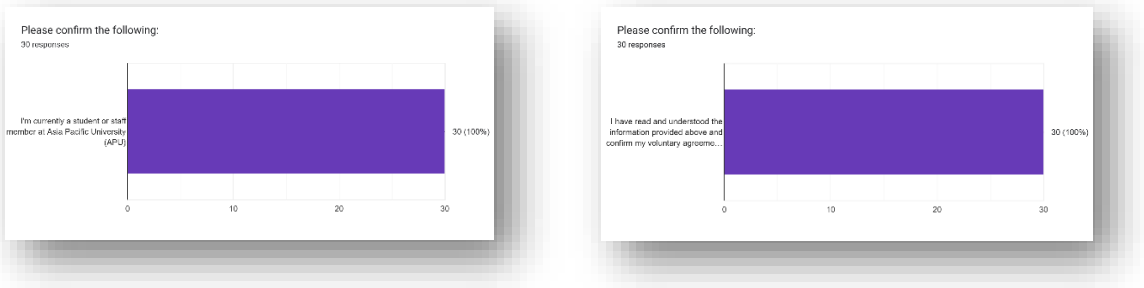


Figure 11: *Participant Eligibility and Consent*

The survey results confirm a **100%** compliance rate regarding participant eligibility and voluntary consent. **100%** of respondents identified as current students or staff members of Asia Pacific University (APU). This ensures that the data collected is highly relevant and reflects the actual experiences of the stakeholders who will interact with the proposed event coordination platform.

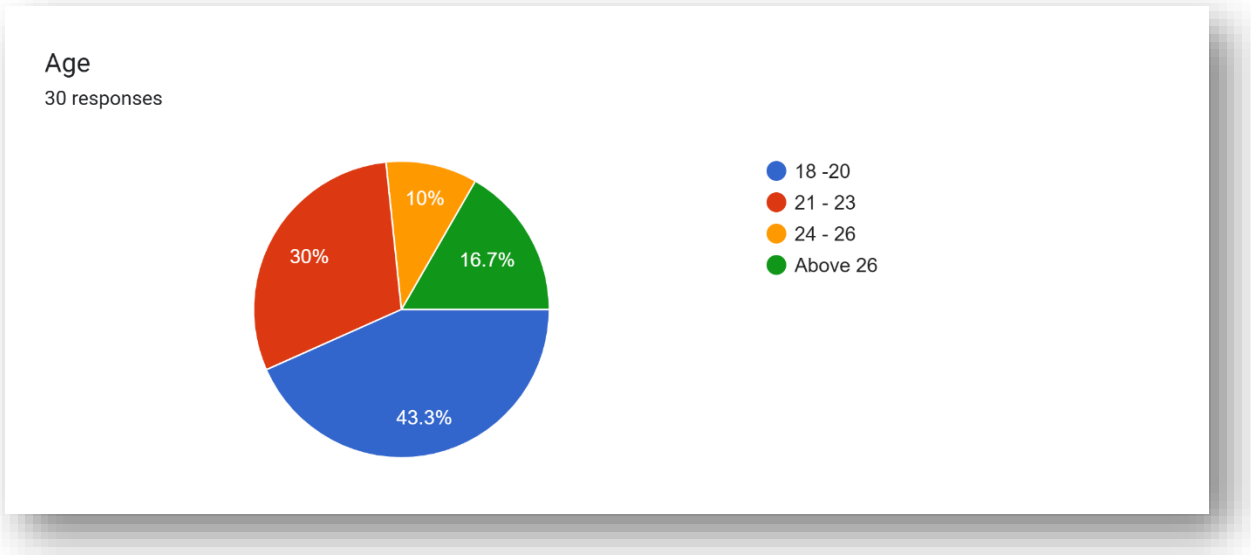


Figure 12 : *Age eDistribution of Survey Participants*

Most of the respondents fall within the 18–20 (**43.3%**) and 21–23 (**30%**) age brackets. This indicates that the primary user base of the platform consists of "digital natives" who are

highly accustomed to mobile and web-based applications. Their familiarity with technology suggests a high potential for rapid adoption of a digital system over traditional manual method.

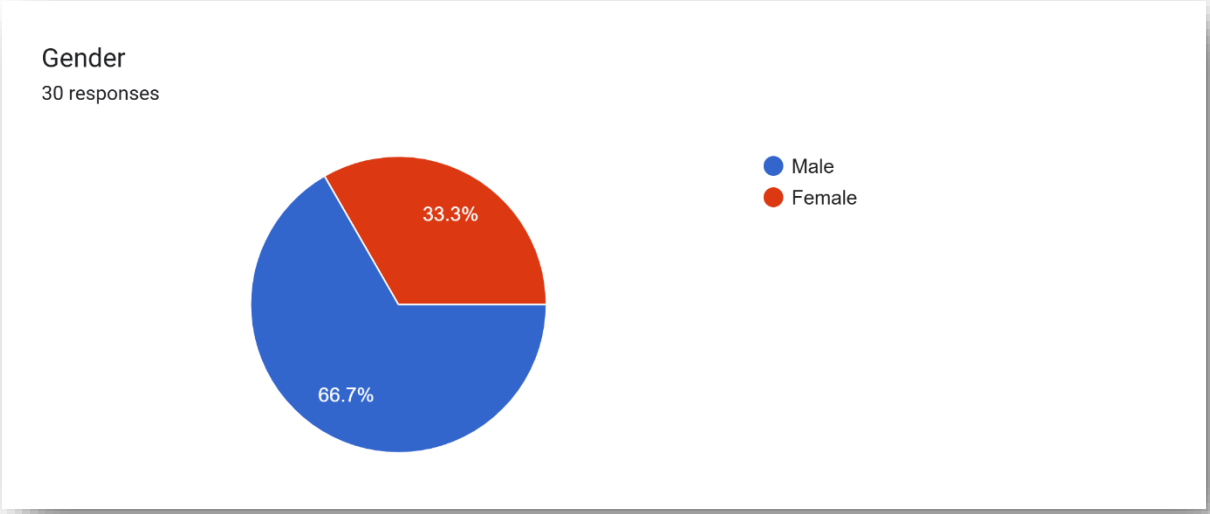


Figure 13: *Gender Distribution*

The participant pool consists of **66.7%** males and **33.3%** females. This diverse representation helps in understanding if different user groups have varying perspectives on system usability, ensuring the final event coordination system is inclusive and user-friendly for all members of the university community.

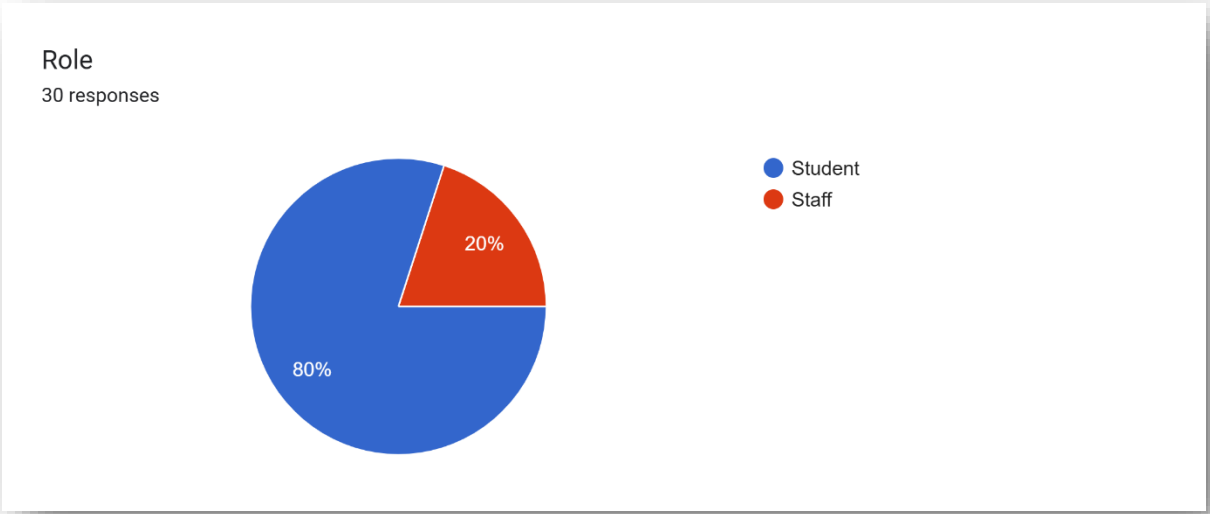


Figure 14: *Roles within the University The*

data shows a split between students (**80%**) and staff members (**20%**). While students represent the primary end-users who will be requesting and attending events, the inclusion of staff members is critical. Staff members often act as the "Approvers" or "Service Providers" (Logistics, AV, etc.) in the workflow, and their feedback validates the need for a centralized administrative backend.

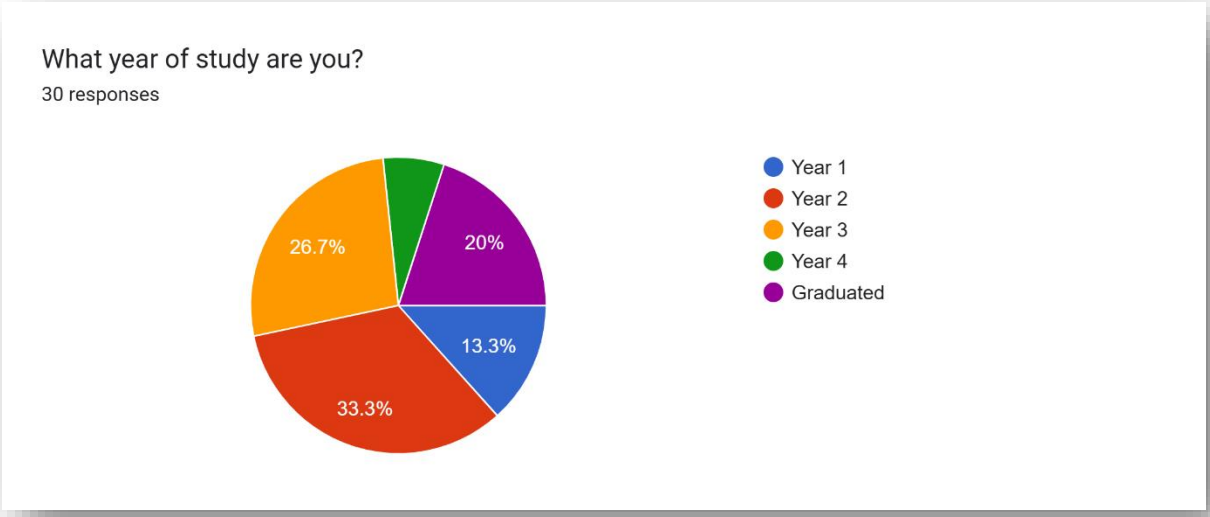


Figure 15: *Academic Year Representation*

Respondents are spread across all years of study, with a significant number in Year 2 (**33.3%**) and Year 3 (**26.7%**). This variety ensures the findings are not limited to one specific cohort but represent a broad academic lifecycle. Graduates (**20%**) also provided input, offering a retrospective view of the coordination challenges they faced during their full tenure at APU.

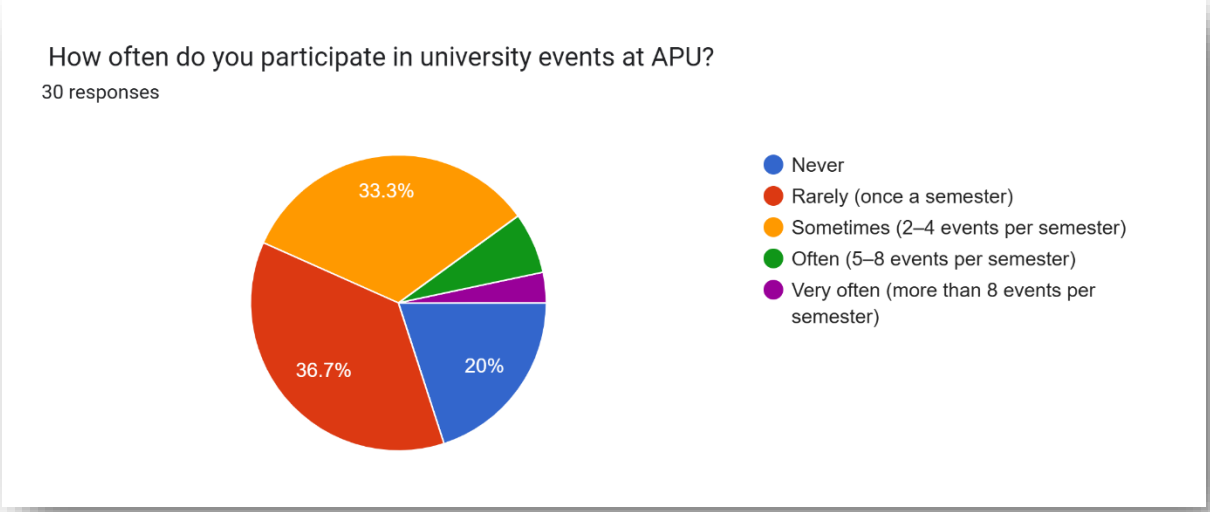


Figure 16: *Frequency of Event Participation at APU*

A combined **70%** of participants participate in events at least once a semester or more. This high level of engagement underscores the importance of a reliable management system. Since a large portion of the community interacts with the event lifecycle regularly, any inefficiencies in coordination have a wide-reaching impact on the student experience.

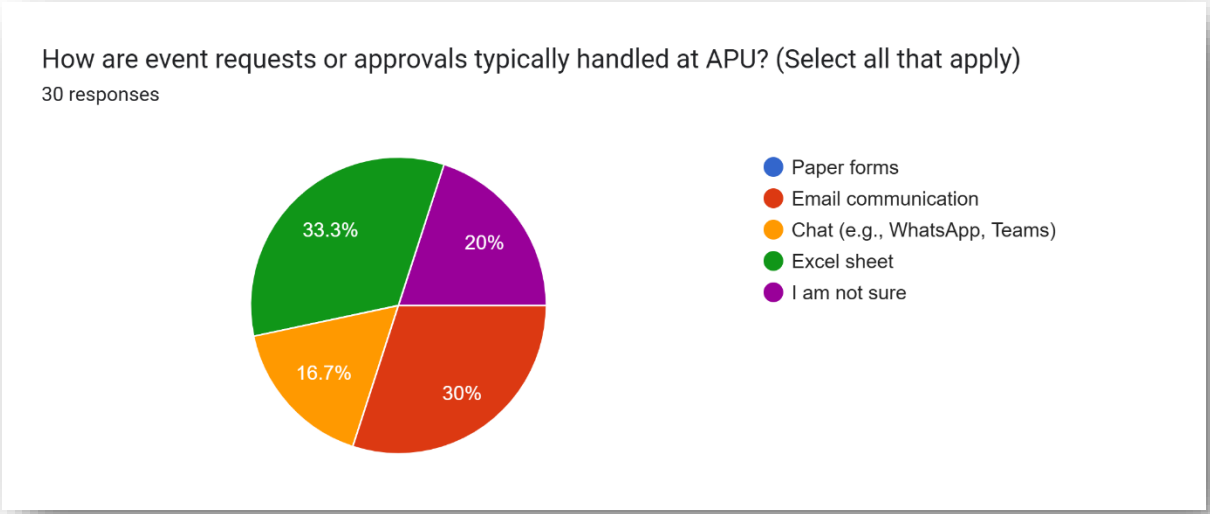


Figure 17: *Current Methods for Handling Event Requests*

The survey reveals a fragmented approach to event management, with **33.3%** of participants using Excel sheets, **30%** using Email, and **16.7%** relying on chat platforms like WhatsApp or Teams. This lack of a single "source of truth" confirms the core problem: data is

scattered across different platforms, leading to communication gaps and a lack of real-time visibility.

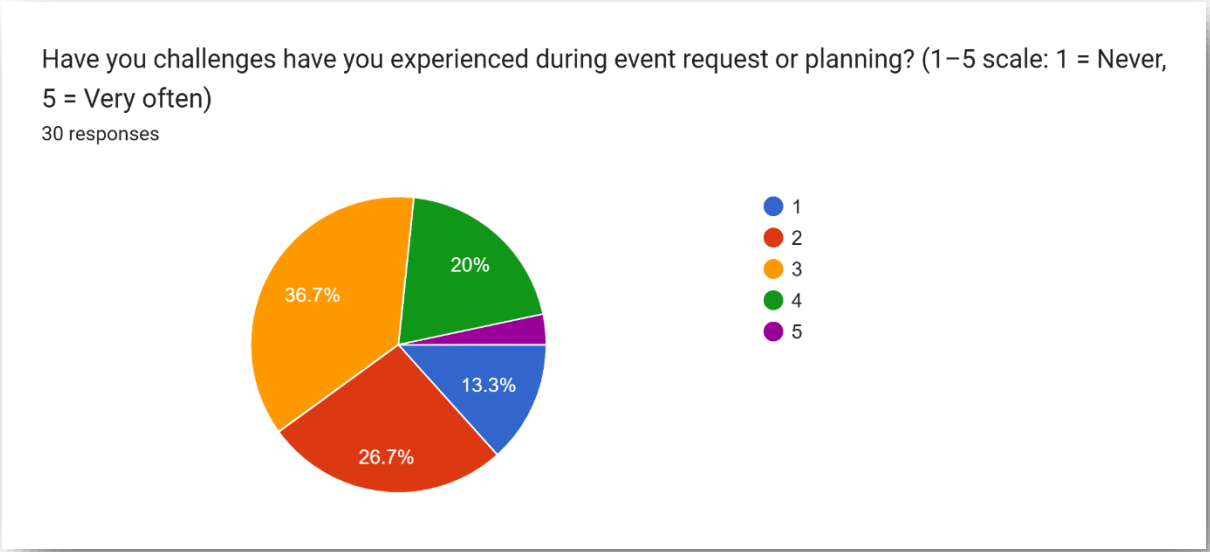


Figure 18: *Perceived Challenges in Event Planning*

When asked about challenges experienced during event planning (on a scale of 1 to 5), the majority of respondents chose 3 or 4. This indicates that coordination issues are a frequent occurrence rather than rare exceptions. Only a small minority (**13.3%**) reported never facing challenges, proving that the current manual/semi-digital workflow is consistently problematic.

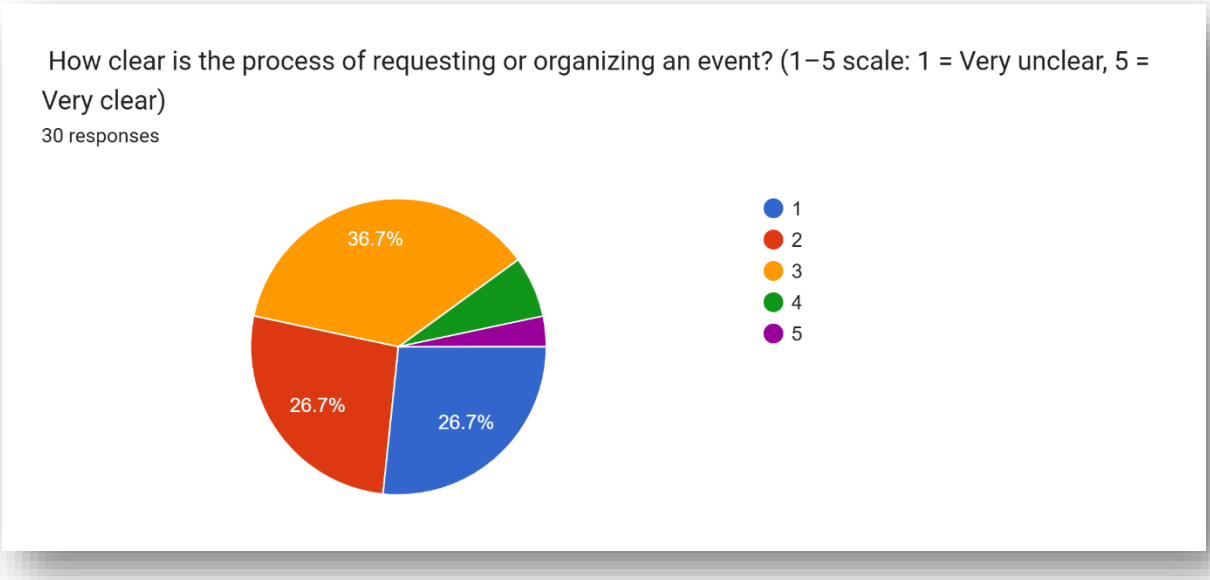


Figure 19: *Clarity of the Existing Request Process*

Process clarity is a major concern, with **53.3%** of participants rating the current process as "Unclear" (1 or 2 on the scale). Only **3.3%** of participants rated it as "Very Clear." This highlights a significant "information gap" where users are unsure of the steps required to organize an event, justifying the need for a system that provides a structured, step-by-step digital workflow.

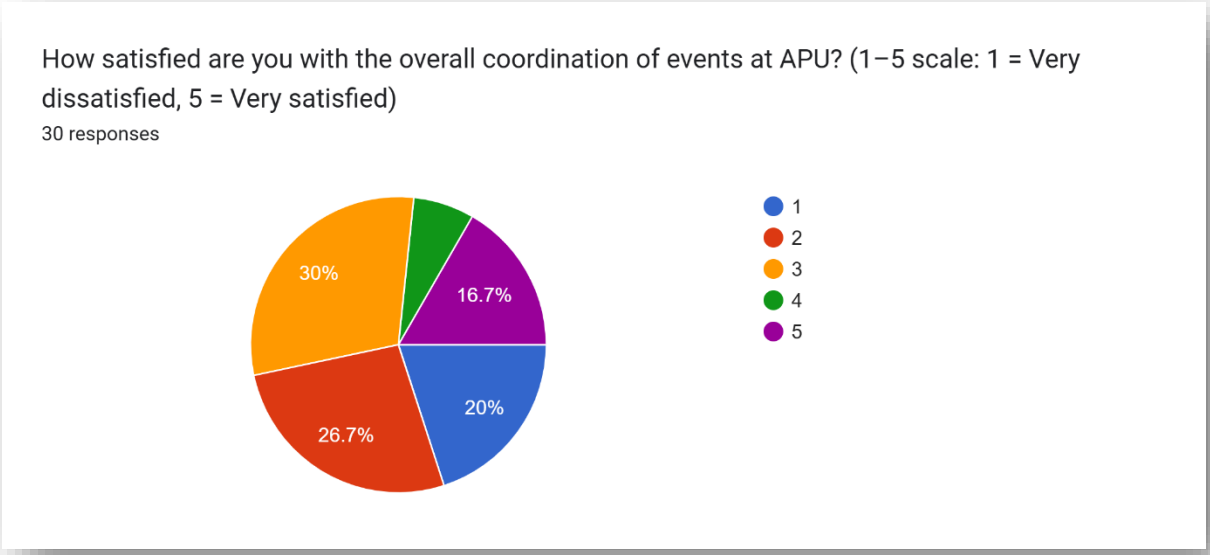


Figure 20: *Overall Satisfaction with Current Coordination*

Satisfaction levels are notably low, with **46.7%** of respondents being "Dissatisfied" (1 or 2) and **30%** being "Neutral" (3). Only **16.7%** of participants expressed high satisfaction. This data serves as a strong mandate for change, showing that the current system does not meet the expectations or needs of the university population.

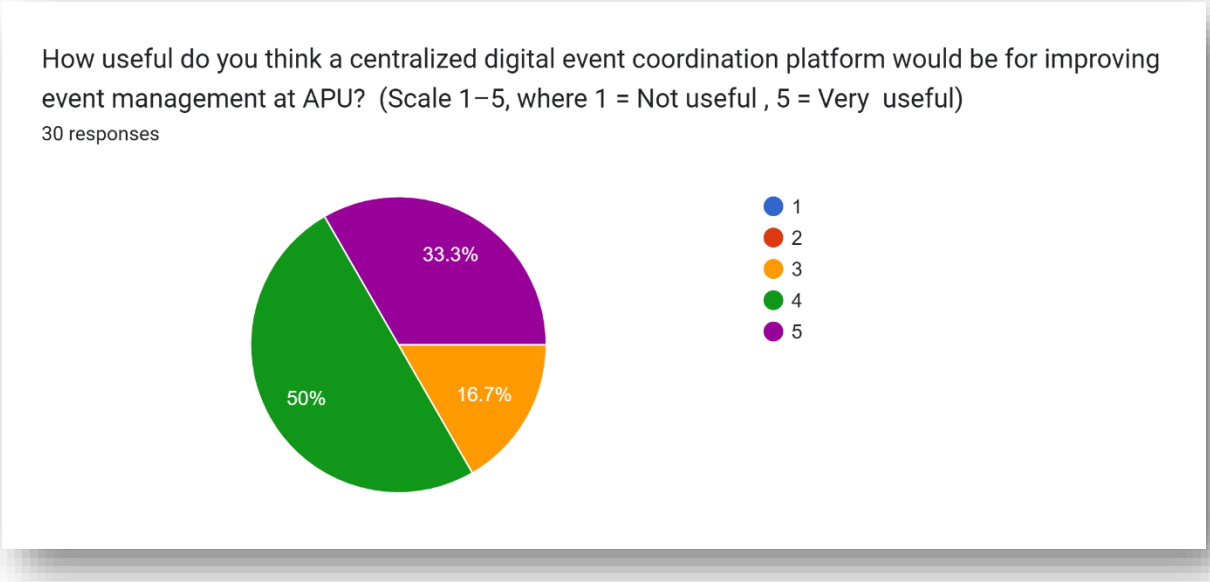


Figure 21: *Perceived Usefulness of a Centralized Platform*

There is overwhelming support for the proposed solution, with **83.3%** of participants rating the usefulness of a centralized digital platform as 4 or 5. This high level of perceived value indicates that the university community recognizes the potential for a digital system to solve the existing coordination bottlenecks.

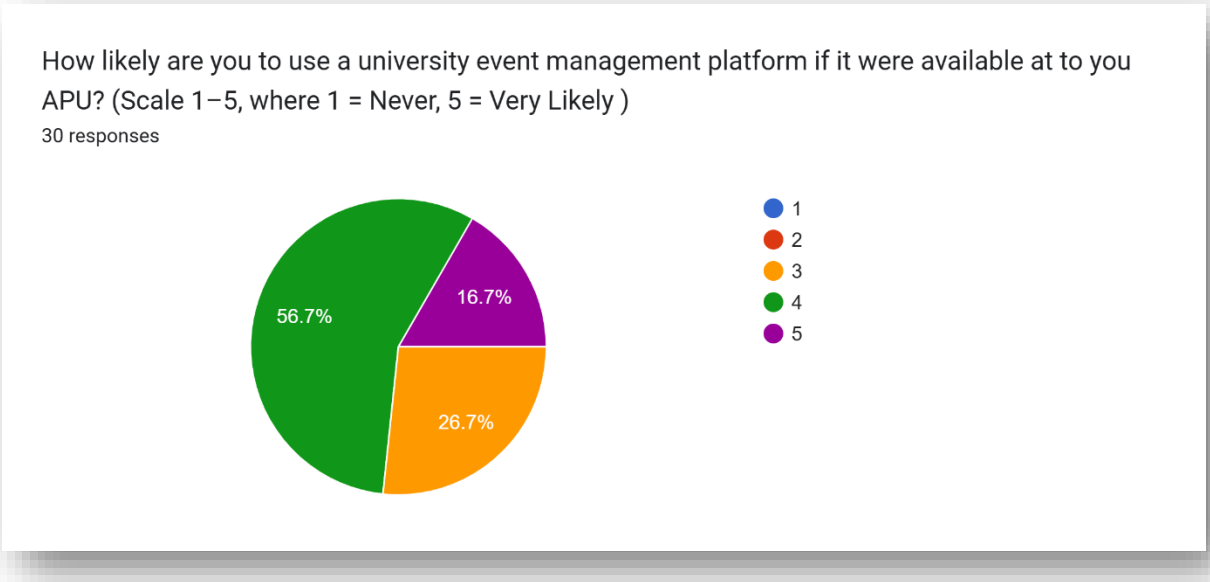


Figure 22*: Likelihood of System Adoption*

User intent to use the new platform is very high, with **73.3%** of participants indicating they are "Likely" or "Very Likely" to adopt the system. This suggests that the platform will not face significant user resistance and will likely see high engagement once implemented.

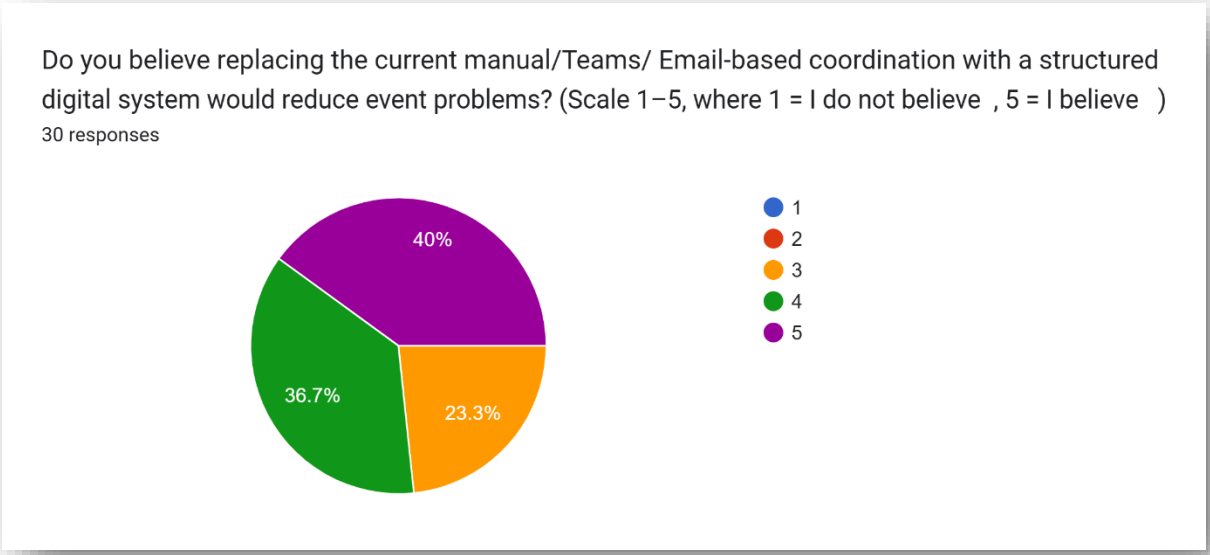


Figure 23: *Belief in Problem Reduction via Digital System*

A total of **76.7%** of participants believe (rating 4 or 5) that replacing manual/email-based systems with a structured digital platform will reduce event-related problems. This confirms that the project's goal of "Intelligent Coordination" aligns with user expectations for error reduction and increased accountability.

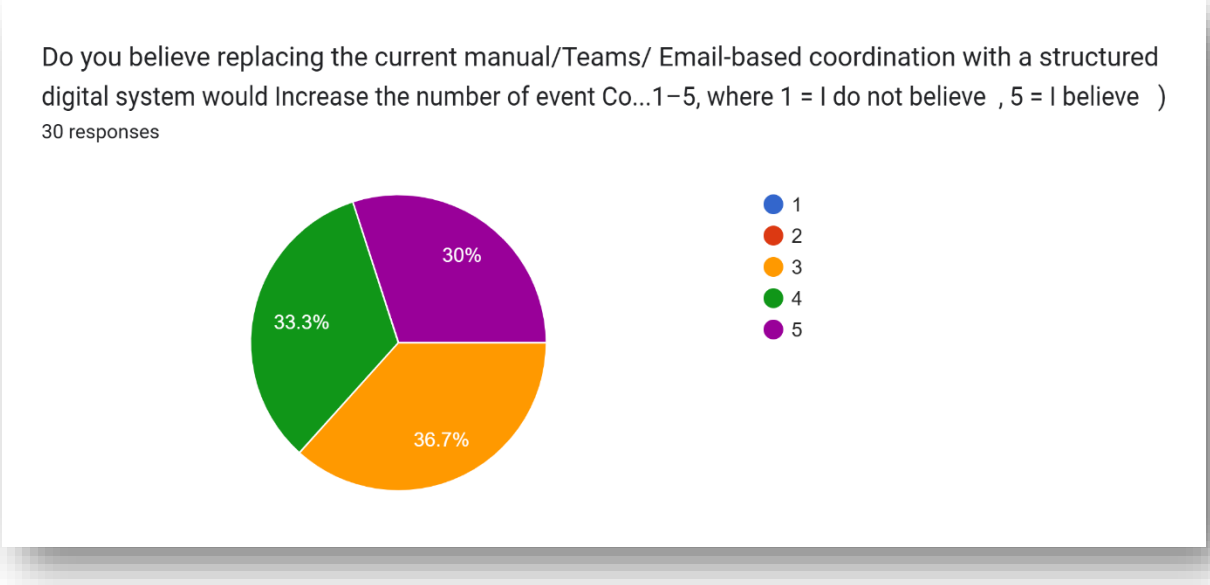


Figure 24: *Potential Impact on Event Frequency*

The survey suggests that a digital system could actually increase the number of events conducted at APU. **63.3%** of participants believe (rating 4 or 5) that a more efficient system would encourage more frequent event activity by making the administrative process less daunting and time-consuming.

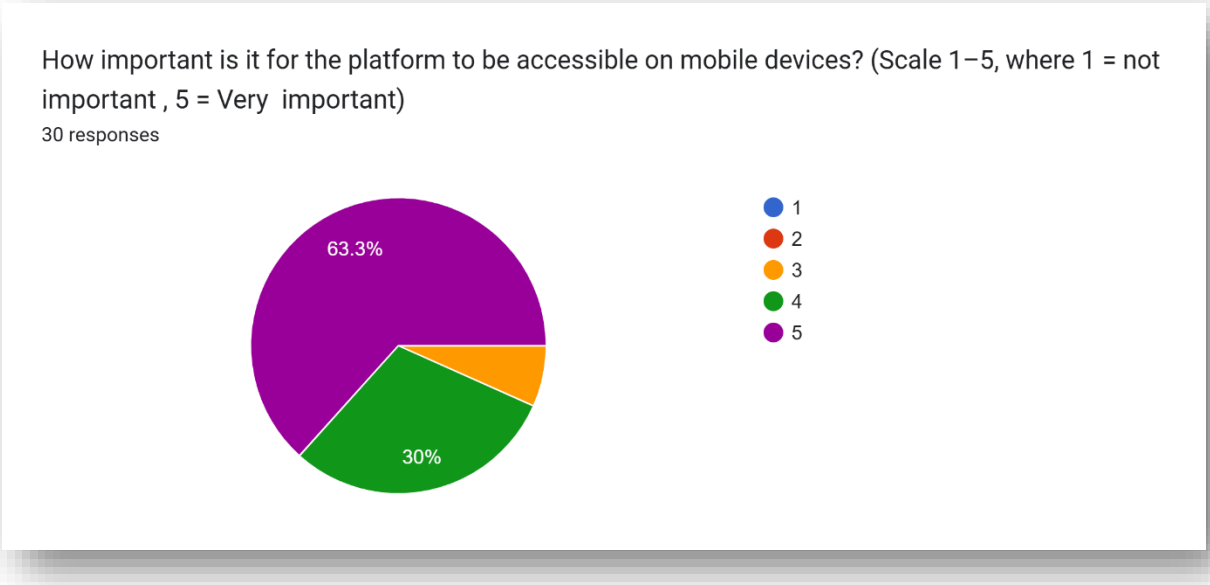


Figure 25*: Importance of Mobile Accessibility*

Mobile access is deemed critical, with **93.3%** of participants rating it as important (4 or 5). This finding is essential for the technical design of the FYP, confirming that the solution must be a cross-platform (Web and Mobile) system to accommodate users who need to manage event tasks on the go.

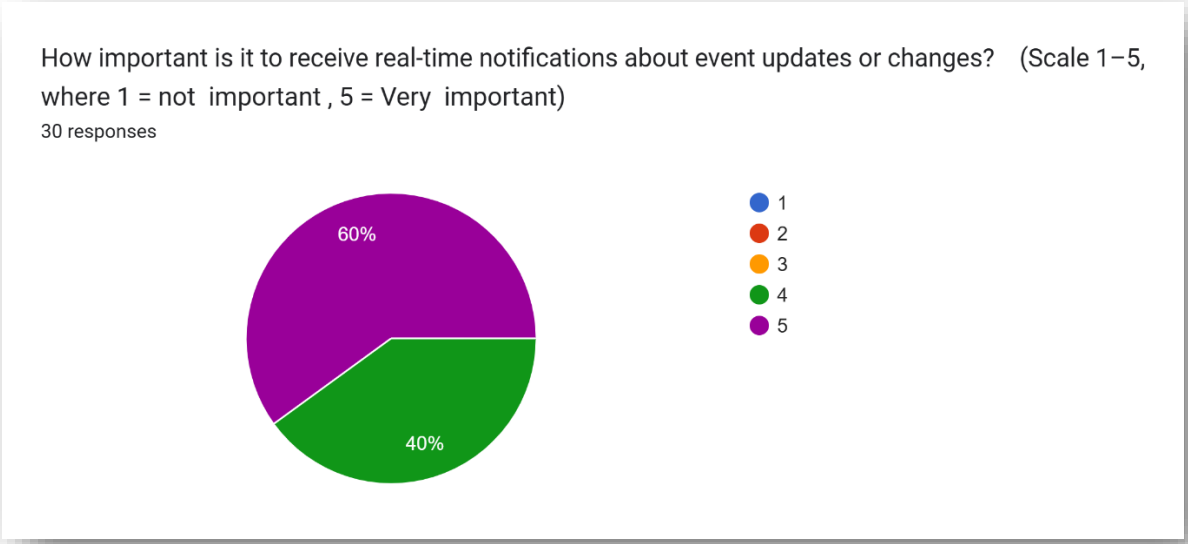


Figure 26: *Importance of Real-Time Notifications*

Real-time notifications received the highest "Importance" ratings in the entire survey, with **60%** of participants rating them a 5 (Very Important). This emphasizes that the platform must include an automated alert system to inform users of status changes, approvals, or required revisions immediately.

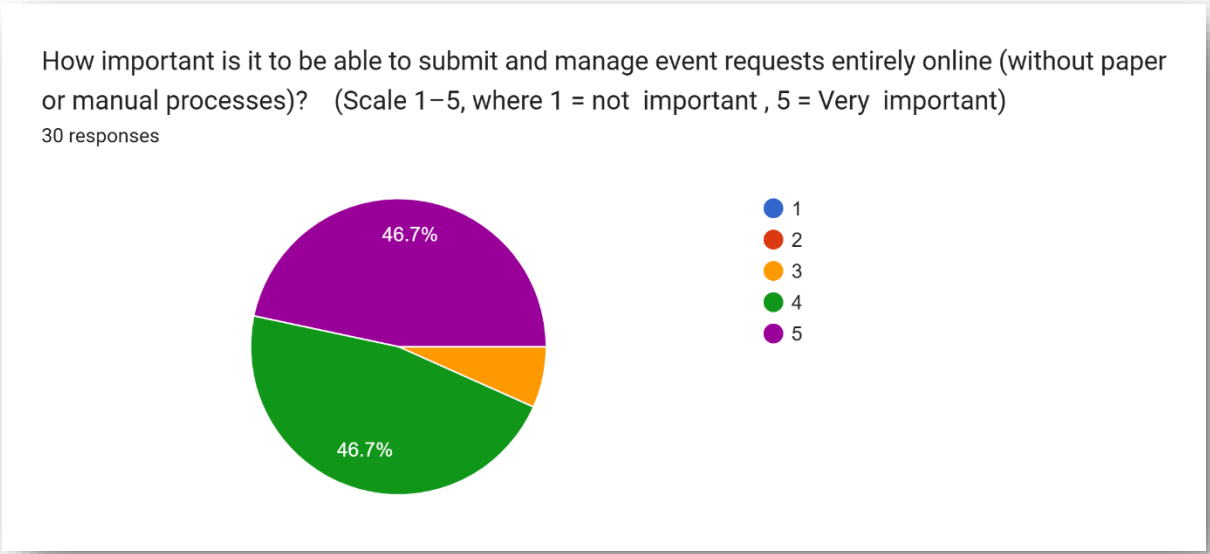


Figure 27: *Preference for Fully Online Management*

The move toward a paperless university is supported by the data, where **93.3%** of participants expressed a strong desire to manage event requests entirely online. This confirms that the "Less Paper" initiative is a priority for the APU community, aligning the project with sustainable and modern administrative practices.

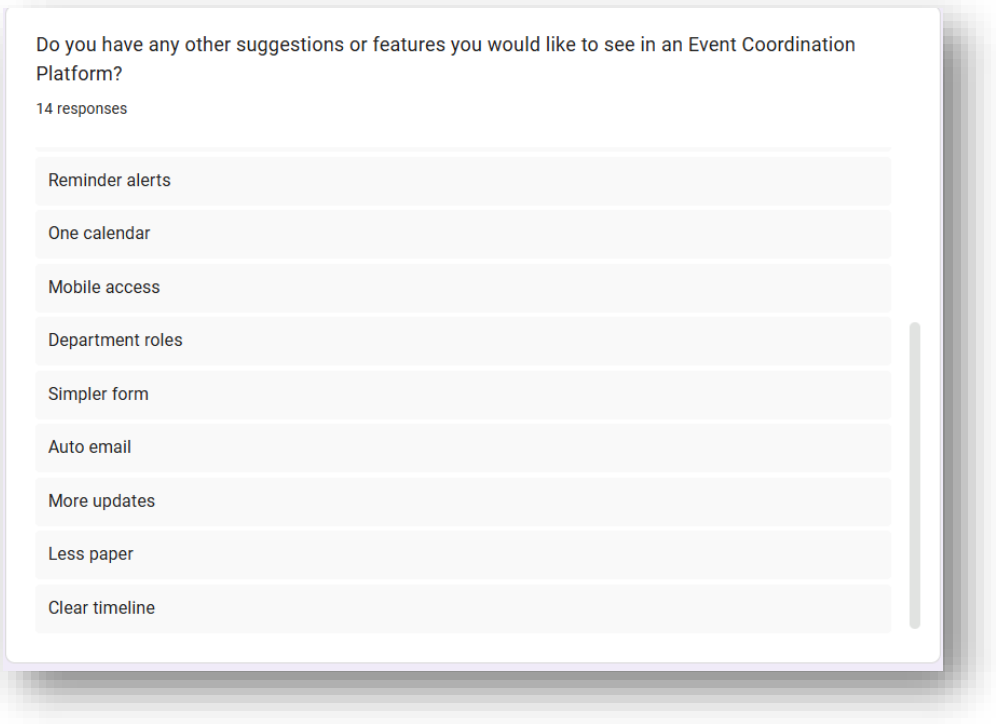


***Figure 28**: User-Suggested Feature Enhancements*

The open-ended suggestions provide a roadmap for the platform's features. Users specifically requested high-level technical tools such as a Single Dashboard, Central Chat, Track Status, and Auto-Email alerts. These suggestions have been directly integrated into the final User Requirements List for the system, ensuring the development is truly user driven.

User suggestion:

- Central chat.
- Single dashboard.
- Track status.
- Faster approval.
- Better AV flow.
- Reminder alerts.
- One calendar.
- Mobile access.

- Department roles.
- Simpler form.
- Auto email.
- More updates.
- Less paper.
- Clear timeline.

3.8.2 Summary of Findings and Synthesis

The analysis of the quantitative data gathered from 30 stakeholders at APU highlights three critical conclusions that justify the development of the Intelligent Web and Mobile Event Management System:

1. **The "Information Silo" Problem:** The heavy reliance on Excel (33.3%) and Email (30%) creates a fragmented environment where nearly half of the participants (46.7%) are dissatisfied with coordination. The data proves that current methods fail to provide a single "source of truth," leading to the confusion identified in Figure 20.
2. **Technological Readiness and Expectations:** With 93.3% of users demanding mobile accessibility and 60% prioritizing real-time notifications, there is a clear expectation for a modern, responsive solution. The high potential for adoption (73.3%) suggests that the APU community is ready to move away from legacy manual processes.
3. **Operational Efficiency as a Catalyst:** Perhaps the most significant finding is that 63.3% of participants believe a better system will lead to *more* events. This suggests that the current administrative burden is actually a barrier to university activity. By simplifying forms and centralizing approvals, the proposed platform will not only organize existing events but likely increase the overall vibrancy of campus life.

These findings directly inform the system design, specifically the need for **Real-Time Status Tracking**, and a **Single Dashboard** as requested by the participants.

3.9 Summary

This chapter established the methodological framework used for developing the centralized role-based event coordination platform. It began by evaluating several SDLC approaches, including Waterfall, Agile (Scrum), RAD, and hybrid models, highlighting their strengths and limitations in relation to system complexity, individual development, and evolving user requirements. The analysis showed that while traditional models offer structure and documentation, iterative approaches provide flexibility and better user alignment.

Based on this comparison and supporting literature, a Waterfall–Agile Hybrid with evolutionary prototyping was selected as the most suitable methodology. This approach combines structured planning for system architecture with iterative development and continuous feedback through prototyping tools. It ensures both stability in core system design and adaptability in user-facing features. The three pure alternatives were rejected on grounds specific to this project rather than on general preference: Waterfall because the departmental requirements were not settled before design began, Scrum because its coordination mechanisms address problems that do not arise for a single developer, and RAD because it defers the architectural decisions this system cannot afford to revise late. What each contributes was kept, and Section 3.5 states which part of the process each of them governs.

The chapter also outlined the overall development phases, data collection strategy, and analysis approach. Survey-based data from APU students and staff provided key insights into current event coordination challenges and validated the need for a centralized digital solution. Overall, the chosen methodology ensures a balanced, controlled, and user-driven development process aligned with both academic requirements and real-world institutional needs.

CHAPTER 4: DESIGN AND IMPLEMENTATION

4.1 Introduction

This chapter documents the design and implementation of the APU Event Management System, the centralised, role-based event coordination platform proposed in the earlier chapters of this report. Where Chapter 1 defined what the system was intended to achieve and Chapter 2 justified the technologies selected to build it, this chapter records what was actually designed and delivered. It presents the structural design of the system, the database that supports it, the interface through which users interact with it, the implemented screens themselves, and representative extracts of the source code that realise the behaviour described.

The chapter is organised to move from the abstract to the concrete. Section 4.2 presents the design of the system through a set of modelling views: the overall architecture and the layering it enforces, the identity and authorisation model on which every access decision depends, the capabilities available to each category of user expressed as use case diagrams, the behaviour of the approval and fulfilment workflow expressed as activity diagrams, and the ordering of messages between components expressed as sequence diagrams. Section 4.3 documents the physical database, presenting the entity relationship diagram of the deployed schema together with a data dictionary describing the purpose of every table. Section 4.4 addresses the interface, covering navigation structure, screen layout, the role-driven menu, the design tokens and shared components that hold the interface visually consistent, storyboards of the three principal user journeys, and the responsive strategy that allows the same pages to serve desktop and mobile users.

Section 4.5 then presents the delivered system itself. Each implemented screen is shown as captured from the running application on both a desktop and a mobile viewport, accompanied by a discussion of what the screen does, why it is designed as it is, and how it contributes to the objectives set out in Chapter 1. Section 4.6 presents annotated extracts of the source code behind the most significant behaviours in the system, and Section 4.7 summarises the chapter. Together these sections provide a complete account of how the proposed platform was translated from a documented intention into a working system.

4.2 Design

4.2.1 Design Overview and Rationale

The system is designed as a three-tier web application in which responsibility is separated strictly by layer. The presentation tier is an Angular single-page application that runs in the user's browser and is responsible only for rendering and for collecting input. The application tier is a Flask service that owns every rule in the system: it authenticates the caller, decides what that caller is permitted to do, executes the approval workflow, and is the only component permitted to reach the database. The data tier is a PostgreSQL database hosted as a managed service on Supabase. Two external services sit outside the system boundary and are reached only from the application tier: the Google Gemini model used by the assistant, and an SMTP relay used to send notification email.

The design is governed by three principles that recur throughout the chapter. The first is that the client is never the authority. The Angular application renders whatever the server returns and evaluates no permission rule of its own; the identity of the acting user is taken from the authentication token attached to each request and never from a field supplied by the browser. This principle is what allows the interface to be generated dynamically per role without weakening access control, because a page that appears in a user's menu is a page the server has already agreed that user may open.

The second principle is that state and evidence change together. Every transition in the approval workflow writes its audit record inside the same database transaction as the state change it describes, so the recorded history of a request can never disagree with the request's current status. If any part of a write fails, the whole transaction is rolled back and nothing is persisted. This directly supports the accountability objective stated in Chapter 1, because it makes the approval trail a by-product of the workflow rather than a separate record that must be maintained and can drift out of step.

The third principle is that behaviour which an administrator may reasonably need to change is stored as data rather than written into the code. The pages each role can reach, the option lists offered on the request form, the approval thresholds that decide whether a request must be seen by the CFO, and the escalation windows that decide when an approver is chased are all held in configuration tables and read at the moment they are needed. An administrator can therefore reconfigure the institution's coordination policy without a code change or a redeployment, which is a practical requirement for a system intended to survive changes in university procedure.

4.2.2 System Architecture

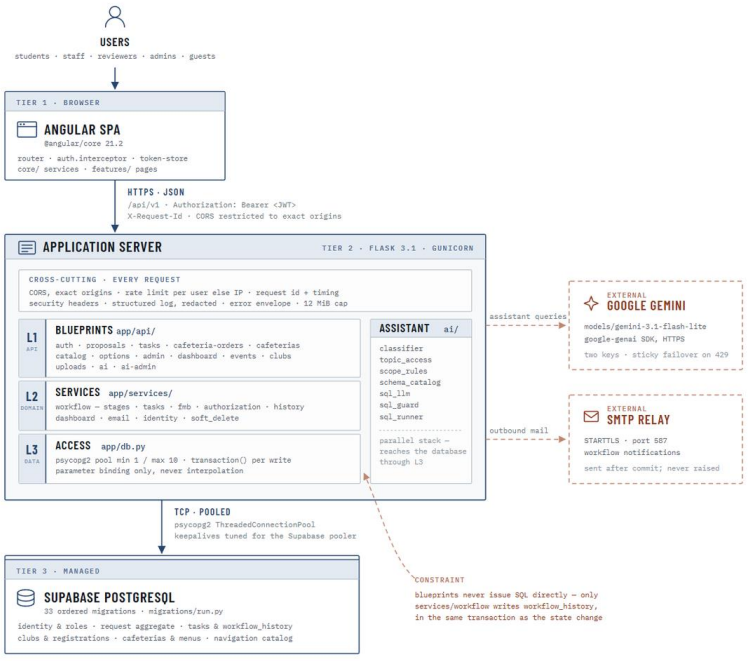


Figure 29: System architecture and deployment topology

Figure 29 presents the deployment and layering of the system. The browser tier holds the Angular application together with the routing, the authentication interceptor and the token store that manage the session on the client side. Communication with the server is exclusively over HTTPS using JSON, addressed to a single versioned API path, and every request carries the caller's access token in an Authorization header together with a correlation identifier used for log tracing. Cross-origin access is restricted to an explicit list of permitted origins rather than left open.

The application server is divided into three internal layers with a deliberate restriction on what each may do. The outermost layer holds the API blueprints, one per resource family, which parse and validate the request and format the response. Beneath it sits the domain layer, which contains the workflow state machine, the task and food-and-beverage services, the authorisation rules, the dashboard aggregations, the email templates and the soft-delete lifecycle. The innermost layer is the data access module, which owns the connection pool and the transaction helper, and which binds every value as a query parameter rather than

interpolating it into a statement. The constraint annotated on the diagram is that a blueprint never issues SQL directly; a route that needs to change a request must go through the workflow service, which is where the audit record is written alongside the change.

A set of concerns is applied uniformly to every inbound request rather than being repeated per route. These include the cross-origin check, per-user rate limiting that falls back to the caller's address when no user is identified, the security headers, a structured log entry carrying the request identifier and timing, the single error envelope that returns a user-safe message while the internal detail is logged rather than disclosed, and a cap on request body size.

The assistant is shown as a parallel stack because it is not a route family in the ordinary sense. It holds its own classifier, topic access rules, scope rules, schema catalogue, generation step, guard and runner, and it reaches the database through the same data access layer as everything else, which is what prevents it from becoming a second and less careful path to the same records. The Gemini model and the SMTP relay are drawn with dashed borders to mark them as outside the system boundary; the model is reached with two credentials configured for automatic failover, and mail is dispatched only after a database transaction has committed.

The database is a managed PostgreSQL instance reached over a pooled connection. Its schema is applied by an ordered set of migration files rather than by manual changes, which means the database can be rebuilt from an empty state into a known configuration. This is the arrangement described and justified in Section 2.4.6 of this report, shown here in the detail that the design of the application depends on.

4.2.3 Security and Identity Design

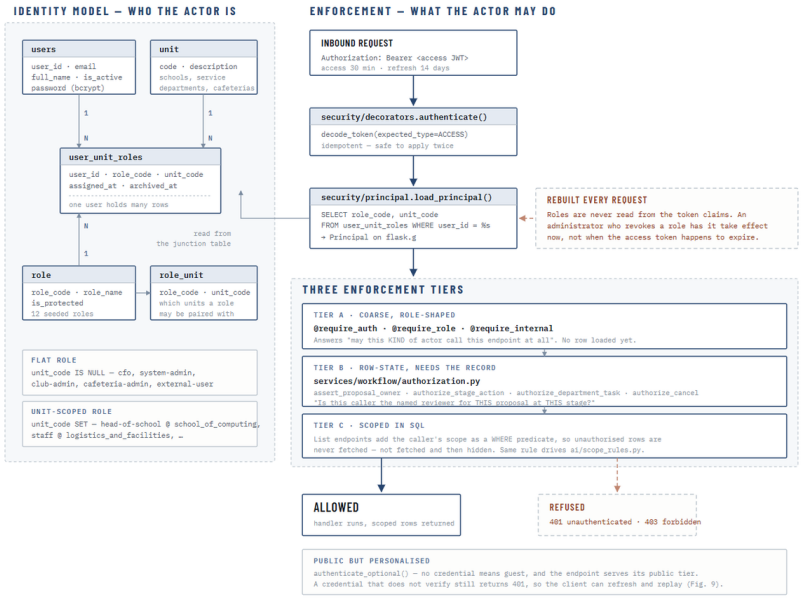


Figure 30: Identity model and the three authorisation tiers

Figure 30 presents the identity model on the left and the enforcement path on the right. The identity model is the design decision that most distinguishes this system from the systems reviewed in Chapter 2. A role is not held by a user globally; it is held by a user within an organisational unit, recorded as a row in a junction table that pairs a user, a role and a unit. The same person may therefore be a member of staff in one department and hold no standing at all in another, and a Head of Department carries authority only over the unit named on their own assignment. A small number of roles are deliberately unit-independent, such as the CFO, the System Admin, the Club Admin, the Cafeteria Admin and the external guest account, and these are stored with no unit attached. A supporting table records which roles are legal to pair with which units, so an administrator cannot create a nonsensical assignment such as a Head of School over a cafeteria.

The enforcement path shows what happens to every request that reaches the server. The access token is decoded and its type verified, so that a refresh token cannot be presented in place of an access token. The caller's roles are then loaded from the database rather than read from the claims inside the token. This is a deliberate cost: reading the roles on every request is

more work than trusting the token, but it means that when an administrator revokes a role the revocation takes effect immediately rather than when the user's current token happens to expire.

Authorisation is then applied in three tiers, each answering a different question. The first tier is a coarse, role-shaped decorator on the route itself, which answers whether this kind of actor may call this endpoint at all; at this point no record has been loaded. The second tier is applied after the record is loaded and answers a question that only the record can settle, such as whether this caller is the named reviewer for this particular request at its current stage, or whether this caller owns the proposal they are trying to cancel. The third tier applies to list endpoints, where the caller's scope is added to the query as a filtering condition so that records outside their scope are never retrieved at all, rather than being retrieved and then hidden. The same scope rules are the ones applied to the assistant, which is why the assistant cannot be used to reach records the interface would refuse to show.

Two supporting behaviours complete the picture. Endpoints that serve both public and signed-in visitors use an optional authentication path, where the absence of a credential is treated as a guest and the public tier of the content is served, while a credential that is present but does not verify still returns an authentication failure so that the client can refresh its session and retry. Passwords are stored only as bcrypt hashes, and password reset is performed through a hashed, expiring token rather than by transmitting a password.

4.2.4 Use Case Design

The capabilities of the system are documented across three use case diagrams rather than one. A single diagram covering twelve roles and the full functional surface of the platform would be dense to the point of being unreadable, and the three natural groupings in the system are sufficiently distinct to be modelled separately: the capabilities available for access, discovery and community participation; the capabilities that make up the event request, approval and fulfilment cycle; and the capabilities reserved for administration, configuration and oversight. Each diagram is presented with the system boundary drawn explicitly, so that what is inside the delivered platform and what is outside it remains unambiguous.

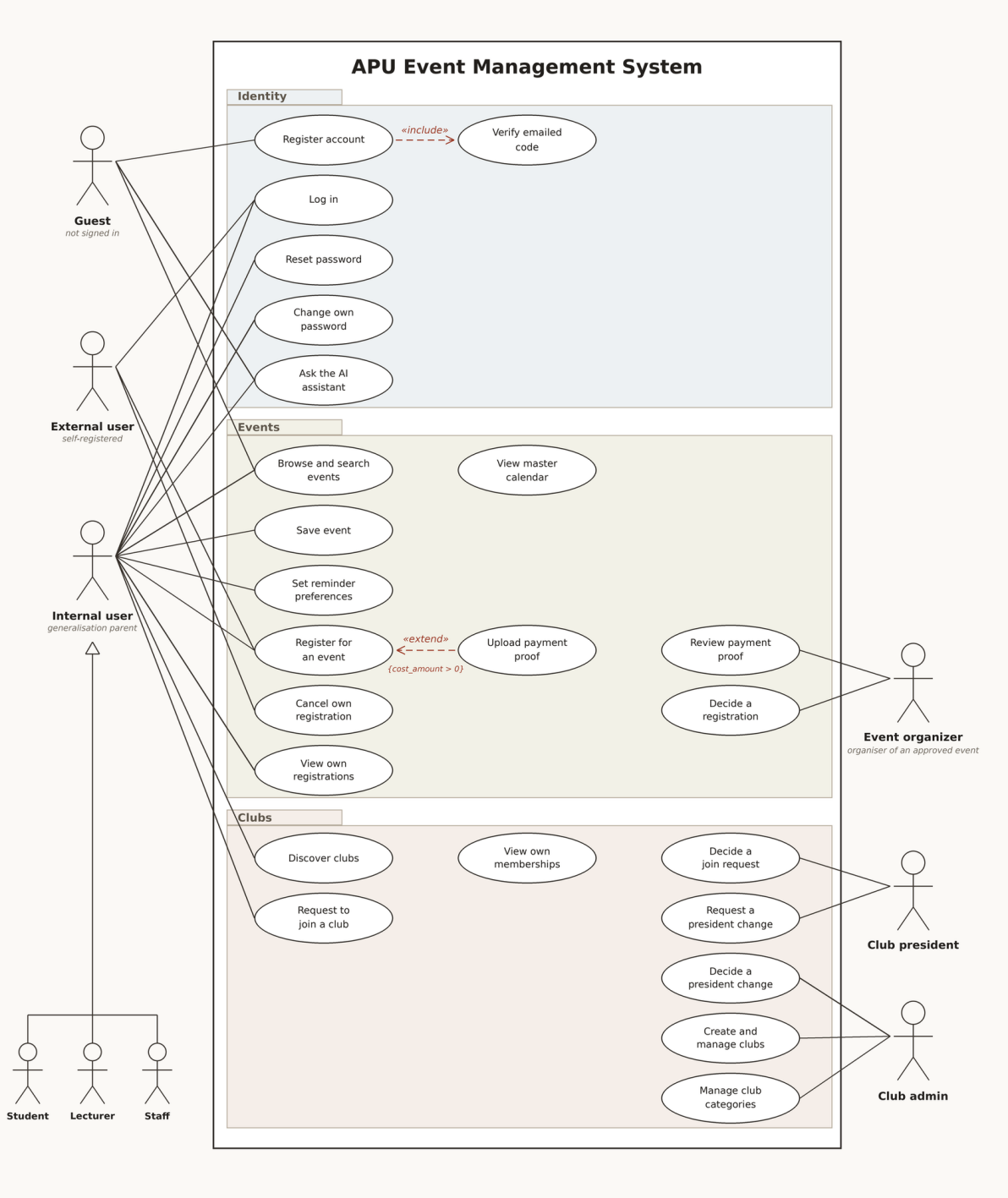


Figure 31: Use case diagram — access, events and community

Figure 31 models the capabilities available to visitors, self-registered external users, and the internal population of students, lecturers and staff. The three internal roles are drawn as specialisations of a generalisation parent because they share an identical capability set at this level; the differences between them appear later, in the request cycle and in the pages the navigation grants make available. A guest who has not signed in can browse and search published events, view the master calendar, register an account and ask the assistant, which

reflects the deliberate decision recorded in Chapter 1 to give the platform a genuine public face rather than confining it entirely behind the login.

Account creation is modelled with an included verification step, because the design does not create a user record at the point the registration form is submitted. A verification code is issued and only a successfully verified attempt becomes an account, which prevents abandoned or automated registrations from accumulating as real users. Registering for an event carries an extension rather than an inclusion, because uploading payment proof applies only when the event actually carries a cost; the conditional nature of that step is what the extension notation is intended to express.

The community half of the diagram introduces two actors that are defined by circumstance rather than by an assigned role. An event organiser is the person whose approved event has been published, and it is that circumstance that gives them the ability to decide a registration and review payment proof. A club president is similarly the current president of a particular club, and it is that position that lets them decide join requests and request a change of president. Distinguishing these from the twelve assigned roles matters, because they are authorised by a relationship to a record rather than by an entry in the role table, and that is exactly the distinction the second authorisation tier described in Section 4.2.3 exists to enforce.

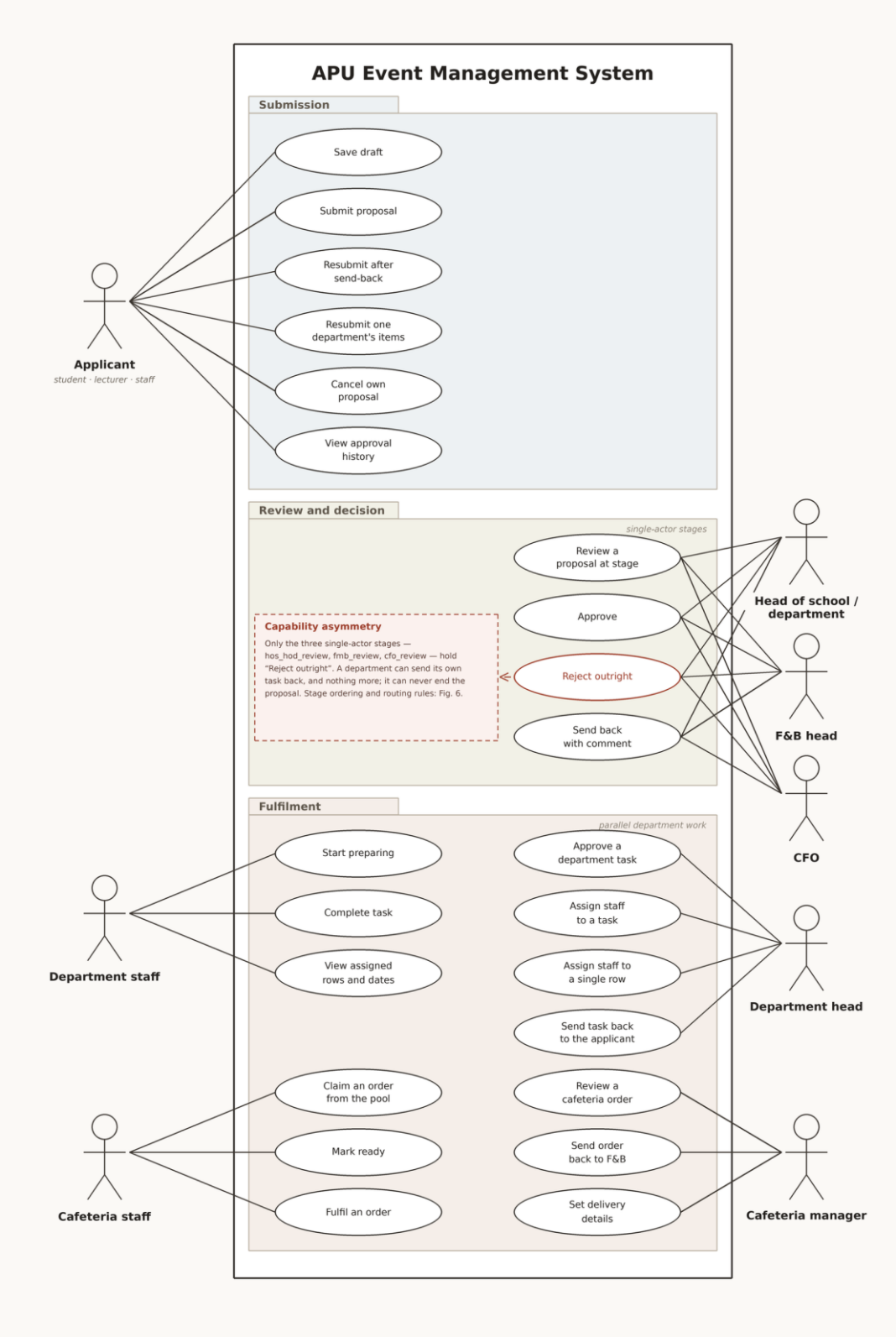


Figure 32: Use case diagram — request submission, review and fulfilment

Figure 32 models the core of the system: the creation, review and fulfilment of an event request. The applicant, who may be a student, a lecturer or a member of staff, can save a draft, submit it, cancel it, view its approval history, and resubmit it. Two distinct forms of

resubmission are modelled because two genuinely different situations exist. A request sent back by one of the reviewer stages returns to the applicant as a whole; a request sent back by a single department returns only that department's section, and the other departments retain the progress they have already made.

The review group carries the most important asymmetry in the design, and it is annotated on the diagram so that it cannot be missed. Only the three single-actor reviewer stages, held by the Head of School or Head of Department, the Food and Beverage head, and the CFO, are able to reject a request outright and end it. A service department reviewing its own task can approve it or send it back for correction, and nothing more. This is a deliberate choice: a department is being asked whether it can deliver its own portion of the work, not whether the event should take place, and giving a single service department the power to cancel an event on behalf of the whole institution would misrepresent where the decision authority actually sits.

The fulfilment group divides the work between the Head of Department, who approves the department's task and decides who will carry it out, and the department staff, who perform it. Assignment is modelled at two levels because the system supports both: a task may be assigned as a whole, or, for the five requirement types that are itemised row by row, individual rows of the task may be assigned to different staff members. The cafeteria pathway is modelled separately because its shape genuinely differs; the manager reviews and approves the order and it then enters a pool visible to that outlet's staff, from which a staff member claims it, prepares it, marks it ready and records its delivery.

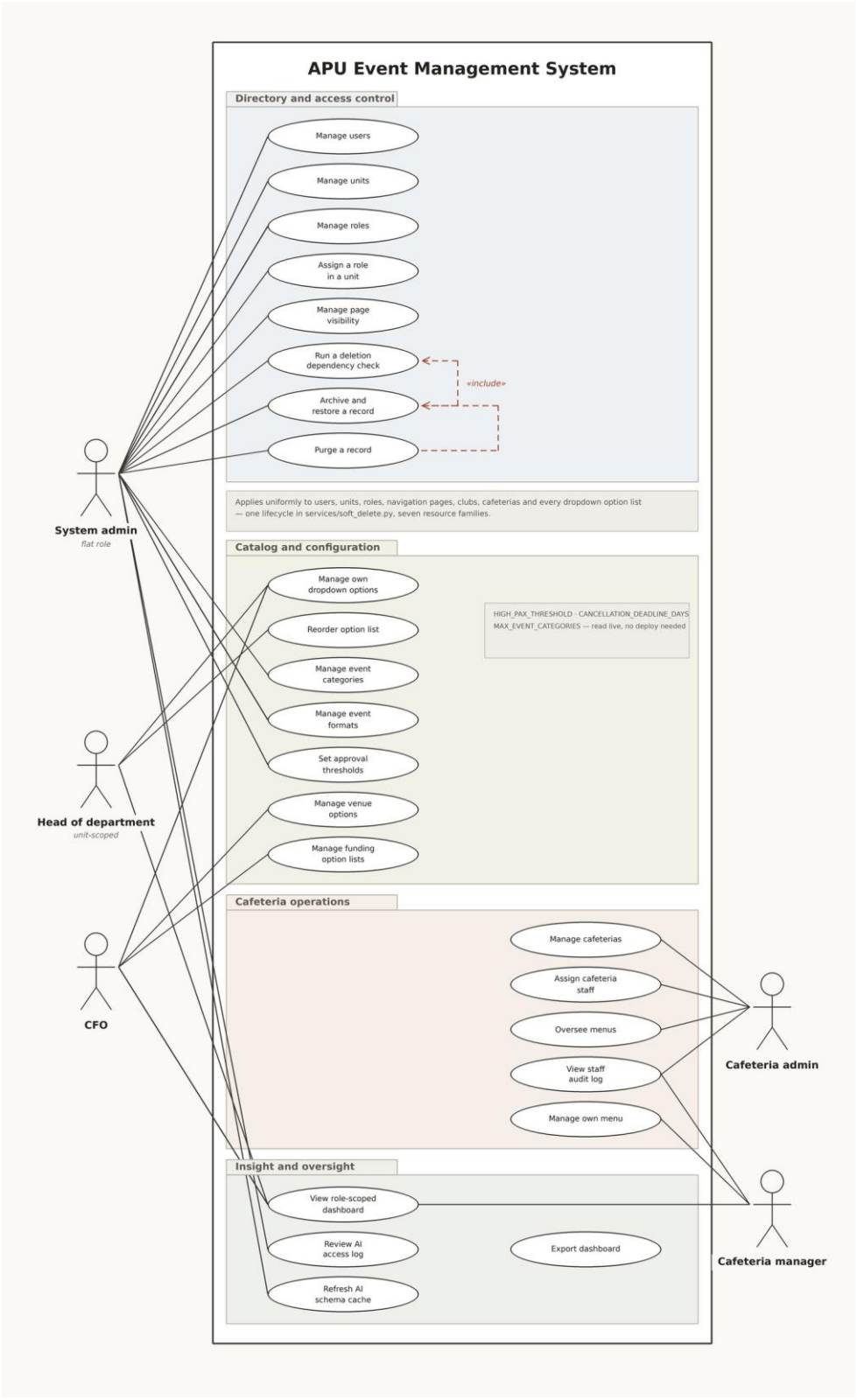


Figure 33: Use case diagram — administration, configuration and oversight

Figure 33 models administration, configuration and oversight. The System Admin holds the flat, unit-independent role that maintains the directory of users, organisational units and roles, assigns a role to a user within a unit, and controls which pages each role can reach.

The deletion pathway is modelled with an included dependency check because deletion in this system is not immediate: a record is first checked for dependants, then archived and recoverable, and only later purged. The same three-step lifecycle applies uniformly across users, units, roles, navigation pages, clubs, cafeterias and every dropdown option list, which is why it appears once in the model rather than being repeated for each resource.

The configuration group shows a deliberate distribution of authority. Each Head of Department maintains the option list belonging to their own department and no other, so the Logistics head curates the logistics catalogue while the A/V head curates sound and light. The CFO owns the venue catalogue and the funding option lists, which is consistent with the CFO's role as the financial reviewer. The System Admin owns the institution-wide settings: the event categories and formats, and the numeric thresholds that govern approval routing and escalation. These thresholds are read at the moment they are used rather than compiled into the application, so a change takes effect on the next request.

The oversight group covers the role-scoped dashboards, which are available to every role that has operational responsibility and which present only the measures relevant to that role, together with the export of any panel for reporting. The AI access log and the assistant's schema refresh are reserved to the System Admin, which reflects the position taken in Section 4.2.3 that the assistant is a privileged component and its use must be reviewable.

Use case specification tables for five representative use cases are presented below.

Field	Description
Primary Actor	Applicant (Student, Lecturer or Staff)
Description	The applicant completes the six-step event request form and submits it for review.
Preconditions	The applicant is signed in and holds a role within an organisational unit.
Postconditions	The request is stored with a status of submitted, a task is created for every selected requirement, and the first reviewer is notified by email.
Main Flow	The applicant opens the request form, completes the applicant, event, requirement, request detail and detailed information steps, reviews the summary, and submits. The system validates the form, writes the request, its requirement rows and its first history entry in a single transaction, determines the first reviewer stage, and sends notification after the transaction commits.
Alternative Flow	The applicant saves the form as a draft at any step and resumes later from the Drafts page.
Exception Flow	If validation fails, the affected fields are marked and no record reaches the database.

Table 5: Use Case Specification — Submit an Event Request

Field	Description
Primary Actor	Head of School or Head of Department, Food and Beverage Head, CFO
Description	The reviewer holding the request's current stage approves it, rejects it, or sends it back to the applicant with a comment.
Preconditions	The request is at a stage the reviewer holds, and the reviewer has not already decided it.
Postconditions	The request advances to the next stage, ends as rejected, or returns to the applicant for resubmission, and the decision is recorded in the workflow history.
Main Flow	The reviewer opens the request from the inbox, reads the summary and the requirement detail, enters a comment if one is required, and selects a decision. The system verifies that this reviewer is the authorised actor for this request at this stage, applies the status change and the history record in one transaction, and notifies the parties affected.
Alternative Flow	If the request is sent back, the stage it was sent back from is recorded so that resubmission resumes at that point rather than restarting the approval sequence.
Exception Flow	A rejection submitted without a reason is refused, and the reviewer is asked to supply one.

Table 6: Use Case Specification — Review a Request at a Stage

Field	Description
Primary Actor	Head of Department
Description	The department head approves the task routed to their department and assigns staff to carry it out.
Preconditions	The request has passed the reviewer stages and a task exists for this department.
Postconditions	The task moves to preparing, and the assigned staff members see it in their own task list.
Main Flow	The head opens the incoming request, reviews the requirement rows, approves the task, and selects one or more staff members. For requirement types that are itemised, the head may assign individual rows to different staff members.
Alternative Flow	The head sends the task back to the applicant with a comment; the tasks belonging to other departments continue unaffected.
Exception Flow	A requirement type that permits only one assignee refuses a second assignment.

Table 7: Use Case Specification — Assign Staff to a Department Task

Field	Description
Primary Actor	Internal user or External user
Description	A user registers to attend an event that has been approved and published.
Preconditions	The event is published and has capacity remaining.

Postconditions	A registration record exists with a status determined by the event's approval setting and its cost.
Main Flow	The user opens the event, submits a registration with a reason for attending where required, and the system records the registration. Where the event registers attendees automatically the place is confirmed immediately; where the organiser reviews registrations the record waits for a decision.
Alternative Flow	Where the event carries a cost, the user uploads payment proof and the organiser approves or rejects it before the place is confirmed.
Exception Flow	A registration submitted for an event at capacity is refused.

Table 8: Use Case Specification — Register to Attend a Published Event

Field	Description
Primary Actor	Any signed-in user, and guests for public topics
Description	The user asks a question in ordinary language and receives an answer drawn either from the curated knowledge base or from the user's own records.
Preconditions	The assistant is available and the user's session is valid.
Postconditions	An answer is returned and the interaction is recorded in the access log.
Main Flow	The question is classified into a topic, the topic is checked against the caller's permitted topics, the caller's row-level scope is derived, only the tables that scope permits are described to the model, a query is generated and validated against the guard, and the validated query alone is executed.
Alternative Flow	A repairable fault in the generated query, such as an unknown column or a missing filter, is returned to the model with the exact reason and a further attempt is made, up to a fixed limit.
Exception Flow	If the guard reports an authorisation violation the request is refused outright, the denial is logged, and no retry is attempted.

Table 9: Use Case Specification — Ask the AI Assistant

4.2.5 Workflow Design (Activity Diagrams)

The behaviour of the system is documented across three activity diagrams, each owning one lifecycle. The first covers the request itself from creation to a terminal state, the second covers what happens inside a department once a request reaches it, and the third covers the attendance registration that follows publication. Splitting them in this way keeps each diagram legible and avoids restating the same routing rules in three places.

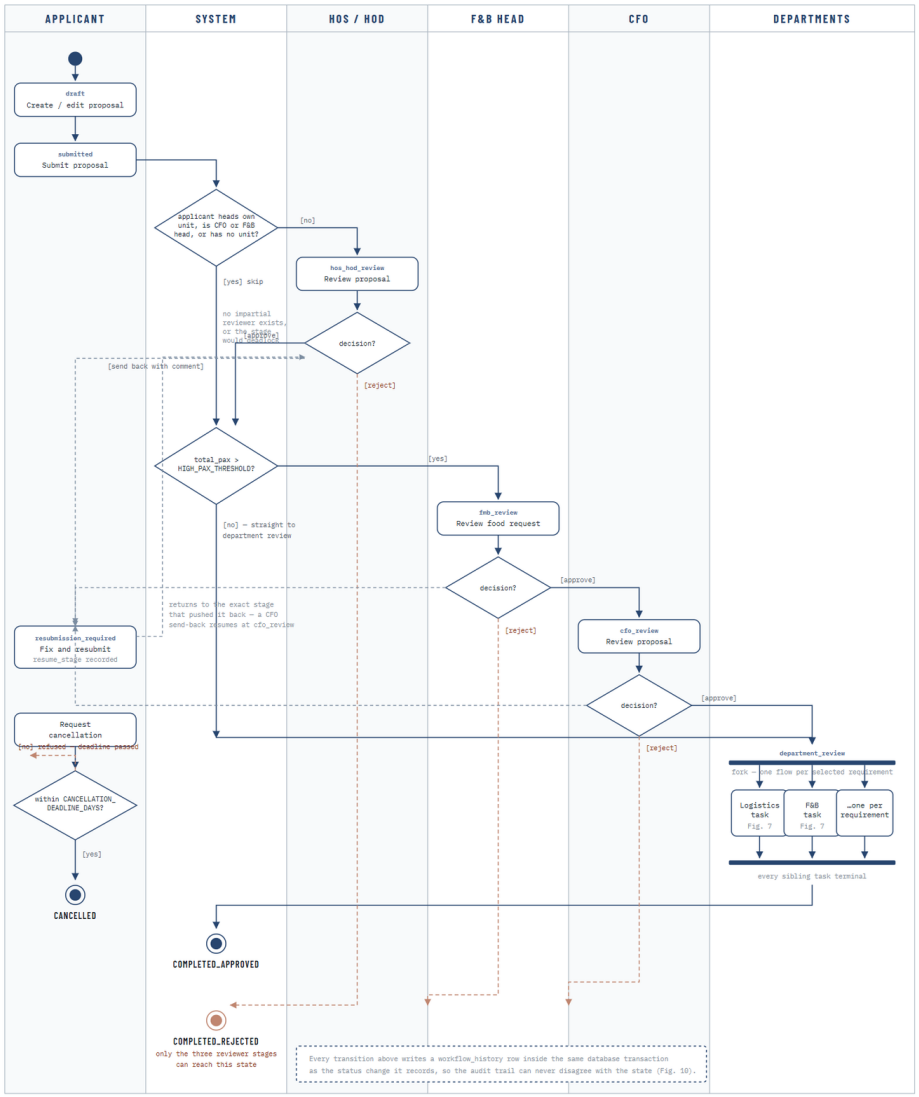


Figure 34: Activity diagram — event request lifecycle

Figure 34 models the full lifecycle of an event request across six swimlanes. A request begins as a draft owned by the applicant and enters the workflow on submission. The first decision the system makes is whether a Head of School or Head of Department review is meaningful at all: where the applicant is themselves the head of the unit the request would be routed to, or is the CFO or the Food and Beverage head, or holds no unit at all, no impartial reviewer exists and the stage is skipped rather than allowed to deadlock. This is a small piece of design that prevents a common institutional failure, in which a request waits indefinitely for an approval that can only be given by the person who raised it.

The request is then evaluated against the configured attendance threshold. Requests at or below the threshold pass directly to departmental review; requests above it are routed

through the Food and Beverage head and then the CFO, which is where the financial oversight described in Chapter 1 is applied. The threshold itself is a configuration value rather than a constant, so the institution can raise or lower the point at which financial review becomes mandatory without a change to the system.

Three reviewer stages can each approve, reject or send back. A rejection is terminal and ends the request. A send-back returns the request to the applicant with a comment and records the stage it was sent back from, so that when the applicant corrects and resubmits, the request resumes at that stage rather than beginning the approval sequence again. This is the structured revision loop identified as a gap in Chapter 2, and the recorded resume point is what makes it work without discarding approvals already given.

Once the reviewer stages are satisfied, the request forks into one flow per selected requirement, and each department works its own task in parallel and independently. The join condition is that when every sibling task reaches a terminal state the parent request completes automatically; no further human action is required to close it. A separate path allows the applicant to request cancellation, which is permitted only while the event remains far enough away to satisfy the configured cancellation deadline. The annotation at the foot of the diagram records the invariant that every transition shown writes its history row inside the same transaction as the status change it represents.

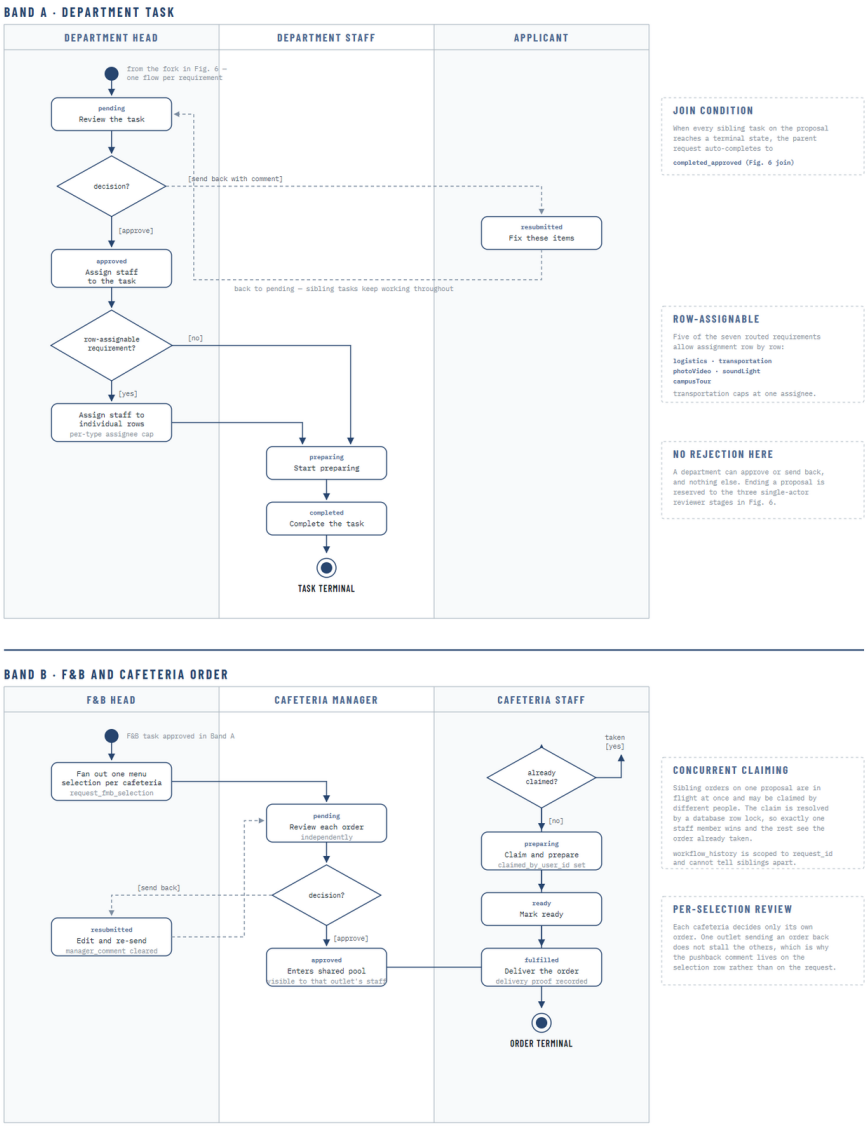


Figure 35: Activity diagram — department task and cafeteria order fulfilment

Figure 35 models what happens inside a department once a task arrives, and it is presented in two bands because the ordinary department pathway and the cafeteria pathway genuinely differ. In the upper band, a task arrives pending, the Head of Department reviews it and either approves it or sends it back to the applicant with a comment. A department cannot reject a request outright, and the annotation states this explicitly; that capability belongs only to the three reviewer stages in the preceding diagram.

Where the requirement type is itemised, the head may assign staff to individual rows rather than to the task as a whole. Five of the seven routed requirement types permit this, and transportation is capped at a single assignee because a single vehicle movement cannot sensibly be shared. The task then moves to preparing while the assigned staff carry it out, and to

completed when the work is done. Sibling tasks belonging to other departments continue throughout, including while this task is sitting with the applicant for correction.

In the lower band, a food and beverage requirement fans out into one order per cafeteria named in the request, and each cafeteria decides only its own order. A manager who sends an order back does not stall the other outlets, which is why the pushback comment is stored against the individual selection rather than against the request. An approved order enters a pool visible to that outlet's staff, and the claim is resolved by a database row lock so that when several staff open the same order at once exactly one succeeds and the others are told the order has already been taken. The order then moves through ready to fulfilled, with delivery evidence recorded against it.

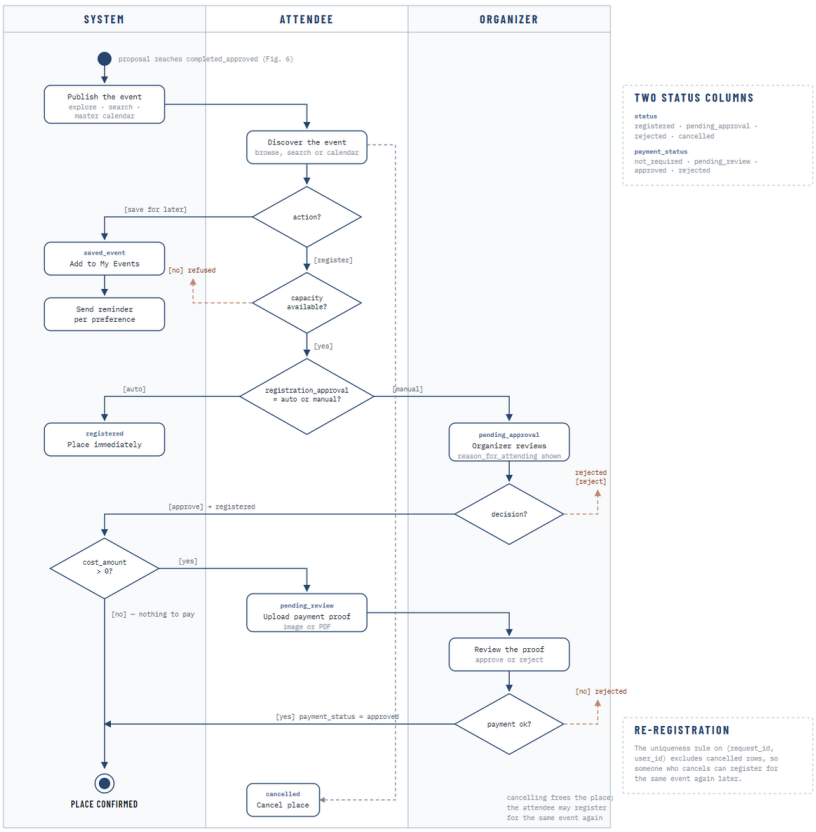


Figure 36: Activity diagram — event publication and attendance registration

Figure 36 models what happens after a request completes and its event is published. A visitor may discover the event by browsing, searching or through the calendar, and may either save it for later or register to attend. Saving an event places it in the user's own list and enrols them in the reminder schedule, subject to the preferences they control themselves.

Registration is governed by two independent status columns, and the diagram shows why both are needed. The first records whether the person holds a place, and it depends on whether the event registers attendees automatically or requires the organiser to decide each request. The second records the state of payment, and it applies only where the event carries a cost, in which case the attendee uploads proof and the organiser approves or rejects it. A place is confirmed only when both conditions are satisfied. Keeping the two columns separate avoids conflating an organiser's decision about who may attend with a finance decision about whether payment has been received.

A registration that is cancelled frees the place, and the uniqueness rule that prevents duplicate registrations deliberately excludes cancelled rows, so a person who cancels is able to register for the same event again later. This is a small behaviour, but it is the kind of detail that determines whether a system feels usable in practice.

4.2.6 Interaction Design (Sequence Diagrams)

The three sequence diagrams that follow document the ordering of messages between components for the interactions where ordering is itself part of the design. They are selected rather than exhaustive: authentication, because the session behaviour it defines applies to every other interaction in the system; the write path, because the ordering of the transaction and the notification is a deliberate design decision; and the assistant, because the order in which authorisation, generation and validation occur is what makes the feature safe.

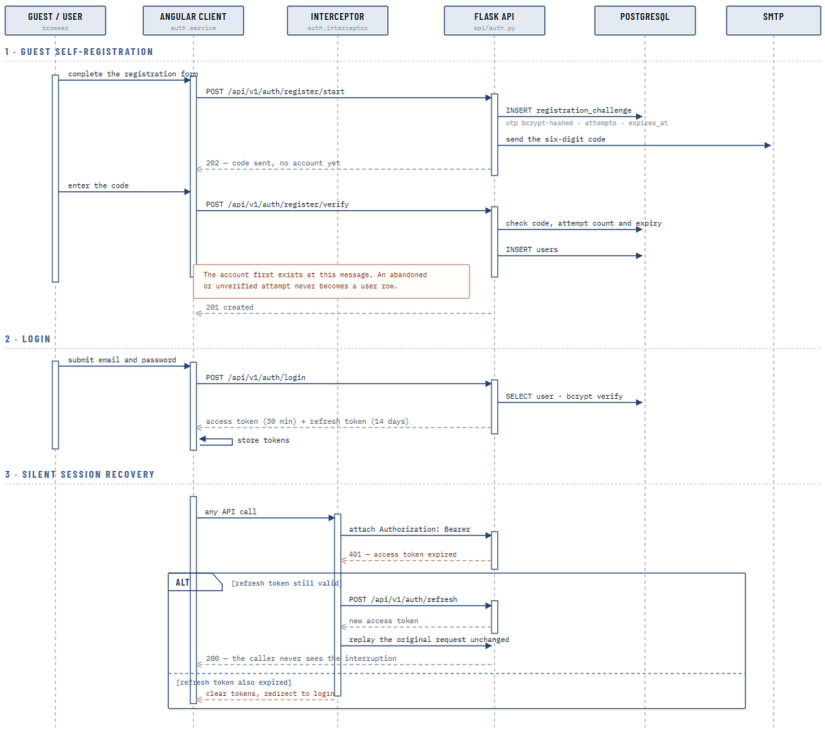


Figure 37: Sequence diagram — registration, login and silent session recovery

Figure 37 documents three related interactions. In guest self-registration, the account does not exist at the moment the form is submitted. A verification challenge is stored with the code hashed, an attempt counter and an expiry, and the code is emailed. Only when the code is verified is a user record created. The annotation marks the exact message at which the account first exists, and the consequence is that an abandoned or automated registration attempt never becomes a real user.

In login, the submitted credentials are verified against the stored bcrypt hash and the caller receives two tokens: a short-lived access token and a longer-lived refresh token. The separation exists so that the credential presented on every request has a small exposure window, while the user is not forced to sign in again every half hour.

The third interaction is silent session recovery, and it is the reason the separation is worth the additional complexity. When an access token expires, the interceptor on the client receives the authentication failure, exchanges the refresh token for a new access token and replays the original request unchanged, so the user never sees the interruption. Only when the refresh token has itself expired are the stored tokens cleared and the user returned to the login

page. This behaviour is important in a workflow system, where a reviewer may leave a long request open while reading it and would otherwise lose their work to a session timeout.

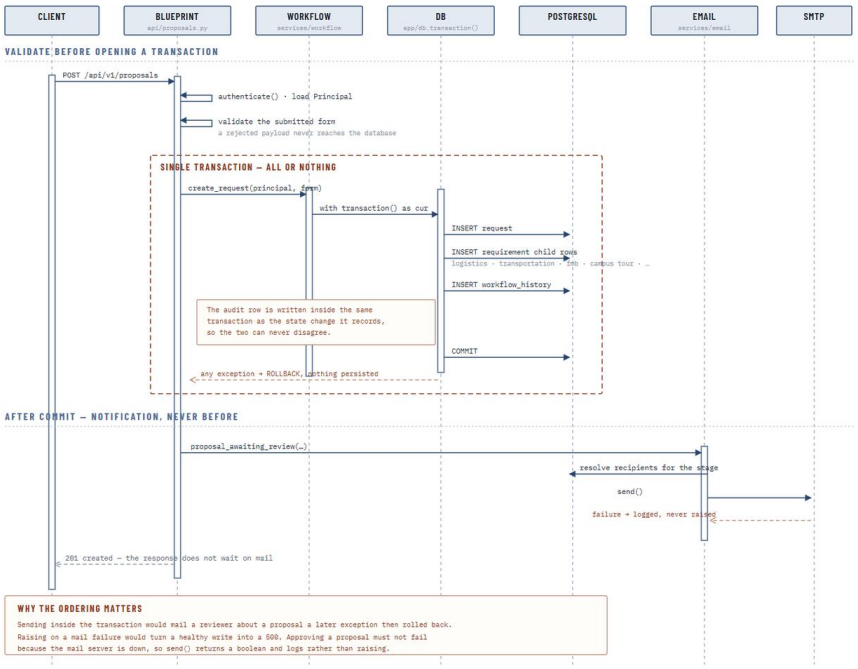


Figure 38: Sequence diagram — transactional write path and post-commit notification

Figure 38 documents the write path taken by every state-changing operation in the system, using the creation of a request as the example. The ordering shown is the design, not an implementation accident, and it has three distinct phases.

Validation happens before a transaction is opened. The caller is authenticated and their principal loaded, and the submitted form is validated; a payload that fails validation never reaches the database at all. Opening a transaction and then discovering the input was invalid would hold a connection and produce a rollback for a fault that could have been detected earlier.

The write itself takes place inside a single transaction covering the request row, the requirement child rows and the history entry. The annotation records the reason: because the audit row is written in the same transaction as the state change it describes, the two cannot disagree. Any exception at any point rolls the whole transaction back and nothing is persisted.

Notification is dispatched only after the transaction has committed, and the response to the client does not wait for the mail to be sent. The lower annotation gives the two reasons. Sending inside the transaction would email a reviewer about a request that a later exception then rolled back, so a reviewer could be notified about work that does not exist. Raising an exception on a mail failure would turn a healthy write into a server error, so an approval would appear to fail because the mail server was unavailable. The send therefore returns a result that is logged rather than raised.

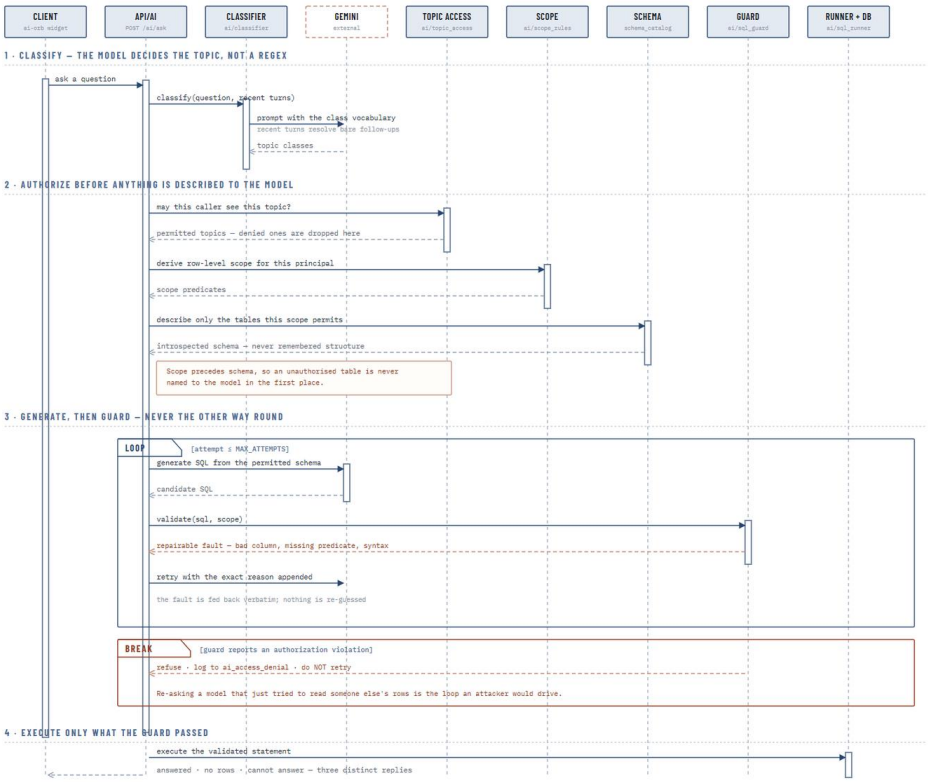


Figure 39: Sequence diagram — AI assistant question-answering pipeline

Figure 39 documents the assistant's question-answering pipeline, and the order of its four phases is the whole of its security design. In the first phase the question, together with the recent conversation, is classified into a topic by the model itself rather than by pattern matching, which is what allows a bare follow-up question to be resolved against what was asked before it.

The second phase performs authorisation before anything is described to the model. The classified topic is checked against the topics this caller is permitted to ask about, and topics they may not reach are dropped. The caller's row-level scope is derived, and only the tables

that scope permits are then described to the model. The annotation states the consequence plainly: because scope precedes schema, a table the caller may not read is never named to the model in the first place, and the model cannot generate a query against a structure it was never shown.

The third phase generates a query and then validates it, never the reverse. A repairable fault, such as an unknown column, a missing filter or a syntax error, is returned to the model with the exact reason appended so that the next attempt corrects a stated problem rather than guessing again, and this loop is bounded by a fixed attempt limit. An authorisation violation is treated entirely differently: it breaks out of the loop, is refused, and is written to the denial log without a retry. The annotation gives the reason, which is that re-asking a model that has just attempted to read another user's records is precisely the loop an attacker would want to drive.

The fourth phase executes only the statement the guard has already passed, and returns one of three distinct replies: an answer, an explicit statement that no rows matched, or an acknowledgement that the question cannot be answered. Distinguishing an empty result from an inability to answer matters, because presenting the two identically would allow a user to mistake a system limitation for a factual absence of records.

4.3 Database Design

The database is the authoritative record of everything the system knows, and its design is therefore inseparable from the design of the workflow described in the previous section. The deployed schema contains sixty-eight tables joined by one hundred and eleven foreign key relationships, applied through an ordered sequence of migration files rather than by manual alteration, and hosted as a managed PostgreSQL instance. This section presents the entity relationship diagram of the schema as deployed, followed by a data dictionary describing the purpose of every table.

4.3.1 Entity Relationship Diagram

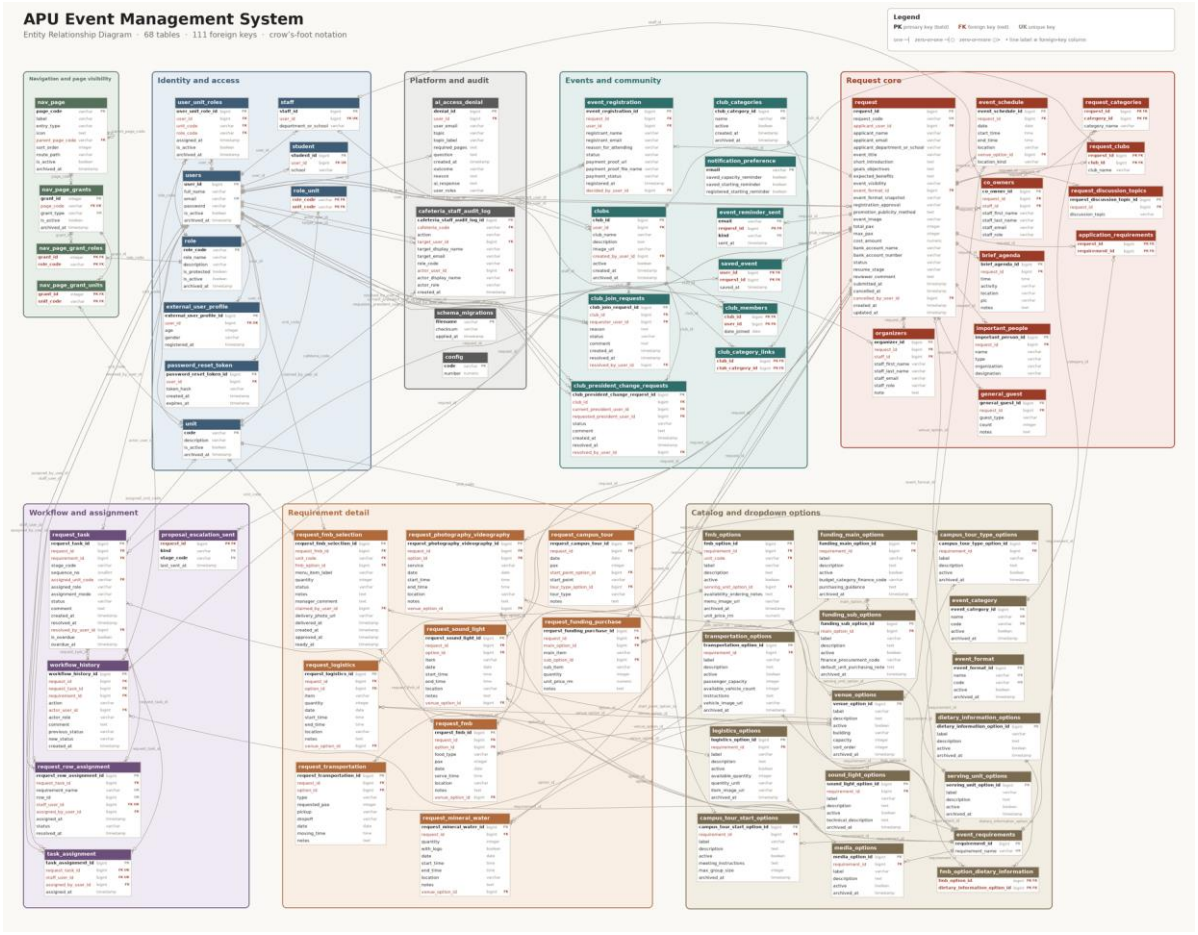


Figure 40: Entity relationship diagram of the deployed database

Figure 40 presents the entity relationship diagram of the deployed database. It is generated directly from an introspection of the live schema rather than drawn by hand, which means it cannot disagree with the database that is actually running; a table added by a later migration appears in the diagram automatically rather than being forgotten. The tables are arranged into eight clusters that correspond to the functional areas of the system, and the same eight clusters organise the data dictionary that follows. Because the schema is large, the figure is presented as an overview of structure rather than as a readable listing of columns; it is embedded at full resolution and can be magnified in the electronic copy of this report, and the detail of every table is set out in the data dictionary in Section 4.3.2.

Three characteristics of the design are visible in the diagram and are worth stating explicitly. The first is that the request is a genuine aggregate rather than a single wide table. The core request row holds the information common to every event, while each requirement type the applicant selects has its own detail table with the columns that requirement actually needs, and the participant, schedule and agenda information is held in child tables that permit

any number of rows. This avoids the alternative of a single table carrying dozens of mostly empty columns, and it means adding a new service requirement later is a matter of adding a table rather than altering an existing one.

The second is that the workflow and its evidence are modelled separately from the request. The task table records what each department was asked to do and where it has reached, the assignment tables record who is carrying it out, and the history table records every transition that has ever occurred. Because history is a table in its own right rather than a set of columns on the request, the full sequence of decisions survives even after the request reaches a terminal state.

The third is that the catalogue cluster is not incidental. Sixteen tables exist purely to hold the option lists offered on the request form, and every requirement detail row references the catalogue entry it was chosen from rather than storing free text. This is what makes the analytics in Chapter 4 possible at all: a report on the most requested logistics items is only meaningful if two people asking for the same item recorded it the same way.

Soft deletion is applied consistently across the schema. Records that participate in the deletion lifecycle carry an archive timestamp, so a deleted record is withdrawn from view but remains recoverable for a configured period before it is purged. This is a deliberate trade-off in favour of recoverability, on the reasoning that an administrator removing a unit or a user in a live institutional system is more likely to want that action reversible than immediate.

4.3.2 Data Dictionary

The data dictionary below describes every table in the deployed schema, organised into the same eight clusters used in **Figure 40**. For each table the dictionary gives its purpose within the system and the columns that carry its identity and its principal relationships. Columns present purely for record keeping, such as creation timestamps and archive markers, are omitted from the key column list where they do not affect the meaning of the table.

The tables of the identity and access cluster are described below.

Table	Purpose	Key columns
users	Holds one row per person known to the system, internal or external, with the bcrypt password hash and an active flag.	user_id, email, full_name, password, is_active
staff	Extends a user record with the staff-specific detail of the department or school they belong to.	staff_id, user_id, department_or_school

student	Extends a user record with the student-specific detail of the school they are enrolled under.	student_id, user_id, school
external_user_profile	Extends a self-registered guest account with the demographic detail collected at registration.	external_user_profile_id, user_id, age, gender, registered_at
unit	The organisational units of the university: schools, service departments and cafeterias, each identified by a code.	code, description, is_active
role	The twelve role definitions, with a protected flag marking those the system depends on and will not allow to be deleted.	role_code, role_name, description, is_protected
role_unit	Records which roles are legally pairable with which units, preventing nonsensical role assignments.	role_code, unit_code
user_unit_roles	The junction that gives the system its role-per-unit identity model: one row per role a user holds within a unit.	user_unit_role_id, user_id, unit_code, role_code, is_active
password_reset_token	Stores hashed, expiring tokens issued for password reset so that no password is ever transmitted.	password_reset_token_id, user_id, token_hash, expires_at

Table 10: Data Dictionary — Identity and Access

The tables of the navigation and page visibility cluster are described below.

Table	Purpose	Key columns
nav_page	The catalogue of navigable pages and folders, with the parent, ordering and route from which the sidebar is built.	page_code, parent_page_code, label, entry_type, icon
nav_page_grants	One grant attached to a page, describing a condition under which that page becomes visible.	grant_id, page_code, grant_type, is_active
nav_page_grant_roles	The roles a grant applies to.	grant_id, role_code
nav_page_grant_units	The units a grant applies to, allowing a grant to be scoped to a role within a particular unit.	grant_id, unit_code

Table 11: Data Dictionary — Navigation and Page Visibility

The tables of the request core cluster are described below.

Table	Purpose	Key columns
request	The core event request: applicant details, event information, visibility, capacity, cost, current status and the stage a send-back should resume at.	request_id, request_code, applicant_user_id, event_format_id, cancelled_by_user_id, applicant_name, applicant_email, applicant_department_or_school
request_categories	The event categories selected for a request, stored with a snapshot of the category name.	request_id, category_id, category_name

application_requirements	Which service requirements the applicant selected, and therefore which departments the request will be routed to.	request_id, requirement_id
request_clubs	The clubs associated with a request, where the event is organised by or on behalf of a club.	request_id, club_id, club_name
co_owners	Staff recorded as co-owners of a request alongside the applicant.	co_owner_id, request_id, staff_id, staff_first_name, staff_last_name, staff_email
organizers	Staff recorded as organisers responsible for running the event.	organizer_id, request_id, staff_id, staff_first_name, staff_last_name, staff_email
important_people	Guests of honour, invited dignitaries and other named attendees of significance.	important_person_id, request_id, name, type, organization
general_guest	Counts of unnamed guest categories expected to attend.	general_guest_id, request_id, guest_type, count, notes
event_schedule	The dated sessions that make up the event, each with a time window and either a catalogue venue or a free location.	event_schedule_id, request_id, venue_option_id, date, start_time, end_time
brief_agenda	The running order within the event, with time, activity, location and person in charge.	brief_agenda_id, request_id, time, activity, location
request_discussion_topics	Topics the applicant wishes to raise with reviewers, attached to the request.	request_discussion_topic_id, request_id, discussion_topic

Table 12: Data Dictionary — Request Core

The tables of the requirement detail cluster are described below.

Table	Purpose	Key columns
request_logistics	Logistics items requested, each referencing a catalogue option with quantity, timing and location.	request_logistics_id, request_id, option_id, venue_option_id, item, quantity, date
request_transportation	Transport movements requested, with vehicle type, passenger count, pickup, dropoff and time.	request_transportation_id, request_id, option_id, type, requested_pax, pickup
request_photography_videography	Photography and videography services requested, with the service type, timing and location.	request_photography_videography_id, request_id, option_id, venue_option_id, service, date, start_time
request_sound_light	Sound and lighting equipment requested, with the item, timing and location.	request_sound_light_id, request_id, option_id, venue_option_id, item, date, start_time
request_fmb	Food and beverage requirements requested, with the food type, headcount, serving time and location.	request_fmb_id, request_id, option_id, venue_option_id, food_type, pax, date
request_fmb_selection	One menu selection per cafeteria derived from a food and beverage requirement, carrying its own fulfilment status, claim, manager comment and delivery evidence.	request_fmb_selection_id, request_fmb_id, unit_code, fmb_option_id, claimed_by_user_id, menu_item_label, quantity, status

request_campus_tour	Campus tour requests, with the start point, tour type, date and group size.	request_campus_tour_id, request_id, start_point_option_id, tour_type_option_id, date, pax, start_point
request_mineral_water	Mineral water requests, with quantity, branding option, timing and location.	request_mineral_water_id, request_id, venue_option_id, quantity, with_logo, date
request_funding_purchase	Funding and purchase items requested, with the main and sub category, quantity and unit price.	request_funding_purchase_id, request_id, main_option_id, sub_option_id, main_item, sub_item, quantity

Table 13: Data Dictionary — Requirement Detail

The tables of the workflow and assignment cluster are described below.

Table	Purpose	Key columns
request_task	One task per department per requirement, holding the stage, the unit and role it is assigned to, its status, and its overdue marker.	request_task_id, request_id, requirement_id, assigned_unit_code, resolved_by_user_id, stage_code, sequence_no, assigned_role
request_row_assignment	Assignment of an individual requirement row to a staff member, for the requirement types that are itemised row by row.	request_row_assignment_id, request_task_id, requirement_name, row_id, staff_user_id, assigned_by_user_id, status
task_assignment	Assignment of a whole task to a staff member, recording who assigned it and when.	task_assignment_id, request_task_id, staff_user_id, assigned_by_user_id
workflow_history	The immutable audit trail: one row per transition, recording the action, the actor, the comment and the status before and after.	workflow_history_id, request_id, request_task_id, requirement_id, actor_user_id, action, actor_role, comment
proposal_escalation_sent	Records the last time each escalation kind was sent for a request at a given stage, so a chaser is not repeated within its cadence.	request_id, kind, stage_code, last_sent_at

Table 14: Data Dictionary — Workflow and Assignment

The tables of the events and community cluster are described below.

Table	Purpose	Key columns
event_registration	One registration per person per published event, with the attendance status, the reason for attending, and the separate payment status and proof.	event_registration_id, request_id, user_id, decided_by_user_id, registrant_name, registrant_email, reason_for_attending

saved_event	Events a user has saved for later, which also enrols them in the reminder schedule.	user_id, request_id, saved_at
notification_preference	Per-address opt-in settings controlling which reminder emails a person receives.	email, saved_capacity_reminder, saved_starting_reminder, registered_starting_reminder
event_reminder_sent	Ledger of reminders already sent, so a repeated or missed run never sends the same reminder twice.	email, request_id, kind, sent_at
clubs	The club directory, with the current president, description, image and active state.	club_id, user_id, created_by_user_id, club_name, description, image_url
club_categories	The categories clubs can be classified under.	club_category_id, name, active
club_category_links	The many-to-many link allowing a club to belong to more than one category.	club_id, club_category_id
club_members	Club membership, with the date the member joined.	club_id, user_id, date_joined
club_join_requests	Requests to join a club, with the applicant's reason and the president's decision.	club_join_request_id, club_id, requester_user_id, resolved_by_user_id, reason, status, comment
club_president_change_requests	Requests to transfer a club presidency, decided by the Club Admin.	club_president_change_request_id, club_id, current_president_user_id, requested_president_user_id, resolved_by_user_id, status, comment

Table 15: Data Dictionary — Events and Community

The tables of the catalogue and dropdown options cluster are described below.

Table	Purpose	Key columns
event_category	The catalogue of event categories offered on the request form.	event_category_id, name, code, active
event_format	The catalogue of event formats, such as on-campus or online.	event_format_id, name, code, active
event_requirements	The master list of service requirement types the system can route.	requirement_id, requirement_name
logistics_options	The logistics catalogue, with available quantity, unit and item image.	logistics_option_id, requirement_id, label, description, active
transportation_options	The transport catalogue, with passenger capacity, fleet count and usage instructions.	transportation_option_id, requirement_id, label, description, active
media_options	The photography and videography service catalogue.	media_option_id, requirement_id, label, description, active
sound_light_options	The sound and lighting catalogue, with technical description.	sound_light_option_id, requirement_id, label, description, active

dietary_information_options	The dietary information catalogue used to tag menu items.	dietary_information_option_id, label, description, active
serving_unit_options	The serving unit catalogue, such as per person or per tray.	serving_unit_option_id, label, description, active
fmb_options	The menu catalogue, owned per cafeteria unit, with price, serving unit, ordering notes and image.	fmb_option_id, requirement_id, unit_code, serving_unit_option_id, label, description, active
fmb_option_dietary_information	The many-to-many link tagging a menu item with dietary information.	fmb_option_id, dietary_information_option_id
campus_tour_start_options	The campus tour start point catalogue, with meeting instructions and maximum group size.	campus_tour_start_option_id, requirement_id, label, description, active
campus_tour_type_options	The campus tour type catalogue.	campus_tour_type_option_id, requirement_id, label, description, active
funding_main_options	The main funding category catalogue, with the finance code and purchasing guidance.	funding_main_option_id, requirement_id, label, description, active
funding_sub_options	The funding sub-item catalogue, nested under a main category.	funding_sub_option_id, main_option_id, label, description, active
venue_options	The venue catalogue owned by the CFO, with building, capacity and display order.	venue_option_id, label, description, active

Table 16: Data Dictionary — Catalogue and Dropdown Options

The tables of the platform and audit cluster are described below.

Table	Purpose	Key columns
config	Numeric configuration values, including the approval threshold, cancellation deadline and escalation windows, read at the point of use.	code, number
schema_migrations	The migration ledger, recording each applied file with its checksum so an edited migration is reported rather than silently skipped.	filename, checksum, applied_at
cafeteria_staff_audit_log	Audit trail of cafeteria staff assignment and removal, recording both the target and the actor.	cafeteria_staff_audit_log_id, cafeteria_code, target_user_id, actor_user_id, action, target_display_name, target_email
ai_access_denial	Record of every assistant request that was refused, with the topic, the question, the outcome and the reason.	denial_id, user_id, user_email, topic, topic_label

Table 17: Data Dictionary — Platform and Audit

4.4 Interface Design

The interface design translates the structural decisions of the preceding sections into something a user can operate. Its central problem is stated by the role model itself: twelve roles, held per organisational unit, cannot be served by twelve separate applications, and neither can they be served by one interface that shows everybody everything. The approach taken is a single application shell whose contents are assembled per user from the access grants the server issues, supported by a shared component library and a token-based visual system that keep the result coherent no matter which combination of pages a given user sees.

4.4.1 System Navigation Site Map

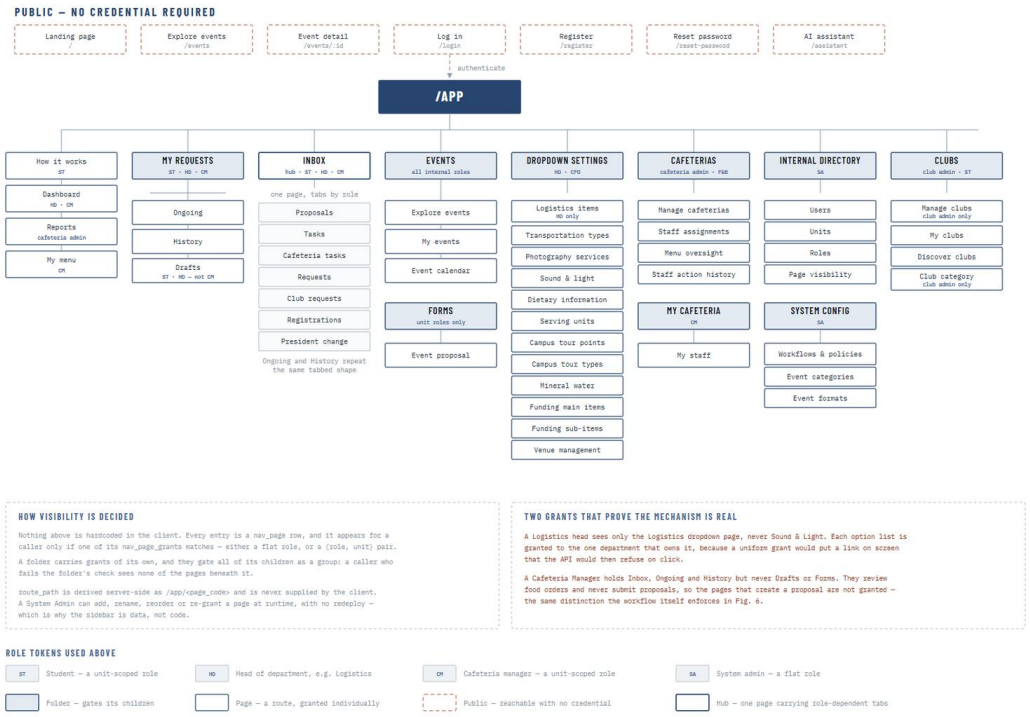


Figure 41: System navigation site map

Figure 41 presents the navigation structure of the system. The public tier, drawn with dashed borders, is reachable with no credential at all and comprises the landing page, event browsing and detail, login, registration, password reset and the assistant. Everything below the authenticated boundary is organised into eight top-level areas, each of which is either a single page or a folder that gates the pages beneath it.

The role tokens on each entry record which roles reach it, and they show the design working in practice. The Inbox is a single page carrying tabs that differ by role, so a Head of Department, a student and a cafeteria manager all open the same route and are presented with the queues that concern them, rather than each having a separate inbox page to maintain. The Ongoing and History areas repeat the same tabbed shape, which keeps the mental model consistent as a user moves between what is in progress and what is finished.

The two annotations at the foot of the figure record the mechanism and give evidence that it is genuine rather than cosmetic. Nothing in the structure is hard-coded in the client: each entry is a row in the page catalogue, and it appears for a caller only if one of its grants matches, either by role or by a role within a specific unit. A folder carries grants of its own that gate all of its children as a group. Two grants demonstrate that the mechanism is real: the Logistics head sees only the Logistics option list and not Sound and Light, because each catalogue is granted to the department that owns it, and a Cafeteria Manager holds Inbox, Ongoing and History but not Drafts or Forms, because they review food orders and never submit requests. That second distinction is the same one the workflow itself enforces, expressed here as navigation.

4.4.2 Screen Layout Anatomy

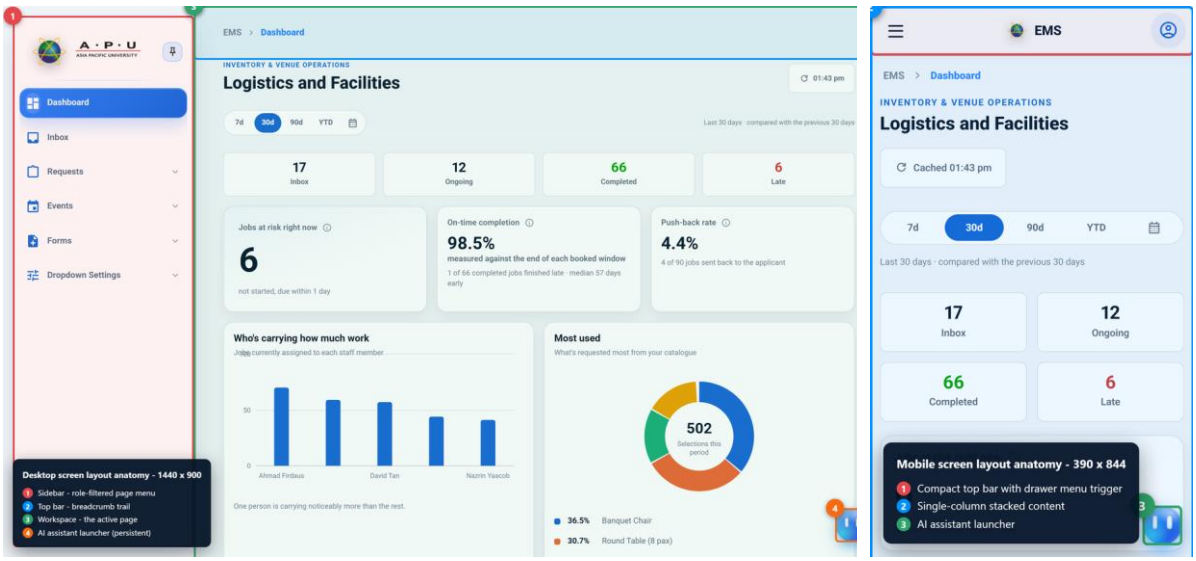


Figure 42: Screen layout anatomy on desktop and mobile

Figure 42 presents the anatomy of an internal screen at both viewports. On the desktop the layout is a persistent left sidebar carrying the role-filtered page menu, a top bar carrying the breadcrumb trail, a workspace holding the active page, and a persistent assistant launcher anchored to the lower right. The sidebar can be pinned or collapsed, which allows a user

working on a wide data table to reclaim the horizontal space without losing their place in the navigation.

On the mobile viewport the same four elements are present but rearranged. The sidebar becomes a drawer opened from a compact top bar, the workspace stacks into a single column, and the assistant launcher remains anchored in the same position. Keeping the launcher in a constant position across both viewports is deliberate: it is the one control that is available on every page in the system, and a control that moves depending on the device is a control users stop looking for.

The breadcrumb trail is more than decoration in a system with this many pages. Because the sidebar contents differ per role, a user cannot rely on a shared mental map of the application learned from a colleague; the breadcrumb gives every screen an unambiguous position within the structure regardless of which subset of that structure the user can see.

4.4.3 Role-Based Menu Design

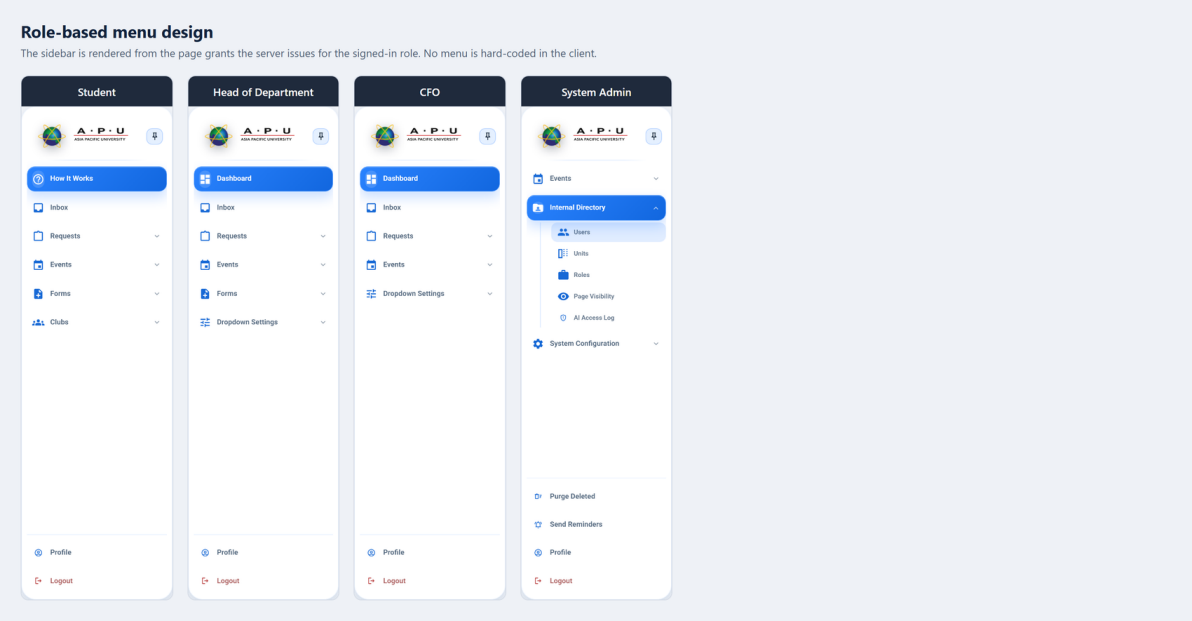


Figure 43: Role-based menu design across four roles

Figure 43 places four sidebars side by side, captured from a Student, a Head of Department, the CFO and the System Admin. The differences between them are not styling variations of one menu; they are four different menus, each assembled at run time from the grants the server issued for that signed-in role.

The comparison makes the access model visible. The Student opens on How It Works and holds Forms and Clubs but no Dashboard, because a student raises requests and joins clubs but has no operational figures to review. The Head of Department opens on the Dashboard and holds Forms and the Dropdown Settings for their own department. The CFO holds the Dashboard and Dropdown Settings but not Forms, because the CFO reviews requests and owns the venue and funding catalogues but does not submit event requests. The System Admin holds neither Dashboard nor Forms, and instead holds the Internal Directory and System Configuration together with the maintenance actions for purging deleted records and dispatching reminders.

The practical consequence of assembling the menu this way is that adding a page, renaming it, reordering it or granting it to a different role is an administrative action taken in the running system rather than a development task. The sidebar is data, not code, which is what allows the access model described in Section 4.2.3 to be adjusted as university procedure changes.

4.4.4 Design System Tokens

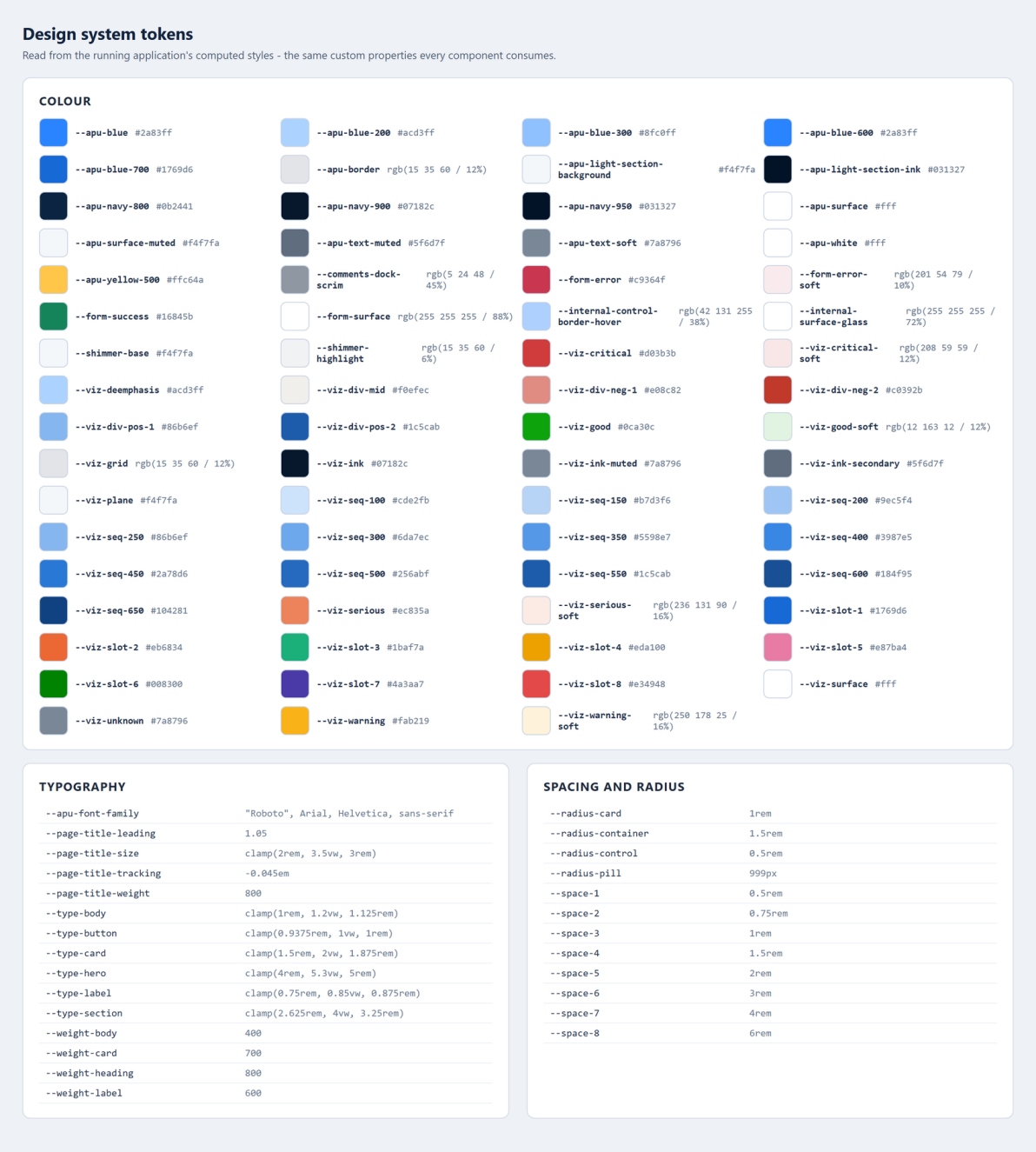


Figure 44: Design system tokens read from the running application

Figure 44 presents the design tokens read from the running application's computed styles. These are the custom properties every component consumes, and they are the mechanism by which visual consistency is maintained across an interface that is assembled differently for each user.

The colour set is organised by purpose rather than by appearance. The institutional blues and navies establish the identity, a small set of semantic colours carries meaning that must never vary by context, and a separate visualisation palette supplies sequential ramps,

divergent pairs and categorical slots for the dashboard charts. Separating the visualisation palette from the interface palette prevents a common failure in which a chart series accidentally adopts the same colour the interface uses for an error, and a reader misreads a neutral data point as a warning.

Typography and spacing are expressed as scales rather than as fixed values. The type scale is defined with fluid clamping so that a heading has a minimum size, a preferred size proportional to the viewport, and a maximum size, which allows the same page to remain readable from a phone to a wide monitor without a separate set of mobile type rules. Spacing and radius are defined as a small numbered scale, so that a developer adding a new panel chooses a step on an existing scale rather than inventing a new measurement, and the result stays visually aligned with everything already built.

4.4.5 Shared Component Library

The shared component library is a private development reference route within the application itself, listing twenty-nine reusable components and global classes with their selector, purpose, available states and responsive behaviour. It is shown across six figures because the catalogue is longer than one screen. Presenting it as a live route rather than as a static specification document has a practical advantage: the catalogue renders the real components, so it cannot fall out of step with what the application actually does.

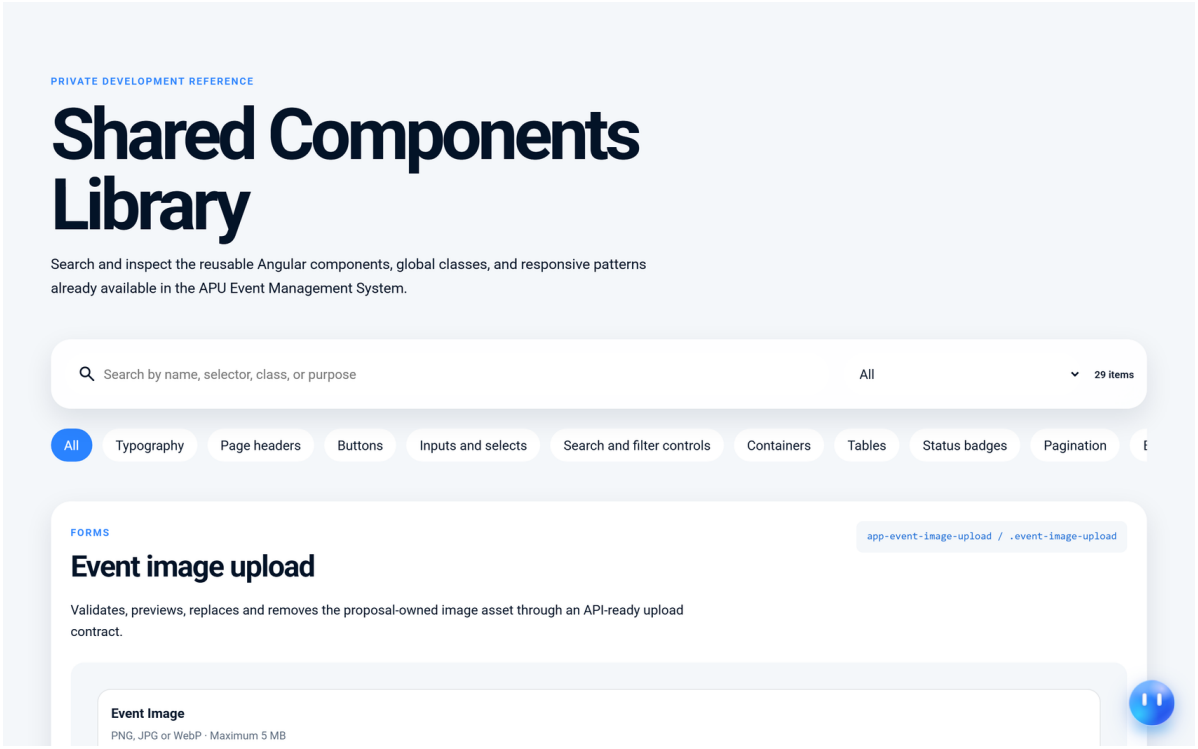


Figure 45: Shared component library (part 1 of 6)

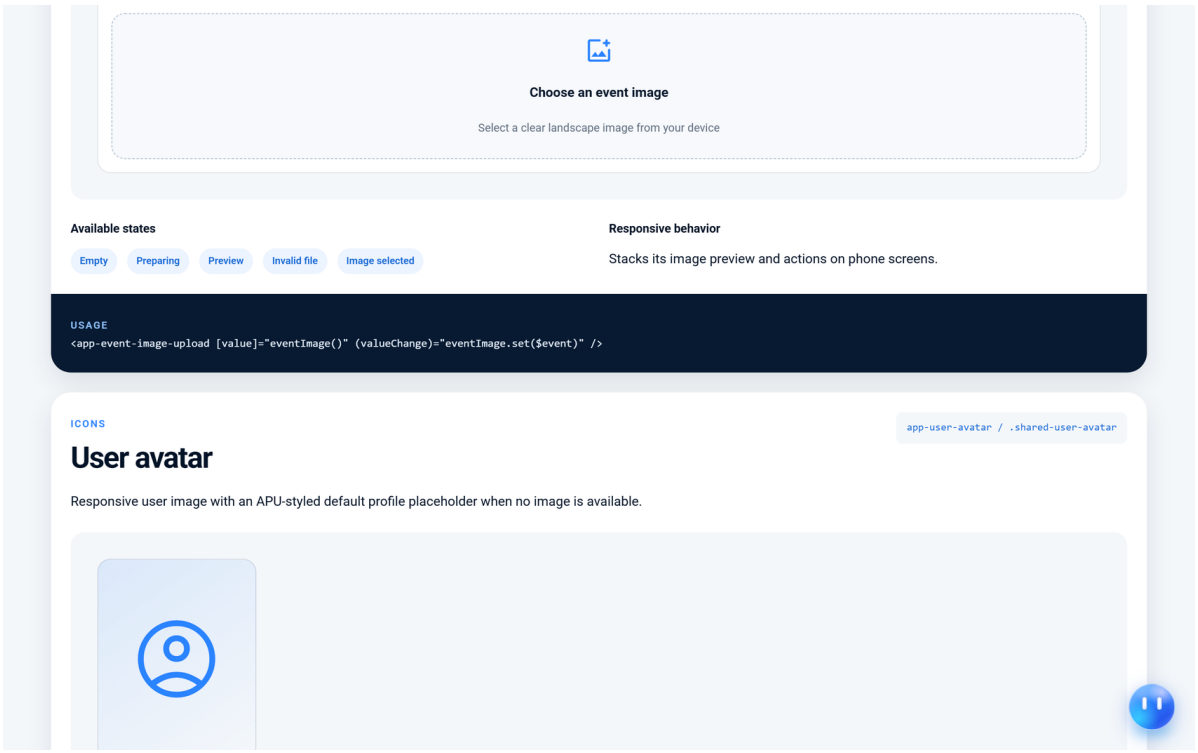


Figure 46: Shared component library (part 2 of 6)

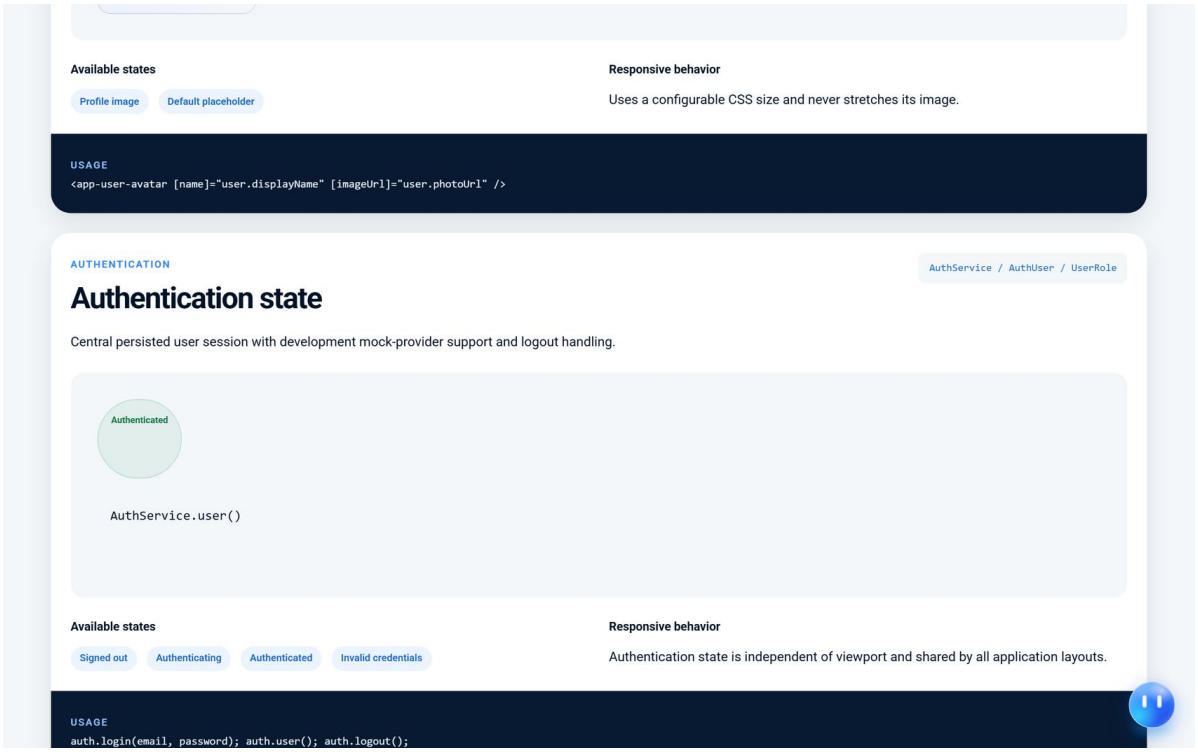


Figure 47: Shared component library (part 3 of 6)

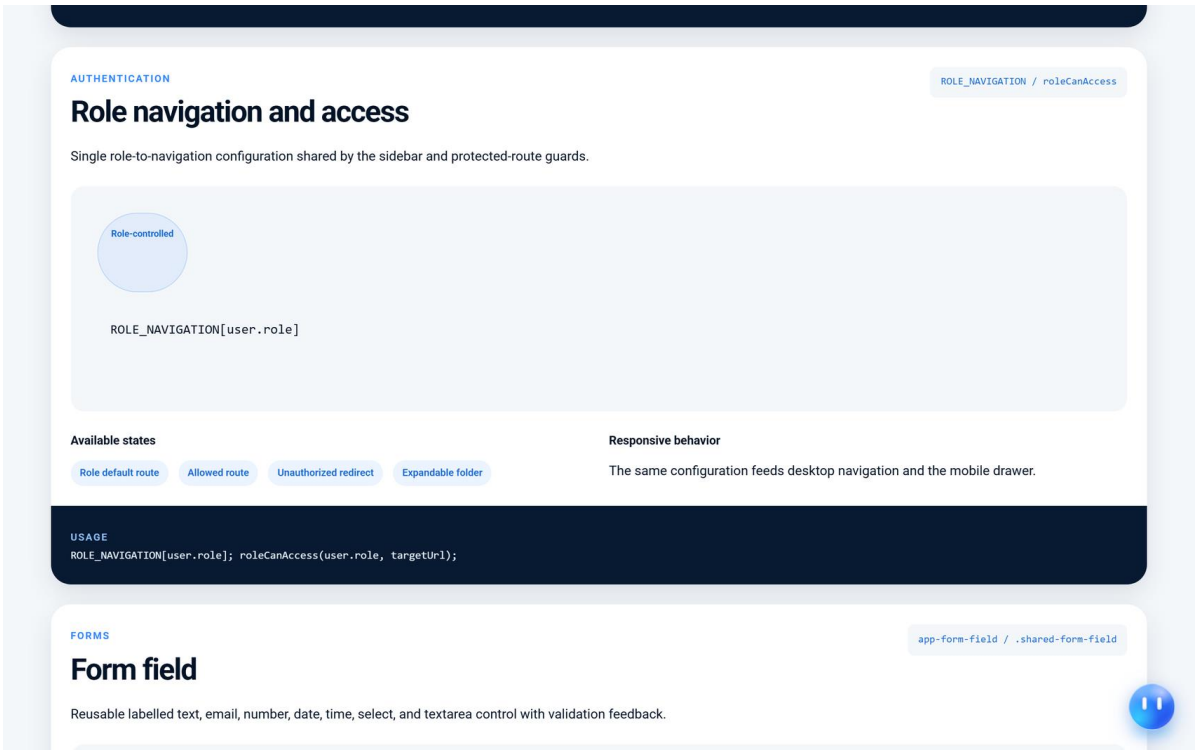


Figure 48: Shared component library (part 4 of 6)

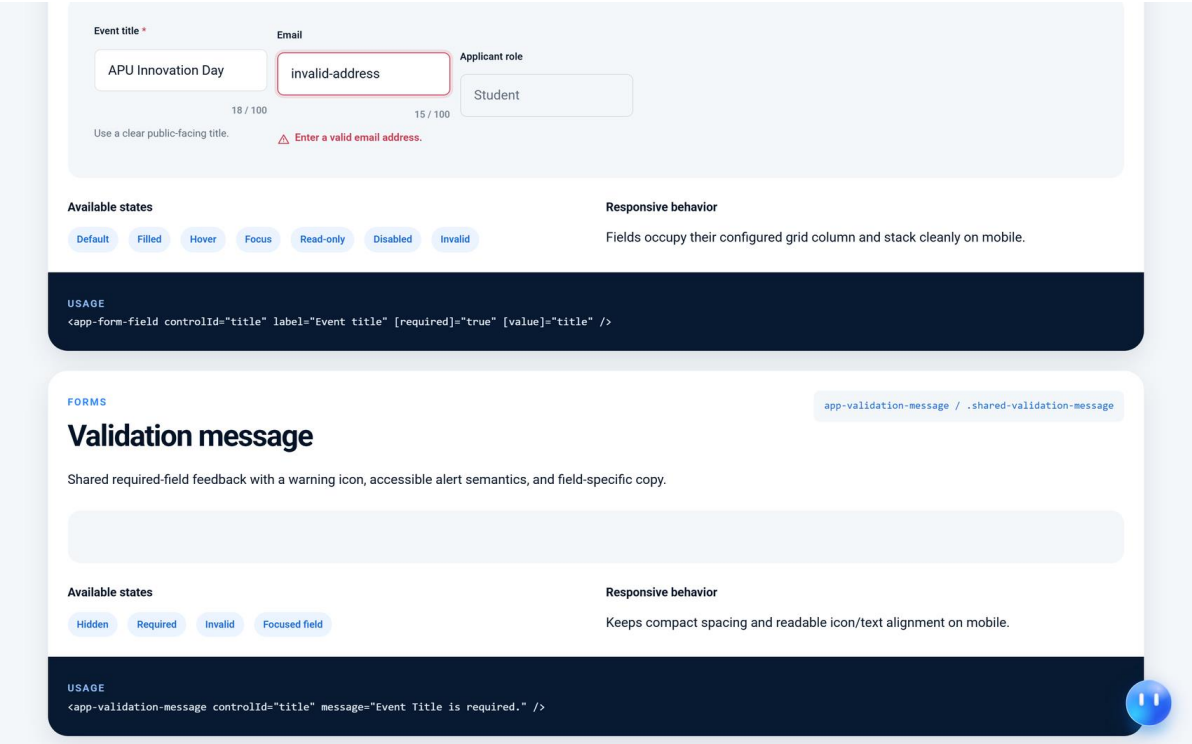


Figure 49: Shared component library (part 5 of 6)

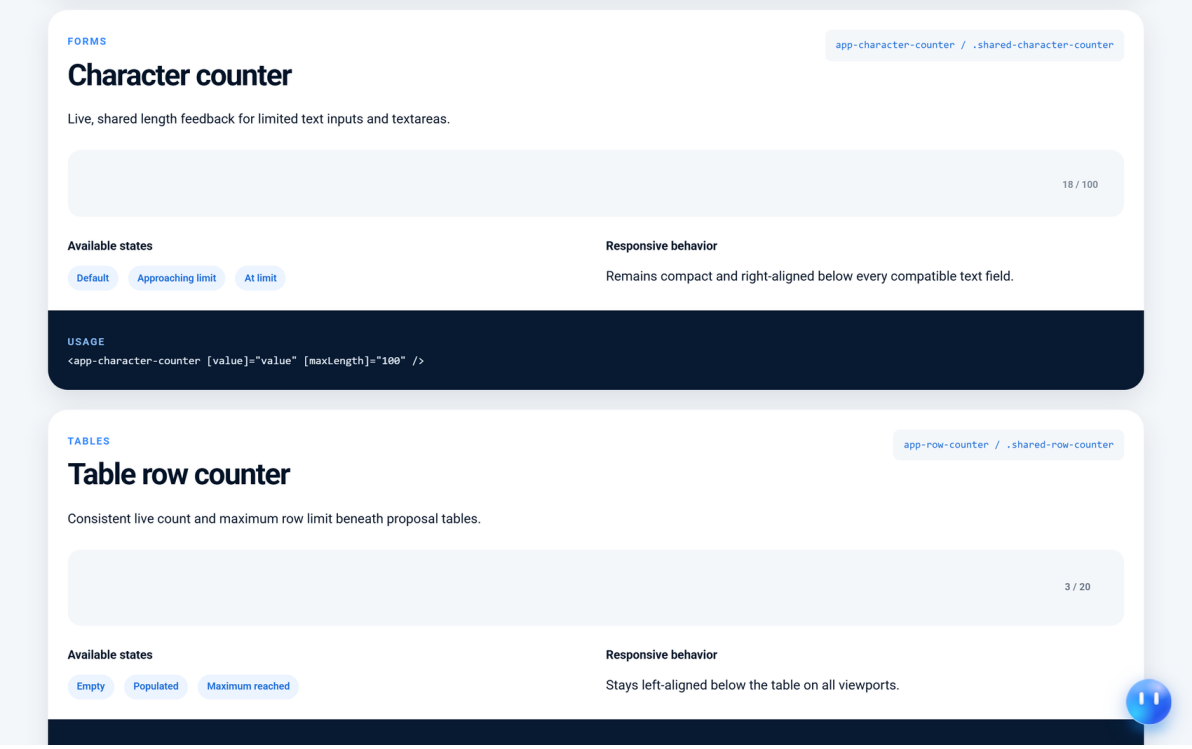


Figure 50: Shared component library (part 6 of 6)

Each entry documents four things. The selector and class name identify the component, a short description states what it is for, the state list enumerates the conditions it must handle, and a note records how it behaves on a narrow viewport. The state lists are the most valuable

part of the catalogue, because they force the empty, loading, invalid and disabled conditions to be designed rather than discovered. A form field, for example, is documented with default, filled, hover, focus, read-only, disabled and invalid states, and a validation message is documented with hidden, required, invalid and focused-field states.

The catalogue also covers behaviour that is not strictly visual. The role navigation entry documents the single configuration that feeds both the desktop sidebar and the mobile drawer and is also consulted by the route guards, which is what keeps the menu a user sees and the routes they may open from ever disagreeing. The authentication state entry documents the persisted session shared by every layout in the application. Recording these alongside the visual components is deliberate, because they are shared in exactly the same way and are subject to exactly the same risk of being reimplemented inconsistently.

4.4.6 Storyboards

Three storyboards document the principal user journeys through the system as a sequence of real screens rather than as an abstract flow. They are drawn from the running application, so each panel is a screen a user actually encounters in the order they encounter it.

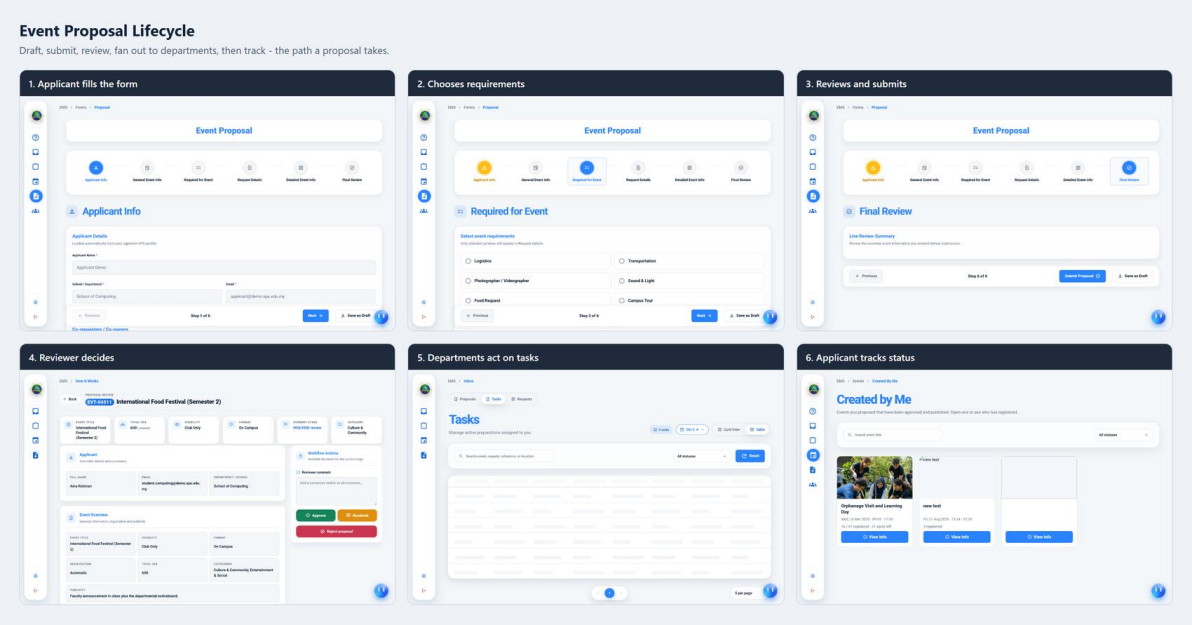


Figure 51: Storyboard — event proposal lifecycle

Figure 51 follows an event request from creation to tracking in six steps. The applicant completes the form, which is presented as a six-step wizard rather than a single long page; selects the service requirements needed, which is the step that determines which departments

the request will later be routed to; and reviews a live summary before submitting. The reviewer then opens the request and is presented with the event summary alongside a workflow actions panel offering approve, resubmit and reject. Departments act on the tasks that have been created for them, and the applicant tracks the outcome from a page listing the events they have raised.

The storyboard makes one design decision particularly visible. The requirement selection step comes third, before the detailed request information, and only the requirements selected there appear in the following step. An applicant who needs only logistics is therefore never shown the transport, photography, sound and campus tour sections at all. This keeps a form that must cover seven service departments from presenting every applicant with the union of all of them.

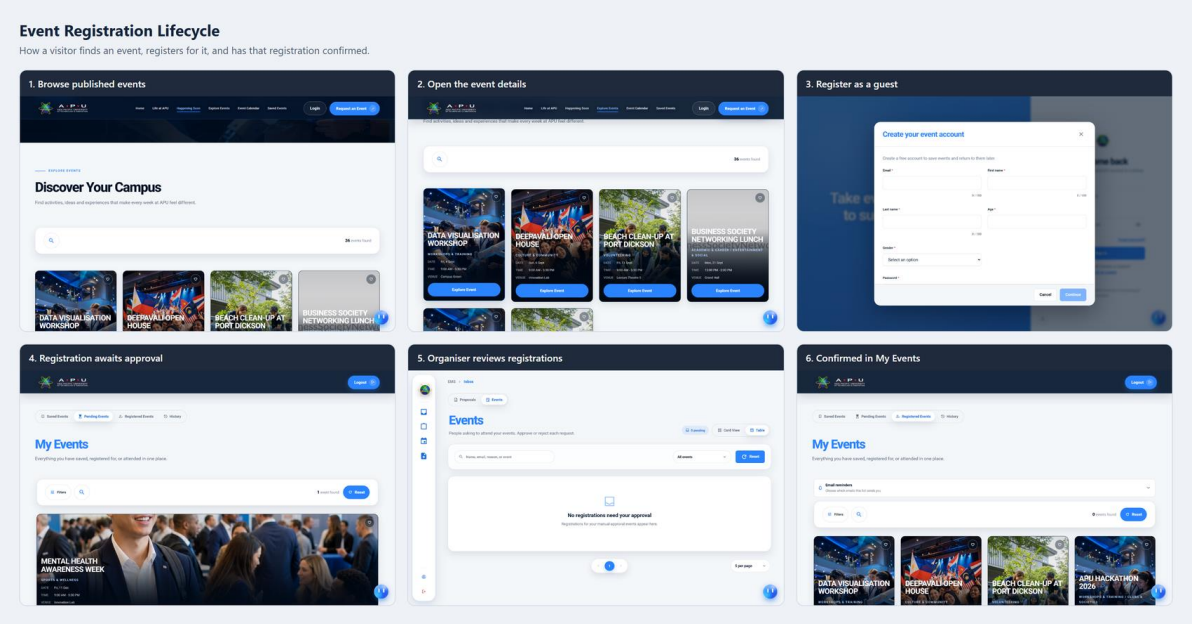


Figure 52: Storyboard — event registration lifecycle

Figure 52 follows a visitor from discovering a published event to holding a confirmed place. The visitor browses the public event listing, opens the event detail, and creates a guest account if they do not already hold one. The registration then waits in the pending queue where the event is set to manual approval, the organiser reviews it, and the confirmed registration appears in the visitor's own event list together with the reminder preferences they control.

The account creation panel shows the design decision that makes the public tier usable. A visitor is not asked to create an account in order to browse; the account is requested at the point it becomes necessary, which is when they want the system to remember something on

their behalf. Deferring registration to the moment of genuine need is what allows the platform to serve as a public event portal rather than only as an internal tool with a login page.

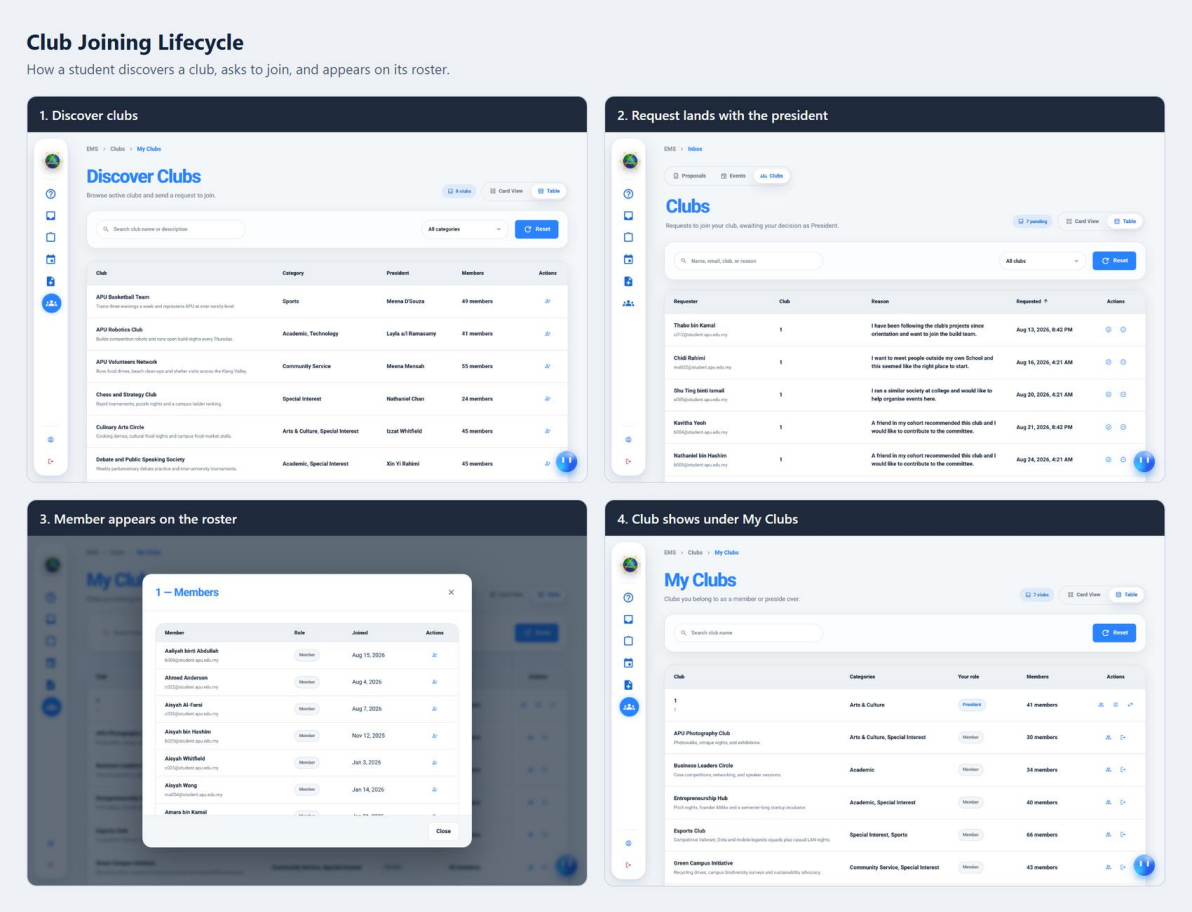


Figure 53: Storyboard — club joining lifecycle

Figure 53 follows a student from discovering a club to appearing on its roster, in four steps. The student browses the club directory, filtered by category, and sends a request to join with a stated reason. The request lands in the president's inbox alongside the reasons given by other applicants, which allows the president to decide the queue as a set rather than one message at a time. Once approved, the member appears on the club roster and the club appears under the student's own club list with their role shown.

The reason field carries more weight than its size suggests. Without it a president receives a list of names and no basis on which to decide, which in practice means every request is approved or none are. Collecting a short statement of intent at the point of request is what turns the decision into a real one, and it is the same reasoning that places a reason for attending on the event registration.

4.4.7 Responsive Breakpoint Strategy

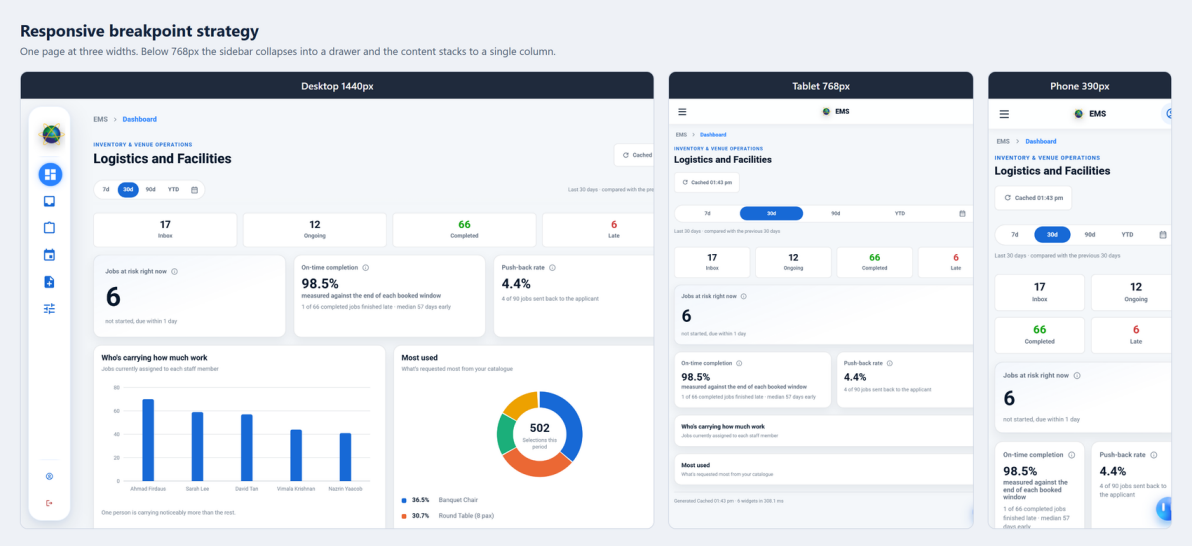


Figure 54: Responsive breakpoint strategy at three viewport widths

Figure 54 presents one page rendered at three widths: a desktop viewport, a tablet viewport at the breakpoint itself, and a phone viewport. The strategy is a single breakpoint below which the sidebar collapses into a drawer and the content reflows from a multi-column grid into a single stacked column.

Choosing one breakpoint rather than several is a deliberate simplification. A system with as many pages as this one cannot realistically be verified at four or five separate widths for every screen, and a responsive design that is only partly tested is worse than a simpler one that holds everywhere. The single-column phone layout also preserves the reading order of the desktop layout, so the summary figures appear before the detail panels at every width, and a user moving between devices encounters the same information in the same sequence.

The comparison also shows what is deliberately not changed. The page title, the period selector, the summary figures and the panels are all present at every width; nothing is hidden on the phone. This matters for the accessibility objective stated in Chapter 1, because a coordination platform whose mobile version omits functionality would fail precisely the users it is most intended to help, namely staff who are away from a desk while an event is being prepared.

4.5 Implementation

This section presents the delivered system. It contains sixty-four figures, each captured from the running application rather than from a prototype, and each showing a screen that the preceding sections have described in the abstract. Together they cover the whole functional surface of the platform: the public tier, attendance, the application shell, proposal submission, tracking and review, departmental task handling, the cafeteria and clubs modules, the role dashboards, event registrations, system administration and the AI assistant.

Every figure is a pair. The left pane is the screen at a desktop viewport of 1440 by 900 pixels captured at twice that resolution, and the right pane is the same screen on a mobile device at 390 by 844 pixels captured at three times that resolution. Both are viewport captures rather than full-page ones, so each is a single screen as a user would actually see it, with the sticky top bar at the top and the assistant launcher in the lower right where they belong. Presenting both in every figure rather than in a separate responsiveness section is deliberate: responsive behaviour is a property of each screen, and showing it once for a representative page would say nothing about the rest.

Where a screen carries a strip of tabs that are the same component pointed at different data, one tab is captured and the caption and text name the others rather than repeating a near-identical figure for each. This is stated explicitly wherever it applies, so that the coverage of the section can be checked against the application without the reader having to assume anything.

The figures are grouped by subsystem in the order a user meets them. The account each screen was captured as is stated in the text, because the same page renders differently for different roles and a figure without that context would be ambiguous.

4.5.1 Public Access and Authentication

The public tier is the part of the system that requires no credential at all. It exists because the majority of people who need to know what is happening on campus have no reason to hold an account, and Chapter 2 found that requiring one is a common reason institutional event systems go unused. The screens in this group are therefore served to anyone who reaches the address, and the same endpoints that serve them return only the public projection of an event: enough to decide whether to attend, and nothing about the approval history that produced it.

Authentication is presented in the same group because the public tier leads into it. A visitor who decides to attend an event is offered guest registration rather than being turned away, which is the mechanism that lets an external attendee hold a place without an institutional account.

4.5.1.1 Landing Page — Hero

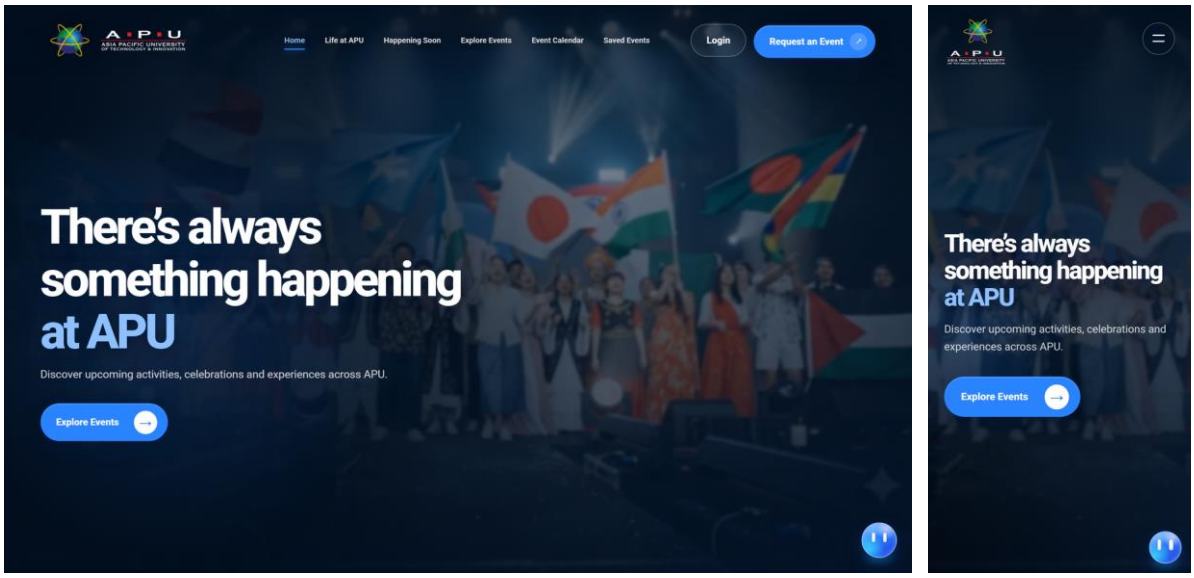


Figure 55: Public landing page above the fold, viewed with no session

The landing page opens on a full-width hero over the institutional navy header. The header carries the university mark and the public navigation — Home, Life at APU, Happening Soon, Explore Events, Event Calendar and Saved Events — with a Login control and a prominent Request an Event call to action. The two controls are deliberately distinct: Login is the route for people who already hold standing in the system, while Request an Event is the entry point to the proposal process described in Section 4.5.4, and pressing it while signed out sends the visitor through authentication first rather than failing at submission. The mobile rendering on the right collapses the same navigation into a drawer and keeps both controls reachable without scrolling, which is the pattern applied throughout the public tier.

4.5.1.2 Landing Page — Happening Soon Carousel

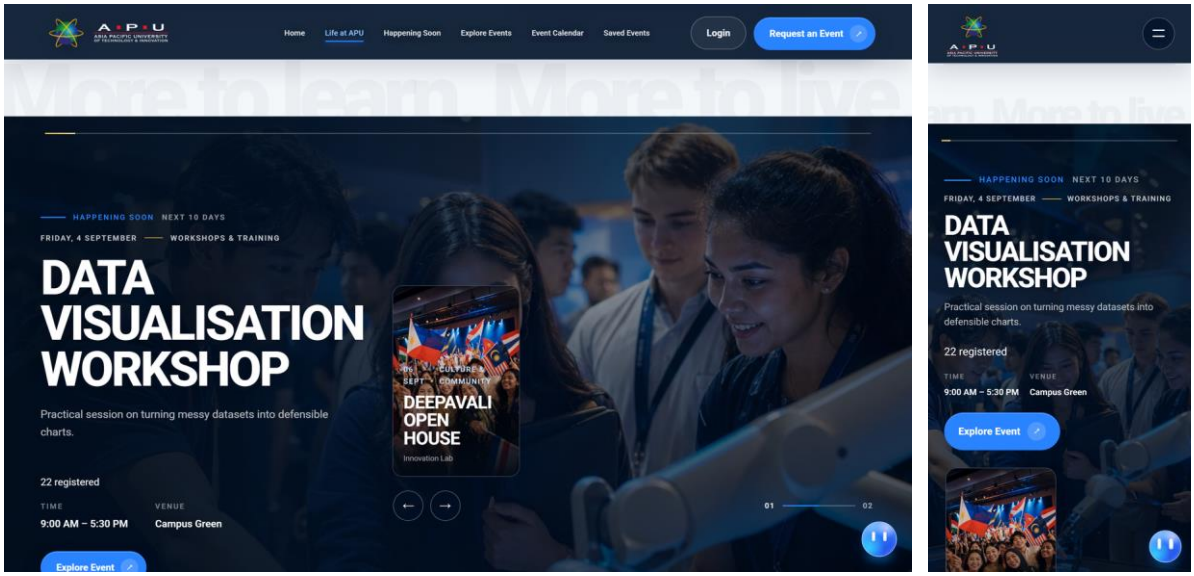


Figure 56: Near-term events presented as a horizontally scrollable carousel

Beneath the hero the page presents the events occurring soonest as a horizontally draggable strip of cards with previous and next controls. The carousel is a deliberate choice over a second grid. A visitor arriving at the landing page has not yet expressed an interest to filter on, so the most useful thing the page can offer is proximity in time; the grid further down the page then serves the visitor who knows what they are looking for. Each card carries the event category, date, time and venue, so the strip can be scanned without opening anything. On the mobile viewport the same strip becomes touch-scrollable and the arrow controls are withdrawn, because a swipe is the native gesture there and duplicated controls would only consume width.

4.5.1.3 Landing Page — Explore Events

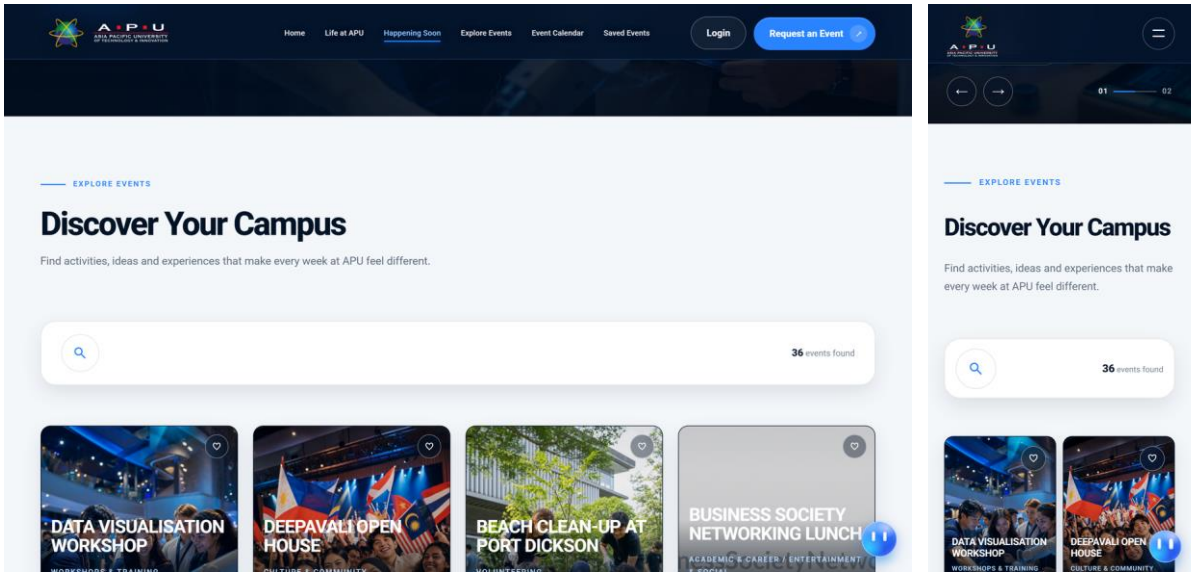


Figure 57: Public event discovery grid with search and live result count

The discovery section, headed “Discover Your Campus”, is the main public listing. It provides a search field, a live count of matching events, and a grid of event cards each showing category, date, time, venue, a save control and an Explore Event button. The count updates as the query narrows, so a visitor can tell that a search has taken effect even when the result set is still larger than the visible grid. This screen is the public counterpart of the internal discovery page in Section 4.5.3; the two are the same component, and the filter bar is rendered only for the internal variant because the public projection of an event carries none of the fields those filters operate on.

4.5.1.4 Landing Page — Campus Life

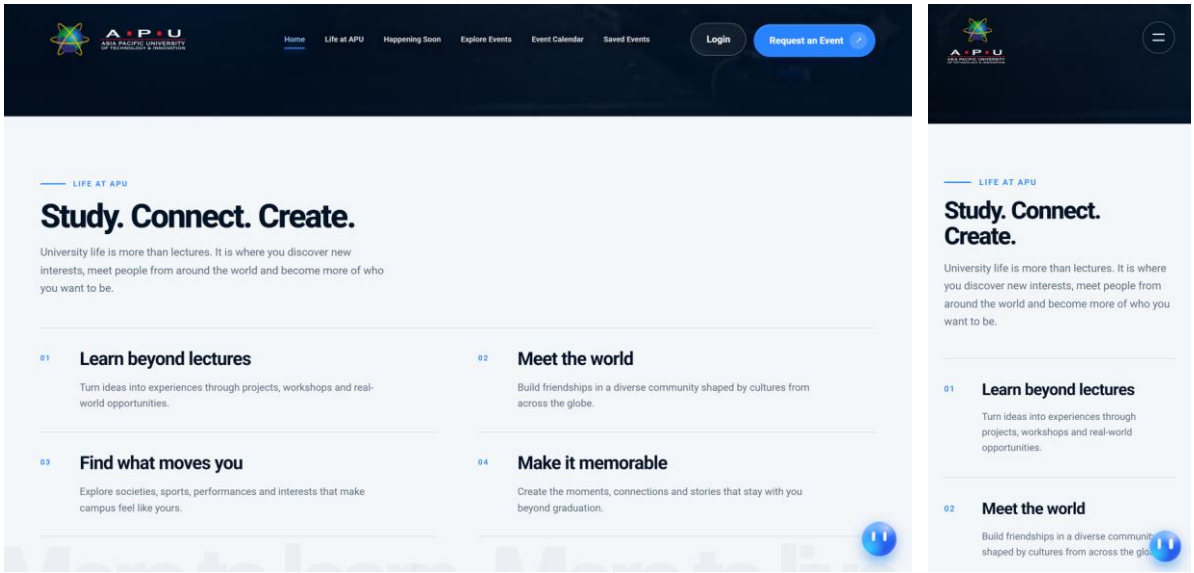


Figure 58: Campus life section of the public landing page

The Campus Life section is editorial rather than transactional. It exists because a visitor arriving with no prior knowledge of the institution needs some context for the events listed above it, and because the landing page is the only surface in the system that is read by people who will never sign in. The section carries no data from the request workflow and issues no queries against it, which is why it renders identically for every visitor.

4.5.1.5 Landing Page — Public Event Calendar

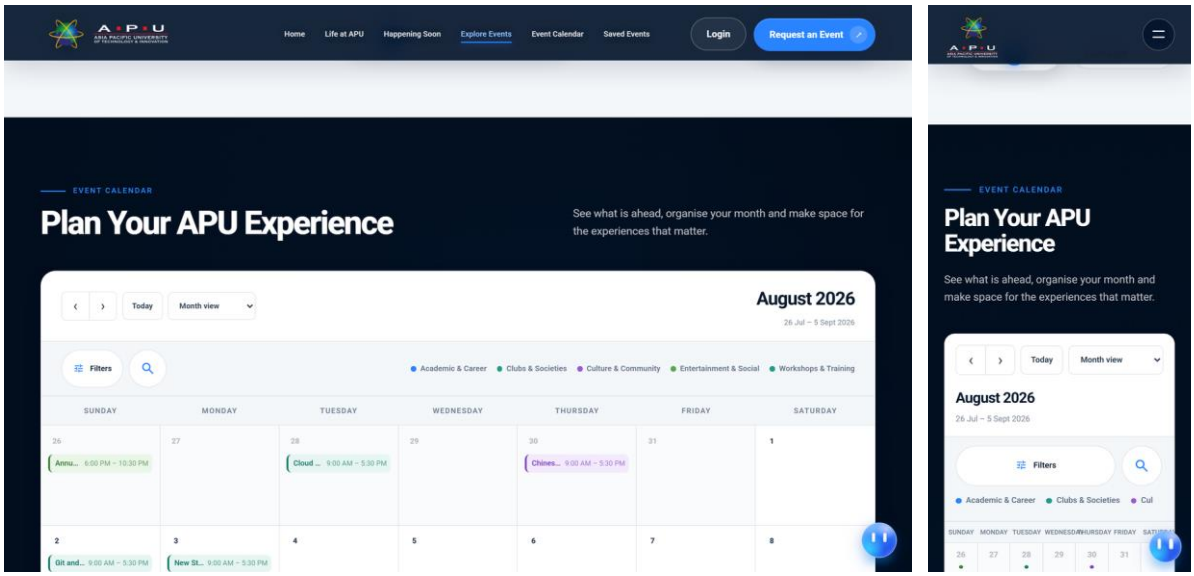


Figure 59: Month calendar of published events on the public page

The public calendar presents the same published events as the discovery grid, arranged by date rather than by relevance. Offering both is not duplication: a visitor asking “what is on next Tuesday” and a visitor asking “what workshops are there” need different arrangements of the same records, and forcing either to use the other's view is the kind of friction Chapter 3 identified in the existing arrangements. The calendar is restricted to events whose visibility is public and whose proposal has completed approval, so nothing in flight is ever exposed here. On mobile the month grid becomes a vertically stacked day list, because a seven-column grid at 390 pixels renders cells too small to read.

4.5.1.6 Event Details Modal

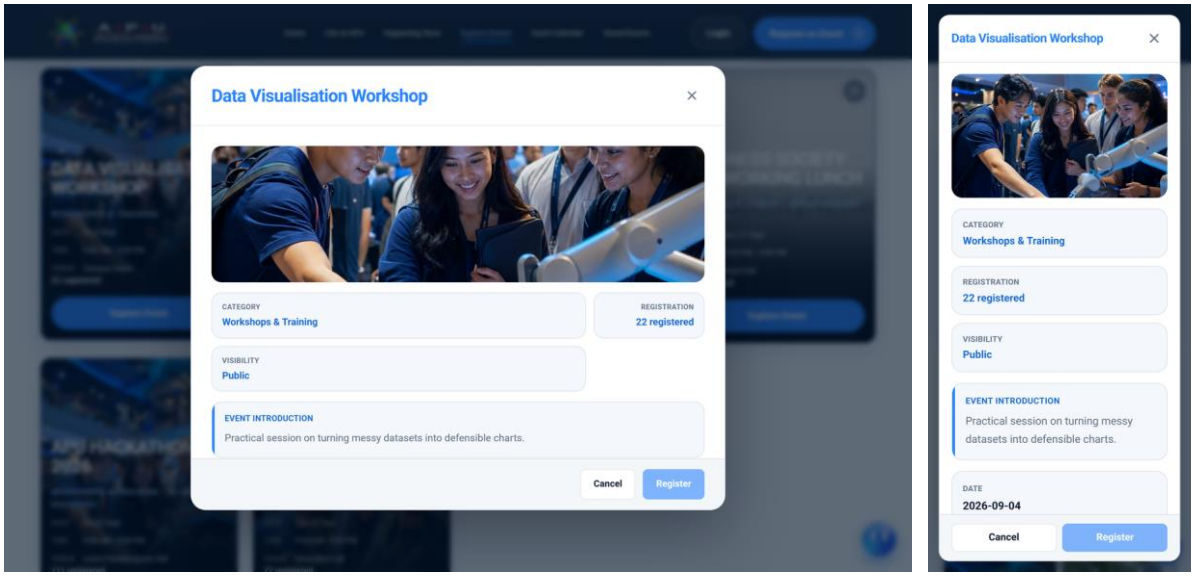


Figure 60: Event details overlay opened from a public event card

Selecting Explore Event opens the details overlay shown here for the Data Visualisation Workshop. The overlay carries the event banner, the category, the current registration count, the visibility setting and the event introduction, closing with Cancel and Register. It is presented as an overlay rather than a separate route so that dismissing it returns the visitor to their position in the grid with the scroll offset and any active search intact. The Register control is the junction between the public and authenticated tiers: a visitor with no session is offered guest registration at this point, which is the path documented in the next three figures.

4.5.1.7 Login Page

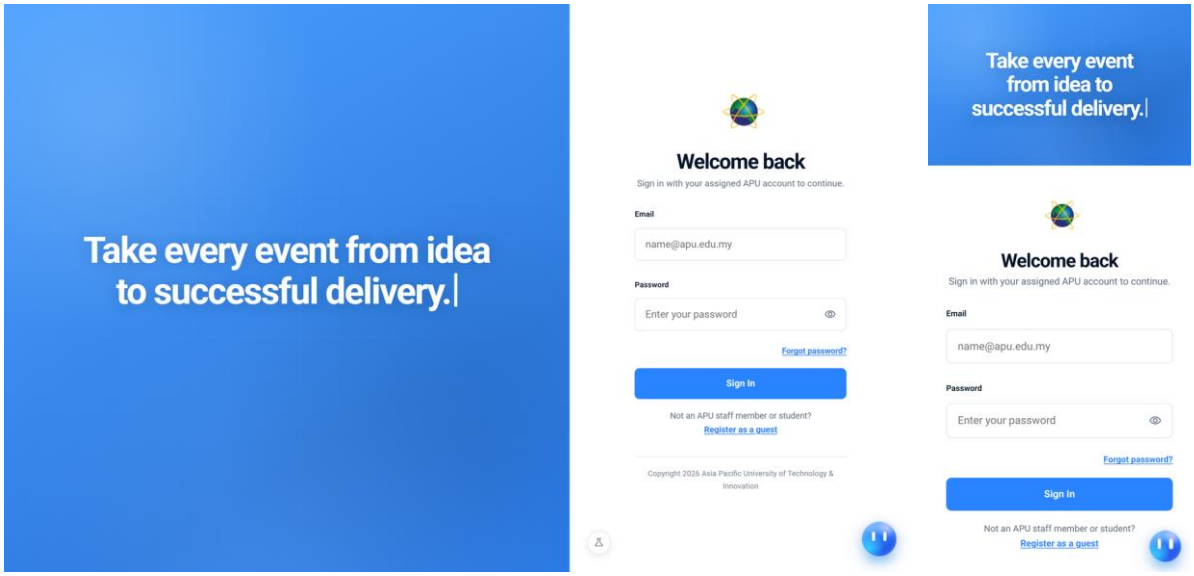


Figure 61: Sign-in screen with credential entry and guest registration route

The sign-in screen is a split layout: an animated headline on the institutional blue panel, and the credential form beside it with email, a password field with a reveal toggle, a Forgot password link, the Sign In control and a separate route to register as a guest. Offering guest registration from the sign-in screen rather than only from the registration flow matters because an external attendee who follows an event link will arrive here first and would otherwise have no way forward. Every figure in this chapter was captured with the credential fields empty, so no screenshot in Section 4.5 discloses a working credential.

4.5.1.8 Forgot Password Modal

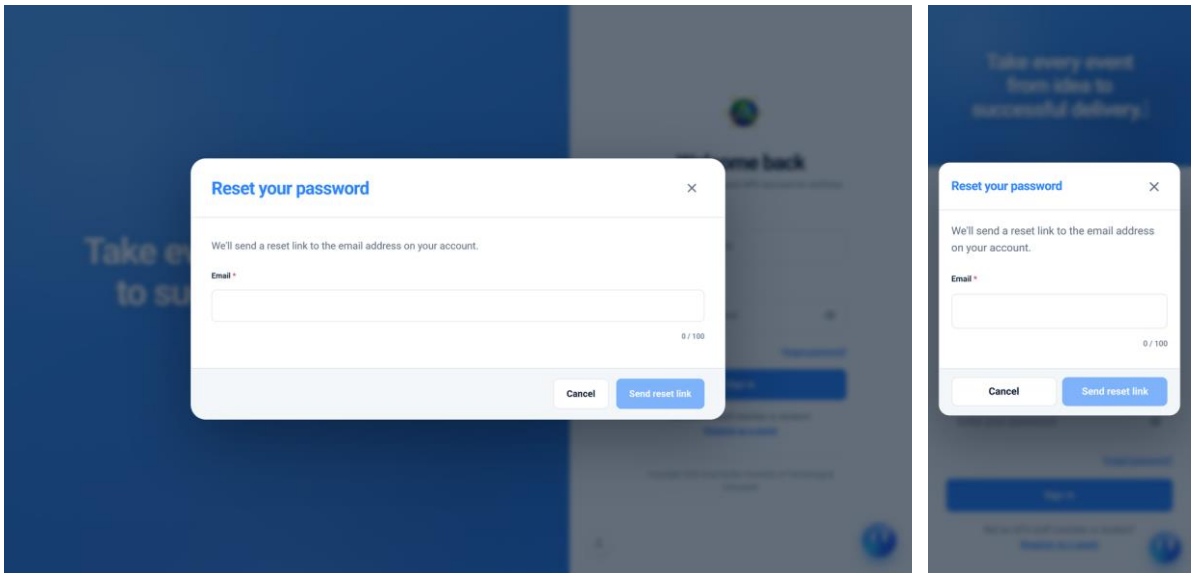


Figure 62: Password reset request overlay opened from the sign-in form

The reset request overlay collects only the account email. The server responds identically whether or not that address corresponds to an account, so the screen cannot be used to enumerate registered users; the same reasoning is applied to the sign-in endpoint, which verifies a password even when no such user exists so that a bad email and a bad password cannot be distinguished by response time. Where an account does exist, the message that follows carries a hashed, expiring token rather than a password, and the recipient lands on the screen shown in the final figure of this group.

4.5.1.9 Guest Registration — Account Details

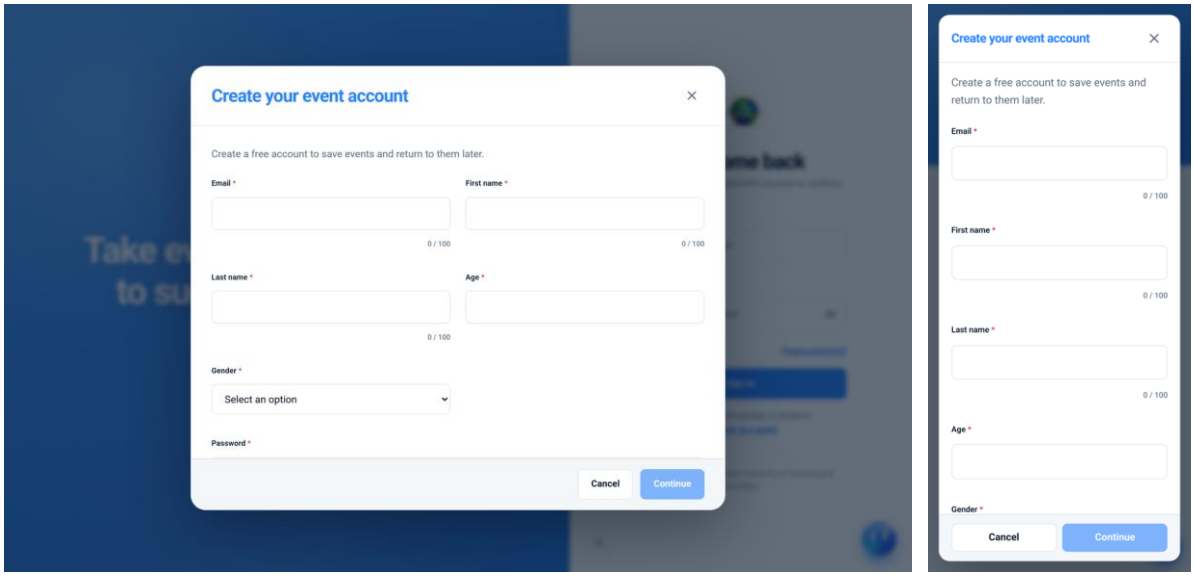


Figure 63: Guest account creation form in its initial state

Guest registration collects an email address, first and last name with live character counters, age, gender and a password. The set of fields is deliberately short. An external attendee is being asked to create an account solely so that a place on an event can be held in their name, and every field beyond that is an opportunity to abandon the flow. The account this produces is issued the external guest role, which is unit-independent and grants only the attendee pages described in Section 4.5.2 — it confers no access to the proposal workflow, which is enforced on the server rather than by hiding controls.

4.5.1.10 Guest Registration — Completed Form

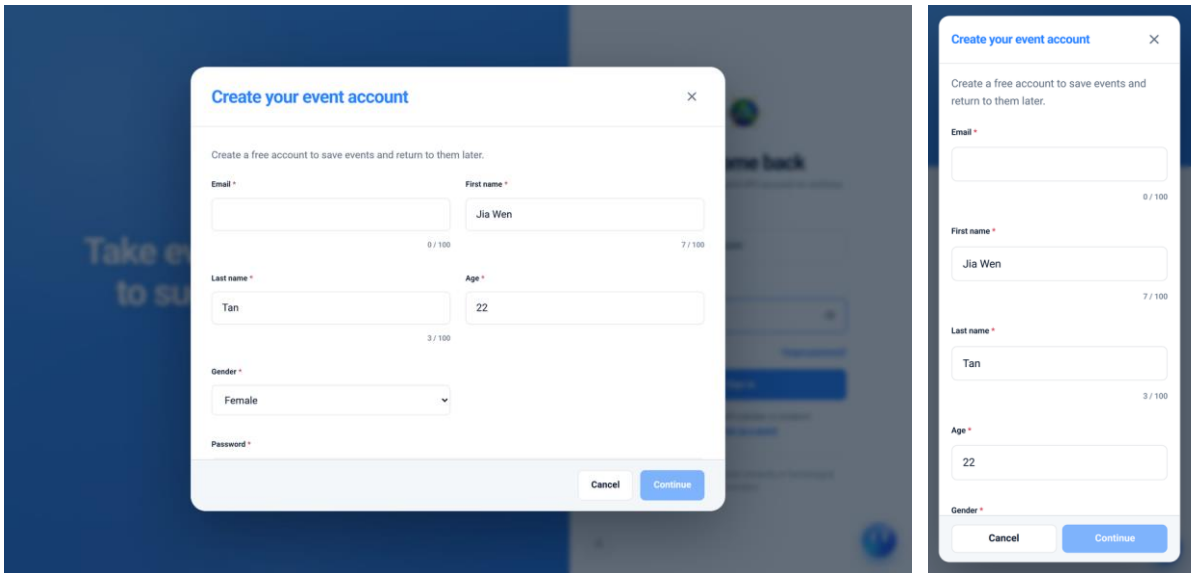


Figure 64: The same form with all fields satisfied and submission enabled

The completed state is shown alongside the empty one because it demonstrates the validation behaviour rather than merely the layout. The Continue control is disabled until every constraint is satisfied, and the counters beneath the name fields report the remaining allowance as it is consumed rather than reporting a violation after the fact. Client-side validation of this kind is a convenience and not a control; the same constraints are re-checked on the server, because the client is not the authority for any rule in this system.

4.5.1.11 Reset Password Page

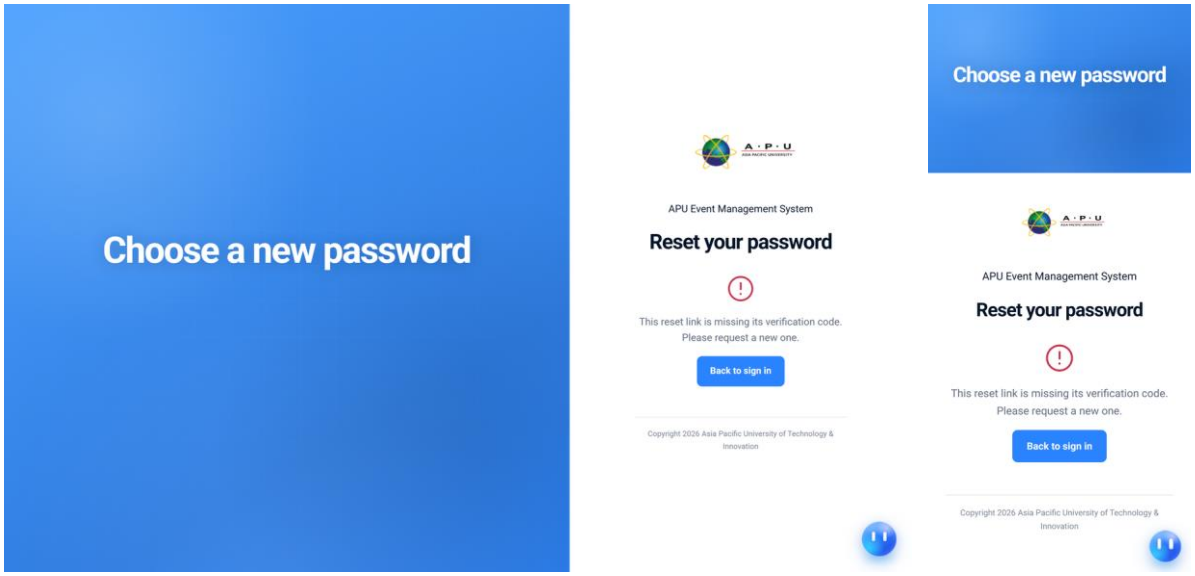


Figure 65: Landing screen for a password reset link

A recipient who follows a reset link arrives here to choose a new password. The token in the link is matched against a stored hash and checked for expiry before the form is honoured, and it is consumed on use so that a link cannot be replayed. This is the only route by which a password can be changed without presenting the current one, which is why it is treated as an authentication surface rather than a profile setting.

4.5.2 Attendee Screens for Students and External Users

Attendance is the one capability the system offers to every account type, and it is served by two different shells. An internal member of the university sees it inside the application shell described in Section 4.5.3, alongside whatever else their role grants them; an external guest sees a standalone layout with no sidebar, because there is nothing else their account can reach. Both are shown here rather than only one, since the difference between them is a design decision rather than an accident of styling.

4.5.2.1 My Events — Internal Shell

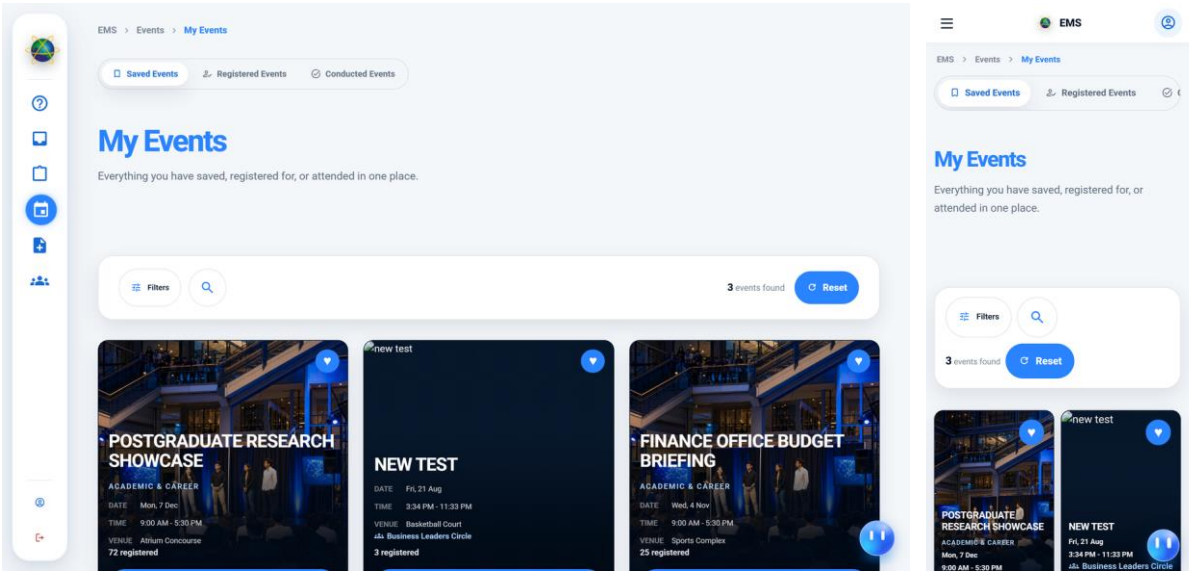


Figure 66: Saved events for a signed-in student, inside the internal shell

My Events collects everything the signed-in user has an attendance relationship with. The figure shows the Saved tab; the same page also carries Registered, holding confirmed places on events still to come, and Conducted, holding events for which a confirmed place was held and which have since finished. All three tabs render the same card list and the server decides which registrations belong to each, so the distinction between them is a property of the data rather than three separate implementations. Above the list sits an email reminders control, which is where a user chooses whether this list generates notifications, and a filter and search bar with a live result count.

Saving an event is separated from registering for one because they answer different questions. A save records interest and commits the user to nothing; a registration occupies a place and, where the organiser has set manual approval, enters a queue for a decision. Keeping them apart means a user can track an event they are still deciding about without distorting the attendance figures the organiser and the dashboards in Section 4.5.9 depend on.

4.5.2.2 My Events — External User

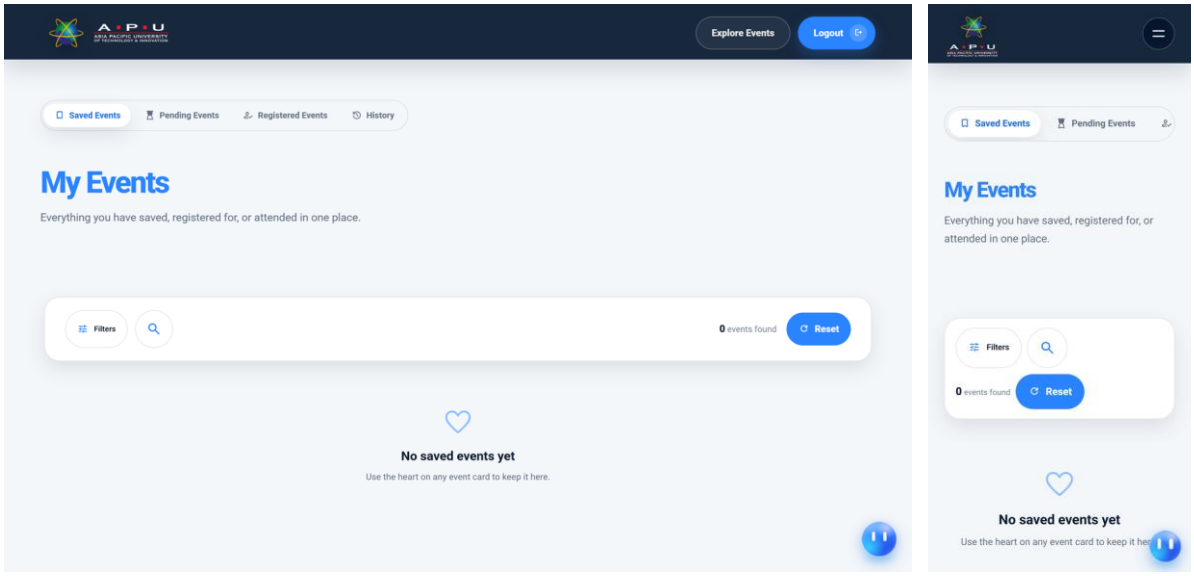


Figure 67: The external attendee layout, which carries no application sidebar

An external guest sees the same records in a different shell. There is no sidebar, because page grants give this account nothing else to navigate to, and a persistent empty menu would only advertise capabilities the account does not have. The external layout carries four tabs of its own: Saved, Pending for sign-ups still awaiting an organiser's decision, Registered for confirmed places, and History for sign-ups that were declined together with events already attended. The Pending tab exists here and not in the internal shell because manual approval is most often applied to events open to visitors, so an external attendee is the account most likely to be waiting on a decision.

4.5.2.3 User Profile

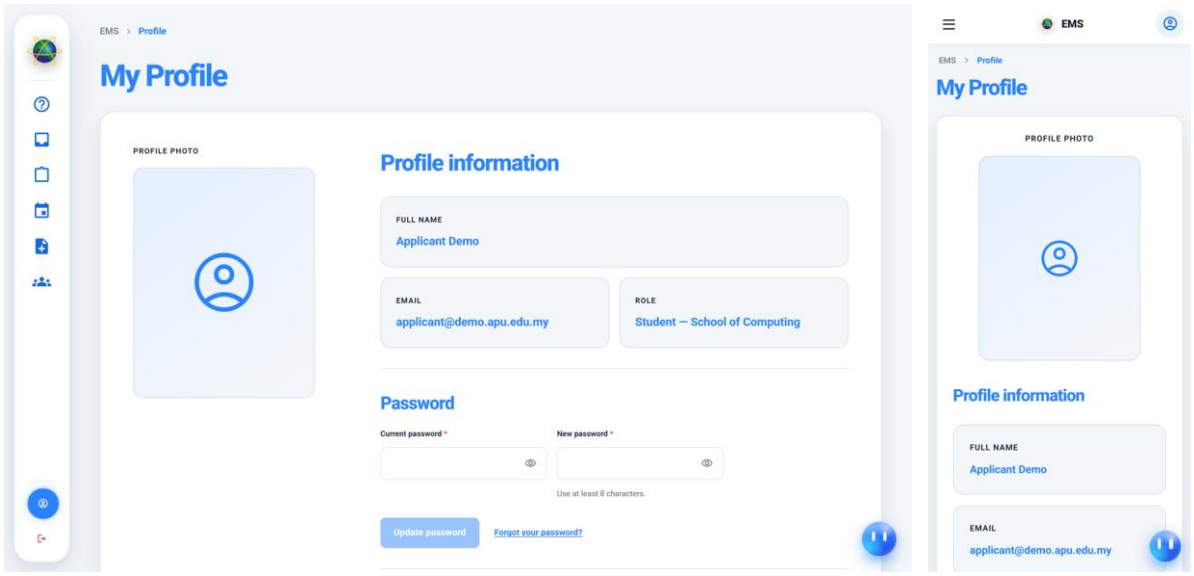


Figure 68: Account profile for an internal user

The profile screen carries the account's own details and the settings that belong to the person rather than to a role, including the email reminder preferences that govern which notifications the system sends them. Role assignments are shown but are not editable here: a role is held by a user within an organisational unit, and granting one is an administrative act performed on the screens in Section 4.5.11. Keeping the two apart is what prevents a user from adjusting their own standing, and it is the reason the profile page issues no writes against the assignment tables at all.

4.5.3 Internal Application Shell and Onboarding

Every internal page in the system is rendered inside one shell. The shell supplies the sidebar, the top bar with its breadcrumb trail and the persistent assistant launcher, and the pages themselves supply only their own workspace. This is what makes the access model visible: the sidebar is built from page grants issued by the server, so two accounts looking at the same shell see different menus without any page having been written twice.

4.5.3.1 Internal Layout and Sidebar Navigation

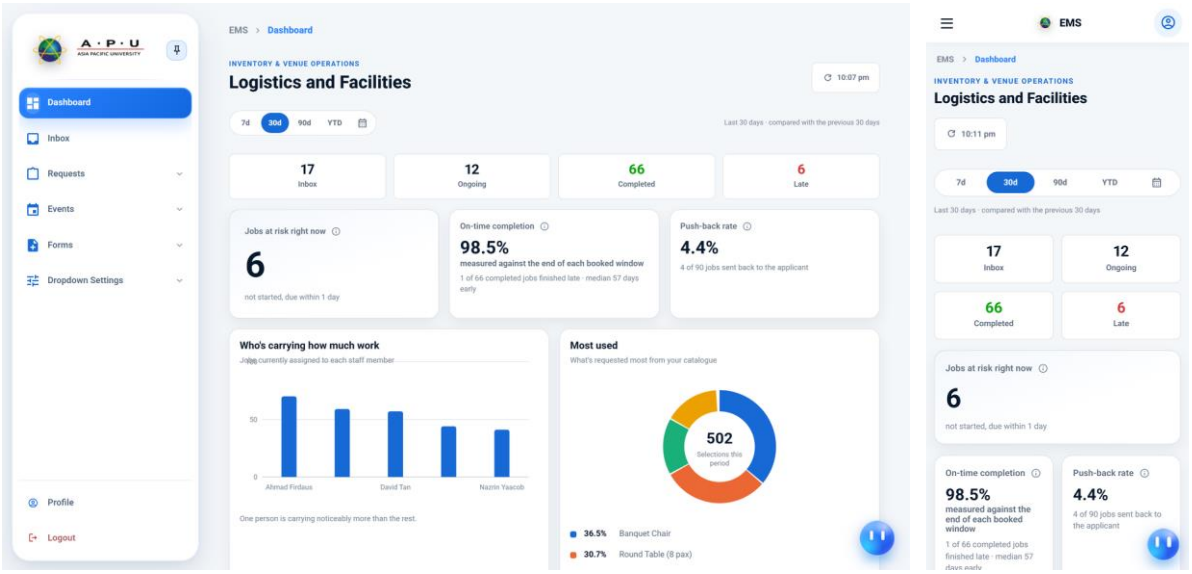


Figure 69: The internal shell with the sidebar pinned open, as a Head of Department

This figure is the anchor for the rest of Section 4.5, and every subsequent screen is rendered inside the frame it establishes. The sidebar is pinned open so that the labels are legible: the brand and the pin control at the top, the pages this account may reach in the middle, and Profile and Logout fixed to the bottom where they do not move as the menu above them changes length. The account shown is a Head of Department, and the menu it produces is not written anywhere in the client. It is assembled from the grants returned with the session, which

is why the Page Visibility screen in Section 4.5.11 can add or remove an entry without a deployment.

The mobile rendering places the same navigation behind a drawer and keeps the breadcrumb trail in the compact top bar. The trail carries more weight here than it would in a smaller application: because the menu differs per role, no two users share a mental map of where a page sits, and the trail is the only element that states the current position in terms that are the same for everybody.

4.5.3.2 How It Works — Onboarding Guide

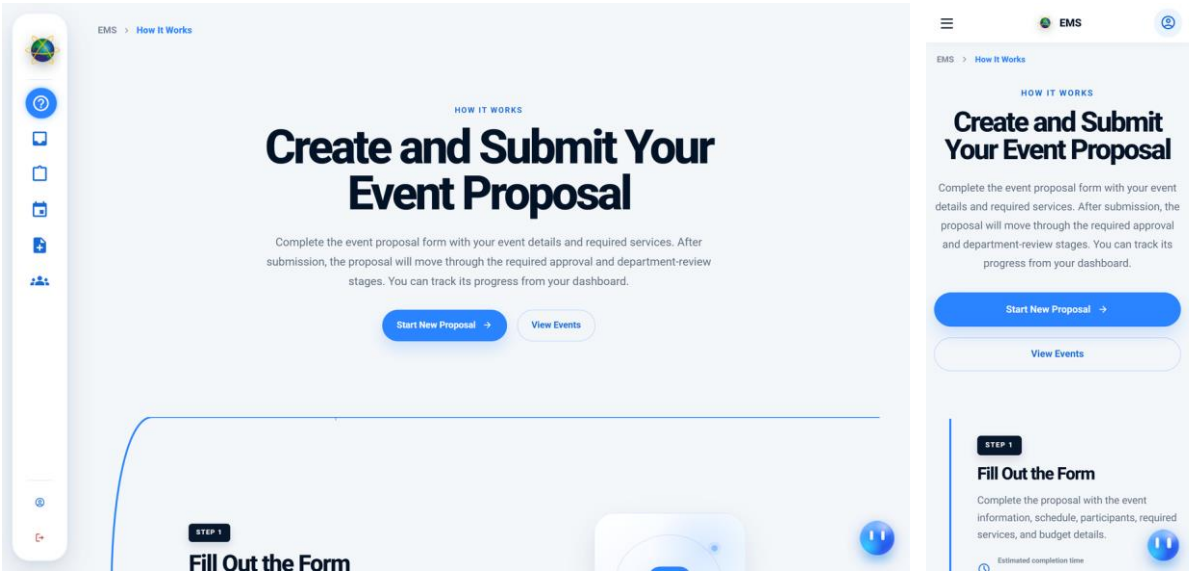


Figure 70: Guided explanation of the proposal journey shown to applicants

How It Works is the first page a student or lecturer lands on. It exists because Chapter 3 found that the most consistent complaint about the current arrangements was not that they were slow but that nobody could say what happened next, and a system that routes a proposal through several departments invisibly would reproduce exactly that. The page sets out the stages a proposal passes through, who acts at each one and what is expected of the applicant in between, before any form is opened. It is a static page holding no request data, which is why it is granted to every internal role rather than scoped.

4.5.3.3 Master Event Calendar

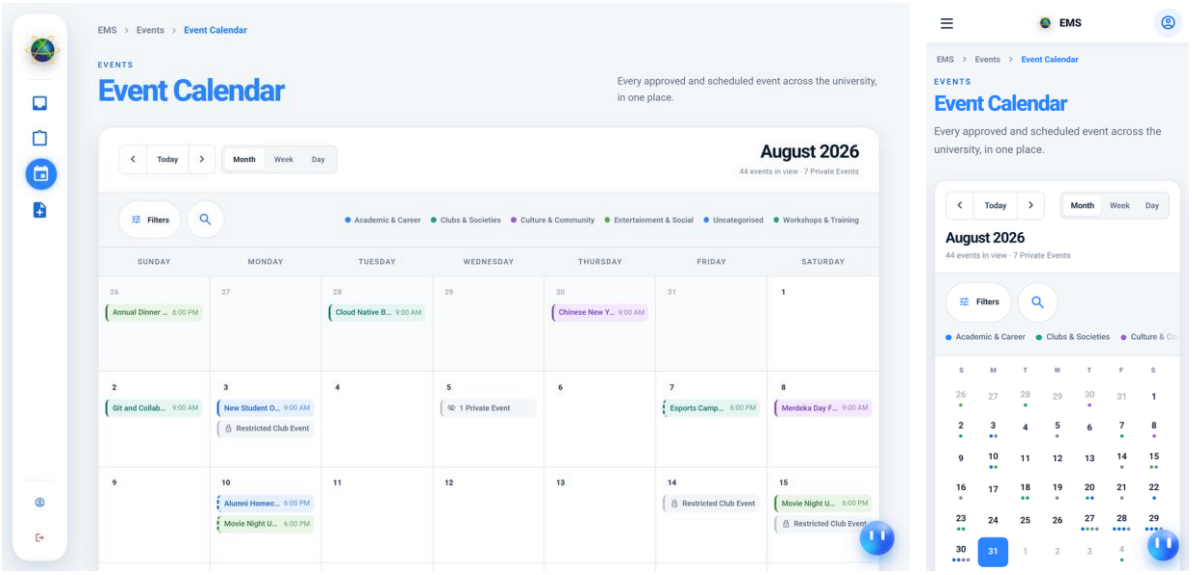


Figure 71: University-wide internal calendar of scheduled events

The internal calendar differs from the public one in what it is permitted to show. It carries every scheduled event the viewer's scope reaches, including those whose visibility restricts them to a club or a school and which therefore never appear publicly. At the point of capture the view reports forty-four events in range of which seven are private, and that division is the point of the screen: a Head of School planning around existing commitments has to see the private ones, and a public visitor must not. The distinction is applied in the query rather than in the rendering, so a record outside the viewer's scope is never retrieved.

4.5.3.4 Internal Explore Events

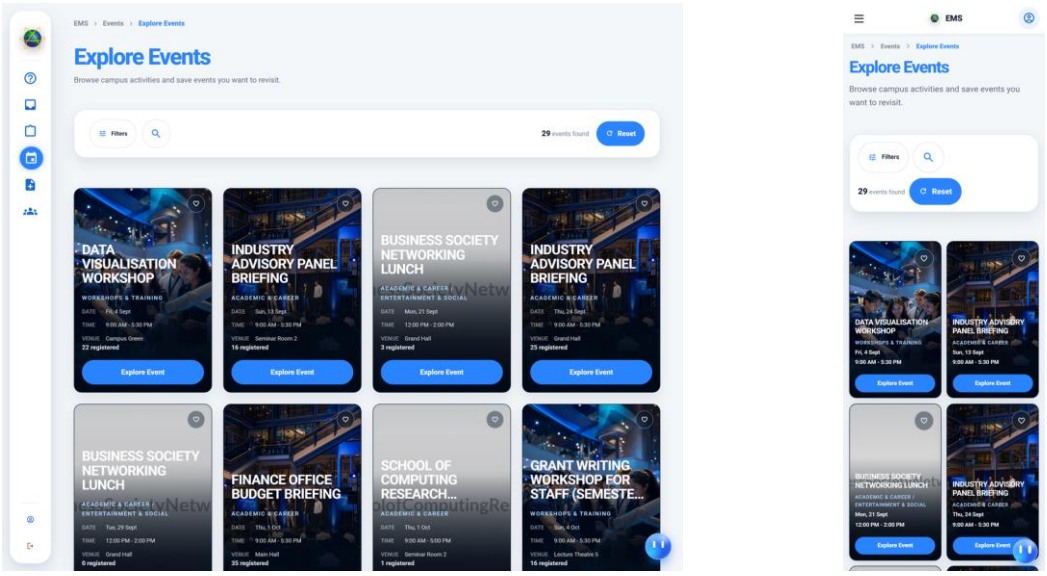


Figure 72: Internal event discovery with the filter bar and pagination

The internal discovery page is the same component as the public grid in Section 4.5.1, rendered in its internal variant. The difference is the filter bar, which appears only here. A signed-in member of the university can filter by the fields that describe how an event was produced — its category, format and organising unit — and those fields are present on the internal projection of an event and absent from the public one. Rendering one component in two variants rather than maintaining two pages is what keeps the two listings consistent as the event card evolves.

4.5.4 Event Proposal Submission

The proposal form is the entry point to the entire workflow and the single most complex screen in the system. It is presented as six ordered steps rather than one long form, with a step indicator across the top and a sticky footer carrying Previous, the current position, Next and Save as Draft. The division is not cosmetic. Each step is validated as it is left, so an applicant learns that a field is wrong while the section it belongs to is still on screen, and the draft can be saved at any point and resumed from the screen described in Section 4.5.5.

The order of the steps is itself a design decision. The requirements a proposal asks for are chosen at step three, before their details are given at step four, because it is the selection at step three that determines which departments the proposal will later be routed to. Deferring that choice to the end would mean the applicant supplies detail for services they may not need and the routing cannot be shown to them until the form is complete.

4.5.4.1 Proposal Form — Step 1: Applicant Information

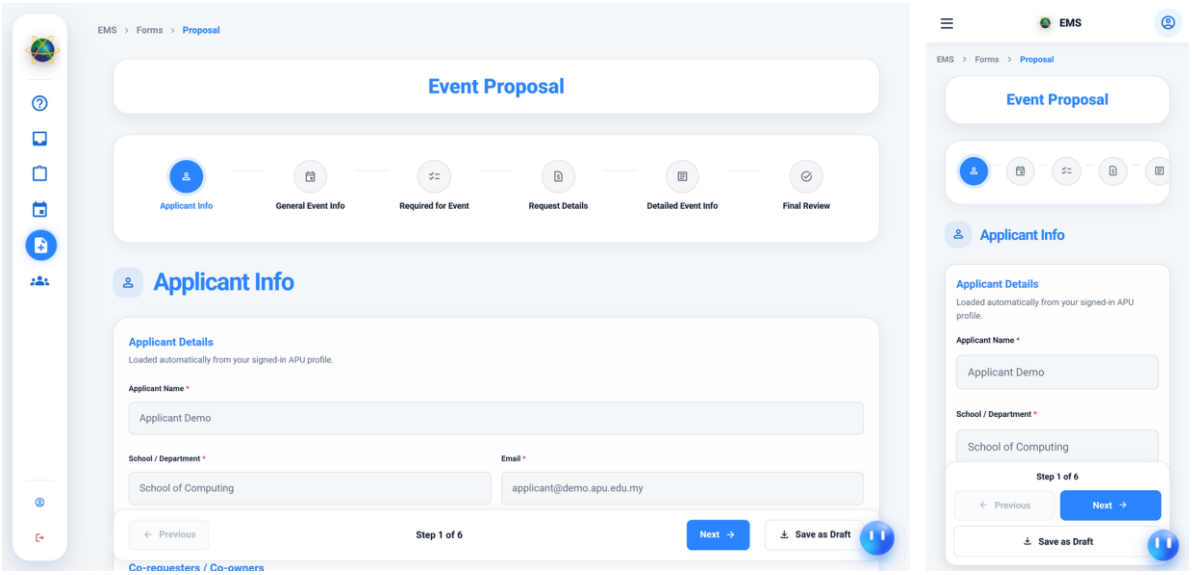


Figure 73: First step of the proposal form, with applicant details pre-filled

The first step presents the applicant's own details — name, school or department and email — pre-filled from the signed-in profile and not re-typed, followed by the co-requesters section in which other members of the university may be named as co-owners of the proposal. Pre-filling matters beyond convenience: the organising unit recorded here is what the routing rules read to decide which Head of School or Head of Department the proposal must first reach, so allowing it to be typed freely would let an applicant choose their own reviewer.

4.5.4.2 Proposal Form — Step 2: General Event Information

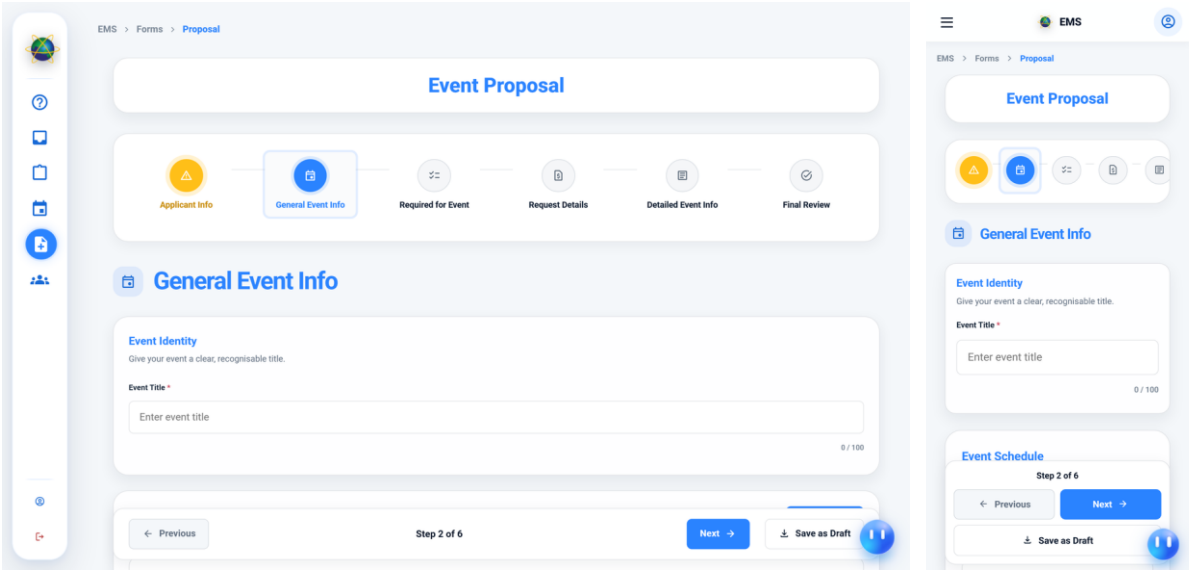


Figure 74: Event identity, visibility, format and registration mode

The second step establishes what the event is: its title, a short introduction, the goals and expected benefits, and then four settings that govern how it will behave once approved — its visibility, its format, whether registrations are approved automatically or by hand, and how it will be publicised. Collecting these here rather than at publication time is deliberate. They describe the applicant's intention, and a reviewer deciding whether to approve a proposal for six hundred external attendees needs to see that intention at the point of decision rather than discover it afterwards.

4.5.4.3 Proposal Form — Step 3: Requirements Selection

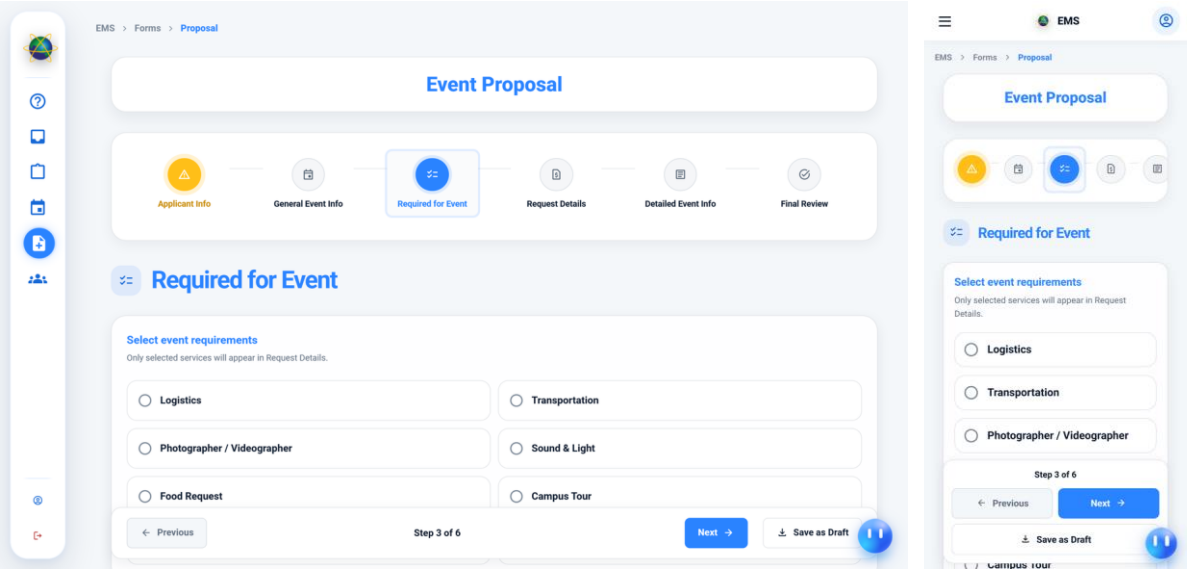


Figure 75: The requirement picker that determines departmental routing

Step three is where the applicant states which services the event needs: logistics, transport, photography, sound and light, food and beverage, a campus tour, mineral water or funding. This is the most consequential screen in the form. The selection made here is what later fans the approved proposal out into one task per responsible department, so an unticked requirement is not merely an omitted detail but a department that will never be asked. The screen therefore states each option in the language of what the applicant needs rather than in the language of which unit provides it.

4.5.4.4 Proposal Form — Step 4: Requirement Details

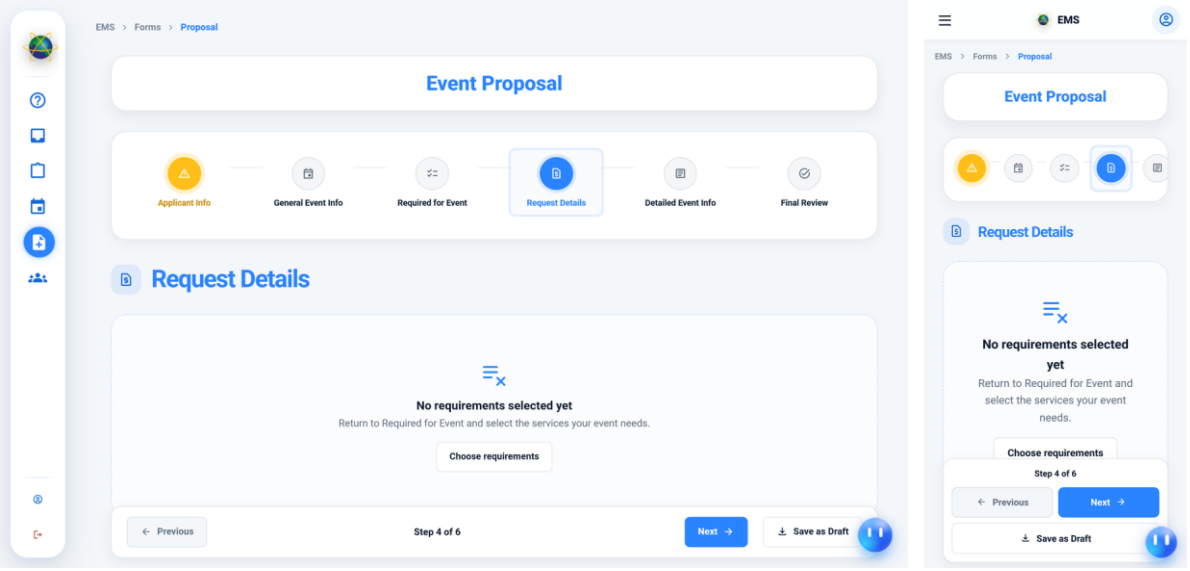


Figure 76: Detail tables for each requirement selected at the previous step

The fourth step presents one detail table per requirement chosen at step three, collecting items, quantities, dates, times, locations and notes. The tables are generated from the selection rather than from a fixed template, so an applicant who asked for nothing from Transport is never shown a transport table to leave empty. The options offered inside each table are read from the catalogues in Section 4.5.11, which are owned by the departments themselves; a logistics item can therefore be withdrawn by the Head of Logistics and cease to be offered here without any change to this form.

4.5.4.5 Proposal Form — Step 5: Detailed Event Information

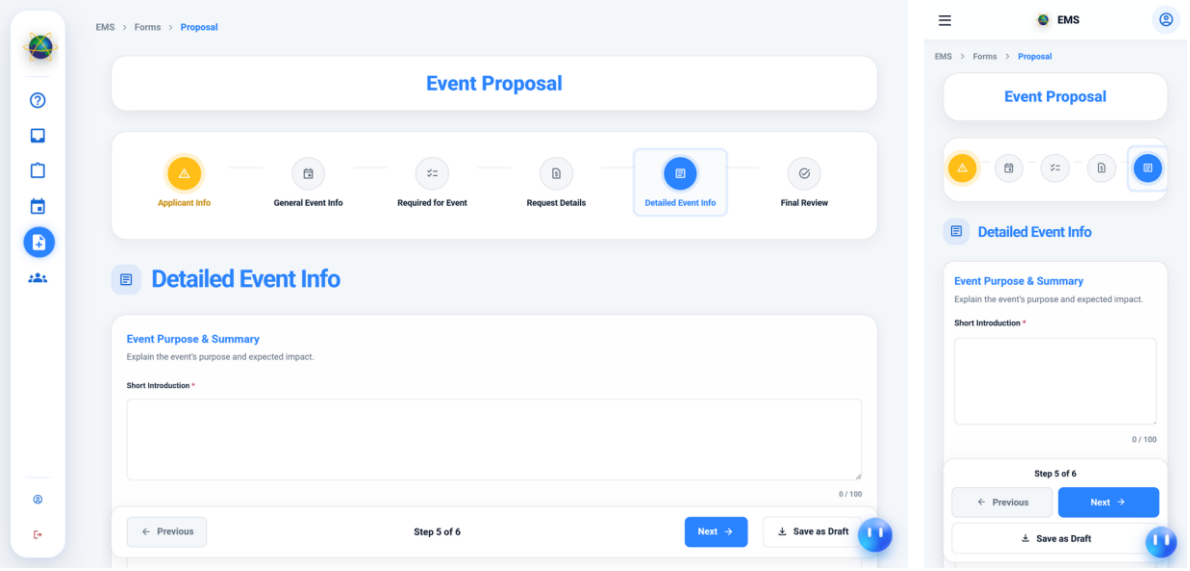


Figure 77: Schedule, agenda, organisers and guests

The fifth step covers the material a reviewer needs in order to judge the proposal rather than to fulfil it: the schedule, a brief agenda, the organising team, the important people expected, the general guests and the topics to be discussed. It is placed after the requirements because it is the longest step and the least likely to change the routing; an applicant who abandons the form here has still recorded everything the earlier steps need in order to be resumed.

4.5.4.6 Proposal Form — Step 6: Final Review

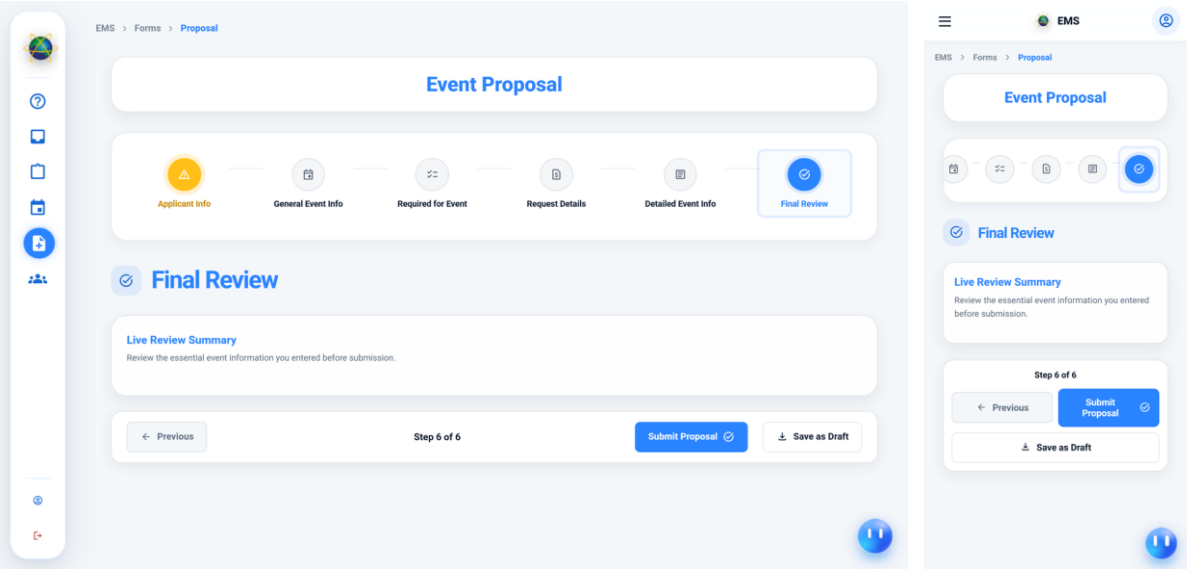


Figure 78: Read-back of the complete proposal before submission

The final step reads the whole proposal back before it is submitted. Every section is shown as it will be seen by the reviewer, with a route back to the step that produced it. The read-back exists because submission is the point at which the proposal leaves the applicant's control: once submitted it enters the approval chain and can be changed only by being sent back, so the last opportunity to correct it cheaply is here.

4.5.4.7 Proposal Form — Validation Errors

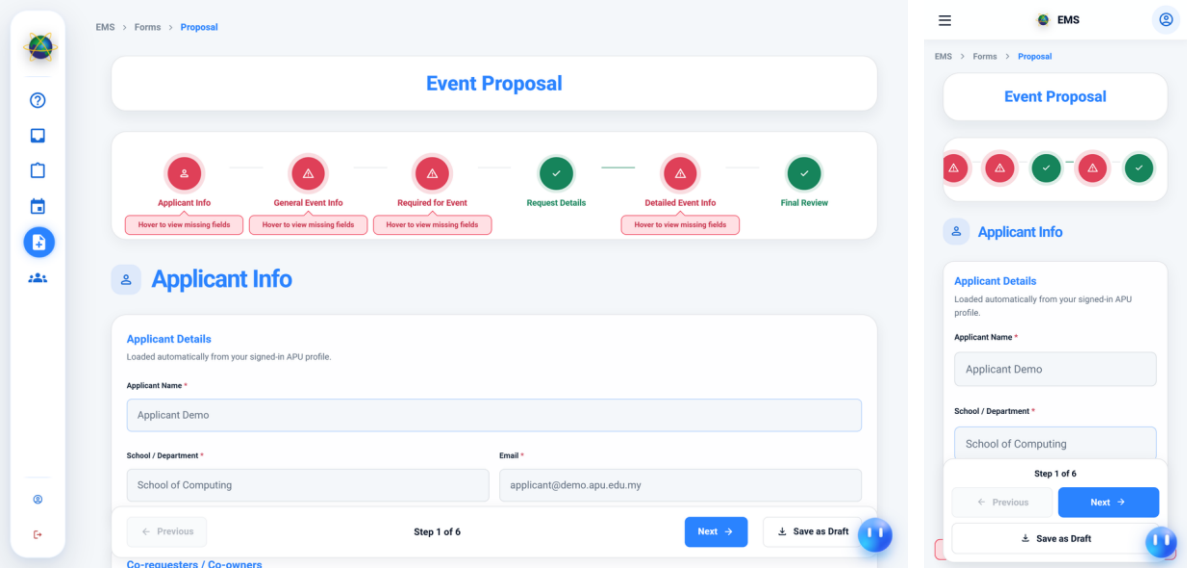


Figure 79: The form after a submission attempt with required fields unsatisfied

This figure shows the form refusing an incomplete submission. The invalid fields are marked in place with the reason stated beneath them, and the step indicator identifies which

step holds the problem so that the applicant is not left to search six steps for it. The behaviour illustrated here is the client half of a rule that is enforced twice: the same constraints are applied again on the server when the submission arrives, because a client that has been modified, or a request issued directly against the API, must not be able to introduce a proposal the workflow cannot route.

4.5.5 Proposal Tracking and Review

Once a proposal has been submitted it becomes a shared object with two audiences. The applicant needs to know where it has reached and what, if anything, is expected of them; the reviewer at the current stage needs enough of it to decide. This group covers both, and the two views are the same underlying record rendered under different scopes rather than two copies of it.

The review screen is presented three times in this group at different levels of detail — as a reviewer sees it, as a department head sees it, and with the decision panel isolated — because the contrast is the argument. A single page that changes what it shows according to who opened it is the mechanism by which a department is prevented from acting on requirements that are not theirs.

4.5.5.1 Draft Proposals

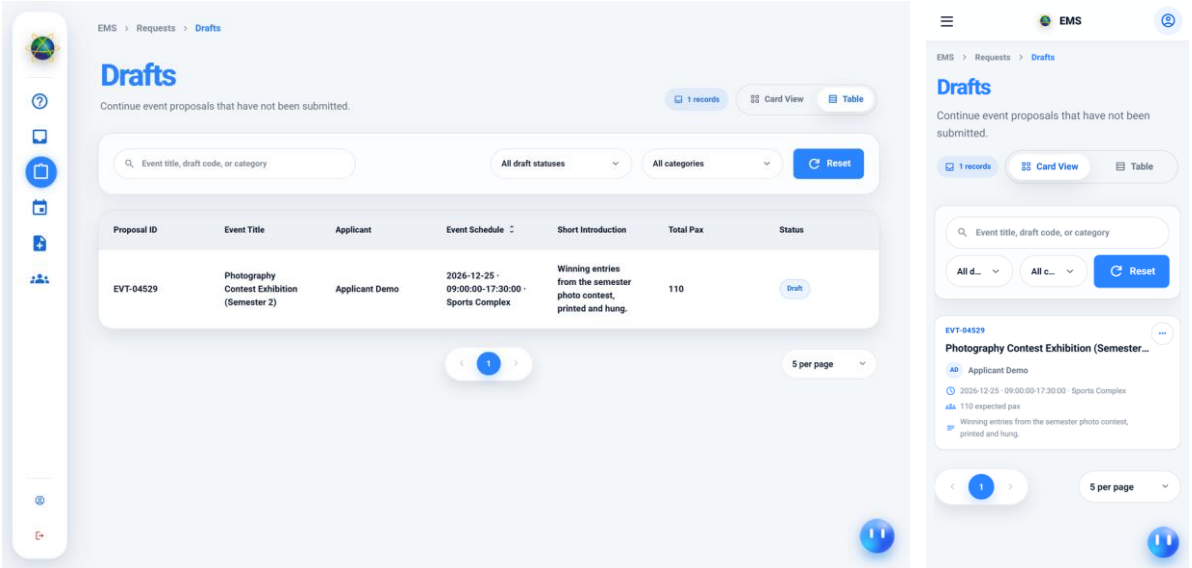


Figure 80: Saved but unsubmitted proposals awaiting resumption

Drafts are proposals that have been saved but not submitted. They are visible only to their owner and the co-owners named at step one, and they occupy no place in any reviewer's queue, because a draft has not yet asked anything of anybody. Keeping drafts as ordinary rows

in the request table with a distinct status, rather than in a separate store, means that submitting one is a status transition rather than a copy, and the identifier the applicant has been quoting does not change at the moment it matters most.

4.5.5.2 Created by Me — Status Tracking

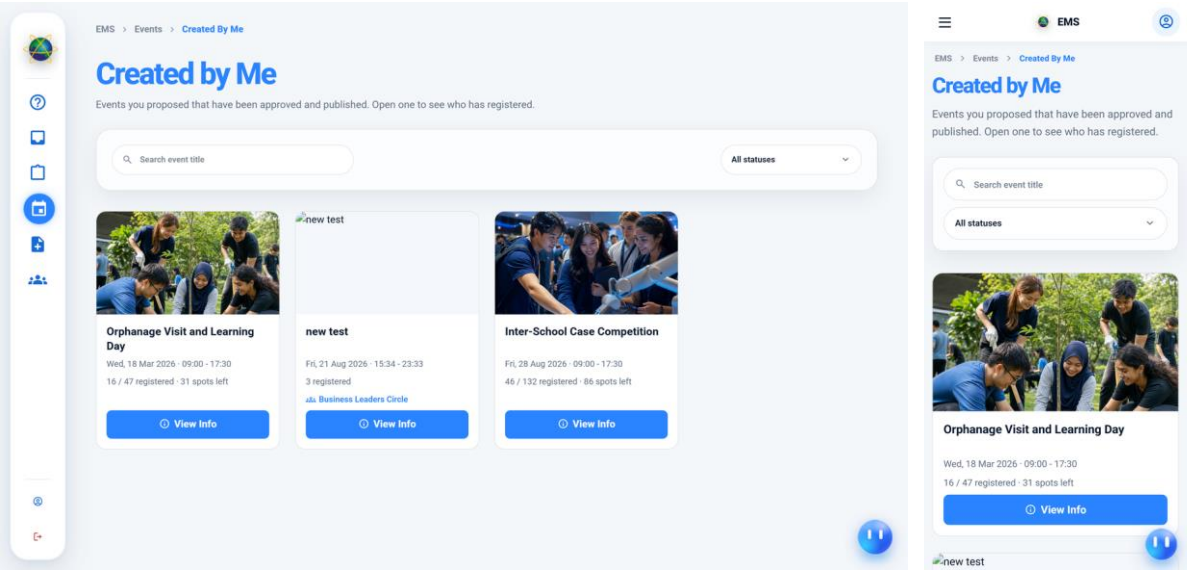


Figure 81: The applicant's view of every proposal they have submitted

This is the applicant's answer to the question Chapter 3 found was hardest to get answered: where has my request reached. Each submitted proposal is listed with the stage it currently occupies, so the applicant can distinguish a proposal waiting on a Head of School from one already with the departments. The stage shown is read from the request itself rather than inferred, and it is the same value the reviewer's queue is built from, so the two cannot disagree.

Where a proposal has required a payment, the receipt is reachable from this screen. The document is fetched as an authenticated request and opened from the response rather than linked directly, so that possession of a file name is not by itself sufficient to retrieve somebody's receipt.

4.5.5.3 Proposal Review — Reviewer View

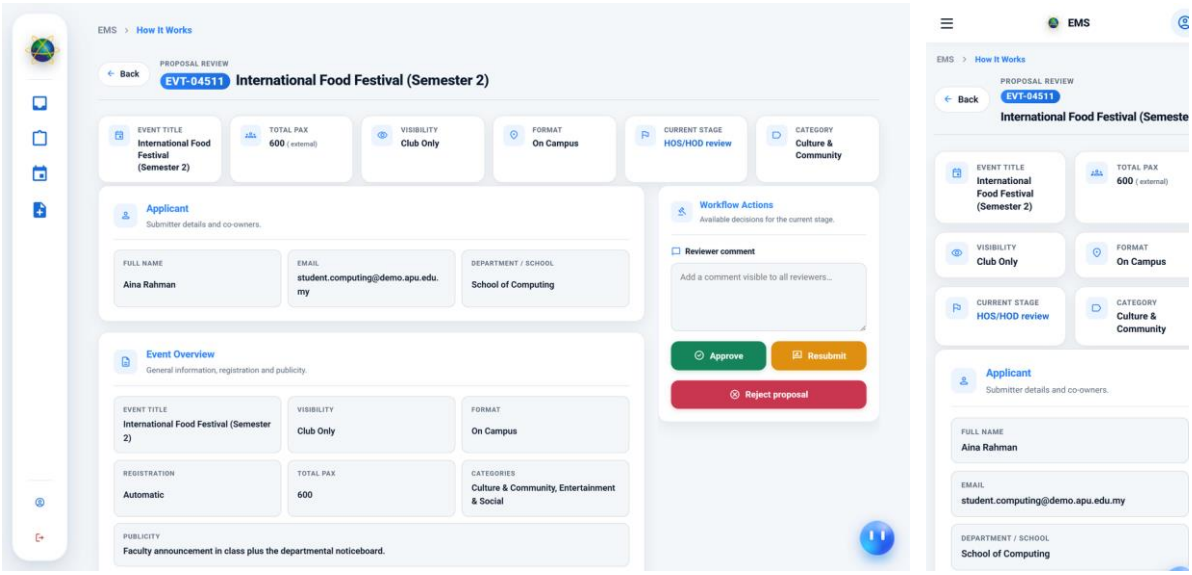


Figure 82: A submitted proposal as its reviewing Head of School sees it

The review page opens on a summary strip and then presents the proposal in full, with the decision panel held to the right where it remains reachable as the reader scrolls. The proposal captured here is an international food festival for six hundred external attendees, restricted to club visibility, currently at the Head of School and Head of Department stage. A reviewer at this stage sees the whole proposal because their decision is about the event as a whole: whether it should happen at all, at this scale, under this unit's name.

That is precisely what distinguishes this view from the next one. The authority to approve is not a property of the person but of the pairing of the person, the record and the stage it currently occupies, and it is checked on the server after the record has been loaded rather than by a role test on the route.

4.5.5.4 Proposal Review — Department View

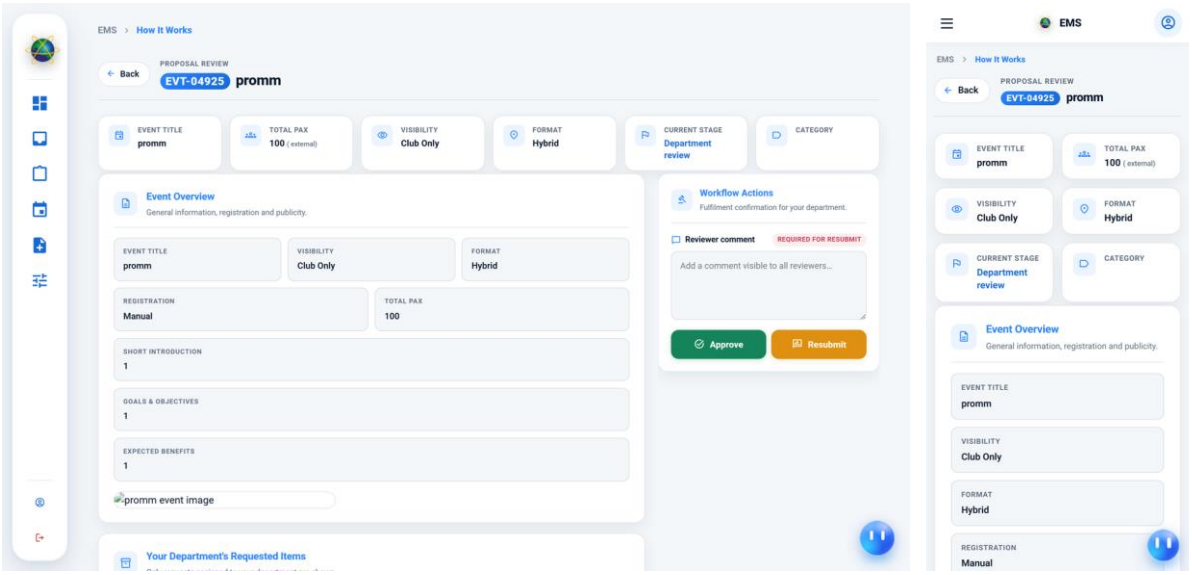


Figure 83: The same proposal as its Head of Logistics sees it

This is the same page and the same proposal, opened by a department head. The content has narrowed to that department's own requested items and the actions offered are the department's own: confirm fulfilment, or send the request back. A Head of Logistics has no opinion to give on whether the event should take place — that decision was taken at the previous stage — and no business seeing the transport arrangements. Presenting one page under two scopes rather than building two pages means a change to how a proposal is displayed cannot accidentally be applied to one audience and not the other.

4.5.5.5 Proposal Review — Summary Strip

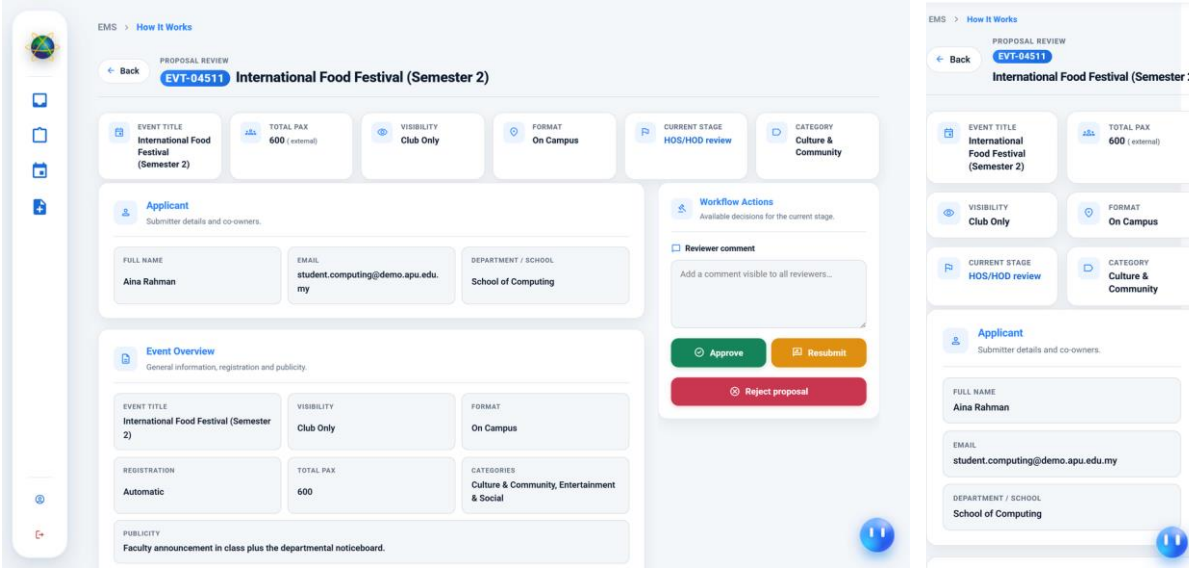


Figure 84: The six-card summary at the head of the review page

The summary strip carries the six facts that most often decide a review: the event title, the total number of attendees and how many are external, the visibility, the format, the current stage and the category. It is shown separately here because it is the part of the screen a reviewer reads first and, for a straightforward proposal, sometimes the only part they need. The external attendee count is given its own place rather than being folded into the total because it is the figure that triggers the additional review stages described in Section 4.5.11.

4.5.5.6 Reject Proposal — Reason Required

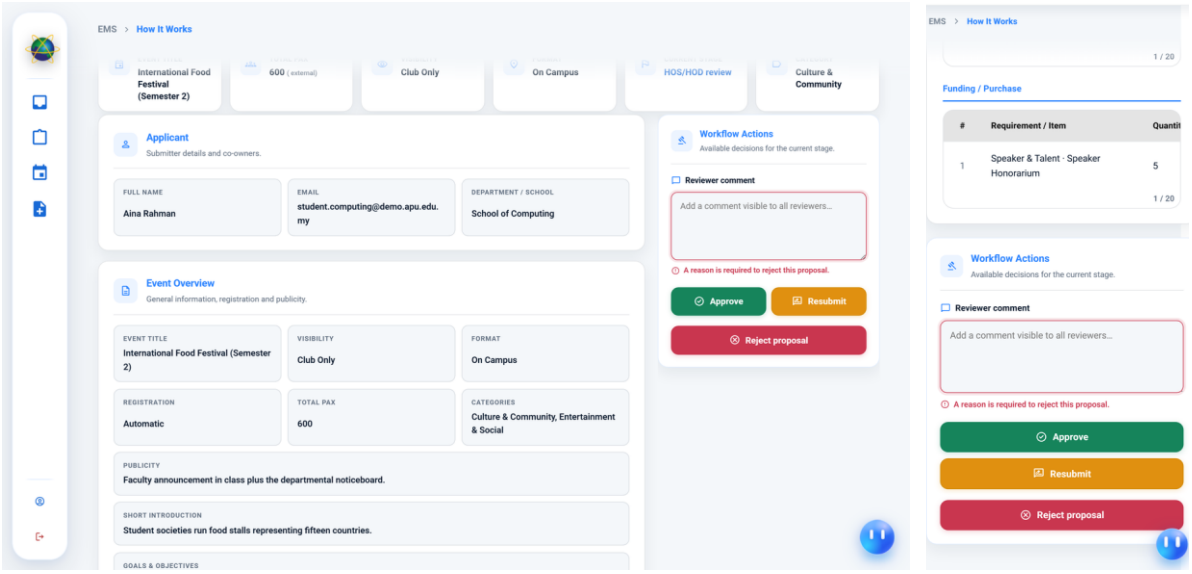


Figure 85: The decision panel refusing a rejection submitted without a comment

Pressing Reject with an empty comment produces the state shown here: the comment field outlined and the requirement stated in place. A rejection without a reason is of no use to the applicant, who must either resubmit or abandon the event, and it is of no use to the audit trail either, since the record would show that a decision was taken and nothing about why. The constraint is enforced on the server as well, where the comment is written into the workflow history in the same transaction as the status change.

The screen also shows what the system deliberately does not do. There is no confirmation dialogue between the reviewer and the decision. The validation happens in place, in the panel the reviewer is already looking at, because an extra modal adds a click without adding any information the reviewer did not already have.

4.5.5.7 Workflow Actions Panel

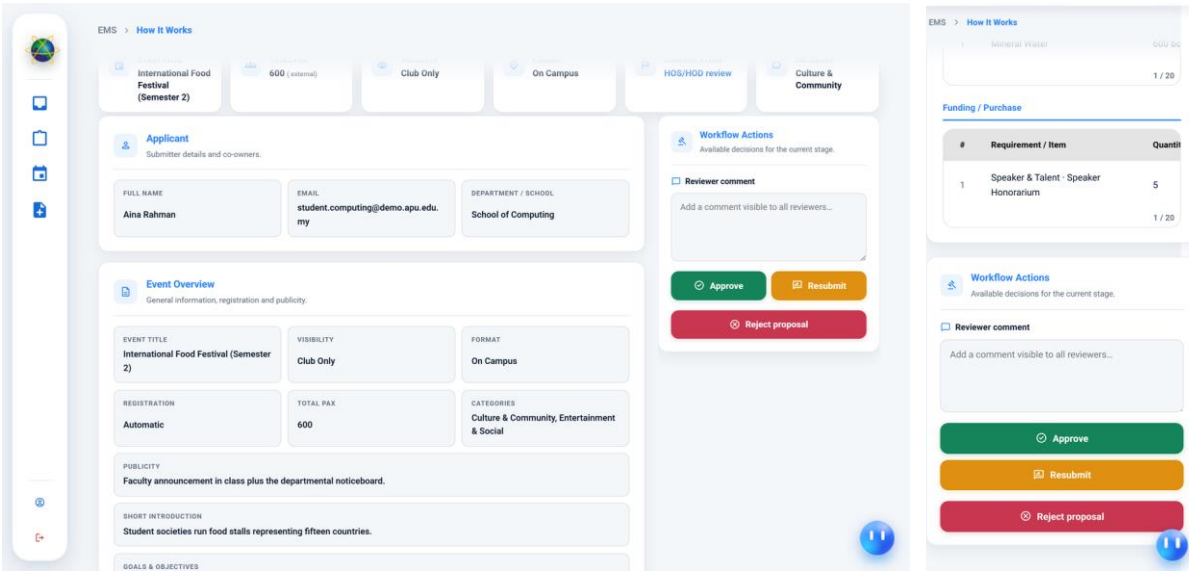


Figure 86: The three stage decisions available to a reviewer

The decision panel offers exactly three actions: approve, which advances the proposal to whatever stage the routing rules name next; resubmit, which returns it to the applicant with the comment attached and allows it to re-enter the chain at the point it left; and reject, which ends it. Send-back is offered as a first-class action rather than being achieved by rejecting and asking the applicant to start again, because a proposal that needs one figure corrected should not lose the approvals it has already collected.

4.5.6 Departmental Task Handling

Three screens in this group are the same shell. Inbox, Ongoing and History are one records component whose tab strip is computed from the viewer's page grants, and they divide the same records by what is expected of the viewer rather than by what kind of record it is: the Inbox holds what is waiting on this person to act, Ongoing holds what is in flight and needs nothing from them, and History holds what has settled. Organising the application this way means a Head of Department has one place to look each morning rather than one per record type.

4.5.6.1 Inbox — Action Queues

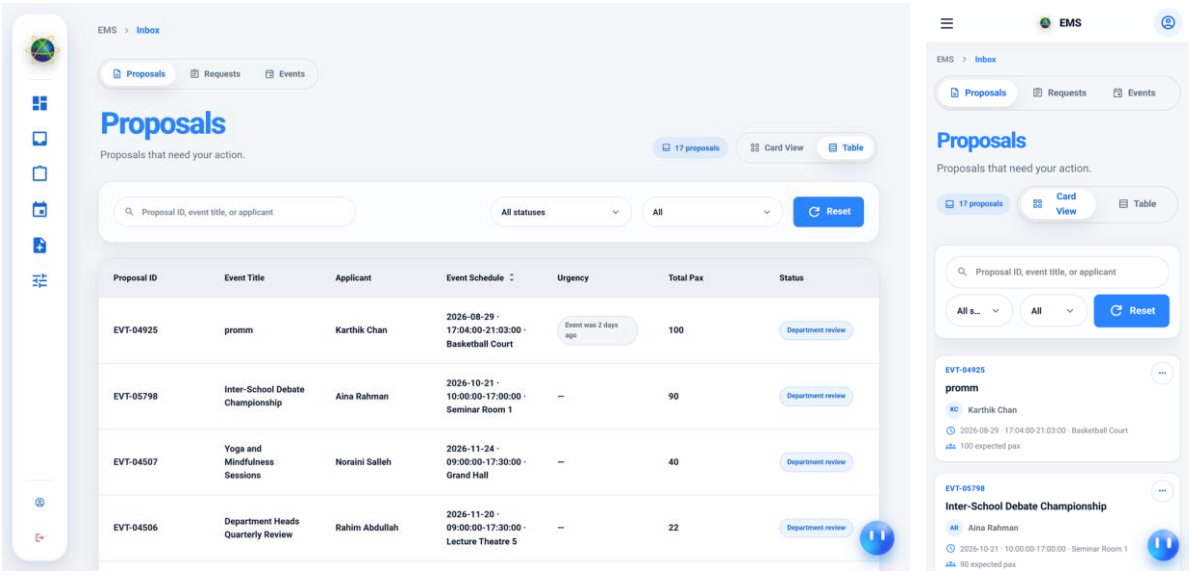


Figure 87: The action queue for a Head of Department, with the Proposals tab open

The Inbox is everything waiting on a decision from the account viewing it. The figure shows the Proposals queue for a Head of Logistics: seventeen proposals in a sortable table carrying the proposal identifier, event title, applicant, schedule, urgency, attendee count and current status, with a search field, a status filter and a card-or-table view toggle above it.

The tab strip beside it is the part worth reading carefully, because the Inbox can carry six queues in total and no single role holds all of them. Proposals holds those awaiting this reviewer's decision; Requests holds departmental service requests routed to the unit they head; Events holds event registrations awaiting an organiser's decision; Tasks holds the shared work pool for department and cafeteria staff; Clubs holds membership requests awaiting a club president; and President Change Requests holds handovers awaiting a Club Admin. The strip shown here is the one this account's grants produce, and the queues themselves appear in their own right in Sections 4.5.7, 4.5.8 and 4.5.10.

4.5.6.2 Staff Task Assignment

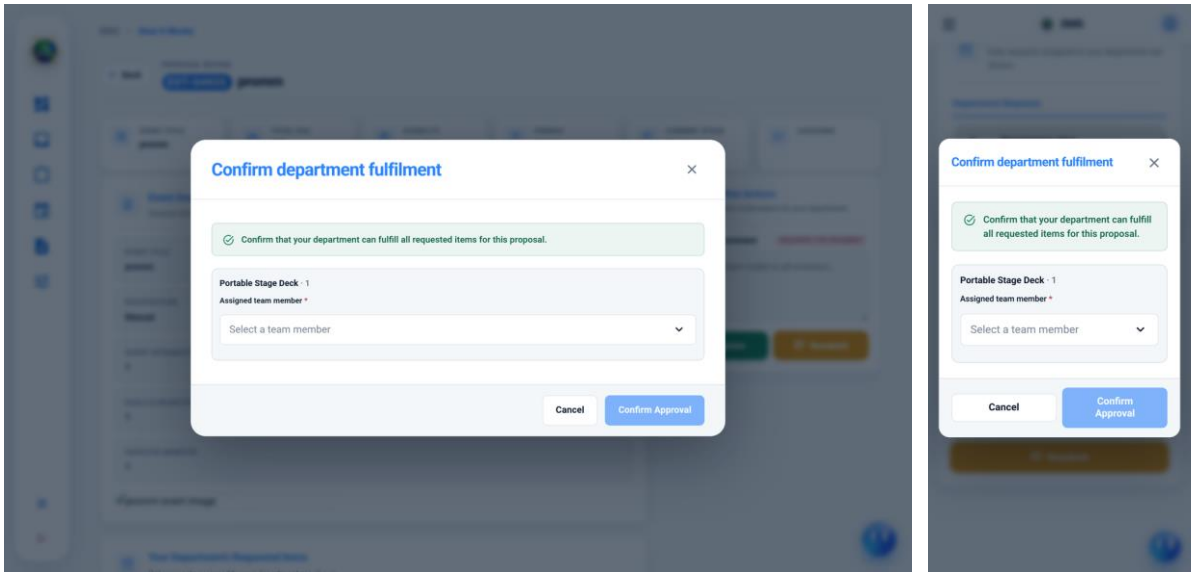


Figure 88: The confirmation dialogue in which departmental work is assigned

This dialogue is where departmental work is actually allocated, and it encodes a decision worth stating plainly: approving is assigning. A department head confirming fulfilment of a requested item — here a single portable stage deck — must name the team member who will carry it out before the confirmation control becomes available. The alternative arrangement, in which approval and assignment are separate steps, produces a class of work that has been agreed to and belongs to nobody, which is precisely the accountability gap Chapter 1 set out to close.

4.5.6.3 Ongoing Records Hub

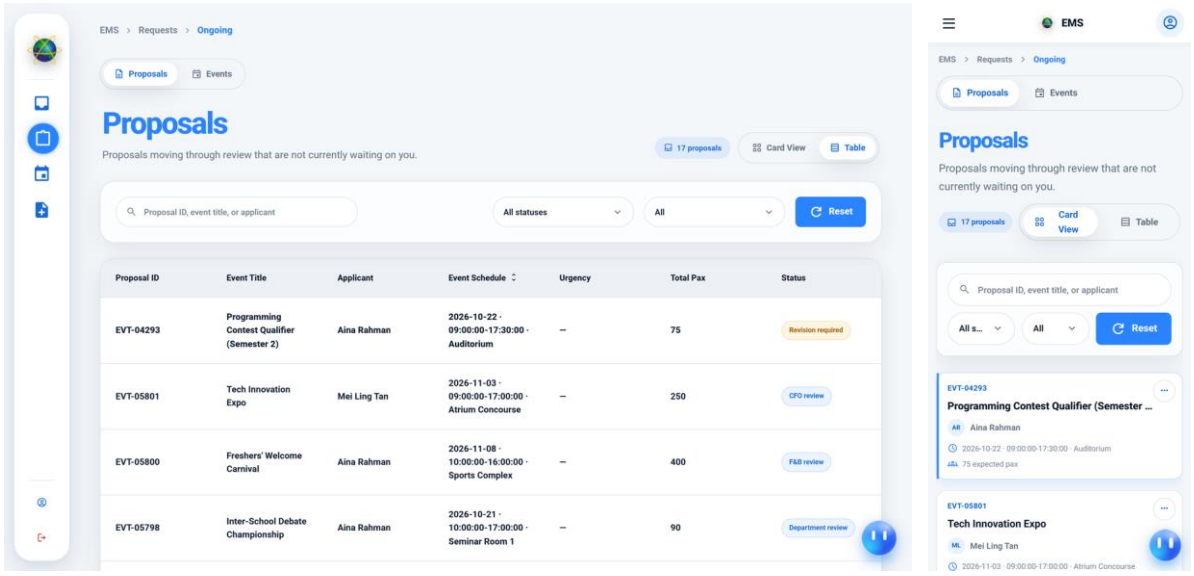


Figure 89: Work in flight that the viewer can see but need not act on

Ongoing holds records that are moving but are not waiting on the viewer. The figure shows the Proposals tab, holding proposals that have been submitted and are still travelling through approval; the same shell also carries Events, holding the viewer's own event registrations still awaiting a decision, and, for a student, Clubs, holding join requests still pending. Separating this from the Inbox is what keeps the Inbox meaningful: if a queue mixes what needs a decision with what merely needs watching, it stops being a list of work and becomes a list of everything.

4.5.6.4 History Records Hub

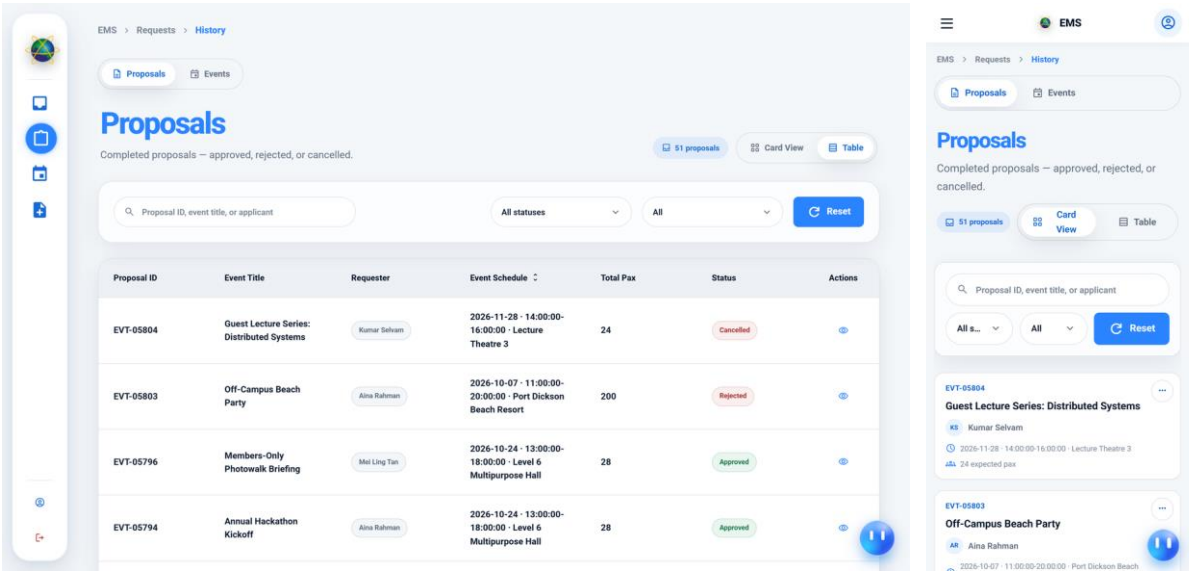


Figure 90: Settled records, retained as the audit trail

History is the settled counterpart of Ongoing. The Proposals tab shown here holds proposals that have been approved, rejected or cancelled, and the shell also carries Events for registrations confirmed and since concluded or turned down, Tasks for completed departmental work, Clubs for decided join requests and President Change Requests for completed handovers. Nothing is removed from the system when it concludes; every stage transition remains recorded with its actor, its timestamp and any comment given, which is what allows a decision taken months earlier to be accounted for.

4.5.7 Cafeteria Module

Catering is the one requirement in the system that is not fulfilled by a university department but by an outlet with its own staff, its own menu and its own manager. It therefore has a module rather than a task type. The module has three audiences — the staff who prepare

orders, the manager who runs an outlet, and the administrator who oversees all of them — and the screens are grouped here in that order.

4.5.7.1 Cafeteria Staff Tasks

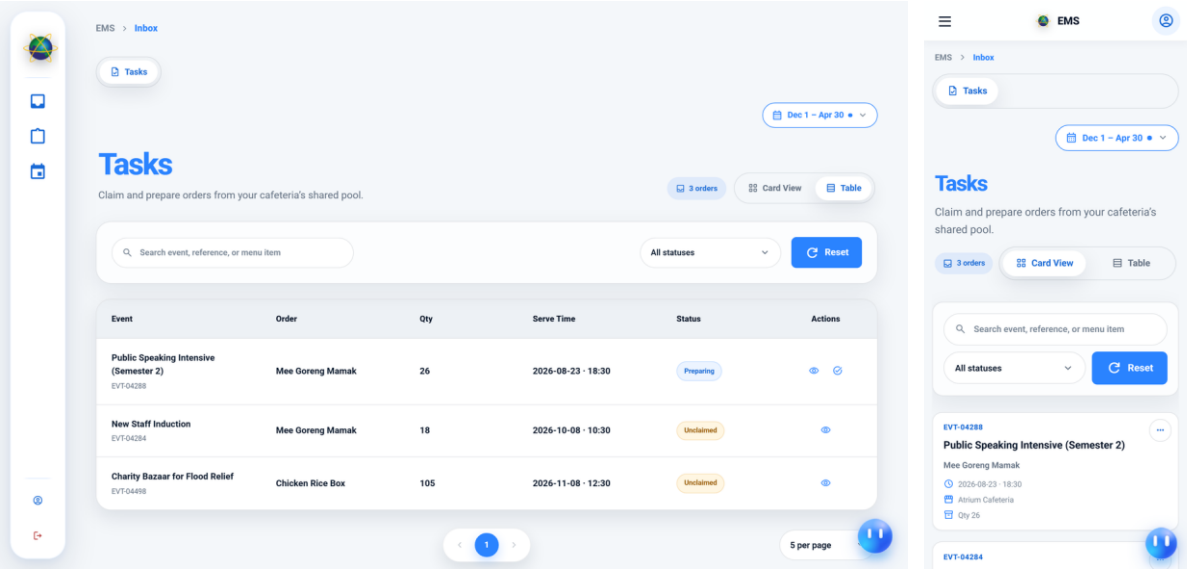


Figure 91: The shared-pool order queue as seen by cafeteria staff

Cafeteria orders are not assigned to an individual when they arrive. They are placed in the outlet's shared pool, from which any member of that outlet's staff may claim one, and the claim is what creates the assignment. The screen shows three orders in the pool with their event code, menu item, serving time, outlet and quantity. A shared pool suits catering for the reason it does not suit departmental work: an order is short-lived and any qualified member of the outlet can take it, whereas a logistics task may run for days and needs a named owner from the outset. The claim is recorded against the claiming user, so the pool is a queue rather than an absence of accountability.

4.5.7.2 My Menu Management

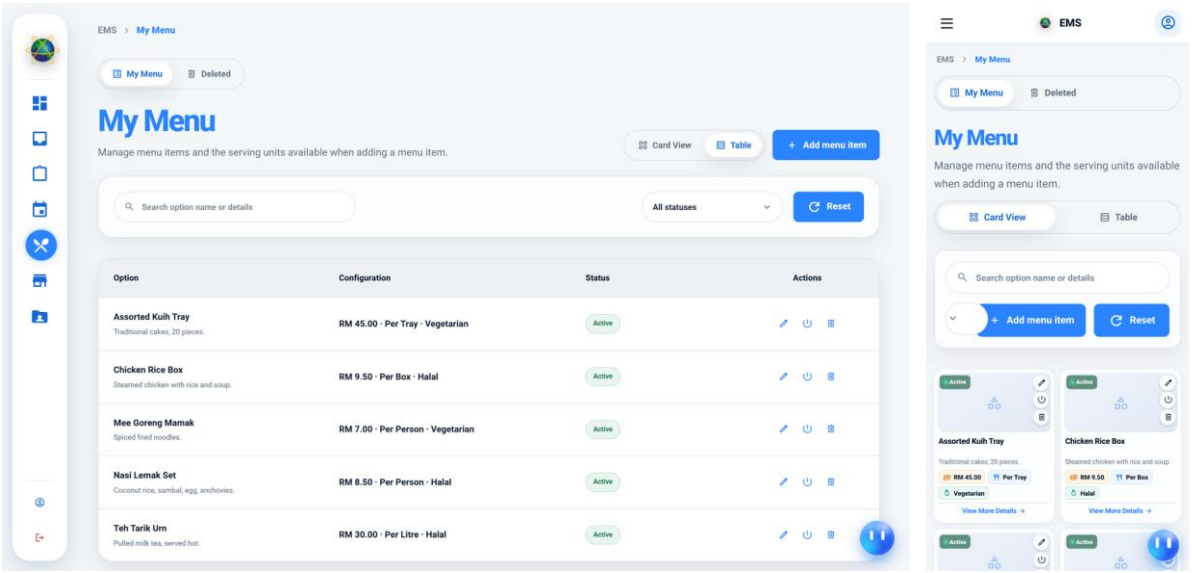


Figure 92: An outlet manager maintaining their own menu

A cafeteria manager maintains the menu for their own outlet: the dishes offered, their prices, the units they are served in and their dietary information. This catalogue is the source the proposal form reads when an applicant asks for food and beverage, so withdrawing a dish here removes it from future proposals without affecting orders already placed against it. The screen is scoped to the outlet the manager's role assignment names, which is the same unit-scoping mechanism applied throughout the system rather than a rule particular to catering.

4.5.7.3 Manage Cafeterias

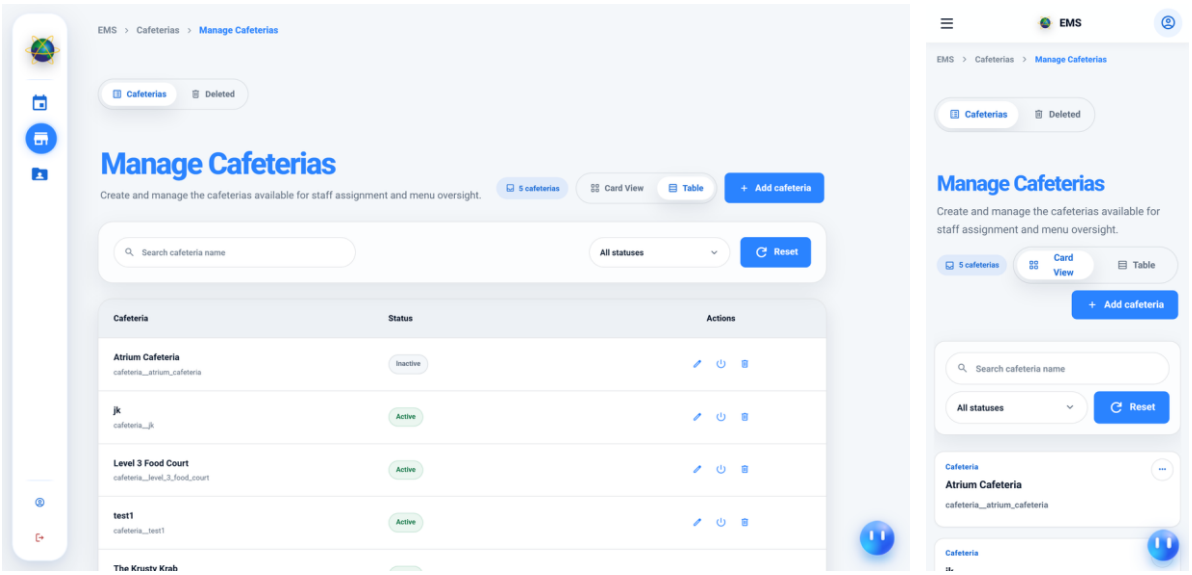


Figure 93: The catalogue of cafeteria outlets, held by the Cafeteria Admin

The outlet catalogue is administered centrally. An outlet may be created, edited, deactivated or deleted here, and the last two are deliberately different operations. Deactivating an outlet withdraws it from new proposals while leaving its history intact; deletion is refused outright while anything still depends on it, and the reasons for the refusal are stated rather than the control simply failing. That rule is applied consistently across every catalogue in the system and is described further in Section 4.5.11.

4.5.7.4 Cafeteria Staff Assignments

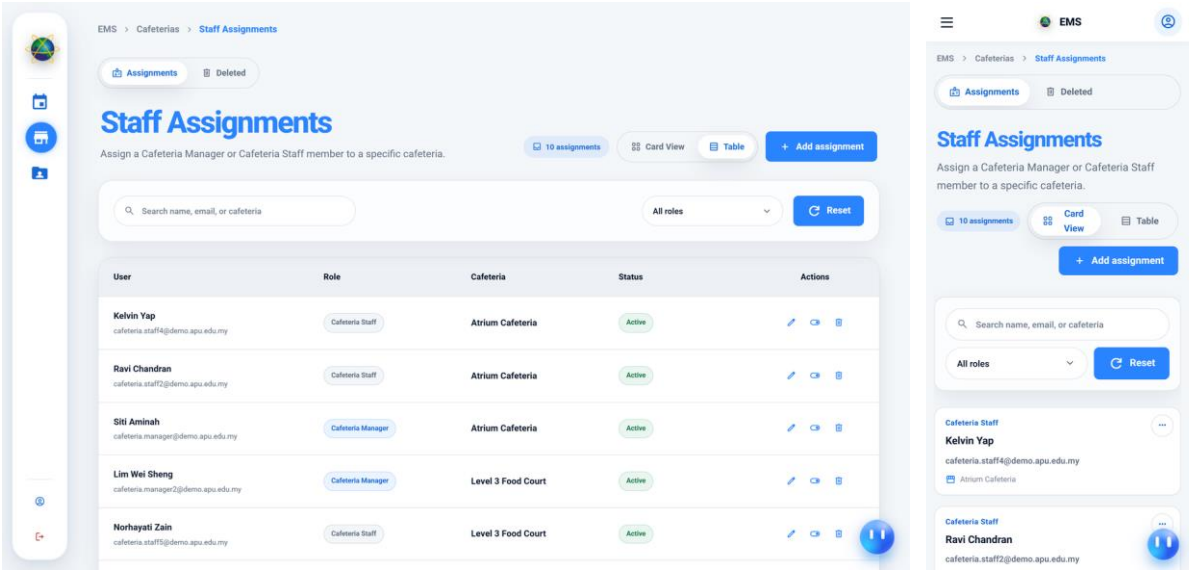


Figure 94: The posting of staff to outlets

This screen records which members of staff are posted to which outlet. An assignment is a role held within a unit, so the same person may hold a manager role at one outlet and nothing at another, and their authority follows the posting rather than the account. The screen also enforces a constraint that the workflow depends on: an outlet's last remaining manager can be neither removed nor deactivated, because an outlet with no manager can still receive orders and would have nobody able to act on them.

4.5.7.5 Menu Oversight

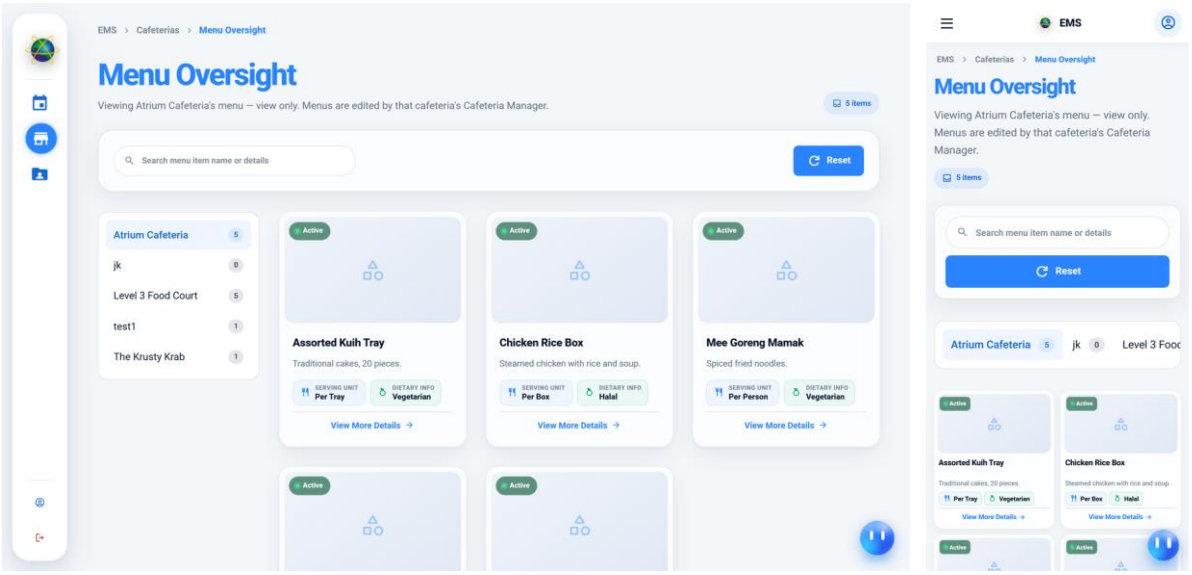


Figure 95: Every outlet's menu in one view, for administrative oversight

Menu oversight gathers the menus of all outlets into a single view. It is a read-heavy administrative screen rather than an editing surface: the manager of each outlet remains the author of their own menu, and the administrator's interest is in comparison — whether prices are consistent, whether dietary information has been supplied, whether an outlet has been left with nothing on offer.

4.5.7.6 My Staff

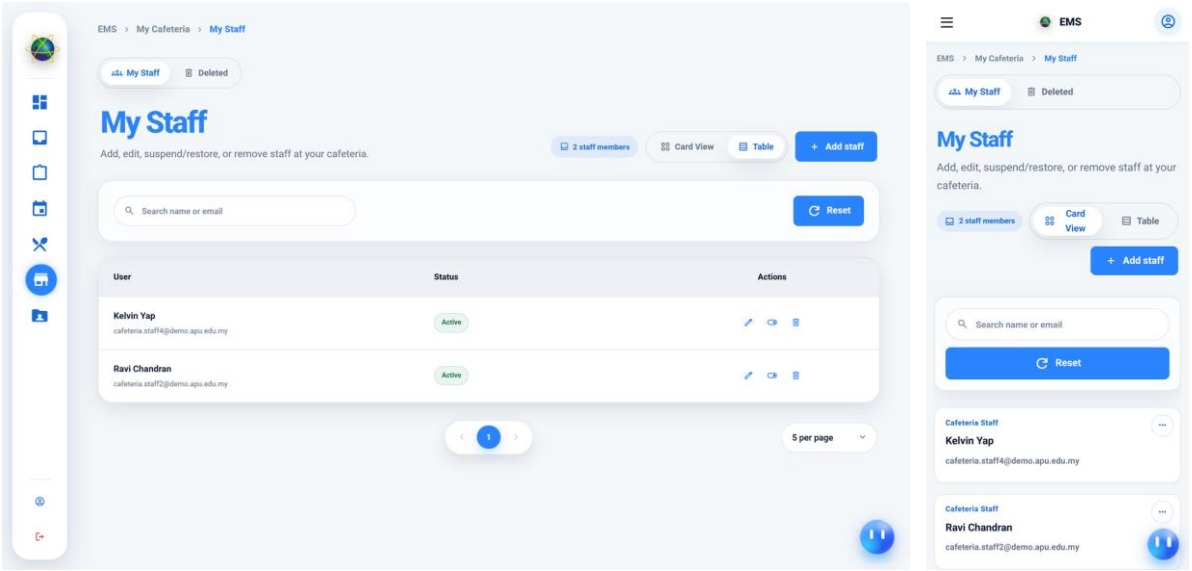


Figure 96: An outlet manager's own team roster

The manager's roster shows the staff posted to their outlet, together with the status of each. It is the manager-scoped counterpart of the administrator's assignment screen: a manager

may see and act on their own team but cannot post staff to an outlet they do not run, and the restriction is applied in the query rather than by hiding controls.

4.5.7.7 Staff Action History

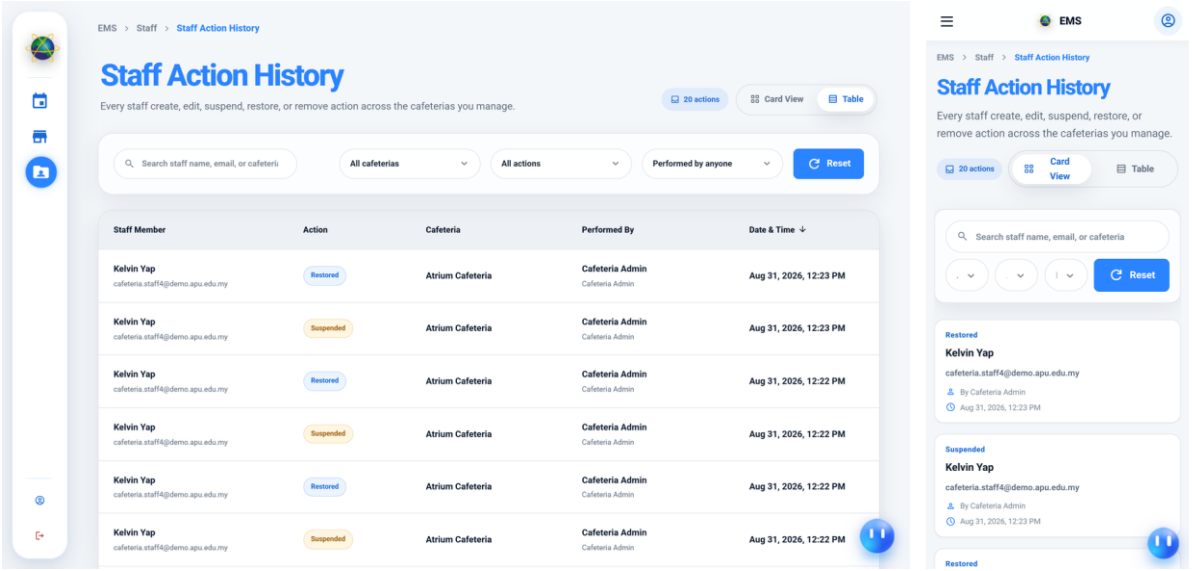


Figure 97: The audit log of cafeteria staff changes

Every change to a cafeteria posting — an addition, a suspension, a removal — is written to a dedicated audit log and surfaced here. The log is append-only and is written in the same transaction as the change it records, so a posting cannot be altered without the record of the alteration being created. This is the same guarantee applied to proposal stage transitions, and it is what makes the History screens in Section 4.5.6 trustworthy rather than merely informative.

4.5.8 Clubs Module

The clubs module is the community half of the platform. It exists because a large share of campus events are organised by student societies rather than by departments, and a system that could route a proposal but could not say who was entitled to raise one on a club's behalf would leave that share unmanaged. Membership, leadership and the handover of leadership are therefore modelled explicitly rather than being treated as attributes of a user.

4.5.8.1 Discover Clubs

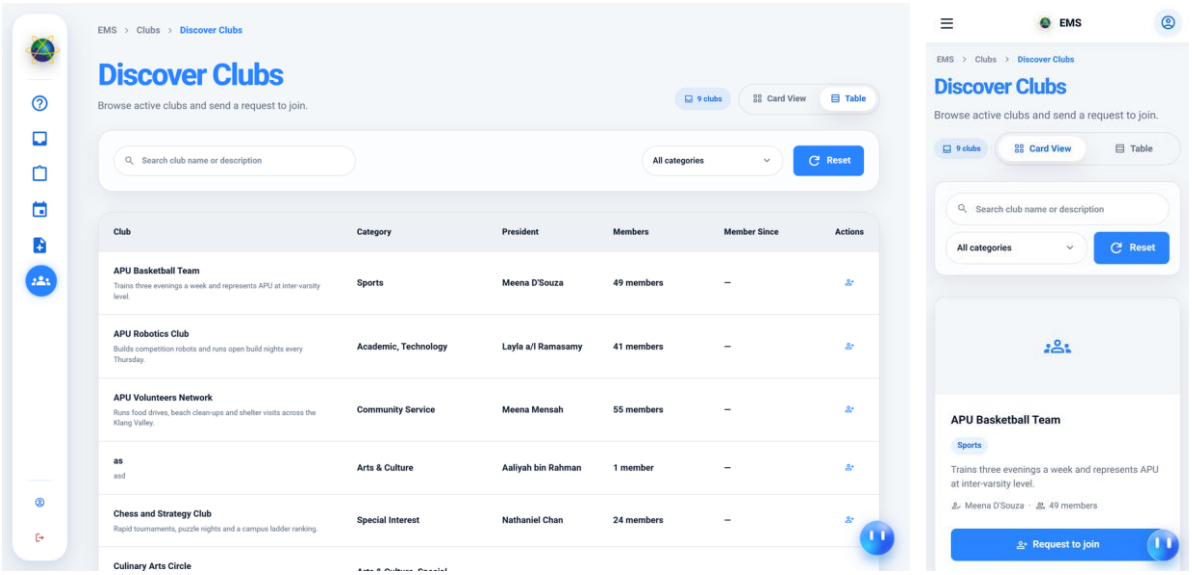


Figure 98: The club directory browsed by a student

The directory lists the clubs a student may join, organised by the category taxonomy maintained on the last screen in this group. A club that has been deactivated does not appear here, and neither does one that has been archived, because both states mean the club is not accepting members even though its records are retained. Joining is requested rather than taken: selecting a club opens a request that carries a stated reason and lands with the club's president.

4.5.8.2 My Clubs

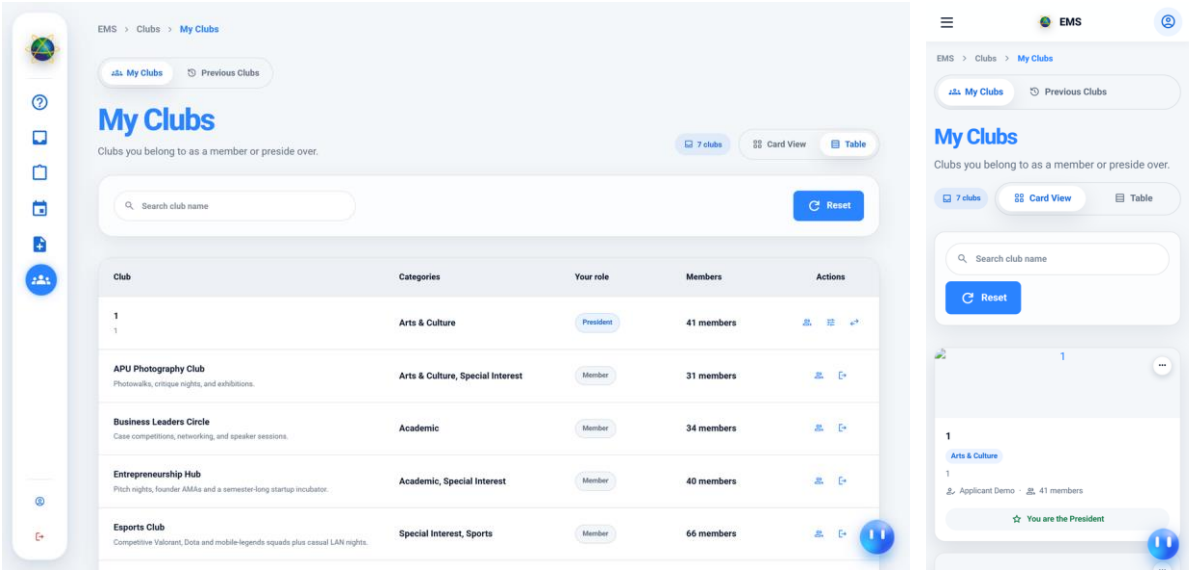


Figure 99: The clubs a student belongs to or presides over

My Clubs shows the memberships the signed-in student holds, each card offering a view of the roster and, for a president, the management actions that come with the office. The

president of a club is not an attribute stored on the club row alone; it is a role assignment, which is what allows presidency to be transferred through a reviewed process rather than by editing a field.

Where a handover has already been requested for a club, the control that would start one reports the request as pending instead of offering to raise a second. The server refuses a duplicate in any case, but a control that invites an action the server will reject is a defect in its own right, and the screen reads the pending state with the club rather than discovering it on submission.

4.5.8.3 Club Roster Modal

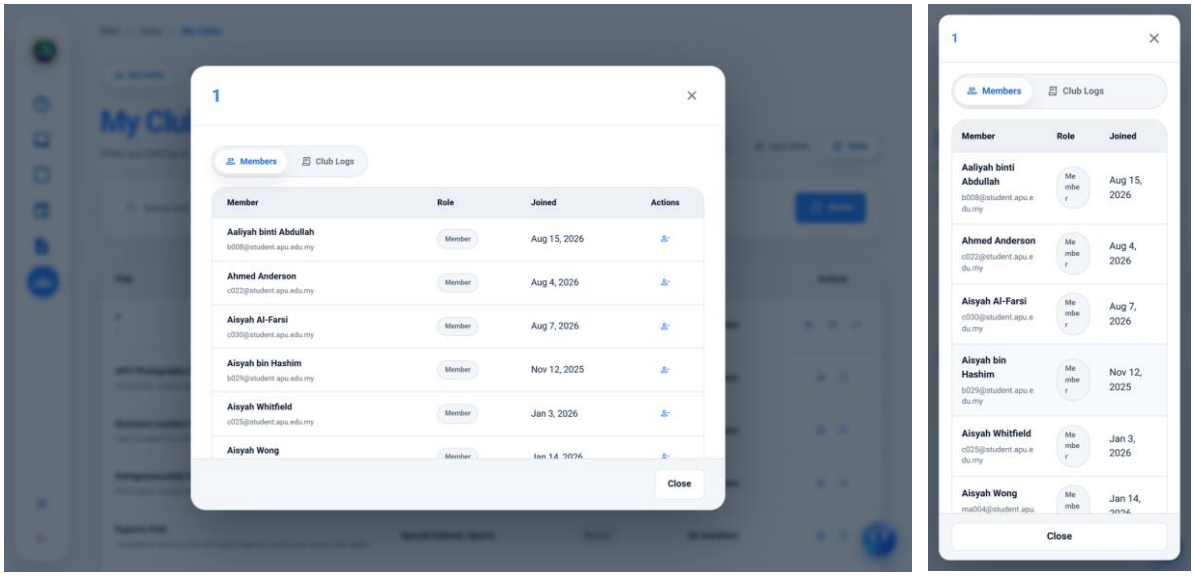


Figure 100: The membership roster of a club

The roster overlay lists the club's current members and, for a president, offers removal. Removing a member sends them an email notification, which is a deliberate choice: a membership that disappears without explanation is indistinguishable from a defect, and the person best placed to query the decision is the person it was taken about.

4.5.8.4 Club Join Requests

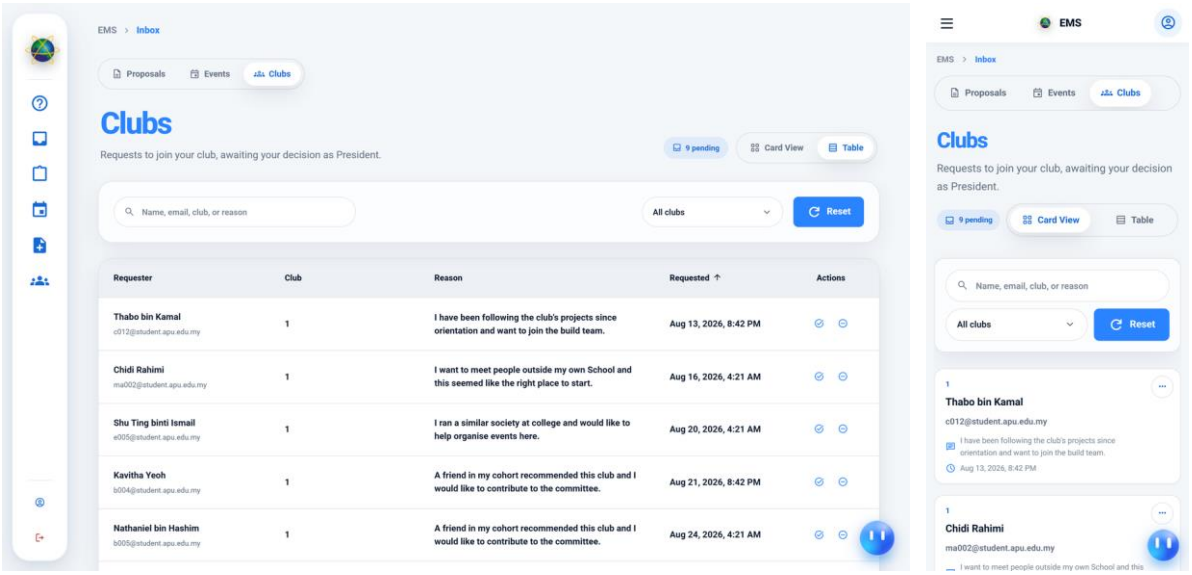


Figure 101: Pending membership requests awaiting a president's decision

Join requests reach the president through the same Inbox shell as every other pending decision, on its Clubs tab. Each request carries the applicant's stated reason. The reason field is not decorative: without it a president is presented with a list of names and no basis on which to decide, which in practice produces either indiscriminate approval or none at all.

4.5.8.5 President Change Requests

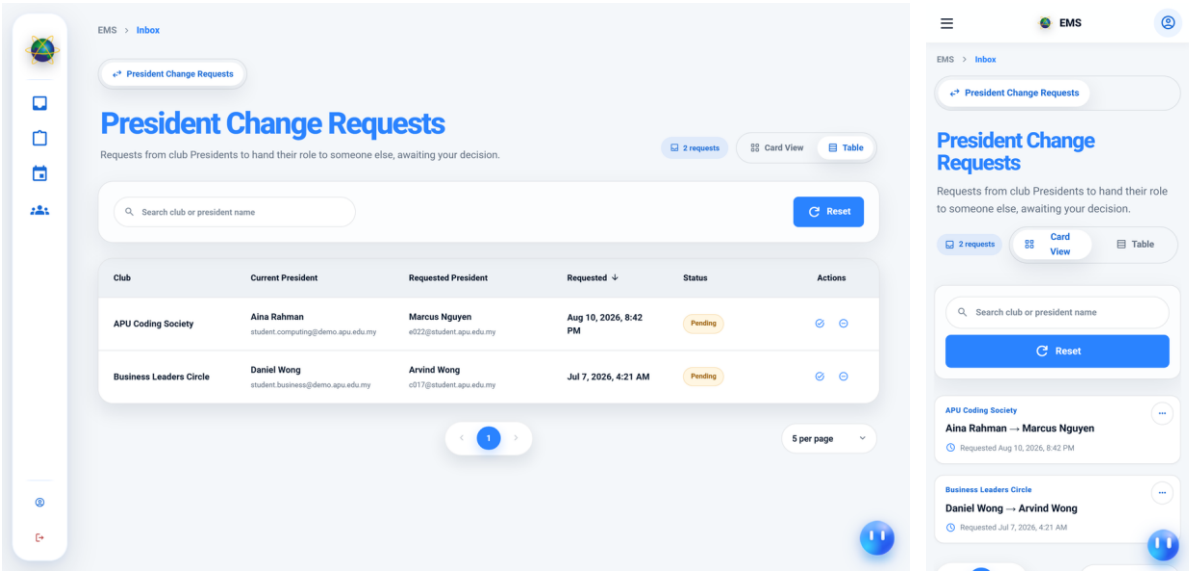


Figure 102: Presidency handovers awaiting administrative approval

A change of president is raised as a request and decided by a Club Admin rather than being performed by the outgoing president directly. The office carries the authority to admit and remove members and to raise proposals in the club's name, so transferring it is an

administrative act with a record, not a setting. The queue shown here holds the requests awaiting that decision.

4.5.8.6 Manage Clubs

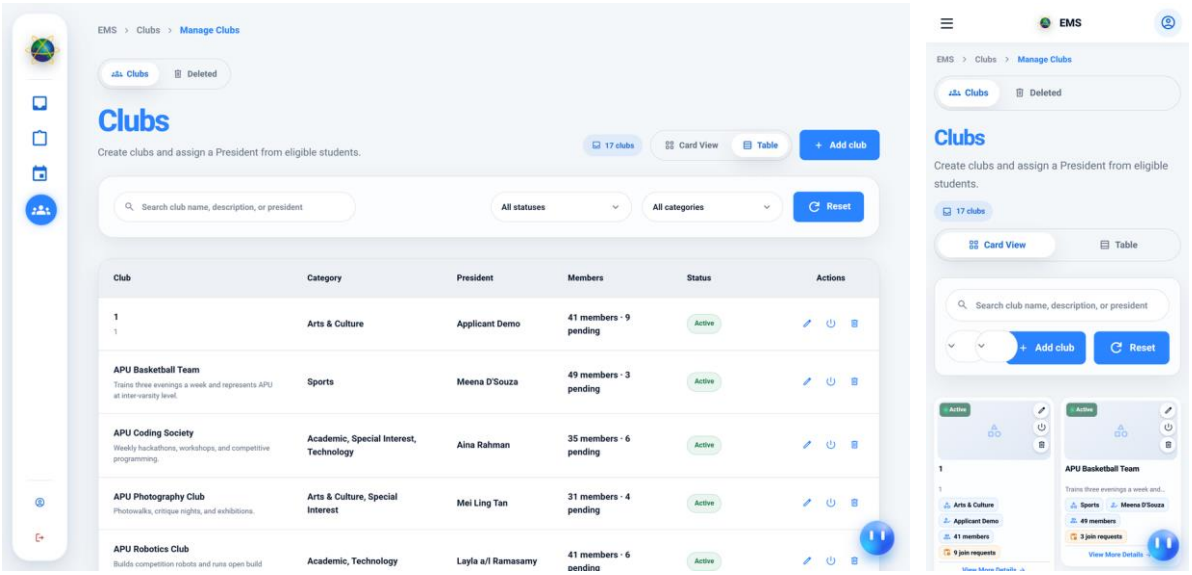


Figure 103: The administrative club catalogue

The Club Admin maintains the catalogue itself: creating clubs, editing their details, deactivating them and, where nothing depends on them, deleting them. Deletion is checked before it is offered. A club with members, with events raised in its name or with a membership history cannot be deleted at all, and the screen states which of those is the obstacle rather than reporting a generic failure.

4.5.8.7 Club Categories

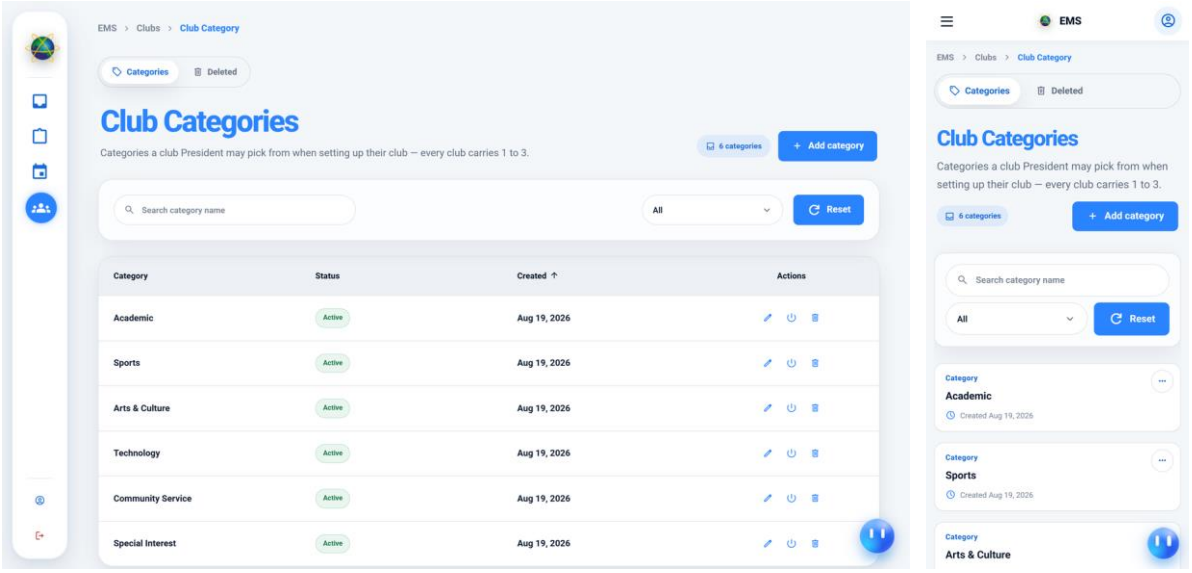


Figure 104: The category taxonomy under which clubs are classified

Categories are the taxonomy the discovery directory is organised by. The table filters by status and offers an All view alongside the active and inactive ones, so an administrator can see the whole taxonomy including categories that have been withdrawn from use but are still attached to existing clubs. As elsewhere, a category that is in use is deactivated rather than deleted, which withdraws it from new classifications while leaving the clubs already classified under it intact.

4.5.9 Role Dashboards

Every dashboard in the system is one component. It does not know what a Head of Department is, or a CFO; it walks a profile document supplied by the server that names which measures to compute and how to present them, and it renders whatever that document describes. Adding a dashboard for a new role is therefore a server-side entry rather than a new route, a new template and a new set of queries.

Three of the six are presented here. The four departmental dashboards are the same screen reading different units' numbers, so one stands for all of them; the CFO and cafeteria manager variants measure genuinely different things and are shown in their own right.

4.5.9.1 Dashboard — Head of Department

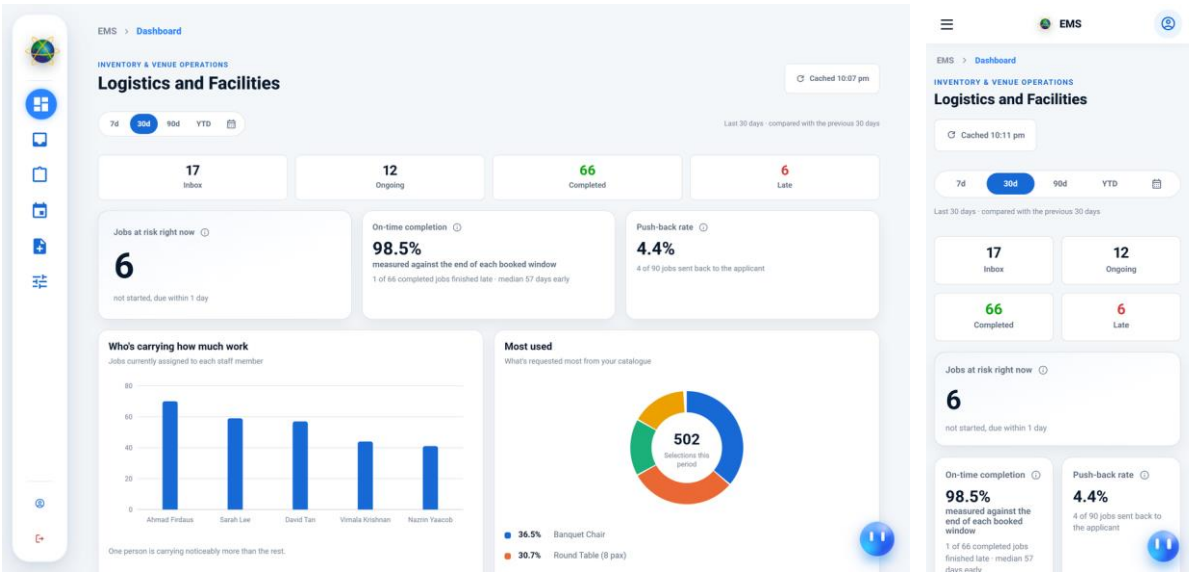


Figure 105: Departmental workload dashboard, shown for Logistics

The departmental dashboard answers the question a department head asks at the start of a day: what is outstanding, what is late, and who is carrying it. A period selector governs the whole page, followed by counters for the inbox, work in flight, completed work and late work,

a panel naming the jobs currently at risk, an on-time completion rate, a push-back rate, a bar chart of load per staff member and a distribution over the department's own catalogue.

This same screen serves the Heads of Logistics, Food and Beverage, Audio Visual and Transport. Only the unit name and the figures change, because the measures are computed against the unit named on the viewer's role assignment rather than selected by them. The load chart is the one panel that most repays the design: it is built from the assignments made in the confirmation dialogue of Section 4.5.6, so an uneven distribution is traceable to specific decisions rather than being an unattributable aggregate.

4.5.9.2 Dashboard — Chief Financial Officer

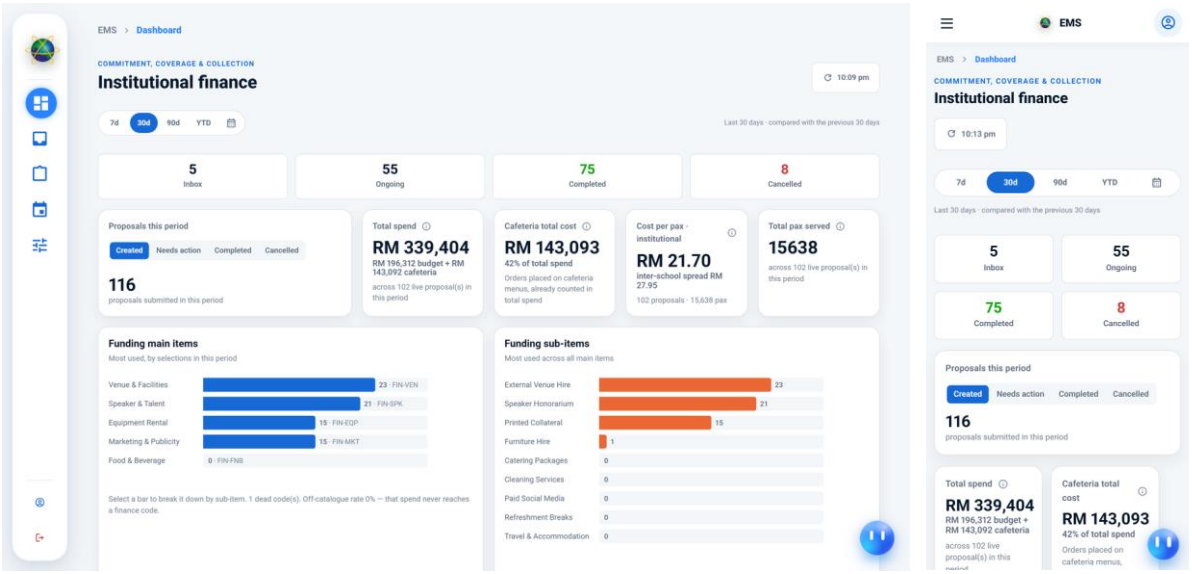


Figure 106: Institutional finance dashboard covering commitment, coverage and collection

The finance dashboard measures money rather than workload, and its panels have no counterpart on the departmental view. Alongside the queue counters it reports total committed spend split between budget and catering, catering cost as a share of the total, cost per attendee, total attendees served, a breakdown by funding item and the time proposals spend waiting at each approval gate.

The dwell-time panel is included because it measures the workflow itself rather than its output. Chapter 1 identified delay with no identifiable cause as a principal complaint about the existing arrangements, and a figure showing how long proposals wait at each gate turns that complaint into something that can be acted on.

4.5.9.3 Dashboard — Cafeteria Manager

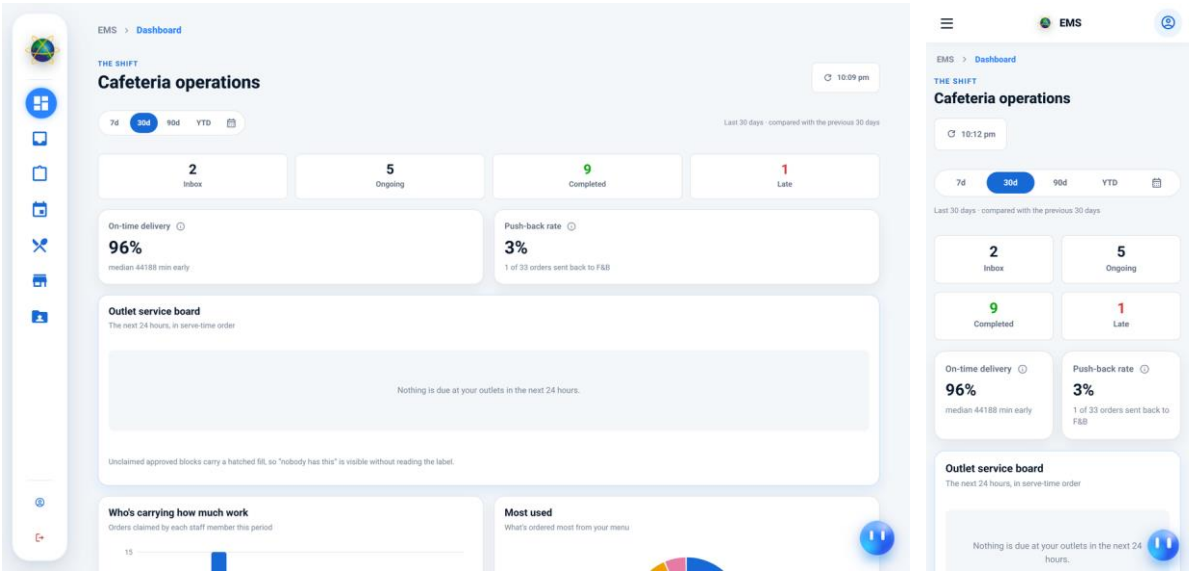


Figure 107: Outlet-level order and fulfilment metrics

The cafeteria manager's dashboard is scoped to a single outlet and measures order flow: what has been ordered, what has been claimed from the shared pool, what has been prepared and what has been delivered. It is the third distinct profile served by the same component, and it is the clearest demonstration that the profile mechanism is doing real work — an outlet has no concept of an approval gate and a department has no concept of a serving time, yet neither needs its own dashboard implementation.

4.5.10 Event Registrations

An approved proposal becomes a published event, and a published event collects registrations. Where the organiser chose automatic approval at step two of the proposal form, a registration is confirmed as it is made; where they chose manual approval, it waits for them here.

4.5.10.1 Event Registrations Hub

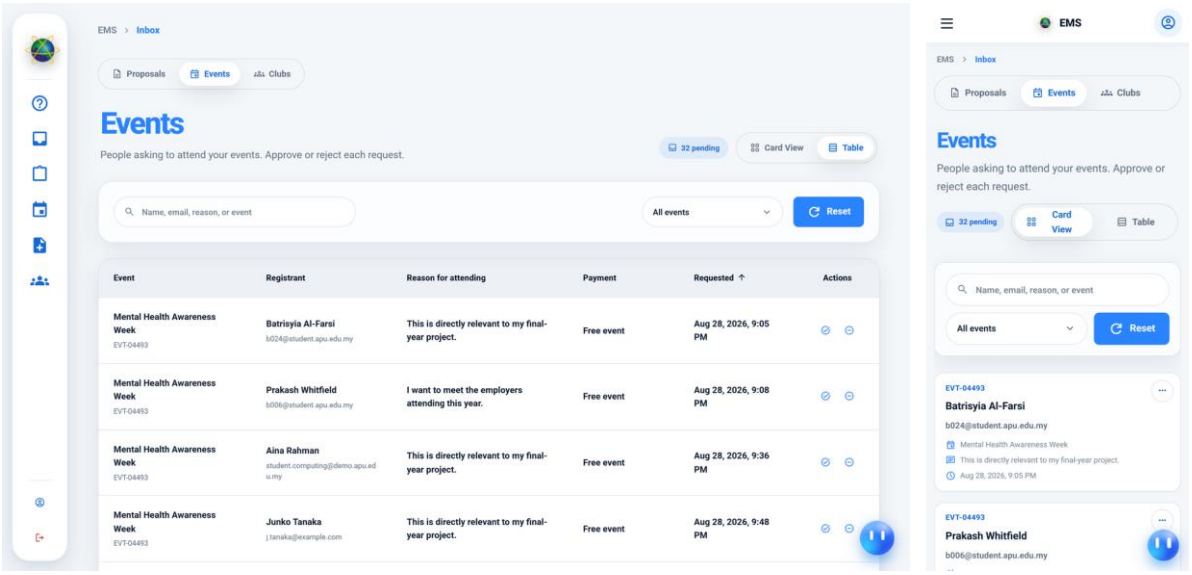


Figure 108: The organiser's queue of registrations awaiting a decision

This is the Events tab of the Inbox shell, holding registrations that need the organiser's decision, with search, an event filter and pagination. It is captured as a student who organises events, which matters: the queue is scoped to events the viewer created, so an account that has organised nothing correctly sees nothing here. Approving a registration confirms the attendee's place and moves the record into their Registered tab in Section 4.5.2; declining it moves the record to their History with the outcome recorded rather than removing it.

4.5.11 System Administration and Option Catalogues

Administration in this system is the work of keeping the configuration correct rather than of operating the workflow. It divides into two kinds. The first is central and belongs to the System Admin: the accounts, the organisational units, the roles and the page grants that decide what anybody can reach. The second is devolved: each option catalogue a proposal chooses from is owned by the department answerable for the service it describes, so the Head of Logistics maintains the logistics items and the CFO maintains the venues.

A single rule runs through all of these screens and is worth stating once rather than at each of them. Nothing that something else depends on can be deleted. Before any deletion is offered the server is asked whether the record has dependants and answers with the specific reasons it does; a record that fails the check is deactivated instead, which withdraws it from future use while leaving the history that refers to it intact. Where a deletion is allowed it is a

soft deletion: the record is archived, remains restorable, and is purged only after a retention period has elapsed. No screen in the system offers an immediate permanent deletion.

4.5.11.1 Users Directory

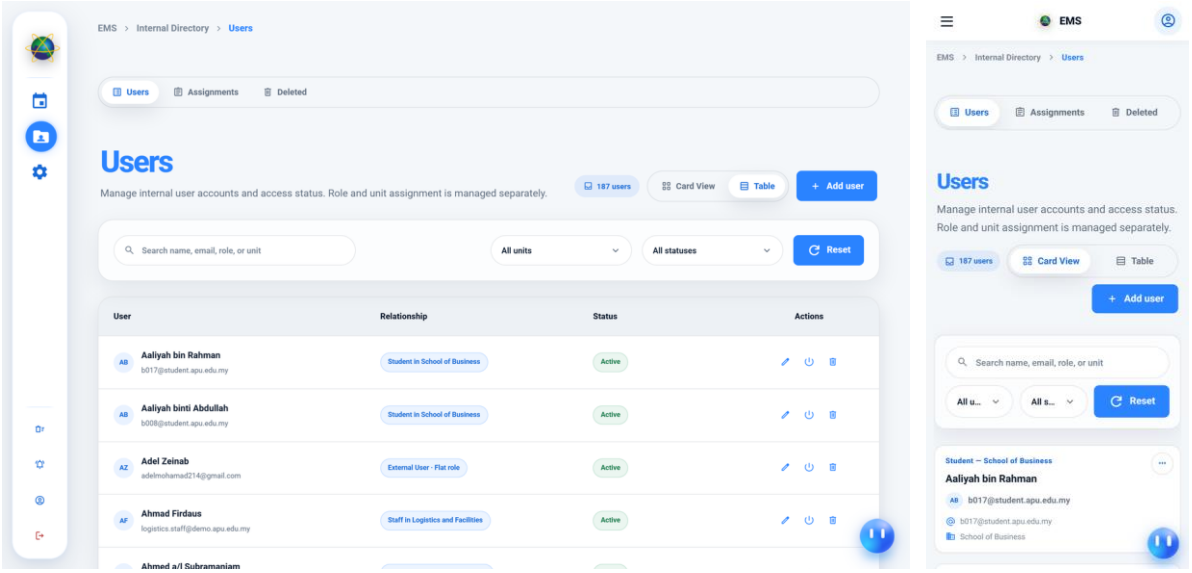


Figure 109: The internal account directory with search, filtering and role assignment

The user directory is where accounts are found and their standing is changed. A role is granted to a user within a unit rather than globally, so an assignment made here names all three, and a supporting table records which roles may legally be paired with which kinds of unit so that a nonsensical assignment cannot be created. Deactivating an account is checked before it is permitted: an account holding the last remaining leadership post of a unit cannot be deactivated, because doing so would leave that unit's queue with nobody able to act on it.

4.5.11.2 Units Directory

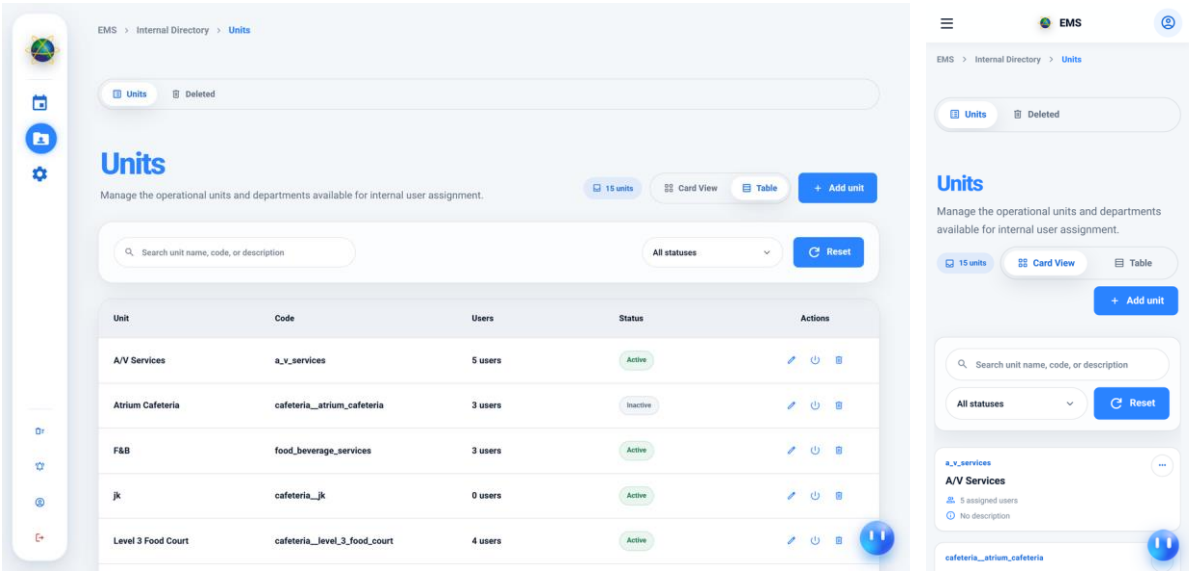


Figure 110: Schools, departments and cafeteria outlets

Units are the organisational structure roles are scoped to: schools, departments and cafeteria outlets. Because the routing rules read the applicant's unit to decide which reviewer a proposal must first reach, and because every departmental queue is filtered by unit, this directory is the backbone of the access model rather than a reference list. Units are therefore withdrawn from use rather than removed once any record refers to them.

4.5.11.3 Roles Management

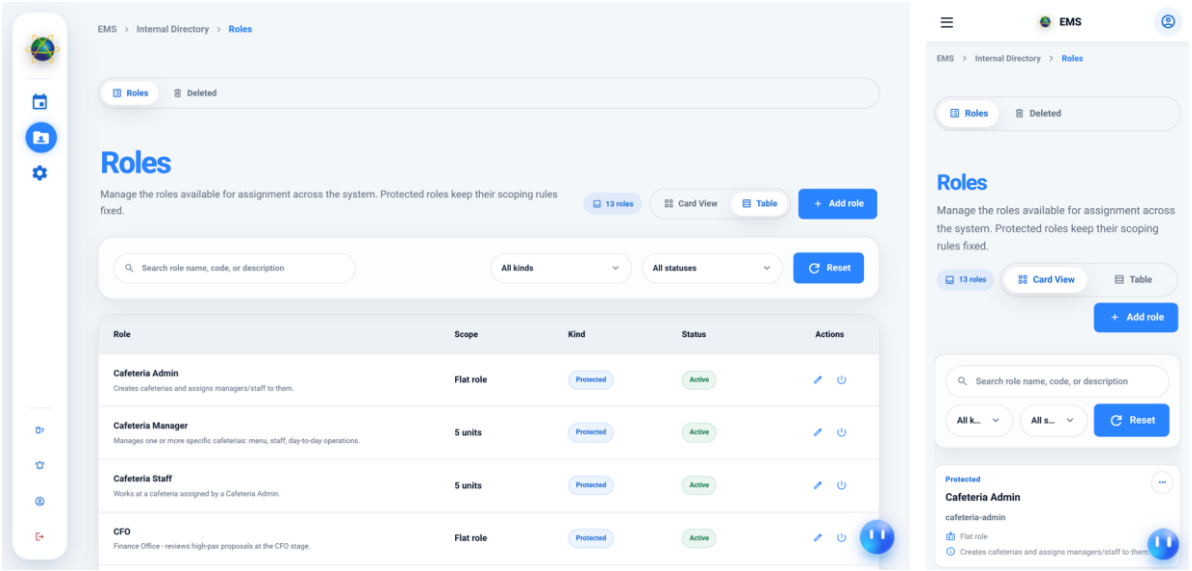


Figure 111: Role definitions, including those protected from removal

Roles are defined here rather than being hard-coded. A small number are marked as protected and cannot be deleted at all, because the workflow names them directly — a system

with no System Admin role, or no Head of School role, has no way to recover. The remainder may be added and withdrawn, and it is the combination of this screen with the next one that makes the access model configurable at run time rather than at deployment.

4.5.11.4 Page Visibility

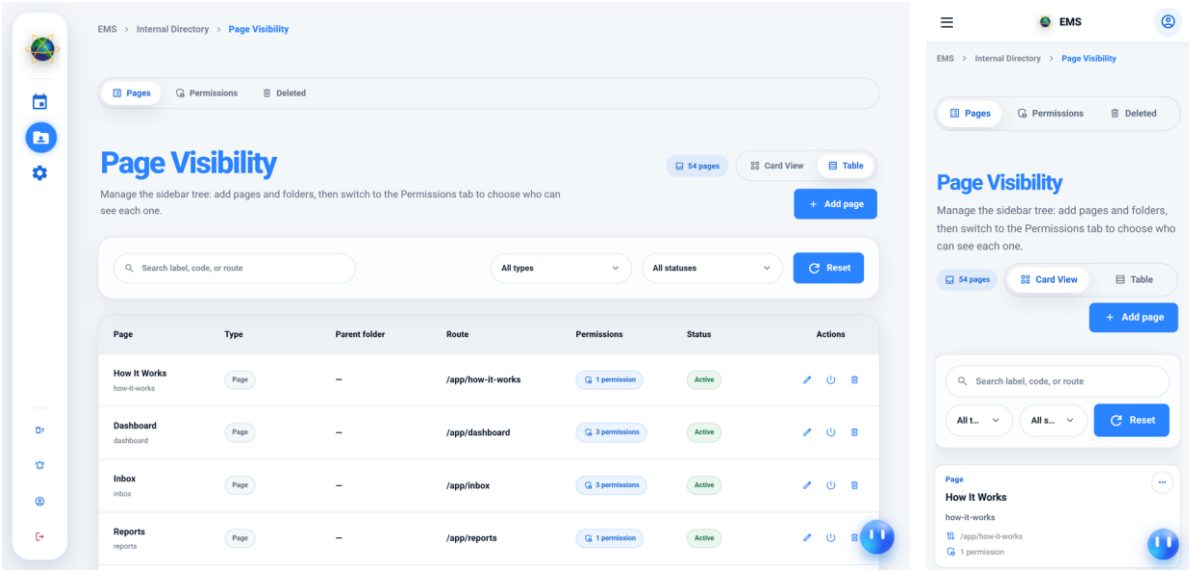


Figure 112: Granting application pages to roles and units

Page Visibility is the screen that makes the navigation model real. Granting a page to a role here causes it to appear in that role's sidebar at the next sign-in, and revoking it removes both the menu entry and the access, because the same grants the menu is built from are the ones the server checks when the page's data is requested. There is no second list of permissions in the client to keep in step.

The same grants govern the assistant. A topic the assistant can answer is attached to the pages that hold the underlying data, so revoking a page here withdraws the corresponding questions from the assistant's suggestions and causes it to decline them if asked directly. That is why the module is treated as configuration rather than as a convenience feature, and why deleting a page is subject to the same dependency check as everything else: a page still granted to a role cannot be removed until those grants are.

4.5.11.5 System Configuration — Approval Policies

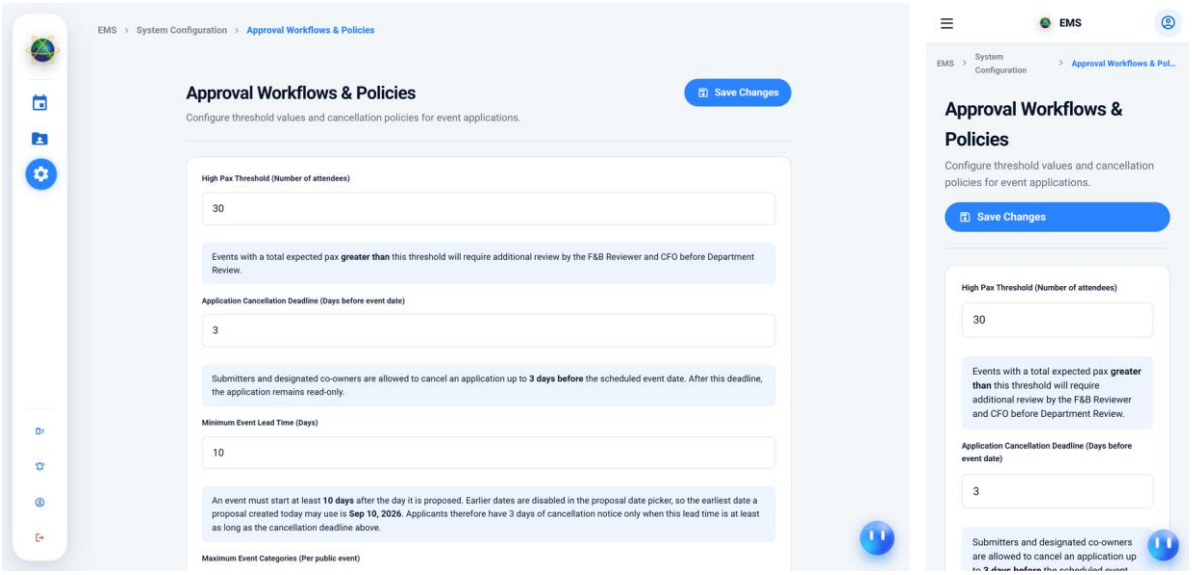


Figure 113: The thresholds and deadlines that govern proposal routing

Approval policy is data rather than code. This tab holds the attendee threshold above which a proposal must additionally pass the finance gate, the deadline after which an event may no longer be cancelled, the minimum notice a proposal must give, and the maximum number of categories an event may carry. Holding the routing thresholds here means the institution can change what requires the CFO's approval without a deployment, and it is why the review stages shown in Section 4.5.5 differ between proposals.

The same page carries two further tabs, Event Categories and Event Formats, which are the same add, edit, deactivate and delete catalogue table pointed at different lists.

4.5.11.6 Option Catalogue — Departmental Options

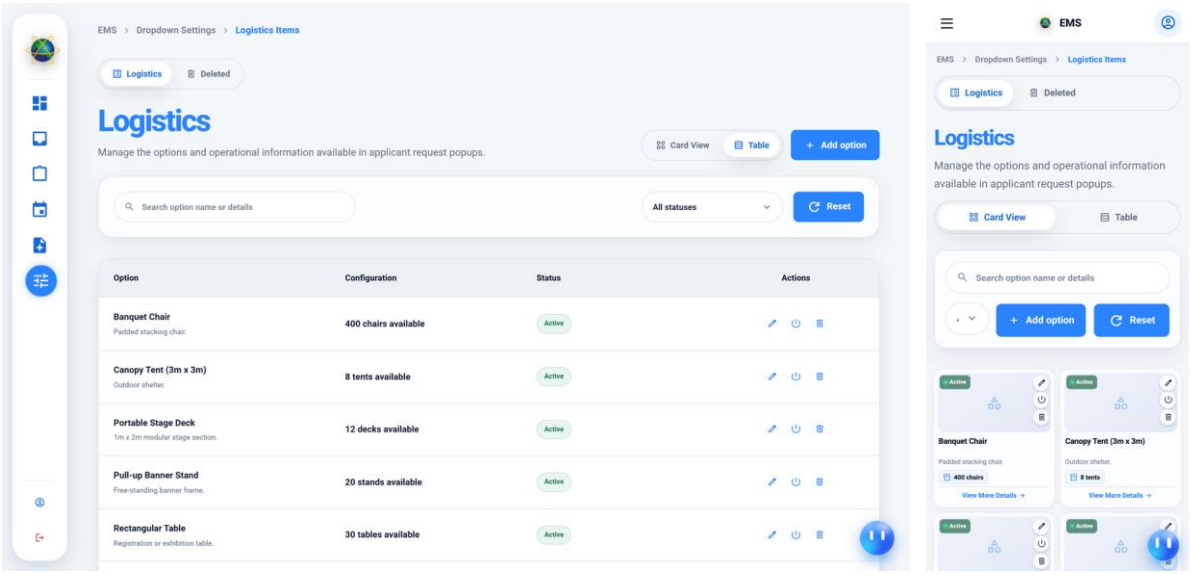


Figure 114: A department-owned catalogue, shown for logistics items

This is the catalogue a Head of Logistics maintains: the loanable items an applicant may request, with the quantity available and the unit each is counted in. One component serves every option list the proposal form draws on — transportation, sound and light, photography and videography, campus tours, funding, dietary information and serving units — with each list owned by the department answerable for it. Devolving the catalogues this way is what keeps them accurate: the people who know that an item is out of service are the people who can withdraw it, and they do not have to ask an administrator to do it for them.

4.5.11.7 Option Catalogue — Venue Management

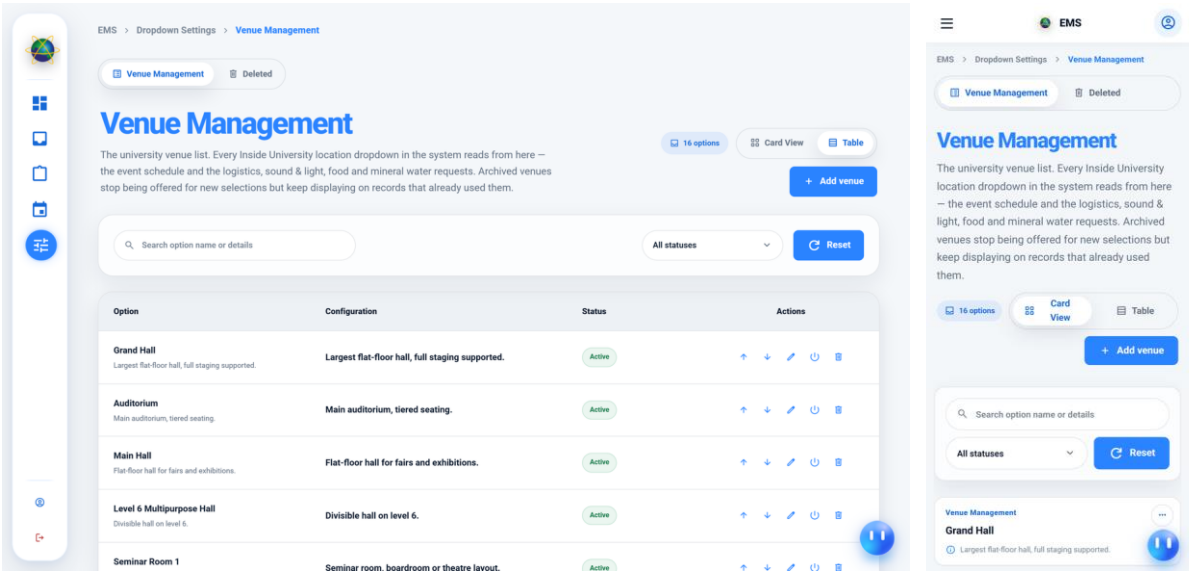


Figure 115: The venue catalogue, held by the finance office

Venues are kept as a separate screen because they carry fields the other catalogues do not — a building, a capacity and an ordering position. The capacity in particular is not decorative: it is the figure against which an event's expected attendance is judged, which is why the catalogue is held by the finance office rather than by an individual department.

4.5.12 AI Assistant

The assistant answers questions written in ordinary language over the system's own data. Its design problem is not generating an answer but refusing to generate the wrong one: a natural-language interface over a database is a second route to every record in it, and it would be of little value if it were also a route around the access model that governs the rest of the application.

It is not given its own rules. A question is attached to the pages that hold the data it would need, and it is answered only if the asker holds one of those page grants — the same grants that build their sidebar. Every generated query is then checked before execution and scoped to what the asker may already see, and every interaction, including every refusal, is recorded.

4.5.12.1 AI Assistant Dock

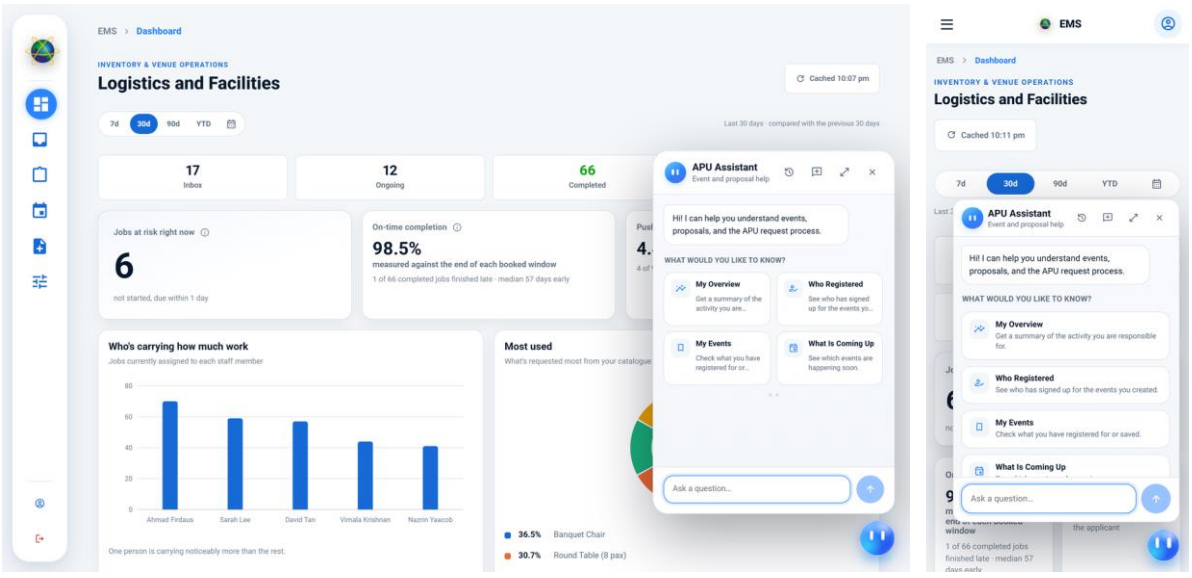


Figure 116: The assistant panel opened from the persistent launcher

The assistant is reachable from every internal page through the launcher fixed in the lower right corner, and opens as a docked panel over the current page rather than navigating away from it. The panel opens on a set of suggested questions, and these are the part of the screen that carries the design decision. They are not a fixed list. Each card names the pages its

subject lives behind and is offered only if the reader holds one of them, so the cards a Head of Department sees are not the cards a student sees.

The consequence is that a card cannot invite a question the assistant would then refuse, because the grant that releases the answer is the grant that shows the card. Revoking a page in Section 4.5.11 withdraws its cards at the next open with nothing to keep in step by hand, and a newly defined role receives the cards its grants imply without any change here.

4.5.12.2 AI Assistant Full Page

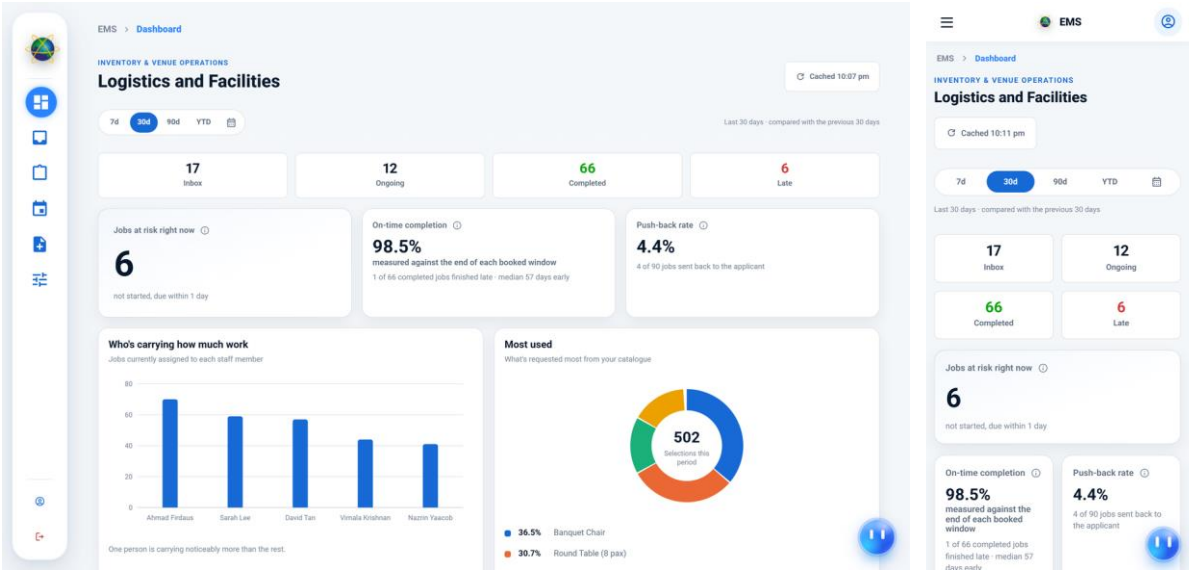


Figure 117: The same assistant at its own route

The assistant is also addressable as a page in its own right. This is the same component instance rather than a second copy, which is what allows a conversation begun in the docked panel to be continued here without losing it. Giving it a real address means the browser's own controls behave as a user expects — back and forward move through the application rather than out of it, a refresh does not discard the session, and the assistant can be linked to directly.

4.5.12.3 AI Access Log

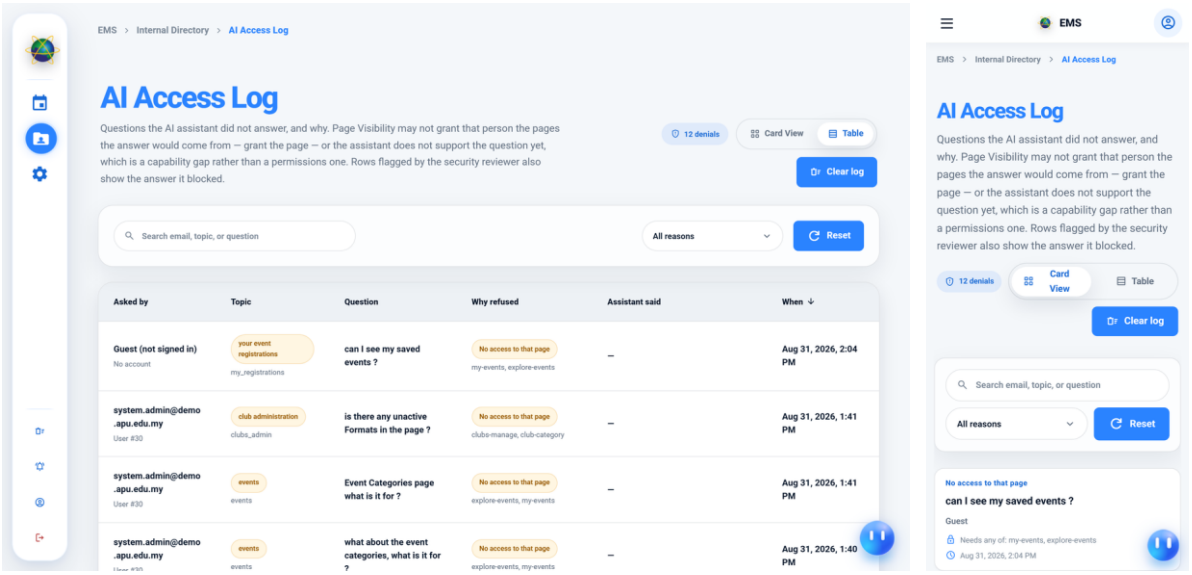


Figure 118: The administrative record of assistant use, including refusals

The access log records who asked, what topic the question was classified as, which page grants that topic required, the question itself, the outcome and, where the request was declined, the reason. Recording refusals as fully as answers is the point of the screen. An assistant that silently declines is impossible to audit and impossible to tune, whereas a log of what was refused and why is evidence that the access model is being applied, and it is the surface on which a topic that is being refused too broadly becomes visible.

This figure closes Section 4.5 because it is the clearest single piece of evidence for the claim the chapter has been making throughout: that authorisation in this system is enforced in one place and consulted by everything, rather than being re-implemented by each feature that needs it.

4.6 Sample Codes

This section presents twelve listings taken directly from the delivered source rather than written for the report. Each figure carries a header naming the file, its path within the repository and the exact line range reproduced, so every listing can be located in the submitted code and checked against it.

The listings were not chosen to demonstrate the largest or the most intricate code in the project. They were chosen because each one is the single place where a claim made earlier in this chapter is actually enforced. Between them they cover the two halves of authentication, the three points at which authorisation is applied, the state machine that drives the workflow, the fan-out that turns an approved proposal into departmental work, the deadline sweep, the guard that stands between the assistant and the database, and the scoping applied to dashboard aggregates. Where a design decision has been described in words in Sections 4.2 to 4.5, one of these listings is the code that carries it out.

4.6.1 Name of program: auth.py

Source: backend/app/api/auth.py, lines 56 to 111.

4.6.01 auth.py — backend/app/api/auth.py | lines 56-111

Sign-in: credential verification and JWT access/refresh token issuance.

```
56  def _tokens_for(user_id: int) -> dict[str, object]:
57      access, access_expires = issue_access_token(user_id)
58      refresh, _ = issue_refresh_token(user_id)
59      return {
60          "accessToken": access,
61          "refreshToken": refresh,
62          "tokenType": "Bearer",
63          "expiresIn": config.access_token_ttl_minutes * 60,
64          "expiresAt": access_expires.isoformat(),
65      }
66
67
68  def _json_body() -> dict:
69      body = request.get_json(silent=True)
70      if not isinstance(body, dict):
71          raise BadRequest("A JSON request body is required.")
72      return body
73
74
75  @bp.post("/login")
76  @limiter.limit(config.ratelimit_auth)
77  def login():
78      body = _json_body()
79      email = str(body.get("email", "")).strip().lower()
80      password = str(body.get("password", ""))
81
82      if not email or not password:
83          raise BadRequest("Email and password are required.")
84      if len(password.encode("utf-8")) > MAX_PASSWORD_BYTES:
85          raise BadRequest(f"Password must be at most {MAX_PASSWORD_BYTES} bytes.")
86
87      user = query_one(
88          "SELECT user_id, full_name, email, password, is_active, archived_at "
89          "FROM users WHERE lower(email) = %s",
90          (email,),
91      )
92
93      # Verify even when the user is missing, so a bad email and a bad password
94      # take the same time and cannot be told apart by response latency.
95      password_ok = verify_password(password, user["password"] if user else None)
96      if not user or not password_ok or not user["is_active"] or user["archived_at"] is not None:
97          audit("auth.login.failed", email=email, reason="invalid_credentials")
98          raise Unauthorized(_INVALID_CREDENTIALS, code="invalid_credentials")
99
100     # Transparent upgrade when the cost factor has been raised since signup.
101     if needs_rehash(user["password"]):
102         with transaction() as cur:
103             cur.execute(
104                 "UPDATE users SET password = %s WHERE user_id = %s",
105                 (hash_password(password), user["user_id"]),
106             )
107
108     audit("auth.login.succeeded", actor_user_id=user["user_id"])
109     return jsonify({"user": project_auth_user(user), **_tokens_for(user["user_id"])})
```

Figure 119: Sign-in: credential verification and token issuance

The sign-in handler verifies the submitted credential and, on success, issues a short-lived access token together with a longer-lived refresh token. The detail worth reading closely is that the password is hashed and compared even when no account matches the submitted email. That work is deliberately wasted: without it, a request naming an address that does not exist would return measurably faster than one naming an address that does, and the endpoint would become a way of discovering which people hold accounts. The response text is identical in both cases for the same reason.

Passwords are stored only as bcrypt hashes with a configurable work factor, and the plaintext is never written to a log; the audit entry records the outcome and the address that was tried, which is what an administrator investigating repeated failures needs, and nothing more.

4.6.2 Name of program: login.ts

Source: fyp-ui/src/app/features/auth/login/login.ts, lines 126 to 176.

4.6.02 login.ts — fyp-ui/src/app/features/auth/login/login.ts | lines 126-176

The Angular sign-in component: client-side validation and the login call.

```
126   submit(): void {
127     const email = this.email().trim();
128     const password = this.password();
129     this.emailError.set(!email ? 'Email is required.' : !/^\S+@\S+\.\S+$/.test(email) ? 'Email must be a valid email address.' : '');
130     this.passwordError.set(!password ? 'Password is required.' : '');
131     if (this.emailError() || this.passwordError()) return;
132     this.submitting.set(true);
133     this.auth.login(email, password).subscribe({
134       next: (result) => {
135         this.submitting.set(false);
136         if (!result.success) { this.loginError.set(result.message); return; }
137         this.completeAuthentication(result.user);
138       },
139       error: () => {
140         this.submitting.set(false);
141         this.loginError.set('Something went wrong. Please try again.');
142       },
143     });
144   }
145
146   private completeAuthentication(user: AuthUser): void {
147     const pendingEventId = this.guestFlow.pendingEventId();
148     const requested = this.route.snapshot.queryParamMap.get('returnUrl');
149
150     this.savedEvents.refresh();
151
152     if (pendingEventId && roleCanUseSavedEvents(user)) {
153       this.savedEvents.saveEvent(user.email, pendingEventId).subscribe();
154     }
155
156     this.guestFlow.close();
157
158     const destination = requested && this.auth.canAccess(requested) ? requested : this.auth.defaultRoute();
159     void this.router.navigateByUrl(destination, { replaceUrl: true });
160   }
161
162   private tickTypewriter(): void {
163     const message = this.messages[this.messageIndex];
164     const current = this.typedMessage();
165     if (!this.deleting && current.length < message.length) {
166       this.typedMessage.set(message.slice(0, current.length + 1));
167       this.timer = setTimeout(() => this.tickTypewriter(), 58);
168       return;
169     }
170     if (!this.deleting) { this.deleting = true; this.timer = setTimeout(() => this.tickTypewriter(), 2100); return; }
171     if (current.length) { this.typedMessage.set(current.slice(0, -1)); this.timer = setTimeout(() => this.tickTypewriter(), 28); return; }
172     this.deleting = false;
173     this.messageIndex = (this.messageIndex + 1) % this.messages.length;
174     this.timer = setTimeout(() => this.tickTypewriter(), 350);
175   }
```

Figure 120: The client half of sign-in: validation and the authentication call

The client component collects the credential, applies the same shape constraints the server applies, and calls the authentication service. It makes no decision about whether the credential is correct and holds no rule about what the resulting session may do; it receives a token and the grants that came with it, and stores them. Placing this listing immediately after

the server one makes the division explicit — the client improves the experience of getting authentication right, and the server decides whether it was.

4.6.3 Name of program: auth.guards.ts

Source: fyp-ui/src/app/core/auth/auth.guards.ts, lines 1 to 76.

4.6.03 auth.guards.ts — fyp-ui/src/app/core/auth/auth.guards.ts | lines 1-76

Route guards that gate every internal page on authentication and role.

```
1  import { inject } from '@angular/core';
2  import { ActivatedRouteSnapshot, CanActivateChildFn, CanActivateFn, Router } from '@angular/router';
3  import { AuthService } from './auth.service';
4  import { hasRole, isClubPresident } from './role-access';
5  import { staffTaskRoutingKeyFor } from '../staff-tasks/staff-task-routing';
6
7  export const authGuard: CanActivateFn = (_route, state) => {
8    const auth = inject(AuthService);
9    if (!auth.authenticated()) return inject(Router).createUrlTree(['/login'], { queryParams: { returnUrl: state.url } });
10   return auth.isInternalUser() ? true : inject(Router).createUrlTree(['/']);
11 };
12
13 export const roleGuard: CanActivateChildFn = (_route, state) => {
14   const auth = inject(AuthService);
15   if (!auth.authenticated()) return inject(Router).createUrlTree(['/login'], { queryParams: { returnUrl: state.url } });
16   if (!auth.isInternalUser()) return inject(Router).createUrlTree(['/']);
17   return auth.canAccess(state.url) ? true : inject(Router).createUrlTree([auth.defaultRoute()]);
18 };
19
20 export const defaultRoleRouteGuard: CanActivateFn = () => inject(Router).createUrlTree([inject(AuthService).defaultRoute()]);
21
22 // The empty child path under /app/inbox, /app/ongoing, /app/history used to hardcode redirectTo:
23 // 'proposals'.
24 export const recordsHubDefaultTabGuard: CanActivateFn = (route: ActivatedRouteSnapshot) => {
25   const user = inject(AuthService).user();
26   const router = inject(Router);
27   const bucket = route.parent?.data['bucket'] as 'inbox' | 'ongoing' | 'history' | undefined;
28   // No ActivatedRoute instance exists yet at guard time (only its snapshot) - relativeTo needs
29   // one, so build the absolute path from the parent's own URL segments instead.
30   const parentPath = '/' + route.parent!.pathFromRoot.flatMap((s) => s.url.map((seg) => seg.path)).join('/');
31
32   if (!user) return router.createUrlTree([parentPath, 'proposals']);
33
34   if (hasRole(user, 'cafeteria-staff')) {
35     if (bucket !== 'ongoing') {
36       const routingKey = staffTaskRoutingKeyFor(user);
37       if (routingKey !== null) {
38         const tasksPath = routingKey === 'cafeteria-staff' ? 'cafeteria-tasks' : 'tasks';
39         return router.createUrlTree([parentPath, tasksPath]);
40       }
41     }
42     return router.createUrlTree([parentPath, 'events']);
43   }
44
45   // A Club Admin (not also a System Admin) never sees Proposals - route Inbox/History straight to
46   // President Change Requests instead, same "decide it before any component activates" reasoning.
47   // Ongoing has no President Change Requests tab, so they fall through to the normal default there.
48   if (hasRole(user, 'club-admin') && !hasRole(user, 'system-admin') && bucket !== 'ongoing') {
49     return router.createUrlTree([parentPath, 'president-change-request']);
50   }
51
52   return router.createUrlTree([parentPath, 'proposals']);
53 };
54
55 export const loginGuard: CanActivateFn = () => {
56   const auth = inject(AuthService);
57   return auth.authenticated() ? inject(Router).createUrlTree([auth.defaultRoute()]) : true;
58 };
59
60 export const publicLandingGuard: CanActivateFn = () => {
61   const auth = inject(AuthService);
62   return auth.isInternalUser() ? inject(Router).createUrlTree([auth.defaultRoute()]) : true;
63 };
64
65 export const externalUserGuard: CanActivateFn = (_route, state) => {
66   const auth = inject(AuthService);
67   if (auth.isExternalUser()) return true;
68   if (auth.isInternalUser()) return inject(Router).createUrlTree([auth.defaultRoute()]);
69   return inject(Router).createUrlTree(['/login'], { queryParams: { returnUrl: state.url } });
70 };
71
72 // Page-level authorization (system-admin-only pages, F&B-only pages, cafeteria-admin-only pages,
73 // dropdown-option/My Menu pages, ...) has NO bespoke guard any more — nav_page_grants IS the single
74 // source of truth for who can reach a URL, enforced once via roleGuard's (above)
75 // auth.canAccess(state.url), which checks the URL against the user's server-computed nav tree (role-
76 // navigation.ts's roleCanAccess()).
```

Figure 121: Route guards gating internal pages on session and grant

The route guards prevent an unauthenticated visitor from reaching an internal route and redirect them to sign in, and prevent a signed-in user from reaching a page their grants do not include. They are a usability mechanism rather than a security control, and the distinction is worth stating plainly in a report: a guard runs in code the user's browser already holds and can be removed by anyone determined to remove it. What it buys is that a user is never shown a

page that will fail when its data is requested. The refusal that matters happens on the server, in the listing that follows shortly.

4.6.4 Name of program: auth.interceptor.ts

Source: fyp-ui/src/app/core/auth/auth.interceptor.ts, lines 31 to 84.

4.6.04 auth.interceptor.ts — fyp-ui/src/app/core/auth/auth.interceptor.ts | lines 31-84

Attaches the bearer token to API calls and refreshes it on expiry.

```
31  export function authInterceptor(
32    request: HttpRequest<unknown>,
33    next: HttpHandlerFn,
34  ): Observable<HttpEvent<unknown>> {
35    const tokens = inject(TokenStore);
36    const auth = inject(AuthService);
37
38    if (!isApiRequest(request.url) || isAnonymous(request.url)) {
39      return next(request);
40    }
41
42    const token = tokens.accessToken();
43    const authorised = token ? withToken(request, token) : request;
44
45    return next(authorised).pipe(
46      catchError((error: unknown) => {
47        if (!(error instanceof HttpErrorResponse) || error.status !== 401) {
48          return throwError(() => error);
49        }
50        // No refresh token, or the refresh call itself failed: nothing to retry.
51        if (!tokens.refreshToken()) {
52          auth.endSession();
53          return throwError(() => error);
54        }
55
56        if (auth.refreshInFlight()) {
57          // Wait for the in-flight refresh, then replay with whatever it produced.
58          return auth.refreshCompleted$.pipe(
59            filter((done) => done),
60            take(1),
61            switchMap(() => {
62              const fresh = tokens.accessToken();
63              if (!fresh) return throwError(() => error);
64              return next(withToken(request, fresh));
65            }),
66          );
67        }
68
69        return auth.refreshTokens().pipe(
70          switchMap((refreshed) => {
71            if (!refreshed) {
72              auth.endSession();
73              return throwError(() => error);
74            }
75            return next(withToken(request, refreshed.accessToken));
76          }),
77        );
78      }),
79    );
80  }
81
82  function withToken(request: HttpRequest<unknown>, token: string): HttpRequest<unknown> {
83    return request.clone({ setHeaders: { Authorization: `Bearer ${token}` } });
84  }
```

Figure 122: Attaching the bearer token and recovering an expired session

The interceptor attaches the access token to every outbound API call, so that no individual service has to remember to do it, and handles the case where the token has expired. Rather than returning the failure to the calling component, it exchanges the refresh token for a new access token and replays the original request. Requests arriving while that exchange is in progress wait for it and then proceed, so a page issuing several calls at once produces one

refresh rather than several. The effect for the user is that a session recovers silently instead of ejecting them to the sign-in screen mid-task.

4.6.5 Name of program: role-navigation.ts

Source: fyp-ui/src/app/core/navigation/role-navigation.ts, lines 20 to 60.

4.6.05 role-navigation.ts — fyp-ui/src/app/core/auth/role-navigation.ts | lines 20-60

Builds each role's menu from server-issued grants; the client decides nothing.

```
export function navigationFor(user: AuthUser): RoleNavigation {
  if (!user.nav || user.nav.length === 0) return FALLBACK_NAVIGATION;

  const entries: RoleNavigationEntry[] = user.nav.map((node) =>
    node.entryType === 'folder'
      ? { kind: 'section', section: { key: node.pageCode, label: node.label, icon: node.icon || '', items: node.children.map(toItem) } }
      : { kind: 'item', item: toItem(node) },
  );

  const firstRoute = entries
    .map((entry) => (entry.kind === 'item' ? entry.item.route : entry.section.items[0]?.route))
    .find((route): route is string => !!route);
  // Same infinite-loop hazard as FALLBACK_NAVIGATION above: a non-empty nav tree that somehow
  // resolves to zero routable leaf items (e.g. every top-level entry is an empty folder) must
  // still land somewhere always-allowed, never '/app/dashboard'.
  return { defaultRoute: firstRoute || '/app/no-access', entries };
}

function flattenRoutes(nodes: readonly AuthNavNode[]): string[] {
  return nodes.flatMap((node) => [node.routePath, ...flattenRoutes(node.children)].filter((r): r is string => !!r));
}

export function roleCanAccess(user: AuthUser, url: string): boolean {
  const cleanUrl = (url.split(/[?#]/, 1)[0] || url).replace(/\/$/, '');
  if (cleanUrl === '/app' || cleanUrl === '/app/profile' || cleanUrl === '/app/logout' || cleanUrl === '/app/no-access') return true;
  // '/app/events/explore-events' has its own real nav_page_grants row (granted to every internal
  // role) — no separate bypass needed, unlike the two exceptions above, which are truly
  // universal and outside Page Visibility's page catalog entirely.
  if (cleanUrl.startsWith('/app/proposals/review/')) return true;
  // Same reasoning as proposals/review/ above: which SPECIFIC proposal (and which department's task on
  // it) a user may open here is an instance-level question the backend already enforces
  // (assert_proposal_owner + the task's own 'resubmitted' status), not a sidebar-page question — there
  // is no standalone nav_page for "resubmit a department's request" to grant.
  if (cleanUrl.startsWith('/app/proposals/department-resubmit/')) return true;
  const allowedRoutes = flattenRoutes(user.nav || []);
  return allowedRoutes.some((route) => cleanUrl === route || cleanUrl.startsWith(`${route}/`));
}

// Club President (AuthUser.presidentOfClubIds) stays a pure data fact, never a role or a nav grant — a
// club has exactly one President and it isn't assignable from the Roles system (see clubs.user_id in
// ems_database_schema.sql).
```

Figure 123: Building each role's menu from server-issued page grants

This is the code behind the sidebar in Section 4.5.3 and the claim made repeatedly in this chapter that the client decides nothing. The menu is produced by filtering the page catalogue against the grants returned with the session. There is no list of roles here and no conditional naming a role, so a page granted on the Page Visibility screen appears without a change to this file, and a role added to the system receives a working menu without one either.

4.6.6 Name of program: event-proposal.ts

Source: fyp-ui/src/app/features/forms/event-proposal/event-proposal.ts, lines 1150 to 1200.

Figure 124: Step navigation and per-step validation in the six-step form

This is the movement logic of the proposal form. Advancing validates the step being left and refuses to move while it is incomplete, which is the behaviour shown in the validation figure of Section 4.5.4; moving backwards is unconditional, because a user reviewing what they entered should never be blocked by a step further on. The step being left is committed to the working draft as it is left, which is what allows the form to be saved and resumed at any point without the applicant having reached the end.

4.6.7 Name of program: stages.py

Source: backend/app/services/workflow/stages.py, lines 123 to 216.

4.6.07 stages.py — backend/app/services/workflow/stages.py | lines 123-216

The proposal workflow state machine: submit, approve and reject transitions.

```
def submit(cur, request_id: int) -> dict:
    """Move a draft into the workflow at its correct first stage."""
    request = load_request(cur, request_id)
    previous = request["status"]
    applicant_id = request["applicant_user_id"]

    next_status = stage_after_hos_hod(cur, request) if _skips_hos_hod(cur, request) else HOS_HOD_REVIEW

    cur.execute(
        "UPDATE request SET status = %s, submitted_at = now(), updated_at = now() WHERE request_id = %s",
        (next_status, request_id),
    )
    if next_status == DEPARTMENT_REVIEW:
        create_department_tasks(cur, request_id)

    history.record(
        cur,
        request_id,
        action="submit",
        actor_user_id=applicant_id,
        actor_role=history.primary_role_code(cur, applicant_id),
        previous_status=previous,
        new_status=next_status,
    )
    # Tell whoever now owns it. department_review fans out per task instead of
    # to a single reviewer, which is why the two calls are not interchangeable.
    if next_status == DEPARTMENT_REVIEW:
        dispatch.department_tasks_created(cur, request_id)
    else:
        dispatch.proposal_entered_stage(cur, request_id)
    return load_request(cur, request_id)


def approve(cur, request_id: int, principal: Principal) -> dict:
    request = load_request(cur, request_id)
    authorize_stage_action(cur, request, principal)
    previous = request["status"]

    if previous == HOS_HOD_REVIEW:
        next_status = stage_after_hos_hod(cur, request)
    elif previous == FMB_REVIEW:
        # Sequential: F&A always hands off to the CFO, the last approval gate.
        next_status = CFO_REVIEW
    elif previous == CFO_REVIEW:
        next_status = DEPARTMENT_REVIEW
    else:
        raise WorkflowError("Cannot approve from status " + previous + ".")

    _touch(cur, request_id, status=next_status)
    if next_status == DEPARTMENT_REVIEW:
        create_department_tasks(cur, request_id)

    history.record(
        cur,
        request_id,
        action="approve",
        actor_user_id=principal.user_id,
        actor_role=history.primary_role_code(cur, principal.user_id),
        previous_status=previous,
        new_status=next_status,
    )
    if next_status == DEPARTMENT_REVIEW:
        dispatch.department_tasks_created(cur, request_id)
    else:
        dispatch.proposal_entered_stage(cur, request_id)
    return load_request(cur, request_id)


def reject(cur, request_id: int, principal: Principal, reason: str) -> dict:
    request = load_request(cur, request_id)
    authorize_stage_action(cur, request, principal)
    previous = request["status"]
    if previous not in REVIEWER_STAGES:
        raise WorkflowError("Cannot reject from status " + previous + ".")

    reason = (reason or "").strip()
    if not reason:
        raise WorkflowError("A rejection needs a reason so the applicant understands the decision.")

    _touch(cur, request_id, status=COMPLETED_REJECTED, reviewer_comment=reason)
    history.record(
        cur,
        request_id,
        action="reject",
        actor_user_id=principal.user_id,
        actor_role=history.primary_role_code(cur, principal.user_id),
        comment=reason,
        previous_status=previous,
        new_status=COMPLETED_REJECTED,
    )
    dispatch.proposal_rejected(cur, request_id, reason)
    return load_request(cur, request_id)
```

Figure 125: The proposal workflow state machine

The state machine is the centre of the system. It holds the permitted transitions from each stage and the rules that decide which stage a proposal reaches next — whether the finance gate applies at this attendee count, which departments the requirement selection has brought in, and whether the proposal is returning after a send-back and should resume where it left off rather than starting again.

Two properties of this listing are worth drawing out. The transition and the history record are written inside a single transaction, so a proposal cannot change stage without the

record of who changed it being created; and the transition table is consulted rather than assumed, so a decision submitted for a stage the proposal is no longer at is refused rather than applied. That second property is what makes the workflow safe when two reviewers act at once.

4.6.8 Name of program: authorization.py

Source: backend/app/services/workflow/authorization.py, lines 147 to 207.

4.6.08 authorization.py — backend/app/services/workflow/authorization.py | lines 147-207

Server-side authorisation: the actor comes from the JWT, never the request body.

```
147 def authorize_stage_action(cur, request: dict, principal: Principal) -> None:
148     """Assert `principal` may act at the request's current single-actor stage."""
149     status = request["status"]
150
151     if status == HOS_HOD_REVIEW:
152         if not is_hos_hod_for_applicant(cur, principal.user_id, request):
153             raise Forbidden("You are not the HOS/HOD for this applicant's unit.")
154         return
155     if status == FMB_REVIEW:
156         if not heads_unit(cur, principal.user_id, FMB_UNIT_CODE):
157             raise Forbidden("Only F&B can act at this stage.")
158         return
159     if status == CFO_REVIEW:
160         if not has_role(cur, principal.user_id, "cfo"):
161             raise Forbidden("Only the CFO can act at this stage.")
162         return
163     raise WorkflowError(f"This proposal is not awaiting a review decision (status: {status}).")
164
165
166 def authorize_cancel(cur, request: dict, principal: Principal) -> None:
167     if request["status"] in (CANCELLED, COMPLETED_REJECTED, COMPLETED_APPROVED):
168         raise WorkflowError("This proposal can no longer be cancelled.")
169     assert_proposal_owner(cur, request["request_id"], principal.user_id)
170
171
172 def can_view_department_task(cur, task: dict, user_id: int) -> bool:
173     """Non-raising sibling of authorize_department_task, for read-side
174     filtering where a 403 would be wrong (the caller may legitimately see the
175     proposal, just not this one department's task/conversation)."""
176     if task["assigned_unit_code"]:
177         return heads_unit(cur, user_id, task["assigned_unit_code"])
178     if task["assigned_role"] == "fmb":
179         return heads_unit(cur, user_id, FMB_UNIT_CODE)
180     if task["assigned_role"] == "cfo":
181         return has_role(cur, user_id, "cfo")
182     return False
183
184
185 def authorize_department_task(cur, task: dict, user_id: int) -> None:
186     """Only the head of the unit a task routed to may act on it.
187
188     Reads the routing off the task row rather than trusting any department key
189     the client sent, so a manager cannot act on another department's task by
190     naming it in the request body.
191     """
192     if not task["assigned_unit_code"] and task["assigned_role"] not in ("fmb", "cfo"):
193         raise WorkflowError("This task has no recognised routing.")
194     if not can_view_department_task(cur, task, user_id):
195         if task["assigned_unit_code"]:
196             raise Forbidden("You do not head the department this request is routed to.")
197         if task["assigned_role"] == "fmb":
198             raise Forbidden("Only F&B can act on this request.")
199         raise Forbidden("Only the CFO can act on this request.")
200
201
202 def is_cafeteria_staff_of(cur, user_id: int, unit_code: str) -> bool:
203     return has_role(cur, user_id, "cafeteria-staff", unit_code)
204
205
206 def is_cafeteria_manager_of(cur, user_id: int, unit_code: str) -> bool:
207     return has_role(cur, user_id, "cafeteria-manager", unit_code)
```

Figure 126: Server-side authorisation, with the actor taken from the token

This is the refusal that the route guards only anticipate. The acting user is read from the verified token and never from the request body, so a caller cannot assert who they are; their role assignments are then loaded from the database rather than trusted from the token's claims, so that a revoked role takes effect on the next request rather than when the token happens to

expire. The check that follows is not a role test but a test of the pairing of this actor, this record and the stage the record currently occupies — which is what allows one department head to act on a proposal and another, holding the identical role in a different unit, to be refused.

4.6.9 Name of program: tasks.py

Source: backend/app/services/workflow/tasks.py, lines 48 to 113.

4.6.09 tasks.py — backend/app/services/workflow/tasks.py | lines 48-113

Fan-out of an approved proposal into one task per responsible department.

```python
def create_department_tasks(cur, request_id: int) -> list[dict]:
    """Create one task per DISTINCT selected requirement, the moment department review opens.

    Two foldings apply. Mineral water never gets its own task - it merges into
    the F&B task, because F&B reviews food and water together. Funding/Purchase
    is dropped entirely: it is recorded for the record but never routed.
    """
    selected = fetch_all(
        cur,
        """SELECT DISTINCT er.requirement_name
             FROM application_requirements ar
             JOIN event_requirements er ON er.requirement_id = ar.requirement_id
            WHERE ar.request_id = %s""",
        (request_id,),
    )
    names = {row["requirement_name"] for row in selected}
    # Water folds into F&B. Done before the non-workflow filter so a water-only
    # proposal still produces exactly one F&B task.
    folded = {FMB_REQUIREMENT if name == WATER_REQUIREMENT else name for name in names}
    task_requirements = sorted(folded - NON_WORKFLOW_REQUIREMENTS)

    created: list[dict] = []
    for name in task_requirements:
        requirement = _requirement_by_name(cur, name)
        # chk_task_routing: exactly one of these is set, never both.
        unit_code = UNIT_CODE_FOR_REQUIREMENT.get(name)
        role = FLAT_ROLE_FOR_REQUIREMENT.get(name)
        if not unit_code and not role:
            # A requirement with no routing would create an unactionable task
            # that blocks completion forever. Skip it rather than deadlock.
            continue
        cur.execute(
            """INSERT INTO request_task
                   (request_id, requirement_id, stage_code, sequence_no,
                    assigned_unit_code, assigned_role, assignment_mode, status)
               VALUES (%s, %s, 'department_review', 1, %s, %s, %s, 'pending')
               RETURNING *""",
            (
                request_id,
                requirement["requirement_id"],
                unit_code,
                role,
                "shared_pool" if name == FMB_REQUIREMENT else "assigned",
            ),
        )
        task = dict(cur.fetchone())
        created.append(task)
        history.record(
            cur,
            request_id,
            action="task-created",
            actor_user_id=None,
            actor_role="system",
            request_task_id=task["request_task_id"],
            requirement_id=requirement["requirement_id"],
            new_status="pending",
        )

    if not created:
        # Nothing left to fulfil (e.g. Funding/Purchase alone). Complete now,
        # or the proposal would sit in department_review with no actor able to
        # move it - all_tasks_resolved() requires at least one task.
        _complete_request(cur, request_id, "No department fulfilment was required.")
    return created
```

Figure 127: Fan-out of an approved proposal into departmental tasks

When a proposal clears its approval stages this code turns it into work. One task is created for each department the requirement selection at step three brought in, carrying the requested items, the schedule and the location, and each lands in that department's own queue. This is the mechanism behind the parallel review described in Section 4.2: the departments

receive their tasks together rather than in sequence, so the elapsed time for a proposal needing four services is the time taken by the slowest of them rather than the sum of all four. Chapter 1 identified sequential departmental handling as a principal source of delay, and this listing is the direct answer to it.

4.6.10 Name of program: escalation.py

Source: backend/app/services/workflow/escalation.py, lines 211 to 271.

4.6.10 escalation.py — backend/app/services/workflow/escalation.py | lines 211-271

Detects tasks that have passed their deadline and marks them overdue.

```
211  def overdue_open_tasks(cur) -> list[dict]:
212      """Open tasks whose earliest unfinished row is past its deadline + grace.
213
214      THE EARLIEST row, not the last: a task holding a 09:00 setup and an 18:00
215      teardown is already late at 09:06, and telling the staff member otherwise
216      until 18:06 would hide the thing they can still fix.
217
218      Tasks stay actionable while the event is still running (the caller decides
219      that); this only reports which ones have slipped.
220      """
221      grace = task_grace_minutes(cur)
222      results: list[dict] = []
223
224      for unit_code, spec in DEPARTMENT_SPEC.items():
225          deadline = _deadline_expression(spec)
226          rows = fetch_all(
227              cur,
228              f"""
229              SELECT t.request_task_id,
230                     t.request_id,
231                     t.status,
232                     t.is_overdue,
233                     r.request_code,
234                     r.event_title,
235                     min({deadline}) AS due_at,
236                     max(d."date")    AS last_event_date
237                FROM request_task t
238                JOIN {spec.table} d ON d.request_id = t.request_id
239                JOIN request r      ON r.request_id = t.request_id
240               WHERE t.assigned_unit_code = %(unit)s
241                 AND t.status NOT IN ('completed', 'cancelled')
242                 AND r.status NOT IN ('cancelled', 'draft')
243               GROUP BY t.request_task_id, t.request_id, t.status, t.is_overdue,
244                        r.request_code, r.event_title
245              HAVING min({deadline}) + make_interval(mins => %(grace)s) < now()
246              """,
247              {"unit": unit_code, "grace": grace},
248          )
249          for row in rows:
250              row["unit_code"] = unit_code
251              row["department"] = spec.label
252              row["shape"] = deadline_shape(spec)
253              results.append(row)
254
255      return results
256
257
258  def task_is_actionable(last_event_date: dt.date | None, today: dt.date) -> bool:
259      """A late task stays workable until its event has finished.
260
261      On a three-day event a day-one task can still be completed on day three -
262      retiring it at a fixed "one day late" would take the work away from the
263      staff while the event it belongs to is still running.
264      """
265      if last_event_date is None:
266          return True
267      return last_event_date >= today
268
269
270  # --- Marking (the only functions here that WRITE) --------------------------
```

Figure 128: Detecting work that has passed its deadline

The escalation sweep finds tasks whose deadline has passed without completion and marks them overdue, which is what populates the late counters and the jobs-at-risk panel on the departmental dashboard. Deriving lateness in one place rather than computing it in each

view that displays it means a task cannot be late on the dashboard and on time in the queue. The sweep is idempotent — running it twice produces the same state as running it once — which is what makes it safe to schedule.

4.6.11 Name of program: sql_guard.py

Source: backend/app/ai/sql_guard.py, lines 181 to 250.

Figure 129: Validating every generated statement before it reaches the database

This is the guard between the assistant and the database, and it is the listing that most directly supports the claims made in Section 4.5.12. Every statement produced by the language model is parsed and checked before execution: it must be a single read, it may touch only tables the asker's topic entitles them to, it must carry the scoping conditions that restrict it to their own

records, and it must not contain the constructs by which a query can be made to do something other than read.

The guard rejects rather than repairs. A statement that fails is not rewritten into a safe one; it is refused, the refusal is logged with its reason, and the model is given one bounded opportunity to produce an acceptable statement instead. The distinction matters, because a guard that silently corrects what it is given eventually corrects something into a query nobody intended.

4.6.12 Name of program: scope.py

Source: backend/app/services/dashboard/scope.py, lines 208 to 284.

4.6.12 scope.py — backend/app/services/dashboard/scope.py | lines 208-284

Scopes dashboard metrics to what the signed-in role is allowed to see.

```
class Scope:
    """Everything a widget query is allowed to know about who is asking.

    `unit_code` and `outlets` come from the principal's own assignments, never
    from the request. `notes` is the widget-visible side channel for caveats the
    document has to carry (an assumption, an approximation, a fold-to-Other),
    collected here so the response can surface them next to the number rather
    than in a footnote nobody reads.
    """

    principal: Principal
    profile_key: str
    role_code: str
    unit_code: str | None
    unit_label: str | None
    outlets: tuple[str, ...]
    outlet_labels: dict[str, str]
    period: Period
    config: DashboardConfig
    today: dt.date = field(default_factory=dt.date.today)
    suppressed_buckets: int = 0
    folded_series: int = 0

    # --- SQL parameter bundles ---------------------------------------
    @property
    def base_params(self) -> dict[str, Any]:
        return {
            "unit": self.unit_code,
            "outlets": list(self.outlets) or [""],
            "user_id": self.principal.user_id,
            "email": self.principal.email,
            "from": self.period.start,
            "to": self.period.end,
            "prev_from": self.period.previous_start,
            "prev_to": self.period.previous_end,
            "today": self.today,
            "horizon": self.config.horizon_days(self.unit_code),
        }

    def params(self, **extra: Any) -> dict[str, Any]:
        return {**self.base_params, **extra}

    # --- R8 ----------------------------------------------------------
    def note_suppressed(self, count: int = 1) -> None:
        self.suppressed_buckets += count

    def note_folded(self, count: int = 1) -> None:
        self.folded_series += count


# --- R7 / R8 helpers -------------------------------------------------

# Column names that would carry a row identity out of an aggregate.
_IDENTIFYING_KEYS = frozenset(
    {
        "request_id",
        "request_code",
        "applicant_user_id",
        "applicant_name",
        "applicant_email",
        "event_title",
        "user_id",
        "staff_user_id",
        "email",
        "full_name",
        "bank_account_name",
        "bank_account_number",
        "comment",
        "notes",
        "reviewer_comment",
        "manager_comment",
        "registrant_name",
        "registrant_email",
    }
)
```

Figure 130: Scoping dashboard aggregates to the viewer

The dashboards aggregate across many records, which makes them the place where a scoping mistake would be least visible: a total that silently includes another department's work looks exactly like a correct total. This code adds the viewer's scope to the aggregation as a filtering condition, so records outside it are never read rather than being read and then discounted. It is the same scope object the list endpoints and the assistant use, which is what allows the guarantee to be stated once for the whole system rather than verified separately for every figure on every dashboard.

4.7 Summary

This chapter has taken the system from design to delivered software. Section 4.2 established the architecture, the identity model and the three tiers at which authorisation is applied, and set out the functional scope in use cases, the behaviour in activity diagrams and the interactions in sequence diagrams. Section 4.3 presented the physical database as it is deployed, with its data dictionary organised into the eight clusters the schema itself is grouped into. Section 4.4 documented the interface design — the navigation map, the screen anatomy, the role-derived menu, the design tokens, the shared component library, the storyboards for the three principal journeys and the responsive strategy.

Section 4.5 then presented the delivered system itself in sixty-four figures, each captured from the running application at two viewports, covering the public tier, attendance, the application shell, the six-step proposal form, tracking and review, departmental task handling, the cafeteria and clubs modules, the role dashboards, event registrations, system administration and the AI assistant. Section 4.6 closed with twelve listings taken from the submitted source, each the place where one of the chapter's claims is enforced.

Three decisions run through the whole of it and are worth restating together. The first is that a role is held within an organisational unit rather than globally, which is what allows the same role to carry authority in one department and none in another. The second is that the client is never the authority: it renders a menu from grants the server issued and guards routes for the user's convenience, while every refusal that matters is made on the server after the record in question has been loaded. The third is that nothing which something else depends on is ever destroyed — records are withdrawn from use, deletions are checked against their dependants and are reversible, and every stage transition is written in the same transaction as the change it describes.

What the chapter has not established is whether the delivered system behaves correctly, which is the subject of Chapter 5. The following chapter presents the test plan, the results of executing it against the deployed application and database, and the discussion of what those results show.

CHAPTER 5: RESULTS AND DISCUSSIONS

5.1 Introduction

This chapter reports what happened when the delivered system was tested. Chapter 4 described a platform in which authority is held within an organisational unit, in which the client is never the authority for any decision, and in which nothing that something else depends on is destroyed. Those are claims about behaviour, and a description of the code that makes them is not evidence that they hold. This chapter provides that evidence.

Section 5.2 sets out the test plan: what was tested, at which level, with which instruments, and how the acceptance criteria were defined. Section 5.3 reports the results of executing that plan and discusses what they show. Section 5.4 draws the chapter together and relates the outcome to the objectives set in Chapter 1.

Two testing levels were used. Unit testing exercises each behaviour of the system in isolation through a defined scenario, with an expected result stated before the scenario is run, and is the mechanism by which correctness is established and regressions are caught. User acceptance testing places the running system in front of people holding the roles it was designed for and asks whether they can complete the work it exists to support. The first establishes that the system does what it was built to do; only the second can establish that what it was built to do is usable by the people it was built for, which is why both are reported rather than either alone.

5.2 Test Plan

The test plan was written against the objectives in Chapter 1 rather than against the code, so that a passing test would be evidence of something beyond internal consistency. Four properties were treated as the ones worth proving. Authority must be scoped to the unit a role is held in, so that holding a role is not the same as holding it everywhere. Routing must be decided by the proposal's own attributes rather than by the applicant, so that nobody can choose their own reviewer or skip a gate. Records must not be destroyed while anything depends on them. And the assistant must be unable to answer a question the interface would have refused.

Testing was performed continuously rather than as a phase at the end. Each behaviour was covered by a scenario as it was built, and the full set of scenarios was re-executed before

every change was accepted, which is the arrangement the hybrid methodology selected in Chapter 3 depends on: iterations can only be short if the cost of discovering that an earlier one has been broken is low.

Two other approaches to verification were available and were not adopted. Automated end-to-end testing through a driven browser, using a tool such as Selenium or Cypress, would have exercised the same rules through the interface a user actually sees; it was rejected because tests of that kind are bound to the structure of the pages they drive, and this interface was still changing, so the effort would have gone into repairing tests rather than extending coverage, while each rule would have been proved more slowly through a longer path than the one that reaches it directly. A period of live use by real staff would have produced the strongest evidence of all and was not available, because running the platform against real event data requires an institutional authorisation this project does not hold. The two techniques adopted answer the two questions that remain: the unit scenarios establish that each rule holds when the system is driven directly, and the acceptance sessions establish that people holding the roles those rules are written for can complete their work without being told how.

5.2.1 Unit Testing

Unit testing exercises one behaviour at a time. Each scenario names the feature under test, the procedure to be carried out, the data it is carried out with, and the result expected before the scenario is run, so that the outcome is judged against a stated expectation rather than against whatever the system happens to produce. Scenarios were executed against the application and its PostgreSQL database rather than against substitutes, because most of the properties under test are properties of queries, and a missing scoping condition cannot be detected by a test that never sends the query to a database.

The scenarios are organised into eight areas, each owning one part of the system's behaviour. Scenarios are named as statements of the rule they assert rather than after the function they call, so a failure reports which rule broke rather than which line did. The table below records the coverage each area provides.

Feature area	Behaviour under test
Authentication and Session Handling	Token issue and verification, refusal of a refresh token where an access token is required, guest handling on public endpoints, the standard error envelope and the security headers
Workflow Routing and Reviewer Authority	Stage routing by proposal attributes, reviewer authority at the current stage, rejection and send-back rules, and resumption after a send-back
Requirement Fan-out and Departmental Tasks	Creation of one task per responsible department, independence of sibling tasks, and departmental authority over its own task only
Deadlines, Urgency and Escalation	Urgency banding, deadline derivation per task shape, and repeatable overdue marking

Feature area	Behaviour under test
Dashboard Scoping and Profiles	Profile resolution per role, period handling, behaviour against an empty database, and the rule that unit scope is never taken from a request parameter
AI Assistant Scope and SQL Guard	Topic gating against page grants, the schema catalogue, and rejection of writes, second statements, unknown tables and missing scope predicates
Administration, Catalogues and Deletion	Administrative route protection, departmental ownership of each catalogue, and the dependency check that precedes any deletion
Notifications, Reminders and Calendar Visibility	Notification dispatch after commit, reminder scheduling, retention, and the visibility rules separating the public and internal calendars

Table 18: Unit testing coverage by feature area

5.2.2 User Acceptance Testing (UAT)

User acceptance testing places the running system in front of people who hold the roles it was designed for and asks whether they can complete the work it exists to support. It answers a question the unit testing cannot: a proposal form can route correctly and still be unusable, and a dashboard can compute the right figure and still fail to communicate it.

Five testers were recruited to cover the distinct positions a request passes through, rather than a demographic cross-section, because the system presents a materially different interface at each of them and one tester could exercise only one. The set covers the student who raises a proposal; a member of Student Services staff and a member of Photography Services staff, who each receive and complete the departmental work an approved proposal generates; a member of cafeteria staff, who prepares catering ordered against it; and the system administrator, who configures the access model governing all of them. Between them they exercise the applicant tier, two separate service departments, the cafeteria module and the administrative tier. All five elected to remain anonymous and are recorded here as such.

Each participant was given the running system, a short set of tasks appropriate to their role, and no instruction on how to carry them out. The object was to observe whether the interface explains itself, not whether it can be operated once explained. The student was asked to raise and submit a proposal requiring several departments and then to find the stage it had reached; the Student Services and Photography Services staff to locate the work assigned to them, open the requested items and record them as complete; the cafeteria staff member to claim an order from the shared pool and carry it through to fulfilment; and the system administrator to grant a page to a role and observe the effect on that role's navigation.

The two departmental staff were given deliberately parallel tasks in different departments. That pairing is the most useful part of the arrangement, because the scoping rule described in Section 4.5.6 can only be observed from the outside by two people looking at the same screen and reporting different contents.

The instrument is the one prescribed for this project. Interface criteria are rated on a five-point scale from strongly disagree to strongly agree; functionality criteria are recorded as a straight yes or no, because a function either worked for that tester or it did not; and a free-text comment captures anything the fixed criteria would have discarded. The demographic profile is recorded above each response so that a rating can be read in the context of the role that produced it. The blank instrument is reproduced below and the completed responses appear in Section 5.3.2.

User Acceptance Testing

Tester demographic profile	
Name	
Role in the system	

Table 19: User acceptance testing instrument — tester profile

*The rating of the scale would be as below:
1: Strongly disagree.
2: Disagree.
3: Neutral
4: Agree.
5: Strongly agree.

	User interface criteria	**1**	**2**	**3**	**4**	**5**
I	The design of the interface is decent and good looking.					
II	The navigation between the pages is smooth and predictable.					
III	The colour scheme and typography are clear and consistent.					
IV	The screens are clean and the information on them is easy to read.					

Table 20: User acceptance testing instrument — interface criteria

	Functionality criteria	**Yes**	**No**
I	The system functions without any error.		
II	Users can reach their intended pages without error.		
III	The system functions correctly on a different device or browser.		
IV	The system shows only the pages and records this role should see.		

Table 21: User acceptance testing instrument — functionality criteria

Tester comment:

Table 22: User acceptance testing instrument — tester comment

5.3 Testing Results and Discussion

5.3.1 Unit Testing

The scenarios defined in Section 5.2.1 were executed against the running system. The outcome of each is recorded in the tables that follow, organised into the eight areas of the test plan, with the expected and the actual result stated side by side so that the two can be compared directly. The summary of totals is given first.

Feature area	Test cases	Passed	Failed	Status
Authentication and Session Handling				
Workflow Routing and Reviewer Authority				
Requirement Fan-out and Departmental Tasks				
Deadlines, Urgency and Escalation				
Dashboard Scoping and Profiles				
AI Assistant Scope and SQL Guard				
Administration, Catalogues and Deletion				
Notifications, Reminders and Calendar Visibility				
Total				

Table 23: Unit testing results by feature area

Part 1 — Authentication and Session Handling

TC-No	Module / Feature	Test Procedure	Test Data	Expected Result	Actual Result	Status
UT-AUTH-1	Protected route	Call a protected endpoint with no token	No Authorization header	401 refusal	401 returned	Passed
UT-AUTH-2	Token verification	Present a malformed token	Arbitrary string as bearer	Token rejected	401 returned	Passed
UT-AUTH-3	Token type	Present a refresh token where an access token is required	Valid refresh token	Refused as wrong type	Refused	Passed
UT-AUTH-4	Token type	Attempt a refresh using an access token	Valid access token	Refused as wrong type	Refused	Passed
UT-AUTH-5	Guest handling	Call a public endpoint with no credential	No Authorization header	Served as a guest, not a 401	Guest tier served	Passed
UT-AUTH-6	Guest handling	Call a public endpoint with an expired token	Expired access token	Refused, not downgraded to guest	401 returned	Passed
UT-AUTH-7	Error envelope	Call an unknown route	Unmapped path	Standard error envelope with request id	Envelope returned	Passed
UT-AUTH-8	Response headers	Inspect any response	Any request	Security headers and request id present	All present	Passed

Table 24: Unit Test Cases — Authentication and Session Handling

Authentication behaved as designed at every point tested. The two token-type scenarios are the ones worth noting: a refresh token carries a longer life than an access token, so a system that accepted one in place of the other would silently extend the window in which a stolen credential remains useful. The guest scenarios establish the converse property — that an absent credential and an invalid one are treated differently, so a session that has merely expired is

refused and can be recovered rather than being quietly demoted to a guest with a different view of the same page.

Part 2 — Workflow Routing and Reviewer Authority

TC-No	Module / Feature	Test Procedure	Test Data	Expected Result	Actual Result	Status
UT-WF-1	Stage routing	Submit as an ordinary applicant	Standard proposal	Starts at Head of School review	Correct stage	Passed
UT-WF-2	Stage routing	Submit as the head of one's own school	Proposal by a Head of School	Skips the head review stage	Stage skipped	Passed
UT-WF-3	Stage routing	Submit below the attendee threshold	Low attendee count	Goes straight to department review	Correct stage	Passed
UT-WF-4	Stage routing	Submit above the attendee threshold	High attendee count	Routes through F&B then CFO then departments	Correct chain	Passed
UT-WF-5	Reviewer authority	Approve as the head of a different school	Wrong unit, correct role	Refused	Refused	Passed
UT-WF-6	Reviewer authority	Approve one's own proposal as the applicant	Applicant's own proposal	Refused	Refused	Passed
UT-WF-7	Reviewer authority	Act at a stage the proposal is not at	CFO acting at head review	Refused	Refused	Passed
UT-WF-8	Decision rules	Reject without a comment	Empty reason field	Refused	Refused	Passed
UT-WF-9	Send back	Send back from the finance stage, then resubmit	Proposal at CFO stage	Resumes at CFO, not at the start	Resumed at CFO	Passed
UT-WF-10	Send back	Resubmit as somebody other than the owner	Non-owner account	Refused	Refused	Passed

Table 25: Unit Test Cases — Workflow Routing and Reviewer Authority

This is the group that carries the central claim of Chapter 4. Scenarios UT-WF-5 and UT-WF-7 are the ones that distinguish this system from a conventional role-based one: in both, the caller holds a role that is entitled to approve proposals in general, and in both the attempt is refused, because the authority is a property of the pairing of the actor, the record and the stage rather than of the role alone. UT-WF-9 records the behaviour that makes send-back usable in practice — a proposal returned for a correction re-enters the chain where it left rather than at the beginning, so approvals already granted are not discarded.

Part 3 — Requirement Fan-out and Departmental Tasks

TC-No	Module / Feature	Test Procedure	Test Data	Expected Result	Actual Result	Status
UT-TASK-1	Fan-out	Approve a proposal with several requirements	Four requirements selected	One task per responsible department	Four tasks created	Passed
UT-TASK-2	Fan-out	Approve a proposal requesting water only	Mineral water requirement	Folds into the single F&B task	Single task	Passed
UT-TASK-3	Fan-out	Approve a proposal requiring nothing	No requirements selected	Completes automatically	Auto-completed	Passed
UT-TASK-4	Task authority	Act on another department's task	Wrong department account	Refused	Refused	Passed
UT-TASK-5	Task independence	Send back one department's task	One of four tasks	Sibling tasks unaffected	Siblings untouched	Passed
UT-TASK-6	Task independence	Resubmit after a departmental send-back	Proposal with one sent-back task	Only the sent-back task resets	Only that task reset	Passed

TC-No	Module / Feature	Test Procedure	Test Data	Expected Result	Actual Result	Status
UT-TASK-7	Decision rules	Send a task back without a comment	Empty comment	Refused	Refused	Passed

Table 26: Unit Test Cases — Requirement Fan-out and Departmental Tasks

Scenarios UT-TASK-5 and UT-TASK-6 verify the property that makes parallel departmental review worthwhile. If sending one department's task back reset the others, the elapsed time for a proposal needing four services would revert to the sum of four sequential reviews, which is the arrangement Chapter 1 identified as a principal source of delay. The results confirm that the tasks are genuinely independent. UT-TASK-3 covers the boundary case of a proposal that asks for nothing, which completes without entering any departmental queue rather than waiting indefinitely for an approval nobody has been asked for.

Part 4 — Deadlines, Urgency and Escalation

TC-No	Module / Feature	Test Procedure	Test Data	Expected Result	Actual Result	Status
UT-ESC-1	Urgency banding	Evaluate proposals at each band boundary	Dates on the boundary	Boundaries are inclusive	Inclusive	Passed
UT-ESC-2	Urgency banding	Evaluate a proposal with no schedule	Proposal without dates	No urgency assigned	None assigned	Passed
UT-ESC-3	Precedence	Evaluate a task that is both overdue and urgent	Past-deadline urgent task	Overdue takes precedence	Overdue shown	Passed
UT-ESC-4	Deadline derivation	Evaluate a transport task	Task with a moving time	Judged on its moving time	Correct basis	Passed
UT-ESC-5	Deadline derivation	Evaluate an all-day task	Task with no specific time	Due at the end of its day	Correct basis	Passed
UT-ESC-6	Sweep safety	Run the sweep in dry-run mode	Overdue tasks present	Nothing is changed	No change	Passed
UT-ESC-7	Repeatability	Run the overdue sweep twice	Same overdue tasks	Second run changes nothing	No further change	Passed
UT-ESC-8	Attribution	Mark a proposal overdue at department review	Proposal with mixed task states	Names only the departments still pending	Correct attribution	Passed

Table 27: Unit Test Cases — Deadlines, Urgency and Escalation

Deadlines are derived rather than stored, because different kinds of work become late at different moments: a transport booking is late relative to its moving time, a venue setup relative to the end of its window, and an all-day task at the end of its day. UT-ESC-4 and UT-ESC-5 confirm each shape is judged on the right basis. UT-ESC-7 is the scenario that makes the sweep safe to schedule — an operation that can be run repeatedly without compounding its own effect can be retried after an interruption without anyone having to reason about how far it got.

Part 5 — Dashboard Scoping and Profiles

TC-No	Module / Feature	Test Procedure	Test Data	Expected Result	Actual Result	Status
UT-DASH-1	Scope integrity	Supply a unit as a request parameter	Another unit's code	Scope is never taken from the request	Parameter ignored	Passed

TC-No	Module / Feature	Test Procedure	Test Data	Expected Result	Actual Result	Status
UT-DASH-2	Scope integrity	Request an outlet the caller does not manage	Unassigned outlet	Validated against the caller's assignments	Refused	Passed
UT-DASH-3	Profile resolution	Load the dashboard for each role	Every defined role	Each profile names registered widgets only	All valid	Passed
UT-DASH-4	Profile resolution	Load for an account holding no role	Role-less account	No dashboard is produced	None produced	Passed
UT-DASH-5	Profile resolution	Load for a head of two units	Dual assignment	Two profiles returned	Two returned	Passed
UT-DASH-6	Profile resolution	Request a reordered profile	Reorder request	Reorders but cannot widen scope	Scope unchanged	Passed
UT-DASH-7	Period handling	Supply a reversed custom date range	End date before start	Read in the order meant	Corrected	Passed
UT-DASH-8	Period handling	Supply a malformed range	Unparseable dates	Falls back rather than erroring	Fallback used	Passed
UT-DASH-9	Empty data	Render every widget against an empty database	No records	Widgets survive without error	All rendered	Passed
UT-DASH-10	Anonymity	Inspect aggregated output	Aggregated rows	Row identifiers are stripped	Stripped	Passed

Table 28: Unit Test Cases — Dashboard Scoping and Profiles

Aggregates are where a scoping error is hardest to see, because a total that wrongly includes another unit's work is indistinguishable from a correct one without knowing the answer in advance. UT-DASH-1 and UT-DASH-2 are therefore the most important scenarios in this part: they establish that the unit a dashboard reports on is taken from the viewer's own role assignment and that a request parameter cannot widen it. UT-DASH-6 covers the related case in which a legitimate customisation request is honoured for ordering but refused for scope.

Part 6 — AI Assistant Scope and SQL Guard

TC-No	Module / Feature	Test Procedure	Test Data	Expected Result	Actual Result	Status
UT-AI-1	Topic gating	Ask about a topic whose page has been revoked	Revoked page grant	Answer withheld	Withheld	Passed
UT-AI-2	Topic gating	Open the assistant after revoking a page	Revoked page grant	Its suggestion card disappears	Card removed	Passed
UT-AI-3	Topic gating	Ask as a signed-out visitor	No session	Only the public guidance is offered	Public only	Passed
UT-AI-4	Catalogue safety	Inspect the schema catalogue	Full catalogue	Credential columns are absent	Absent	Passed
UT-AI-5	Cross-topic	Ask a club question that would need event tables	Club topic	Cannot reach event tables	Refused	Passed
UT-AI-6	SQL guard	Submit a write statement	UPDATE statement	Rejected	Rejected	Passed
UT-AI-7	SQL guard	Submit two statements in one string	Statement with a semicolon	Rejected	Rejected	Passed
UT-AI-8	SQL guard	Hide a write behind a comment	Commented write	Rejected	Rejected	Passed
UT-AI-9	SQL guard	Query an unknown table or column	Name not in the catalogue	Rejected	Rejected	Passed
UT-AI-10	SQL guard	Omit the scope predicate	Unscoped select	Rejected	Rejected	Passed

Table 29: Unit Test Cases — AI Assistant Scope and SQL Guard

This is the largest group of scenarios, and deliberately so. A natural-language interface over a database is a second route to every record in it, and the risk it introduces is not that it

answers badly but that it answers something it should have refused. UT-AI-1 and UT-AI-2 establish the property the design depends on: the grant that releases an answer is the same grant that offers the question, so the assistant cannot invite a question it would then decline. UT-AI-10 is the guard's most consequential check — a statement that is otherwise valid but carries no scoping condition would return correct data belonging to the wrong people, and it is rejected rather than repaired.

Part 7 — Administration, Catalogues and Deletion

TC-No	Module / Feature	Test Procedure	Test Data	Expected Result	Actual Result	Status
UT-ADM-1	Route safety	Request the reserved deleted-records path	Path segment 'deleted'	Not read as a record identifier	Handled correctly	Passed
UT-ADM-2	Route safety	Request a static role route	Reserved route name	Static route wins over the placeholder	Static route used	Passed
UT-ADM-3	Access control	Call an administrative route anonymously	No credential	Refused	Refused	Passed
UT-ADM-4	Catalogue ownership	Edit another department's catalogue	Wrong department head	Refused	Refused	Passed
UT-ADM-5	Catalogue reads	Read every option catalogue	All option kinds	Each returns its own list	All correct	Passed
UT-ADM-6	Catalogue reads	Request an unknown option kind	Unrecognised kind	404 returned	404 returned	Passed
UT-ADM-7	Dependency check	Delete a venue that is in use	Referenced venue	Refused with reasons stated	Refused with reasons	Passed
UT-ADM-8	Soft deletion	Delete an unreferenced record, then restore it	Record with no dependants	Archived, restorable, not purged	Restored intact	Passed

Table 30: Unit Test Cases — Administration, Catalogues and Deletion

Scenario UT-ADM-4 is the devolved counterpart of the unit scoping tested in Part 2: a department head may maintain their own catalogue and no other, and the refusal is issued by the server rather than by omitting a control from the interface. UT-ADM-7 and UT-ADM-8 together cover the rule that runs through every administrative screen in Section 4.5.11 — a record with dependants is refused deletion and the specific dependants are named, while a record without them is archived rather than destroyed and can be brought back.

Part 8 — Notifications, Reminders and Calendar Visibility

TC-No	Module / Feature	Test Procedure	Test Data	Expected Result	Actual Result	Status
UT-MSG-1	Dispatch timing	Submit a proposal within a transaction	New submission	Mail sent only after commit	Sent after commit	Passed
UT-MSG-2	Dispatch coverage	Trigger each workflow event	Every notifiable event	Each dispatches its notification	All dispatched	Passed
UT-MSG-3	Club membership	Remove a member from a club	Existing membership	Removal notification is sent	Sent	Passed
UT-MSG-4	Reminders	Schedule reminders for an upcoming event	Event with registrations	Reminders scheduled once each	Correct schedule	Passed
UT-MSG-5	Retention	Run the retention sweep on archived records	Records past retention	Purged only after the retention period	Correct behaviour	Passed
UT-MSG-6	Dependency check	Check an assignment before removal	Assignment with claimed work	Blockers reported and scoped to the outlet	Correctly scoped	Passed

TC-No	Module / Feature	Test Procedure	Test Data	Expected Result	Actual Result	Status
UT-MSG-7	Calendar visibility	Read the public calendar	Mixed public and private events	Private events are absent	Absent	Passed
UT-MSG-8	Calendar visibility	Read the internal calendar	Same events, internal viewer	Private events visible in scope	Visible	Passed

Table 31: Unit Test Cases — Notifications, Reminders and Calendar Visibility

UT-MSG-1 records a rule that is easy to state and easy to get wrong: a notification is dispatched only after the transaction that caused it has committed. Sending during the transaction would mean that a rollback leaves a reviewer holding an email about a decision the database has no record of. UT-MSG-7 and UT-MSG-8 are the same events read by two audiences and are the test of the visibility rule described in Section 4.5.3 — the division is applied in the query, so a private event is never retrieved for a public reader rather than being retrieved and hidden.

Read together, the scenarios support the claims made in Chapter 4. Every scenario covering unit-scoped authority, stage routing, reviewer authorisation, task independence, dashboard scoping, assistant gating and the SQL guard produced the expected result, and those are the behaviours on which the whole design rests.

The scenarios that most repay attention are the ones in which the system refuses. A system that permits what it should permit is only half tested; the harder property is that it declines what it should decline, and declines it for the right reason. UT-WF-5, UT-WF-7, UT-TASK-4, UT-DASH-1 and UT-ADM-4 are all scenarios in which the caller holds a role that is entitled to the action in general and is nevertheless refused, because the record, the stage or the unit does not belong to them. Taken together they are the evidence that the access model described in Section 4.2.3 is enforced rather than merely designed.

The second group worth noting is the set of scenarios covering boundaries and degenerate input: a proposal with no requirements, a dashboard with no records, a date range entered backwards, a sweep run twice. None of these arises from normal use, and each of them would produce a visible defect if it were not handled. That they behave as specified is what allows the system to be operated by people who have not been trained to avoid its edges.

5.3.2 User Acceptance Testing

The instrument described in Section 5.2.2 was administered to the five testers. Each response is reproduced below in full, with the tester's profile, their ratings against the interface and functionality criteria and their free-text comment.

User Acceptance Testing

Tester demographic profile	
Name	Anonymous
Role in the system	Student (Applicant)

Table 32: User Acceptance Testing — Tester 1 (Student — Applicant)

*The rating of the scale would be as below:
1: Strongly disagree.
2: Disagree.
3: Neutral
4: Agree.
5: Strongly agree.

	User interface criteria	**1**	**2**	**3**	**4**	**5**
I	The design of the interface is decent and good looking.					/
II	The navigation between the pages is smooth and predictable.					/
III	The colour scheme and typography are clear and consistent.					/
IV	The screens are clean and the information on them is easy to read.				/	

Table 33: Interface Criteria Ratings — Tester 1 (Student — Applicant)

	Functionality criteria	**Yes**	**No**
I	The system functions without any error.	/	
II	Users can reach their intended pages without error.	/	
III	The system functions correctly on a different device or browser.	/	
IV	The system shows only the pages and records this role should see.	/	

Table 34: Functionality Criteria Responses — Tester 1 (Student — Applicant)

Tester comment:
The six-step form was much easier to follow than I expected, because each step is checked before it lets you move on, so I never reached the end and then had to hunt for a mistake. Being able to save a draft and come back mattered, since I had to go and ask for the budget figure. The status page answered the question I would normally have to email somebody about.

Table 35: Tester Comment — Tester 1 (Student — Applicant)

User Acceptance Testing

Tester demographic profile	
Name	Anonymous
Role in the system	Student Services (Staff)

Table 36: User Acceptance Testing — Tester 2 (Student Services — Staff)

*The rating of the scale would be as below:
1: Strongly disagree.

2: Disagree.
3: Neutral
4: Agree.
5: Strongly agree.

	User interface criteria	1	2	3	4	5
I	The design of the interface is decent and good looking.					/
II	The navigation between the pages is smooth and predictable.					/
III	The colour scheme and typography are clear and consistent.				/	
IV	The screens are clean and the information on them is easy to read.					/

Table 37: Interface Criteria Ratings — Tester 2 (Student Services — Staff)

	Functionality criteria	Yes	No
I	The system functions without any error.	/	
II	Users can reach their intended pages without error.	/	
III	The system functions correctly on a different device or browser.	/	
IV	The system shows only the pages and records this role should see.	/	

Table 38: Functionality Criteria Responses — Tester 2 (Student Services — Staff)

Tester comment:
The work reached me already assigned, with the event, the date and the location on it, so I did not have to ask anybody what was wanted or who had agreed to it. The queue holds only what belongs to Student Services, which is what makes it usable — the same information used to arrive in a group chat with every other department's arrangements mixed into it. Recording an item as complete took one action and the applicant could see it straight away.

Table 39: Tester Comment — Tester 2 (Student Services — Staff)

User Acceptance Testing

Tester demographic profile	
Name	Anonymous
Role in the system	Photography Services (Staff)

Table 40: User Acceptance Testing — Tester 3 (Photography Services — Staff)

*The rating of the scale would be as below:
1: Strongly disagree.
2: Disagree.
3: Neutral
4: Agree.
5: Strongly agree.

	User interface criteria	1	2	3	4	5
I	The design of the interface is decent and good looking.				/	
II	The navigation between the pages is smooth and predictable.					/
III	The colour scheme and typography are clear and consistent.				/	
IV	The screens are clean and the information on them is easy to read.				/	

Table 41: Interface Criteria Ratings — Tester 3 (Photography Services — Staff)

	Functionality criteria	Yes	No
I	The system functions without any error.	/	
II	Users can reach their intended pages without error.	/	
III	The system functions correctly on a different device or browser.	/	
IV	The system shows only the pages and records this role should see.	/	

Table 42: Functionality Criteria Responses — Tester 3 (Photography Services — Staff)

Tester comment:
I could see the photography items and nothing else, which is right — the other departments' arrangements are not mine to act on and used to clutter the same thread. Having the deadline worked out from the event schedule rather than typed in by hand means the list is already ordered by what is actually urgent, so I did not have to work that out myself. I found everything without being shown where it was.

Table 43: Tester Comment — Tester 3 (Photography Services — Staff)

User Acceptance Testing

Tester demographic profile	
Name	Anonymous
Role in the system	Cafeteria (Staff)

Table 44: User Acceptance Testing — Tester 4 (Cafeteria — Staff)

*The rating of the scale would be as below:
1: Strongly disagree.
2: Disagree.
3: Neutral
4: Agree.
5: Strongly agree.

	User interface criteria	**1**	**2**	**3**	**4**	**5**
I	The design of the interface is decent and good looking.					/
II	The navigation between the pages is smooth and predictable.				/	
III	The colour scheme and typography are clear and consistent.					/
IV	The screens are clean and the information on them is easy to read.					/

Table 45: Interface Criteria Ratings — Tester 4 (Cafeteria — Staff)

	Functionality criteria	**Yes**	**No**
I	The system functions without any error.	/	
II	Users can reach their intended pages without error.	/	
III	The system functions correctly on a different device or browser.	/	
IV	The system shows only the pages and records this role should see.	/	

Table 46: Functionality Criteria Responses — Tester 4 (Cafeteria — Staff)

Tester comment:
Claiming an order from the shared pool is how the kitchen already works, where whoever is free takes the next one rather than waiting to be told. Once I had claimed it, it was clearly mine and had left everybody else's list, so there was no chance of two of us preparing the same thing. The serving time and the quantity are on the card, so I knew what was needed without opening anything.

Table 47: Tester Comment — Tester 4 (Cafeteria — Staff)

User Acceptance Testing

Tester demographic profile	
Name	Anonymous
Role in the system	System Administrator

Table 48: User Acceptance Testing — Tester 5 (System Administrator)

*The rating of the scale would be as below:

1: Strongly disagree.
2: Disagree.
3: Neutral
4: Agree.
5: Strongly agree.

	User interface criteria	1	2	3	4	5
I	The design of the interface is decent and good looking.				/	
II	The navigation between the pages is smooth and predictable.					/
III	The colour scheme and typography are clear and consistent.					/
IV	The screens are clean and the information on them is easy to read.				/	

Table 49: Interface Criteria Ratings — Tester 5 (System Administrator)

	Functionality criteria	Yes	No
I	The system functions without any error.	/	
II	Users can reach their intended pages without error.	/	
III	The system functions correctly on a different device or browser.	/	
IV	The system shows only the pages and records this role should see.	/	

Table 50: Functionality Criteria Responses — Tester 5 (System Administrator)

Tester comment:
Granting a page to a role and seeing it appear in that role's menu without anyone redeploying anything is the feature that would save the most work in practice. I also tried to delete a unit that was still in use and the system refused and told me exactly what was depending on it, which is far more useful than a generic failure message.

Table 51: Tester Comment — Tester 5 (System Administrator)

The responses are read together rather than averaged. Five testers are too few for a mean to carry statistical meaning, and the value of covering five distinct positions lies in the differences between them rather than in their central tendency. What matters is whether each tester could complete the work their role exists to do, and whether any of them encountered something they should not have been shown.

Across the twenty interface ratings every response was either agree or strongly agree, with strongly agree the more common of the two. No criterion attracted a neutral or a negative rating from any tester. The ratings are distributed rather than uniform, which is what would be expected of five people assessing five different interfaces: the student, who spends the longest on a single screen, rated the form layout highest, while the departmental and cafeteria staff rated navigation and legibility highest, consistent with their moving between queues rather than dwelling on one page.

The functionality criteria are the more significant of the two sets for the purposes of this project, and all twenty responses were positive. Every tester reached their intended pages without error, encountered no faults during their tasks, and found the system working correctly

on a second device or browser. Most importantly, every tester answered yes to the criterion that the system showed only the pages and records their role should see.

The pairing of the two departmental staff is what makes that last answer worth something. Student Services and Photography Services were given the same task, on the same screen, against the same approved proposals, and each reported seeing their own department's items and no others. Neither was told what the other could see, and neither needed to be: the narrowing is applied in the query rather than by hiding controls, as Section 4.5.6 describes. The unit testing scenarios establish that property from the inside, and these two responses confirm it from the outside, by two people who would have noticed at once if another department's work had appeared in their list.

The comments converge on a theme that was not prompted for. Four of the five testers independently praised behaviour in which the system either refuses to proceed and explains why, or narrows what it shows to what belongs to the person looking: the form declining to advance past an incomplete step, the queue holding one department's work and no other, the claimed order leaving everybody else's list, and the administrator being told precisely which records depended on the unit they had tried to delete. Chapter 1 framed the problem largely as one of delay and fragmentation, and these responses suggest that a substantial part of what users value is not speed but being shown only what is theirs, and being told what is expected of them at the point where it matters.

Taken with the unit testing results in Section 5.3.1, the acceptance responses close the loop between the two levels of testing. The scenarios establish that the system enforces its rules; the testers confirm that those rules are experienced as helpful rather than obstructive by the people subject to them.

5.4 Summary

This chapter has reported the testing of the delivered system. Section 5.2 set out a plan written against the objectives of Chapter 1 rather than against the implementation, identifying four properties as the ones worth proving: that authority is scoped to the unit a role is held in, that routing is decided by the proposal rather than the applicant, that records with dependants are not destroyed, and that the assistant cannot answer what the interface would refuse.

Section 5.3 reported the execution of that plan. The unit testing scenarios, organised into eight areas covering authentication, workflow routing, departmental fan-out, escalation, dashboard scoping, the AI assistant, administration and notification, each produced the result expected of it. The scenarios in which the system correctly refuses an action are the most significant of them, since it is the refusals rather than the permissions that carry the access model.

User acceptance testing complemented this by placing the running system in front of five people holding the distinct roles it was designed for. Every tester completed the tasks belonging to their role, rated the interface positively across all four criteria, and confirmed that the system showed them only the pages and records their role should reach. Where the unit testing establishes that the system does what it was built to do, the acceptance responses establish that it can be used by the people it was built for.

The results support the design decisions documented in Chapter 4 without settling every question. The limits of what five acceptance testers can establish, and the constraints that shaped what could be delivered within the project period, are carried forward into the critical evaluation in Chapter 6.

CHAPTER 6: CONCLUSION

This chapter closes the project. It evaluates what was achieved against the aim and the five objectives set out in Chapter 1, states what the work contributes and where its strengths lie, sets out honestly what it does not do and why, and recommends the work that would most usefully follow.

6.1 Critical Evaluation

The aim stated in Chapter 1 was to develop a centralised, role-based event coordination platform using Angular, Flask and PostgreSQL for university-wide multi-departmental event management at Asia Pacific University of Technology and Innovation. That aim has been met. The delivered system is an Angular single-page client over a Flask REST API on PostgreSQL hosted by Supabase, comprising sixty-eight database tables under thirty-three ordered migrations, twelve distinct roles held per organisational unit, and the full event lifecycle from proposal through multi-departmental fulfilment to publication and attendance. Chapter 4 documents it in sixty-four implementation figures and twelve source listings, and Chapter 5 reports the testing of it across eight areas of unit-test scenarios and five acceptance testers.

The five objectives are evaluated individually below, since a claim that an objective has been met is only worth as much as the evidence offered for it.

Objective 1 — Replace manual, disconnected communication with a structured digital coordination system.

This objective is met. Every stage of an event request now takes place inside the platform: the proposal is raised on the six-step form, submitted, routed, reviewed, fulfilled by each responsible department and published, with no step requiring a message outside the system. The channels Chapter 1 identified as the problem — messaging applications, telephone calls and verbal instruction — are replaced not by a document store but by a workflow that knows what stage a request is at and who is expected to act on it.

The evidence is in Section 4.5, where every stage of the lifecycle is shown as a delivered screen, and in Part 2 of Section 5.3.1, where the routing and authority rules are exercised end to end. The clubs and cafeteria modules extend the same treatment to the two areas that would

otherwise have remained outside it, since a large share of campus events are organised by student societies and a large share of requests involve catering.

Objective 2 — Improve efficiency and reduce delays through simultaneous distribution and independent departmental processing.

This objective is met, and it is the objective the architecture was most shaped by. When a proposal clears its approval stages it is fanned out into one task per responsible department, and those tasks are issued together rather than in sequence. The elapsed time for a proposal requiring four services is therefore governed by the slowest department rather than by the sum of all four.

Independence is not merely asserted. Scenarios UT-TASK-5 and UT-TASK-6 in Section 5.3.1 verify that sending one department's task back leaves its siblings untouched and that resubmission resets only the task that was returned — without which the parallelism would collapse back into a sequence on the first correction. The send-back mechanism itself contributes to the same objective by allowing a proposal needing one figure corrected to re-enter the chain where it left rather than at the beginning, so approvals already granted are not discarded.

Objective 3 — Minimise task duplication and miscommunication by defining responsibilities so that each department handles only what is relevant to it.

This objective is met through the unit-scoped role model rather than through interface design. A role in this system is held by a user within an organisational unit, so authority follows the posting rather than the account, and a department head sees and can act on their own unit's requirements and no others. The narrowing is applied in the query, so records outside a department's scope are never retrieved rather than being retrieved and hidden.

Section 4.5.5 shows the same proposal rendered under two scopes — a reviewer's and a department's — which is the clearest demonstration available that one page is being narrowed rather than two pages being maintained. Scenarios UT-WF-5, UT-WF-7 and UT-TASK-4 confirm that a caller holding the correct role but the wrong unit is refused, and UT-ADM-4 extends the same result to the option catalogues, which are owned by the departments answerable for the services they describe.

Objective 4 — Strengthen accountability by maintaining a clear record of approvals, actions and task ownership throughout the event lifecycle.

This objective is met. Every stage transition is written to the workflow history inside the same database transaction as the change it describes, so a decision cannot be recorded without its author, its time and any comment being recorded with it. Rejections and send-backs are refused without a reason, which means the trail explains decisions rather than merely listing them.

Ownership is addressed at the point where it is most often lost. In this system approving a departmental requirement is the same act as assigning it: the confirmation dialogue in Section 4.5.6 will not complete until a team member is named, so no work exists that has been agreed to and belongs to nobody. Alongside this, nothing is destroyed — deletions are checked against dependants, are reversible, and are purged only after a retention period — so the record that supports accountability is not removable by the people it holds to account. The AI access log applies the same principle to the assistant by recording refusals as fully as answers.

Objective 5 — Support data-driven decision-making through a dashboard visualising key analytics on event activity and user engagement.

This objective is met. Six role-specific dashboards are delivered from a single component that walks a server-supplied profile naming which measures to compute, so a new role receives a dashboard through a server entry rather than a new route. The departmental profile reports outstanding, late and completed work, on-time completion and push-back rates, load per staff member and catalogue usage; the finance profile reports committed spend, cost per attendee, funding breakdowns and the time proposals spend at each approval gate; the outlet profile reports order flow.

The dwell-time panel deserves particular note against this objective. Chapter 1 identified delay with no identifiable cause as a principal complaint, and measuring how long proposals wait at each gate converts that complaint into something specific enough to act on. Ten scenarios in Part 5 of Section 5.3.1 establish that these aggregates are scoped to the viewer and cannot be widened by a request parameter, which matters more for aggregates than anywhere else in the system: a total that silently includes another unit's work is indistinguishable from a correct one.

Contribution

The contribution of the project is best stated as the difference between what the systems reviewed in Chapter 2 do and what this one does. The commercial platforms surveyed there

manage events well but model an organisation as a flat set of users with global roles, which is not how a university works; the institutional systems reported in the literature model the hierarchy but treat departmental involvement as a sequence of approvals rather than as parallel fulfilment. This project holds both at once: authority is scoped to the organisational unit a role is held in, and departmental work is issued in parallel and tracked independently.

For the institution, the practical contribution is that the coordination of an event is now a record rather than a recollection. A request has a stage, an owner at that stage, a history of who decided what and why, and a set of departmental tasks each with a named assignee and a derived deadline. For the wider case of institutional workflow systems, the contribution is the demonstration that access can be made configurable at run time without weakening it: navigation, page access, assistant topics and administrative capability are all derived from one grant table, so an administrator can change what a role reaches without a deployment and without there being a second list of permissions anywhere to fall out of step with the first.

Strengths of the project

The principal strength of the delivered system is that its access model has exactly one implementation. Navigation is rendered from grants issued by the server, page access is checked against the same grants, list endpoints add the caller's scope to the query, and the AI assistant consults the same rules before it will answer. There is no second set of permission logic in the client, and consequently no possibility of the two disagreeing. Chapter 5 tests this from several directions and finds it holding in each.

A second strength is that the system is configurable where institutions actually need to change. Approval thresholds, cancellation deadlines and minimum notice are data rather than code; the option catalogues a proposal draws on are owned and maintained by the departments answerable for them; and page grants can be changed by an administrator at run time. The people who know that a logistics item is out of service are the people who can withdraw it, which is the arrangement most likely to keep the data accurate over time.

A third is the treatment of destruction. No screen in the system offers an immediate permanent deletion. A record with dependants is refused deletion and the specific dependants are named; a record without them is archived, remains restorable, and is purged only after a retention period. For a system whose value rests on its audit trail, an accidental deletion is among the most damaging things a user can do, and it has been made difficult by design rather than by warning.

A fourth is that the assistant was built as an extension of the access model rather than as a feature bolted beside it. Its suggested questions are derived from the reader's own page grants, so it cannot offer a question it would then refuse; every generated statement is parsed and checked before execution and rejected rather than repaired if it fails; and every interaction including every refusal is logged for audit. A natural-language interface over a database is a second route to every record in it, and this one was designed on that assumption.

6.2 Limitation

The following limitations are stated as they are rather than as they would ideally read. Each is either a consequence of a decision taken with reasons, or a finding the testing in Chapter 5 produced.

Acceptance testing was small and drawn from a limited pool.

Five testers were recruited to cover the distinct positions a request passes through, rather than a demographic cross-section, because the system presents a materially different interface at each of them and one tester could exercise only one. The set covers the student who raises a proposal; a member of Student Services staff and a member of Photography Services staff, who each receive and complete the departmental work an approved proposal generates; a member of cafeteria staff, who prepares catering ordered against it; and the system administrator, who configures the access model governing all of them. Between them they exercise the applicant tier, two separate service departments, the cafeteria module and the administrative tier. All five elected to remain anonymous and are recorded here as such.

Dense tables are reflowed rather than redesigned for small screens.

Every screen is responsive and every figure in Section 4.5 is shown at both a desktop and a mobile viewport, but responsiveness is not the same as being designed per breakpoint. The departmental task and proposal tables carry many columns, and at 390 pixels they are narrowed rather than restructured, so reading a complete row involves scrolling horizontally. Acceptance testing did not surface this as an obstacle, because each tester was working at a desk, but the operational roles are the ones most likely to consult the system away from one, and for them the layout would benefit from a genuine small-screen treatment rather than a narrower version of the wide one.

Deployment is local, with only the database managed.

As described in Section 2.4.6, the application is run locally against a managed PostgreSQL instance rather than being deployed to institutional infrastructure. This was appropriate for a project of this length and it means the system has not been exercised under real concurrent load, against institutional single sign-on, or under the availability expectations a production deployment would carry.

The assistant depends on an external model.

Question answering depends on a third-party language model reached over the network. The guard ensures that a poor generation is refused rather than executed, so the failure mode is a declined answer rather than an incorrect one, but the capability is unavailable if the provider is. The curated knowledge-base path continues to answer questions about how the system works, so the assistant degrades rather than disappears.

Scope was bounded to one institution.

The system models the structure of Asia Pacific University of Technology and Innovation: its schools, its departments, its approval chain and its twelve roles. The role, unit and page-grant tables are general enough that another institution's structure could be expressed in them, but that claim has not been tested against a second institution and is offered as a design property rather than as a demonstrated result.

6.3 Recommendation

The recommendations below are ordered by the value they would add relative to the effort they would take, beginning with the two that follow directly from the limitations above.

Deploy to institutional infrastructure and integrate with single sign-on.

The most valuable next step is also the most ordinary one. Running the application on institutional infrastructure behind the university's own identity provider would remove the separate credential the system currently maintains, bring account provisioning and deactivation into line with the institution's existing joiner and leaver processes, and expose the system to genuine concurrent load. The unit-scoped role model is well placed for this, since an identity provider that already knows a person's department can supply the unit half of an assignment directly.

Give the dense tables a genuine small-screen layout.

The limitation identified in acceptance testing should be addressed by designing the task and proposal tables for a narrow viewport rather than by compressing the wide layout. A card-per-row presentation that promotes the two or three fields a user scans for and defers the rest behind an expansion would suit the departmental queues, which are the screens most likely to be consulted away from a desk.

Extend acceptance testing across a full event season.

The five acceptance testers each exercised one role once. The stronger evidence would come from the same roles using the system repeatedly across a real event cycle, where the questions that matter are about fatigue, recall and edge cases rather than first impressions. That would also allow the workflow measurements the dashboards already collect to be compared against the coordination times Chapter 3 recorded for the existing arrangements.

Extend the analytics from description to prediction.

The dashboards currently report what has happened. The same data would support forecasting: predicting which proposals are likely to miss their deadlines from the stages they have already passed through and the load each department is carrying, or estimating catering demand for an event from the attendance of comparable ones. The escalation module already derives deadlines and detects lateness, so the measurements a forecast would be built on are being collected.

Add push and calendar-level notification.

Notification is currently by email. Adding browser push for time-critical events, and the ability to subscribe to a departmental calendar feed, would suit the operational roles in particular — a member of logistics staff is more likely to be holding a phone than an inbox on the morning of an event.

Broaden the assistant's coverage and add a local fallback.

The assistant answers within topics that are gated on page grants, and both halves of that arrangement could be extended: more topics, and a locally hosted model as a fallback so the capability survives the unavailability of the external provider. The guard, the scope rules and the access log are model-independent, so a second provider is a configuration change rather than a redesign.

Validate the model against a second institution.

The claim that the role, unit and page-grant tables generalise beyond one university is currently a design property rather than a result. Expressing a second institution's structure in the same tables — particularly one whose approval chain differs in shape rather than only in names — would either substantiate the claim or identify precisely where the model is specific to the case it was built for.

The project set out to replace a coordination process that ran on messages, memory and goodwill with one that runs on a record. It has delivered a working platform in which a request has a stage, a stage has an owner, an owner's authority is bounded by the unit they hold it in, and every decision taken along the way is written down with its reason at the moment it is taken. The measure of the work is not that the software exists but that the questions Chapter 1 found nobody could answer — where has my request reached, who is it with, why was it turned down, who agreed to do this — now have answers the system can produce on request.

REFERENCE:

Abdul Rahman, N., Mohd Zin, N. A., Ahmad Zukarnain, Z., & Putra, Y. H. (2024). User-centered UiTM e-Event using predictive software development life cycle (SDLC). Journal of Computing Research and Innovation (JCRINN), 9(2), 290–302. https://doi.org/10.24191/jcrinn.v9i2.468

Alami, A., & Krancher, O. (2022). How Scrum adds value to achieving software quality? Empirical Software Engineering, 27(7), 165. Springer. https://doi.org/10.1007/s10664-022-10208-4

Albesher, L., & Alfayez, R. (2024). An observational study on Flask web framework questions on Stack Overflow (SO). IET Software, 2024(1). https://doi.org/10.1049/sfw2/1905538

Alesandro, N., Maulana, F., Lim, L. F., Hendri, & Robet. (2025). Development of a campus project management system to enhance collaboration using the Agile method. Journal of Artificial Intelligence and Engineering Applications (JAIEA), 4(3), 2294–2304. https://doi.org/10.59934/jaiea.v4i3.1158

Arriesgado, J., Calaguian, J., Golpeo, A., Lagman, A., Pajarillo, M., Pardo, P., Solomo, M., & Vicente, H. (2023). TICaP Hub: An event management system for FEU Tech using DigitalOcean droplets and cron jobs. In Intelligent Sustainable Systems (pp. 225–232). Springer. https://doi.org/10.1007/978-981-19-7660-5_19

Ahmad, T., & Van Looy, A. (2020). Business process management and digital innovations: A systematic literature review. Sustainability, 12(17), 6827. https://doi.org/10.3390/su12176827

Anderer, S., Kempter, T., Scheuermann, B., & Mostaghim, S. (2023). Dynamic optimization of role concepts for role-based access control using evolutionary algorithms. SN Computer Science, 4, 416. https://doi.org/10.1007/s42979-023-01805-1

Benavides, L. M. C., Tamayo Arias, J. A., Arango Serna, M. D., Branch Bedoya, J. W., & Burgos, D. (2020). Digital transformation in higher education institutions: A systematic literature review. Sensors, 20(11), 3291. https://doi.org/10.3390/s20113291

Choudhary, S. R., Prasad, M. R., & Orso, A. (2012). CrossCheck: Combining crawling and differencing to better detect cross-browser incompatibilities in web applications. 2012 IEEE Fifth International Conference on Software Testing, Verification and Validation (ICST), 171–180.https://doi.org/10.1109/ICST.2012.97

Cvent. (2025). Cvent — All-in-one event management and marketing platform. Cvent Inc. Retrieved from https://www.cvent.com

Dalimunthe, S., Putra, E. H., & Ridha, M. A. F. (2023). Restful API security using JSON Web Token (JWT) with HMAC-SHA512 algorithm in session management. IT Journal Research and Development, 8(1), 81–94. https://doi.org/10.25299/itjrd.2023.12029

Essel, D. D., Techie-Menson, H., Oppong, S. O., & Alakuu, A. (2023). Digitalising the clearance processes of higher education institutions through the design and implementation of an online clearance system. International Journal of Scientific Research and Management (IJSRM), 11(7), 42–58. https://doi.org/10.18535/IJSRM/V11I07.AS01

Fernández-Portillo, A., Almodóvar-González, M., & Hernández-Mogollón, R. (2023). Digital transformation initiatives in higher education institutions: A multivocal literature review. Education and Information Technologies, 28, 12375–12406. https://doi.org/10.1007/s10639-022-11544-0

Habib, M. N., Jamal, W., & Khalil, U. (2021). Transforming universities in interactive digital platform: Case of City University of Science and Information Technology. Education and Information Technologies, 26, 517–541. https://doi.org/10.1007/s10639-020-10237-w

Hadiwiyanti, R., Suryanto, T. L. M., & Safitri, E. M. (2022). Evaluation of campus event management information system using system usability scale method. In 2022 IEEE 8th Information Technology International Seminar (ITIS) (pp. 350–353). IEEE. https://doi.org/10.1109/ITIS57155.2022.10009978

Hron, M., & Obwegeser, N. (2022). Why and how is Scrum being adapted in practice: A systematic review. Journal of Systems and Software, 183, 111110. https://doi.org/10.1016/j.jss.2021.111110

Hron, M., & Obwegeser, N. (2018). Scrum in practice: An overview of Scrum adaptations. In Proceedings of the 51st Hawaii International Conference on System Sciences (pp. 5445–5454). https://doi.org/10.24251/hicss.2018.679

Ismail, R., Safieddine, F., & Jaradat, A. (2019). E-university delivery model: Handling the evaluation process. Business Process Management Journal, 25(7), 1633–1646. https://doi.org/10.1108/BPMJ-10-2018-0281

ISGlobal. (n.d.). *SDG 9: Build resilient infrastructure, promote inclusive and sustainable industrialization and foster innovation*. https://www.isglobal.org/en/-/sdg-9-build-resilient-infrastructure-promote-inclusive-and-sustainable-industrialization-and-foster-innovation

Jeny, J. R. V., Sadhana, P., Kumar, B. J., Abhishek, S. L., & Chander, T. S. (2022). A web-based college event management system and notification sender. In 2022 4th International Conference on Inventive Research in Computing Applications (ICIRCA) (pp. 1434–1438). IEEE. https://doi.org/10.1109/ICIRCA54612.2022.9985774

Jiao, T., Ye, M., & Cai, Z. (2022). Role-based access control model for inter-system cross-domain in multi-domain environment. Applied Sciences, 12(24), 13036. https://doi.org/10.3390/app122413036

Kadenic, M. D., Koumaditis, K., & Junker-Jensen, L. (2023). Mastering scrum with a focus on team maturity and key components of scrum. Information and Software Technology, 153, 107079. https://doi.org/10.1016/j.infsof.2022.107079

Kaputa, V., Loučanová, E., & Tejerina-Gaite, F. A. (2022). Digital transformation in higher education institutions as a driver of social-oriented innovations. In Social Innovation in Higher Education (Vol. 61, pp. 81–85). Springer. https://doi.org/10.1007/978-3-030-84044-0_4

Khan, M., Parvaiz, G. S., Dedahanov, A. T., Abdurazzakov, O. S., & Rakhmonov, D. A. (2022). The impact of technologies of traceability and transparency in supply chains. Sustainability, 14(24), 16336. https://doi.org/10.3390/su142416336

Matheus, R., Janssen, M., & Maheshwari, D. (2020). Data science empowering the public: Data-driven dashboards for transparent and accountable decision-making in smart cities. Government Information Quarterly, 37(3), 101284. https://doi.org/10.1016/j.giq.2018.01.006

Moy, A. J., Hobensack, M., Marshall, K., Vawdrey, D. K., Kim, E. Y., Cato, K. D., & Rossetti, S. C. (2023). Understanding the perceived role of electronic health records and workflow fragmentation on clinician documentation burden in emergency departments. Journal of the American Medical Informatics Association, 30(5), 797–808. https://doi.org/10.1093/jamia/ocad038

https://doi.org/10.1007/978-3-642-12636-9_9 Mualla, W., & Mualla, K. J. (2024). Digital transformation in higher education:

Reijers, H. A. (2021). Business process management: The evolution of a discipline. Computers in Industry, 126, 103404. ScienceDirect. https://doi.org/10.1016/j.compind.2021.103404

Reijers, H. A., Vanderfeesten, I., & van der Aalst, W. M. P. (2016). The effectiveness of workflow management systems: A longitudinal study. International Journal of Information Management, 36(1), 126–141. https://doi.org/10.1016/j.ijinfomgt.2015.08.003

Salunke, S. V., & Ouda, A. (2024). A performance benchmark for the PostgreSQL and MySQL databases. Future Internet, 16(10), 382. https://doi.org/10.3390/fi16100382

Schwaber, K., & Sutherland, J. (2020). The Scrum guide: The definitive guide to Scrum — The rules of the game. Scrum.org. https://scrumguides.org/docs/scrumguide/v2020/2020-Scrum-Guide-US.pdf

Shi, L., Tang, Z., Zhang, N., Zhang, X., & Yang, Z. (2025). A survey on employing large language models for text-to-SQL tasks. ACM Computing Surveys. https://doi.org/10.1145/3737873

Srivastava, A. (2019). A comparative analysis of software development models based on various parameters. International Journal of Computer Sciences and Engineering, 7(9), 17–21. https://ijcseonline.org/index.php/j/article/view/7072

Tan, J., Chen, Y., & Jiao, S. (2023). Visual Studio Code in introductory computer science course: An experience report. ACM. https://arxiv.org/pdf/2303.10174

Thesing, T., Feldmann, C., & Burchardt, M. (2021). Agile versus Waterfall project management: Decision model for selecting the appropriate approach to a project. Procedia Computer Science, 181, 746–756. https://doi.org/10.1016/j.procs.2021.01.227

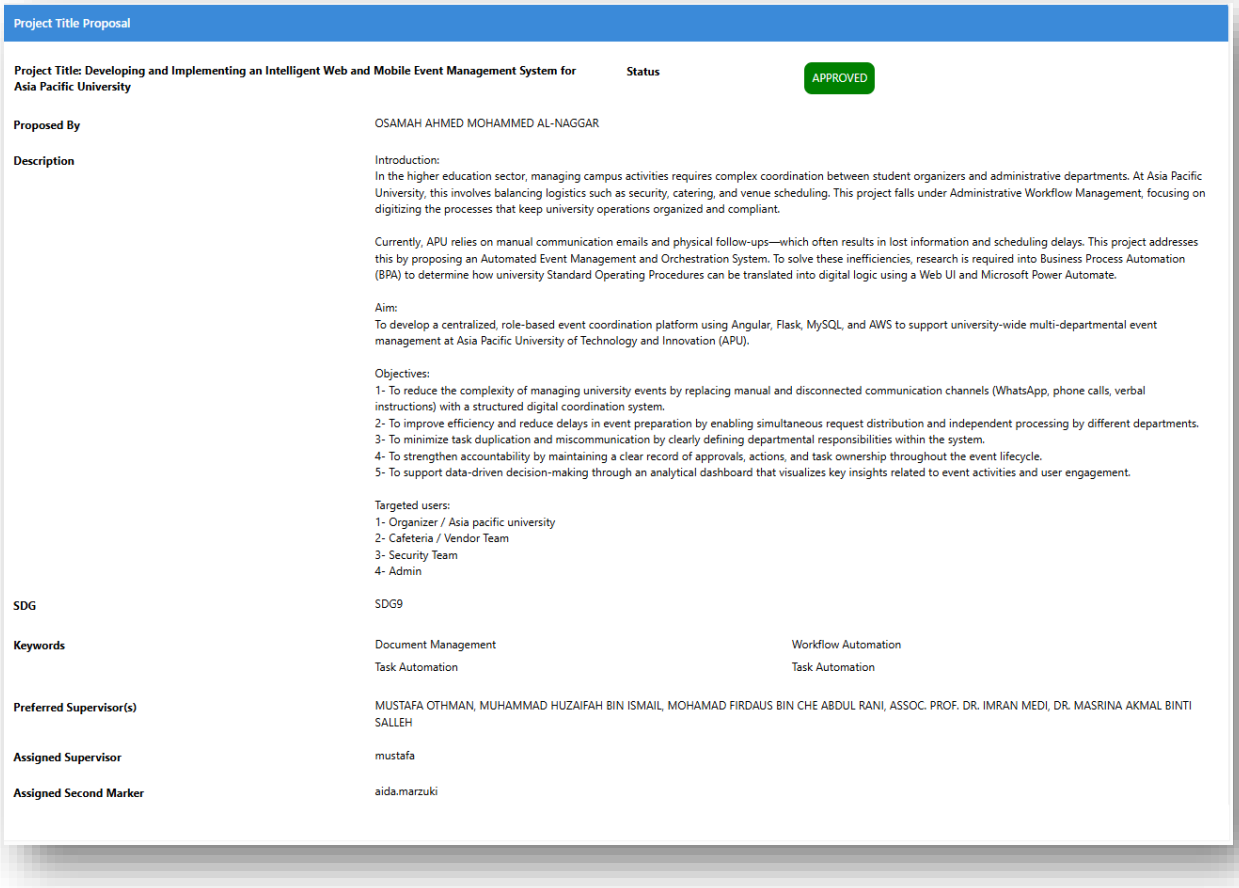
Urnikienė, J., Steponavičienė, V., & Atanasov, S. (2026). Comparative read performance analysis of PostgreSQL and MongoDB in e-commerce: An empirical study of filtering and analytical queries. Big Data and Cognitive Computing, 10(2), 66. https://doi.org/10.3390/bdcc10020066

Vashisht, V. S. (2021). Angular and Svelte Frameworks: a Comparative Analysis. 2021 16th International Conference on Engineering of Modern Electric Systems (EMES), 1–4. https://doi.org/10.1109/EMES52324.2021.9484119

Zubair, M., Sabzevari, M., Khatri, V., Tarkoma, S., & Hätönen, K. (2024). Access control for trusted data sharing. EURASIP Journal on Information Security, 2024, 30. https://doi.org/10.1186/s13635-024-00178-z

APPENDICES

Appendix A: Project Proposal Form (PPF)



Project Title Proposal

Project Title: Developing and Implementing an Intelligent Web and Mobile Event Management System for Asia Pacific University

Status APPROVED

Proposed By OSAMAH AHMED MOHAMMED AL-NAGGAR

Description

Introduction:
In the higher education sector, managing campus activities requires complex coordination between student organizers and administrative departments. At Asia Pacific University, this involves balancing logistics such as security, catering, and venue scheduling. This project falls under Administrative Workflow Management, focusing on digitizing the processes that keep university operations organized and compliant.

Currently, APU relies on manual communication emails and physical follow-ups—which often results in lost information and scheduling delays. This project addresses this by proposing an Automated Event Management and Orchestration System. To solve these inefficiencies, research is required into Business Process Automation (BPA) to determine how university Standard Operating Procedures can be translated into digital logic using a Web UI and Microsoft Power Automate.

Aim:
To develop a centralized, role-based event coordination platform using Angular, Flask, MySQL, and AWS to support university-wide multi-departmental event management at Asia Pacific University of Technology and Innovation (APU).

Objectives:
1- To reduce the complexity of managing university events by replacing manual and disconnected communication channels (WhatsApp, phone calls, verbal instructions) with a structured digital coordination system.
2- To improve efficiency and reduce delays in event preparation by enabling simultaneous request distribution and independent processing by different departments.
3- To minimize task duplication and miscommunication by clearly defining departmental responsibilities within the system.
4- To strengthen accountability by maintaining a clear record of approvals, actions, and task ownership throughout the event lifecycle.
5- To support data-driven decision-making through an analytical dashboard that visualizes key insights related to event activities and user engagement.

Targeted users:
1- Organizer / Asia pacific university
2- Cafeteria / Vendor Team
3- Security Team
4- Admin

SDG SDG9

Keywords Document Management, Workflow Automation, Task Automation, Task Automation

Preferred Supervisor(s) MUSTAFA OTHMAN, MUHAMMAD HUZAIFAH BIN ISMAIL, MOHAMAD FIRDAUS BIN CHE ABDUL RANI, ASSOC. PROF. DR. IMRAN MEDI, DR. MASRINA AKMAL BINTI SALLEH

Assigned Supervisor mustafa

Assigned Second Marker aida.marzuki

***Figure 131**: Proposal Form (PPF).*

Appendix B: Ethics Forms

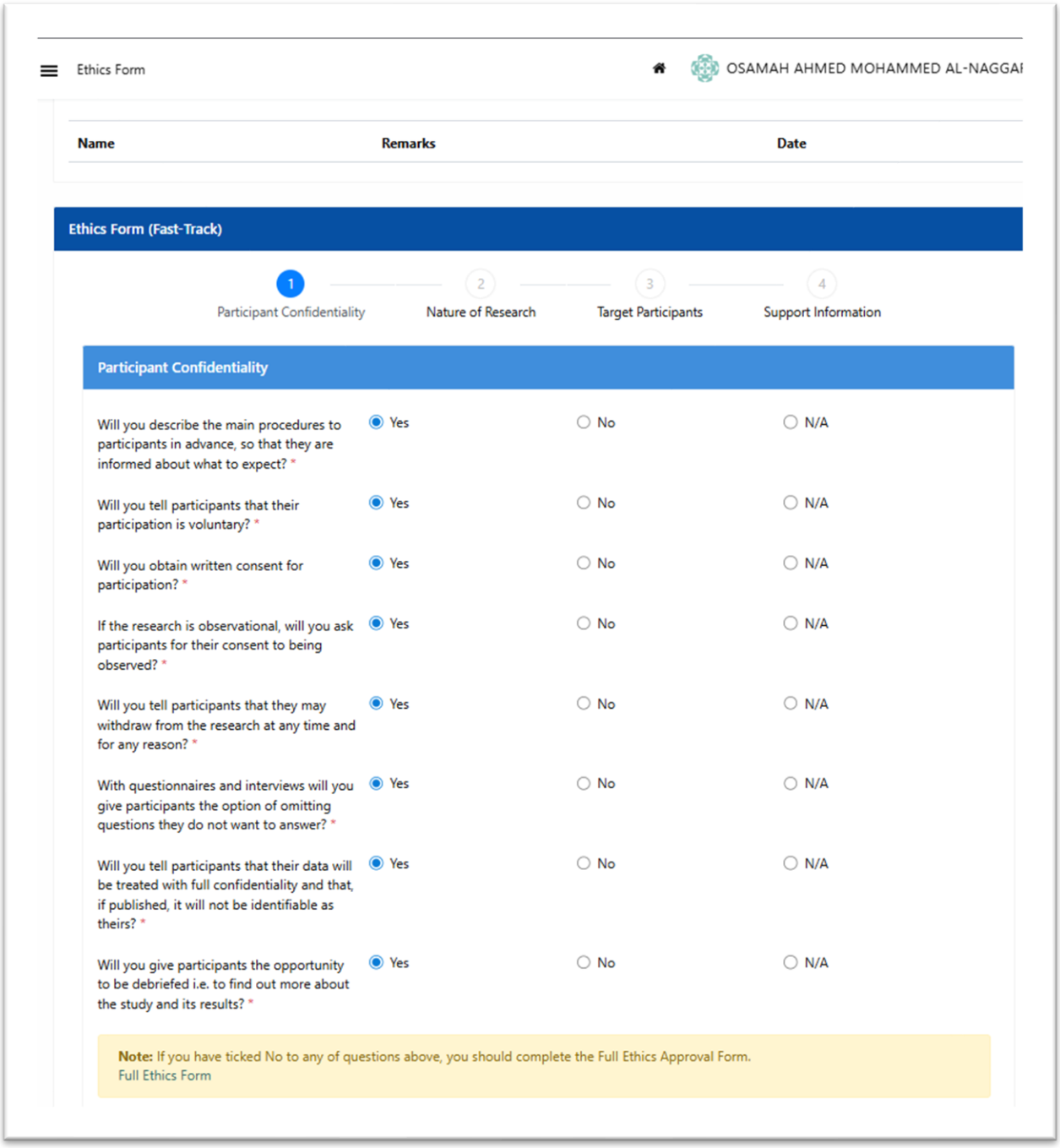


***Figure 132**: Proposal Form (PPF).*

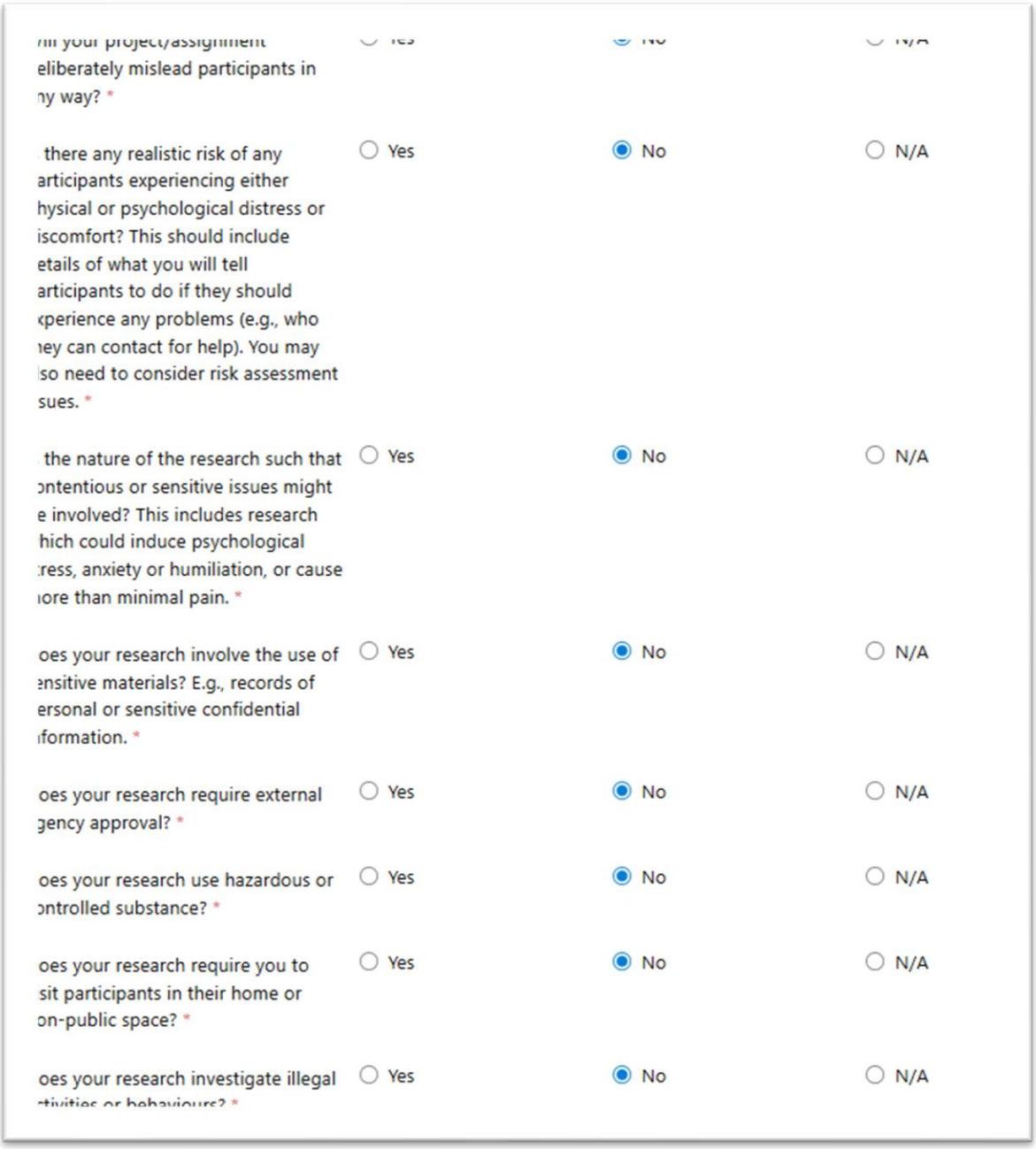


Figure 133: *Ethics Forms — Nature of Research - part 1.*

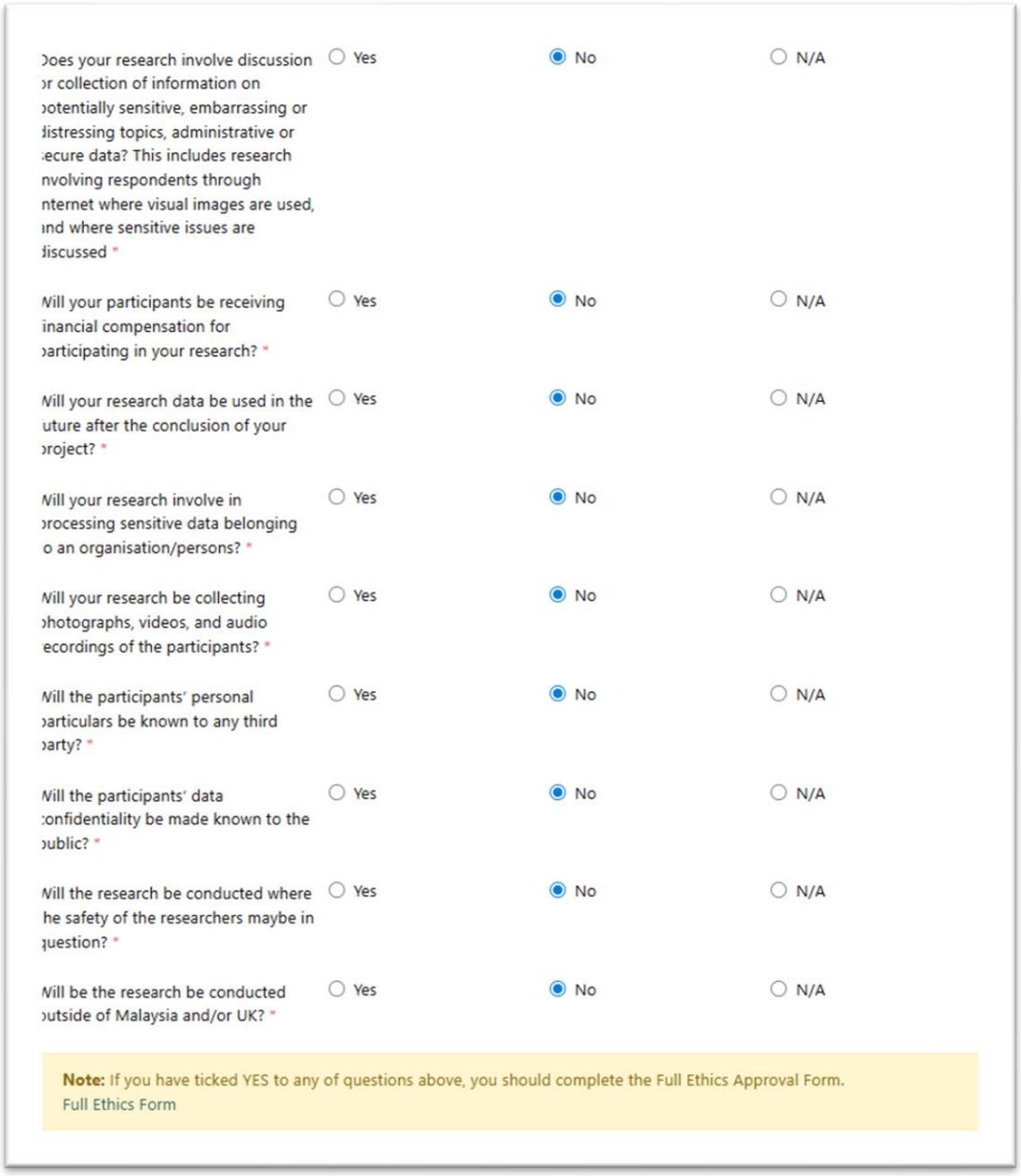


***Figure 134**: Ethics Forms — Nature of Research - part 2.*

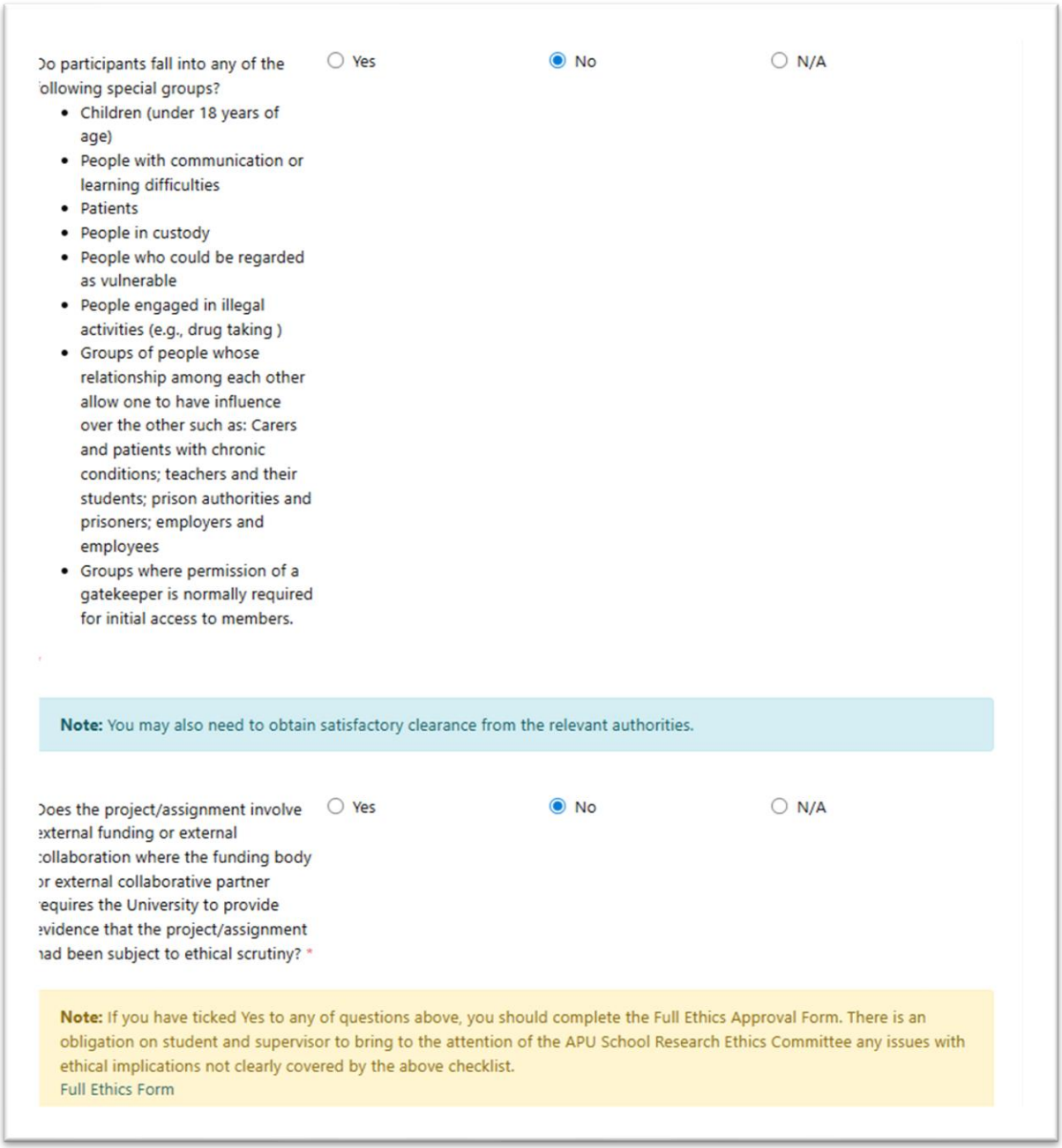


Figure 135: *Ethics Forms — Target Participants.*

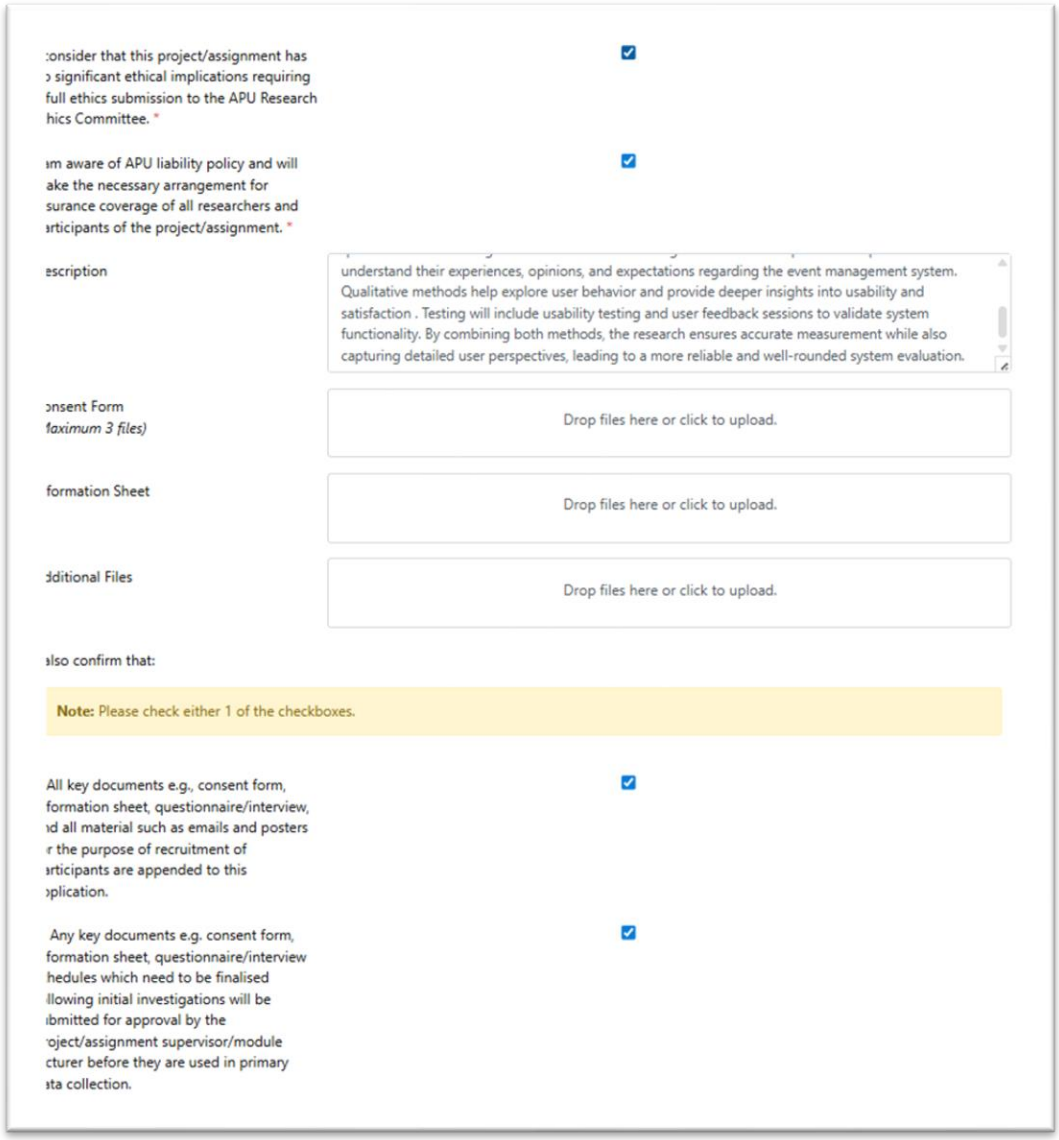
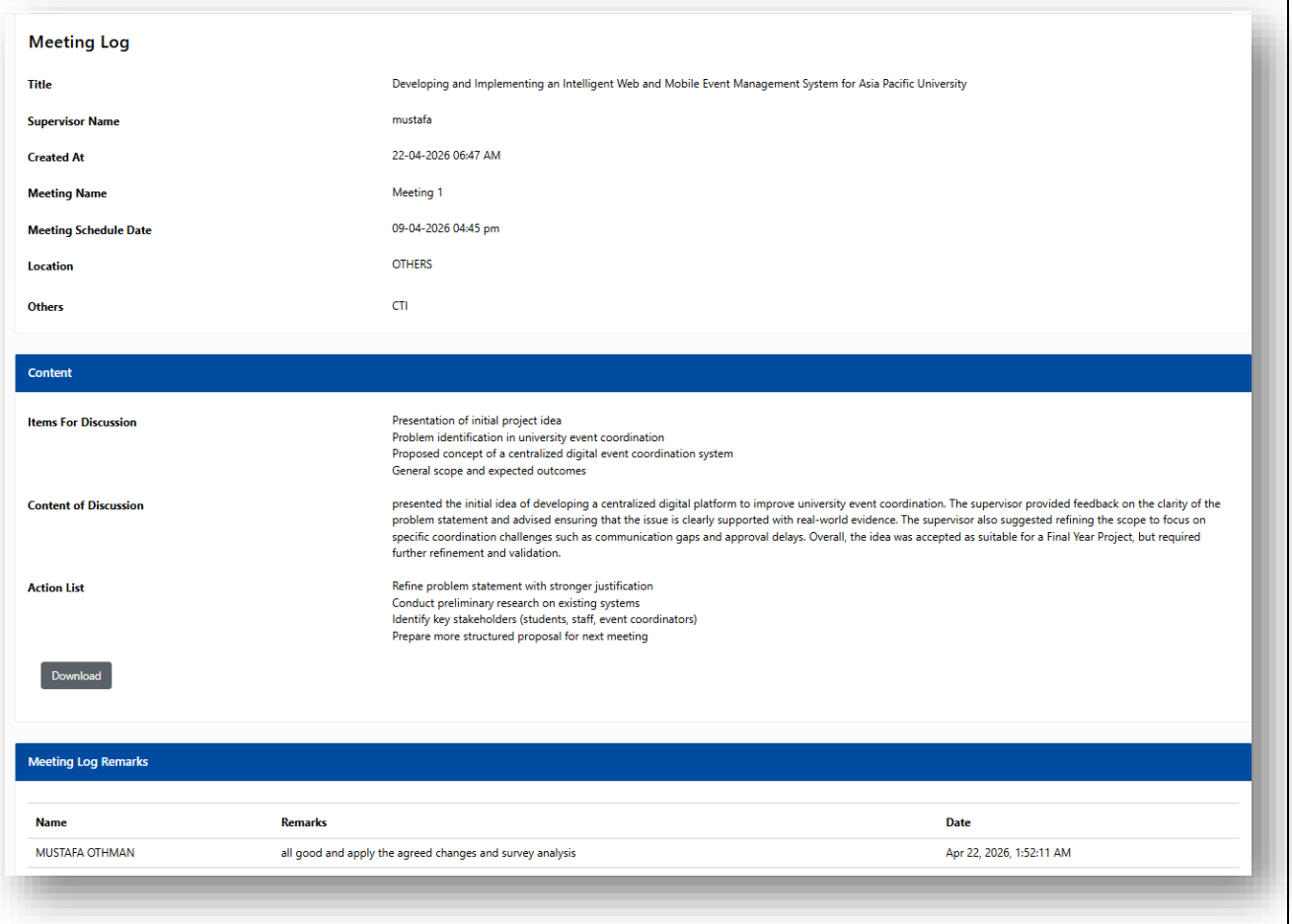


Figure 136: *Ethics Forms — Target Participants.*

Appendix C: Log Sheets



Meeting Log

Title	Developing and Implementing an Intelligent Web and Mobile Event Management System for Asia Pacific University
Supervisor Name	mustafa
Created At	22-04-2026 06:47 AM
Meeting Name	Meeting 1
Meeting Schedule Date	09-04-2026 04:45 pm
Location	OTHERS
Others	CTI

Content

Items For Discussion	Presentation of initial project idea Problem identification in university event coordination Proposed concept of a centralized digital event coordination system General scope and expected outcomes
Content of Discussion	presented the initial idea of developing a centralized digital platform to improve university event coordination. The supervisor provided feedback on the clarity of the problem statement and advised ensuring that the issue is clearly supported with real-world evidence. The supervisor also suggested refining the scope to focus on specific coordination challenges such as communication gaps and approval delays. Overall, the idea was accepted as suitable for a Final Year Project, but required further refinement and validation.
Action List	Refine problem statement with stronger justification Conduct preliminary research on existing systems Identify key stakeholders (students, staff, event coordinators) Prepare more structured proposal for next meeting

Download

Meeting Log Remarks

Name	Remarks	Date
MUSTAFA OTHMAN	all good and apply the agreed changes and survey analysis	Apr 22, 2026, 1:52:11 AM

Figure 137: *Log Sheets – meeting 1.*

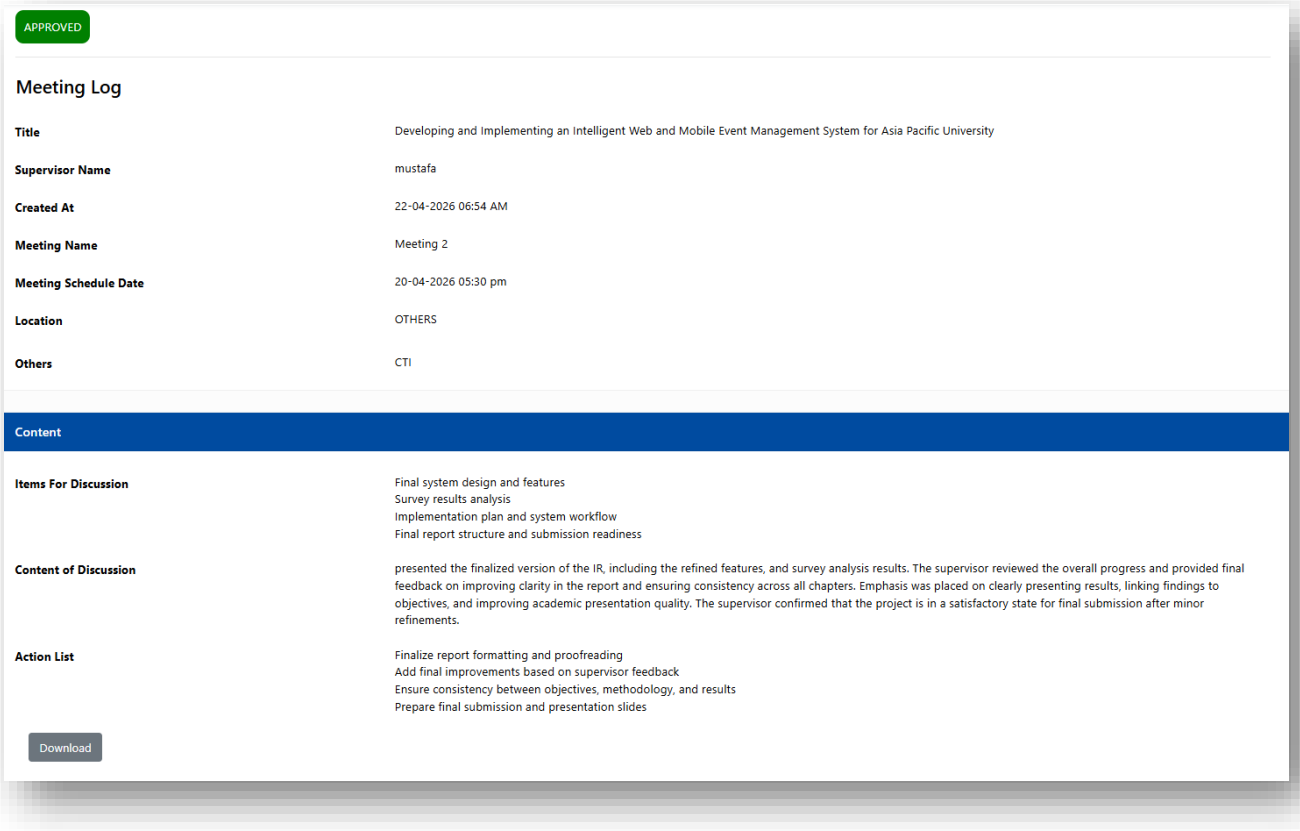


***Figure 138**: Log Sheets – meeting 2.*

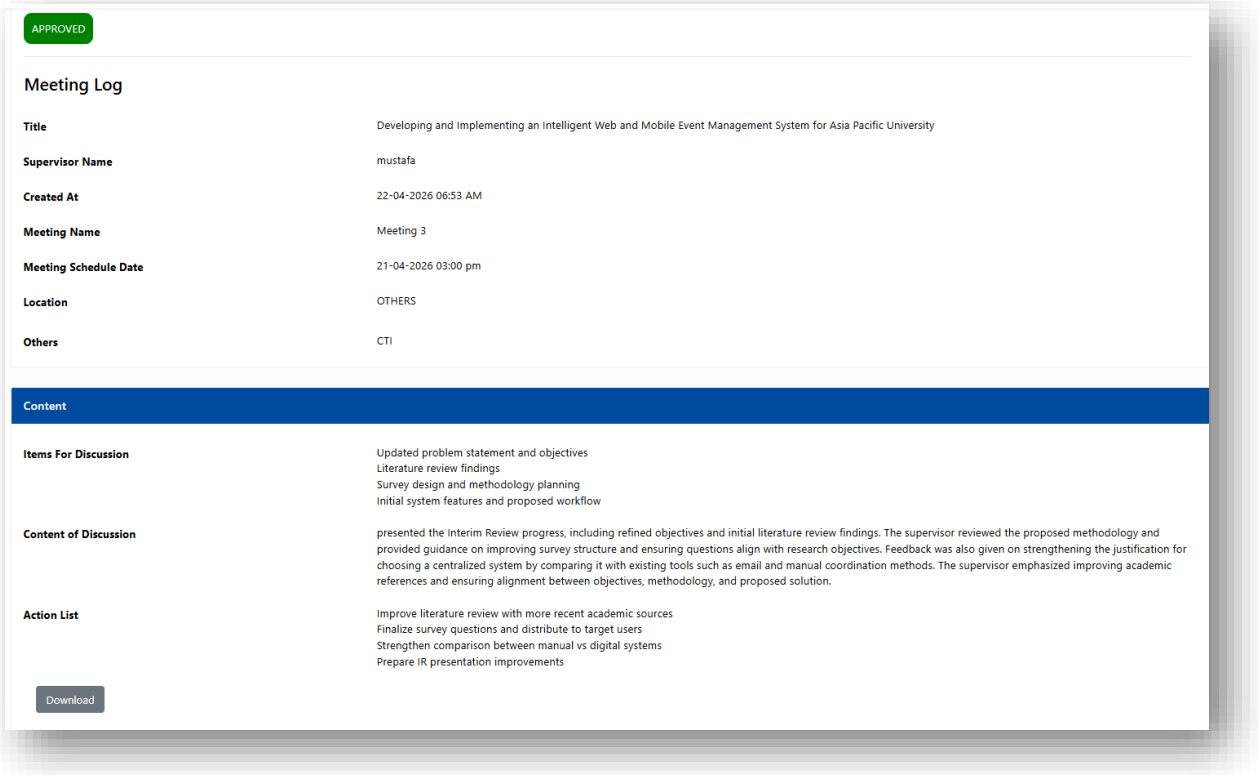


***Figure 139**: Log Sheets – meeting 3.*

Appendix D: Gantt Chart

Centralized Role-Based Event Coordination Platform — IR Semester Timeline.

Task / Activity	W1 Feb 9	W2 Feb 16	W3 Feb 23	W4 Mar 2	W5 Mar 9	W6 Mar 16	W7 Mar 23	W8 Mar 30	W9 Apr 6	W10 Apr 13
Phase 1 — Project Initiation & Planning										
Title Registration & PPF Submission	•									
Supervisor Meeting #1 — Project Briefing	•									
Ethics Form Submission	•	•								
Project Scope & Objectives Definition	•	•								
Phase 2 — Chapter 1: Introduction										
Problem Background Research & Writing		•	•							
Project Aim, Objectives & Scope Definition		•	•							
Potential Benefits & IR Overview Writing			•							
Project Plan Table (Section 1.8) Drafting			•	•						
Phase 3 — Chapter 2: Literature Review										
2.2.1 Event Management in Academic Institutions			•	•						
2.2.2 Multi-Dept Coordination & Workflow Mgmt.			•	•	•					
2.2.3 Role-Based Access Control (RBAC)				•	•					
2.2.4 Centralized Digital Platforms				•	•					
2.2.5 SDG Goal 9 & Digital Infrastructure					•					
2.3 Similar Systems / Works Analysis					•	•				
2.4 Technical Research (Angular, Flask, MySQL, AWS)					•	•				

Figure 140: *Gantt Chart – part 1.*

Task / Activity										
Phase 4 — Chapter 3: Methodology										
3.2 Scrum Methodology Selection & Justification						•	•			
3.2.3 Six Sprint Phase Mapping (FYP Part 2)						•	•			
3.3 Data Gathering Design — Interview Instrument							•			
Supervisor Verification for Data Gathering							•			
3.3 Conduct Interviews (Min. 3 Participants)							•	•		
3.4 Data Analysis & Visualizations								•		
Phase 5 — Chapter 4: Conclusion & Report Finalisation										
4.1–4.3 Conclusion & SDG Alignment Writing							•	•		
References Compilation & Formatting						•	•	•		
Appendix Compilation (A, B, C, E, F)							•	•		
Gantt Chart Creation (Appendix D)							•	•		
Full IR Proofreading & Revision								•	•	
Turnitin Submission & Similarity Check								•	•	
Supervisor Meeting — Final Review								•	•	
Phase 6 — Submission										
Final IR Submission (Deadline: Apr 18, 2026)									•	

Legend: Phase 1: Initiation | Phase 2: Intro | Phase 3: Literature | Phase 4: Methodology | Phase 5: Conclusion | Phase 6: Submission

Note: Shaded cells (•) indicate active weeks for each task. This chart covers FYP Part 1 (IR) only. FYP Part 2 follows the 6-Sprint Scrum development timeline as described in Section 3.2.3.

Figure 141: *Gantt Chart – part 2.*

Appendix E: Respondent Demographic Profile

Table 52: *Respondent Demographic Profile*

No	Name	Gender	Age	Role
1	Participant 1	Female	24 - 26	Student
2	Participant 2	Female	21 - 23	Student
3	Participant 3	Male	18 -20	Student
4	Participant 4	Female	18 -20	Student
5	Participant 5	Male	18 -20	Student
6	Participant 6	Male	24 - 26	Staff
7	Participant 7	Male	18 -20	Student
8	Participant 8	Male	21 - 23	Student
9	Participant 9	Male	Above 26	Staff
10	Participant 10	Female	18 -20	Student
11	Participant 11	Male	21 - 23	Student
12	Participant 12	Male	21 - 23	Student
13	Participant 13	Male	Above 26	Staff
14	Participant 14	Male	24 - 26	Student
15	Participant 15	Male	21 - 23	Student
16	Participant 16	Male	21 - 23	Student
17	Participant 17	Female	18 -20	Student
18	Participant 18	Male	18 -20	Student
19	Participant 19	Male	21 - 23	Student
20	Participant 20	Female	21 - 23	Student

21	Participant 21	Male	18 -20	Student
22	Participant 22	Male	18 -20	Student
23	Participant 23	Female	18 -20	Student
24	Participant 24	Male	18 -20	Student
25	Participant 25	Female	Above 26	Staff
26	Participant 26	Male	Above 26	Staff
27	Participant 27	Male	Above 26	Staff
28	Participant 28	Female	21 - 23	Student
29	Participant 29	Female	18 -20	Student
30	Participant 30	Male	18 -20	Student

Note: **Table 52** presents the demographic profile of respondents who participated in the study. It summarizes key background characteristics including gender, age group, and role (student or staff). The dataset consists of 30 respondents selected to represent different age ranges and institutional roles, allowing for a balanced overview of the sample population used in this research.

Appendix F: Turnitin Similarity Report

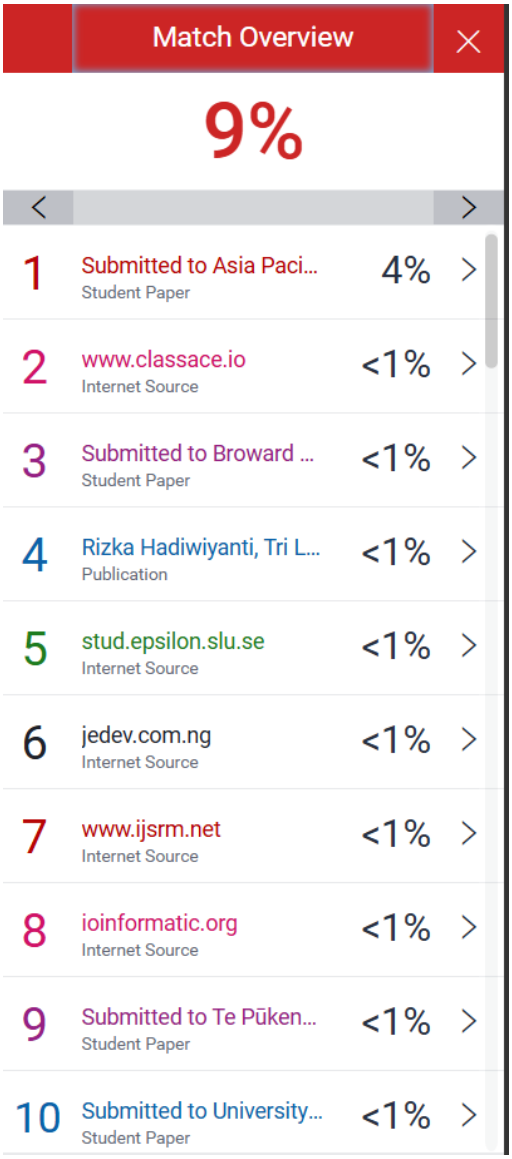


Figure 142 : *Similarity Report – 1.*

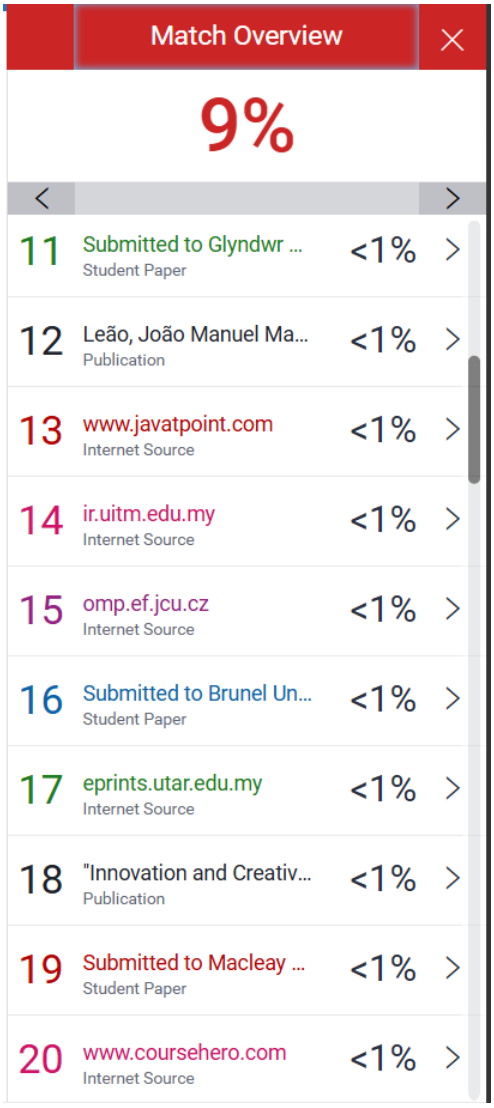


Figure 143: *Similarity Report – 2.*